

Table of Contents

1. Introduction	1
1.1 Introduction to the Project	1
1.2 Purpose of the Project.....	2
1.3 Audience of the Project.....	2
1.4 Aim and Objectives.....	3
1.5 Feature List	4
2. Database Design and Development.....	8
2.1 Entity Relationship Diagram (ERD) with Business Rules	8
2.2 Normalization (up to 3NF) with Table Attributes	11
2.3 Database Development: Data Dictionary with Sample Data	18
2.3.1 Table 1: User	19
2.3.2 Table 2: Plant_Product.....	20
2.3.3 Table 3: Aquarium_Product.....	22
2.3.4 Table 4: Order.....	23
2.3.5 Table 5: Order_Item.....	25
2.3.6 Table 6: Review	27
2.3.7 Table 7: Wishlist.....	29
2.3.8 Table 8: Cart	31
2.3.9 Table 9: Payment.....	33
2.3.10 Table 10: Admin	35
2.4 Relationship Diagram Screenshot (from XAMPP/phpMyAdmin)	37
3. Wireframe and Design	39
3.1 Landing Page.....	39
3.1.1 Research Reference	39
3.1.2 Wireframe Design.....	42
3.1.3 Actual Design.....	43
3.1.4 Design Development	46
3.2 Login Page(s)	47
3.2.1 Research Reference	47
3.2.2 Wireframe Design.....	49
3.2.3 Actual Design.....	50

3.2.4	Design Development	55
3.3	Register Page(s)	56
3.3.1	Research Reference	56
3.3.2	Wireframe Design.....	58
3.3.3	Actual Design.....	59
3.3.4	Design Development	65
3.4	Home Page.....	66
3.4.1	Research Reference	66
3.4.2	Wireframe Design.....	68
3.4.3	Actual Design.....	69
3.4.4	Design Development	72
3.5	Shop Page	72
3.5.1	Research Reference	73
3.5.2	Wireframe Design.....	75
3.5.3	Actual Design.....	76
3.5.4	Design Development	79
3.6	Track-Order Page	80
3.6.1	Research Reference	81
3.6.2	Actual Design.....	83
3.6.3	Design Development	86
3.7	Wishlist Page	86
3.7.1	Research Reference	86
3.7.2	Wireframe Design.....	89
3.7.3	Actual Design.....	90
3.8	Contact Page.....	93
3.8.1	Research Reference	93
3.8.2	Wireframe Design.....	96
3.8.3	Actual Design.....	97
3.8.4	Design Development	100
3.9	Care Guide Page.....	101
3.9.1	Research Reference	101
3.9.2	Actual Design.....	103
3.9.3	Design Development	106
3.10	About Page	106

3.10.1	Research Reference	107
3.10.2	Actual Design.....	108
3.10.3	Design Development	111
3.11	User Dashboard	111
3.11.1	Wireframe Design.....	112
3.11.2	Actual Design.....	113
3.11.3	Design Development	116
3.12	User Orders	117
3.12.1	Wireframe Design.....	117
3.12.2	Actual Design.....	118
3.12.3	Design Development	120
3.13	User Wishlist	121
3.13.1	Wireframe Design.....	122
3.13.2	Actual Design.....	123
3.14	User Reviews	126
3.14.1	Wireframe Design.....	126
3.14.2	Actual Design.....	127
3.14.3	Design Development	130
3.15	User Profile.....	130
3.15.1	Wireframe Design.....	131
3.15.2	Actual Design.....	132
3.15.3	Design Development	135
3.16	User Notification.....	135
3.16.1	Actual Design.....	136
3.16.2	Design Development	139
3.17	User Settings.....	139
3.17.1	Wireframe Design.....	140
3.17.2	Actual Design.....	141
3.17.3	Design Development	144
3.18	Admin Dashboard	144
3.18.1	Wireframe Design.....	145
3.18.2	Actual Design.....	146
3.18.3	Design Development	149
3.19	Admin Analytics	149

3.19.1	Wireframe Design.....	150
3.19.2	Actual Design.....	151
3.19.3	Design Development	154
3.20	Admin Product Management.....	154
3.20.1	Wireframe Design.....	155
3.20.2	Actual Design.....	156
3.20.3	Design Development	159
3.21	Admin Order Management.....	159
3.21.1	Actual Design.....	160
3.21.2	Design Development	163
3.22	Admin Customer Management.....	163
3.22.1	Wireframe Design.....	164
3.22.2	Actual Design.....	165
3.22.3	Design Development	167
3.23	Admin Review Moderation	168
3.23.1	Wireframe Design.....	169
3.23.2	Actual Design.....	169
3.23.3	Design Development	172
3.24	Admin Settings	172
3.24.1	Actual Design.....	173
3.24.2	Design Development	175
4.	Java Classes and Methods.....	176
4.1	Overall Class Diagram.....	177
4.2	DAO Methods	180
5.	Test Cases.....	204
5.1	Test Plan Table.....	204
5.1.1	Session & Cookie Test.....	214
6.	Coursework Development with Analysis	220
6.1	Team Contribution Table	220
6.2	Team Member 1: Dibyant Bhatta (Group Leader)	221
6.2.1	Completed Tasks with Evidence	221
6.3	Team Member 2: Smarika Mainali.....	224
6.3.1	Completed Tasks with Evidence	224

6.4	Team Member 3: Stuti Adhikari.....	227
6.4.1	Completed Tasks with Evidence	227
6.5	Team Member 4: Rakshya Bhandari.....	230
6.5.1	Completed Tasks with Evidence	230
6.6	Team Member 5: Aayush Shrestha.....	233
6.6.1	Completed Tasks with Evidence	233
7.	Conclusion.....	237
8.	References	Error! Bookmark not defined.

Table of Figures

Figure 1: Entity Relationship Diagram (ERD)	9
Figure 2: Relational Model	18
Figure 3: User Table.....	20
Figure 4: Plant_Product Table.....	21
Figure 5: Aquarium_Product Table	23
Figure 6: Oder Table	25
Figure 7: Order_Item Table	27
Figure 8: Review Table	29
Figure 9: Wishlist Table.....	31
Figure 10: Cart Table	33
Figure 11: Payment Table	35
Figure 12: Admin Table	37
Figure 13: Relationship Diagram Screenshot.....	38
Figure 14: Landing	40
Figure 15: landing 2.....	41
Figure 16: Wireframe.....	42
Figure 17: Actual	43
Figure 18:Actual 2	44
Figure 19: Actual 3	45
Figure 20: Final design.....	46
Figure 21: Reference.....	48
Figure 22	49
Figure 23	50
Figure 24	51
Figure 25	52
Figure 26	53
Figure 27	53
Figure 28	54
Figure 29	55
Figure 30	55
Figure 31	57
Figure 32	58
Figure 33	59
Figure 34	60
Figure 35	61
Figure 36	62
Figure 37	63
Figure 38	64
Figure 39	65
Figure 40	65
Figure 41	67
Figure 42	68
Figure 43	69

Figure 44	70
Figure 45	71
Figure 46	72
Figure 47	74
Figure 48	75
Figure 49	76
Figure 50	77
Figure 51	78
Figure 52	79
Figure 53	80
Figure 54	82
Figure 55	83
Figure 56	84
Figure 57	85
Figure 58	86
Figure 59	87
Figure 60	89
Figure 61	90
Figure 62	91
Figure 63	92
Figure 64	94
Figure 65	96
Figure 66	97
Figure 67	98
Figure 68	99
Figure 69	100
Figure 70	102
Figure 71	103
Figure 72	104
Figure 73	105
Figure 74	106
Figure 75	107
Figure 76	108
Figure 77	109
Figure 78	110
Figure 79	111
Figure 80	112
Figure 81	113
Figure 82	114
Figure 83	115
Figure 84	116
Figure 85\	117
Figure 86\	118
Figure 87	119
Figure 88	120
Figure 89	121

Figure 90	122
Figure 91	123
Figure 92	124
Figure 93	125
Figure 94	126
Figure 95	127
Figure 96	128
Figure 97	129
Figure 98	130
Figure 99	131
Figure 100	132
Figure 101	133

Table of Tables

Table 1: Customer-side features	4
Table 2: Admin-side features	6
Table 3: User Table	19
Table 4: Plant_Product Table	20
Table 5: Aquarium_Product Table	22
Table 6: Oder Table	23
Table 7: Order_Item Table	25
Table 8: Review Table	27
Table 9: Wishlist Table	29
Table 10: Cart Table	31
Table 11: Payment Table	33
Table 12: Admin Table	35
Table 13	180
Table 14	181
Table 15	183
Table 16	184
Table 17	185
Table 18	186
Table 19	187
Table 20	188
Table 21	189
Table 22	190
Table 23	191
Table 24	192
Table 25	193
Table 26	194
Table 27	195
Table 28	197
Table 29	198
Table 30	200
Table 31	201
Table 32	202
Table 33	203
Table 34	204
Table 35	205
Table 36	206
Table 37	207
Table 38: Session & Cookie Test	214
Table 39: Team Contribution	220

Abstract

This report presents the design, development, and evaluation of Finora, a full-stack e-commerce web application built as part of the CS5054NI Advanced Programming and Technologies module. Finora is a purpose-built platform dedicated to the sale of indoor plants and aquarium products, consolidating two related hobby domains into a single, unified shopping experience.

The system is implemented using Java Servlets and JSP for the backend, MySQL for relational data storage, and plain CSS with Flexbox for a responsive, framework-free frontend. The application follows the Model-View-Controller (MVC) architectural pattern to ensure a clean separation between business logic, data access, and presentation layers.

The project covers the full software development lifecycle. The database was designed and normalised to Third Normal Form (3NF), encompassing ten relational tables for users, products, orders, payments, cart sessions, wishlists, reviews, and admin accounts. A data dictionary with sample data was produced to document each table's attributes, constraints, and relationships. The entity-relationship diagram and relational model capture the structural design of the system.

Wireframes and design references were produced for all major pages, including the landing page, home page (in plant and aquarium states), customer profile, and multiple admin management pages covering products, orders, customers, and review moderation. Java DAO classes, utility classes, and controller servlets were documented alongside an overall class diagram to illustrate the backend architecture.

Security features include BCrypt password hashing, session-based authentication with configurable timeouts, persistent cookie-based login, and role-based access control separating customer and administrator privileges. Structured test cases covering user registration, login, session handling, and admin CRUD operations were executed and documented with expected and actual outcomes.

Individual contributions from all five team members are recorded with evidence of completed tasks and analysis of challenges encountered during development. The result is a functional, secure, and maintainable e-commerce prototype that demonstrates practical application of advanced programming concepts in a real-world context.

Acknowledgement

We would like to express our sincere gratitude to our Module Leader, Ms. Rabina Lama and Mr. Nishan Poudel for their continuous guidance, constructive feedback, and support throughout the duration of this coursework. Their clear instruction and availability to address our questions were instrumental in shaping both our understanding of the subject and the quality of our final submission.

We are grateful to London Metropolitan University and the faculty of computing for providing the academic framework, resources, and learning environment that made this project possible. The module structure gave us the opportunity to apply theoretical knowledge to a practical, real-world development challenge.

We would also like to acknowledge the collaborative effort of every member of our group. Each team member Dibyant Bhatta, Smarika Mainali, Stuti Adhikari, Rakshya Bhandari, and Aayush Shrestha contributed meaningfully across different aspects of the project, from database design and backend development to frontend implementation, testing, and documentation. The spirit of cooperation and mutual support within the team made this work achievable.

Finally, we would like to thank the developers and communities behind the open-source tools and reference platforms used throughout this project, whose publicly available resources provided valuable inspiration for the design and functionality of Finora.

1. Introduction

1.1 Introduction to the Project

Finora is a web-based e-commerce platform dedicated exclusively to the sale of plants and aquarium products. Unlike general online marketplaces, Finora consolidates these two related hobby domains into a single purpose-built system. A customer can browse moisture-retaining substrates for a Monstera, purchase a sponge filter for a shrimp tank, and complete both transactions in one checkout flow without switching between multiple applications or tabs.

The conceptual origin of Finora stems from a simple observation: individuals setting up indoor gardens or aquatic environments often waste time navigating disparate platforms, each with inconsistent product information. Finora eliminates that friction by offering a unified catalogue where plant care instructions and aquarium specifications coexist under one roof.

The platform supports two distinct user roles. Regular customers use the front-end interface to search, review, and purchase items. Administrators operate on the back end, managing inventory, processing orders, overseeing user accounts, and moderating reviews. Finora is implemented in Java, JSP, CSS, and MySQL following the Model-View-Controller (MVC) architecture. This separation of business logic, presentation and request handling ensures maintainability and allows future updates to any component without breaking the rest. The underlying relational database is normalised to store user profiles, product details (differentiated for plants and aquarium goods), orders, payments, cart contents, reviews, and wish lists in a consistent manner. Performance and scalability are considered from the outset which makes Finora suitable for steady growth in both product catalogue size and concurrent user traffic.

1.2 Purpose of the Project

Enthusiasts of indoor plants and aquarium keep facing a recurring problem which is that there is no dedicated, specialised platform for their combined needs.

Existing options such as Daraz or Hamrobazar are generalist marketplaces flooded with unrelated categories such as: clothing, electronics, furniture and vehicle parts. These platforms rarely provide the depth of product-specific information that plant or aquarium buyers require. A plant purchaser, for instance, needs to know light requirements, watering frequency and toxicity to pets. An aquarium buyer needs filter flow rates, tank material and heater wattage. General marketplaces often reduce such items to a single paragraph of generic description.

Finora directly addresses this gap. The platform offers detailed care guidelines for each plant alongside technical specifications for aquarium equipment. Beyond the customer side, Finora equips administrators with a comprehensive toolkit for operational management. This includes systematic product inventory updates, stock level oversight, processing of incoming orders and precise tracking of financial transactions per order. A secondary purpose is to reduce the overhead for small plant nurseries or aquarium shops that cannot afford custom e-commerce development as Finora can be adopted as a ready to-use solution with minimal configuration.

1.3 Audience of the Project

Finora serves two primary user categories with clearly defined characteristics along with a secondary tertiary audience:

- *Customer Audience:* Individuals aged 18 to 55 who actively engage in indoor horticulture or freshwater aquarium maintenance. This age range is selected because it covers young adults who might be renters with small spaces through mid-career adults who might be homeowners with dedicated plant or fish rooms. Within this group, two sub-audiences exist: beginners seeking reliable beginner-friendly products and care instructions and experienced hobbyists with specific technical requirements. Geographically, the platform assumes users with basic digital literacy and access to a desktop or mobile browser.

- *Administrative Audience:* Store owners, nursery managers or designated staff members responsible for day-to-day platform operations. These users typically have prior e-commerce management experience and need to update product catalogues, fulfil orders, moderate reviews, and monitor sales analytics.
- *Tertiary audience:* Small plant nurseries or aquarium shops lacking the resources to build a custom e-commerce solution. In this scenario, the owner acts as the sole administrator, and the platform serves as a turn-key online storefront. Additionally, casual gift buyers form a minor but incidental audience segment.

1.4 Aim and Objectives

The overall aim of Finora is to design and implement a fully functional, role-based e-commerce web application that streamlines the purchase of plant and aquarium products for customers while providing administrators with efficient, intuitive tools to manage inventory, orders, and user data; all within a secure, responsive, and maintainable MVC architecture. The objectives for the aim are:

- *Relational database design:* Build a normalised MySQL database that stores user accounts, product details, orders, payments, cart sessions, reviews, and wish lists without data redundancy.
- *Secure authentication system:* Implement password hashing, session management with configurable timeouts, cookie-based persistent login, and role-based access control to protect customer areas and admin panels from unauthorised entry.
- *Admin product management (CRUD):* Develop a full create-read-update-delete interface for both plant and aquarium products including category filtering, bulk stock adjustments and image uploads.
- *Customer front-end portal:* Enable user registration, login, product browsing with faceted search, cart management, order placement with automatic stock deduction, review submission and wish list persistence across sessions.
- *Responsive user interface:* Build all JSP views with CSS Flexbox and media queries to ensure usable layouts on desktop, tablet and mobile screen sizes without relying on external front-end frameworks.

- *MVC architecture compliance:* Maintain strict separation between models, views and controllers throughout the entire codebase, enabling independent testing and future modifications.
- *Validation and exception handling:* Apply client-side and server-side validation to all forms. Implement custom exception handling with user-friendly error messages that do not expose stack traces or database internals.

1.5 Feature List

The following features are planned for implementation in Finora:

i. Customer-side features:

Table 1: Customer-side features

Feature	Description	Associated Page(s)
User registration & login	Secure account creation with email verification, password hashing and session-based sign-in.	Login, Register
Product browsing & search	Filter by category, keyword search, price slider, and sorting.	Shop, Home, Landing
Product details with specialised info	Plant care or aquarium specs tab on each product page. Also shows stock status.	Shop
Shopping cart	Add/remove items, update quantities, view subtotal, proceed to checkout. Cart persists after login/logout.	Cart

Order placement & stock update	Multi-step checkout. On order finalisation, system atomically reduces stock levels and creates order record.	Checkout flow
Order history & tracking	View past orders, track current status (pending, confirmed, shipped, delivered, cancelled).	Orders
Wishlist	Save products to a personal wishlist. Two separate pages and items remain until moved to cart or manually removed.	Wishlist
Product reviews & ratings	Write text reviews and assign 1-5 star ratings for purchased products. Reviews appear after admin approval to prevent spam.	Reviews
Profile management	Update personal information, change password, view order history summary and delete account.	Profile
Notifications	In-app and email notifications for order status changes, price drops on wishlisted items and restocked products.	Notification
Settings	Configure email preferences, session timeout duration, and privacy options.	Settings

Contact form	Send inquiries to platform administrators.	Contact
--------------	--	---------

ii. **Admin-side features:**

Table 2: Admin-side features

Feature	Description	Associated Page(s)
Admin dashboard	Graphical overview of orders, low-stock products, recent user registrations, and top-selling items. Quick action buttons for common tasks.	Dashboard
Product management	Add new plant/aquarium products with category-specific fields. Edit, delete or upload product images.	Products
Order management	View all customer orders, filter by status, update status in bulk, add internal notes, cancel orders with automatic refund flag.	Orders
Customer management	Browse user list with search by name/email. View individual order history.	Customer
Review moderation	Approve, edit, or delete customer reviews before they appear publicly.	Reviews

Analytics & reporting	Sales reports, revenue charts, top-selling products, stock turnover rate and customer acquisition chart.	Analytics
System settings	Configure admin account permissions and admin works.	Settings

2. Database Design and Development

2.1 Entity Relationship Diagram (ERD) with Business Rules

An Entity Relationship Diagram (ERD) is a visual map of the database that shows all the entities, their attributes, and how they are connected to each other.

For Finora, the ERD includes ten main entities. Each of these represents a real-world concept in the system. The relationships between these entities are defined using primary keys and foreign keys. Some of the key relationships in the database include:

- The CUSTOMER is the core of all operations, as all the operations in the database, whether they are in the form of a cart, order, review, or wish list, are all associated with a single customer.
- CATEGORY: PRODUCT is used to organize all the products in the database into various groups, with each product belonging to only one category.
- PRODUCT: FISH_COMPATIBILITY allows for the smart compatibility feature, which enables various species of fish to be associated with a product.
- The product table itself contains columns for Water_Level and Sunlight_Req, which allow for the Plant Care Guide to function without the use of an extra table.
- The CART & CART_ITEM tables are used to store the current session of the user, with CART_ITEM referencing the specific products in the user's cart with their price.
- The ORDER & ORDER_ITEM tables are used to store permanent records of all the operations in the database, including the price at which they were purchased.
- The REVIEW table contains a constraint to allow for only one review per product per customer.
- The WISHLIST table contains a constraint to allow for only one wish list of the same product by the same customer.

Entity Relationship Diagram (ERD)

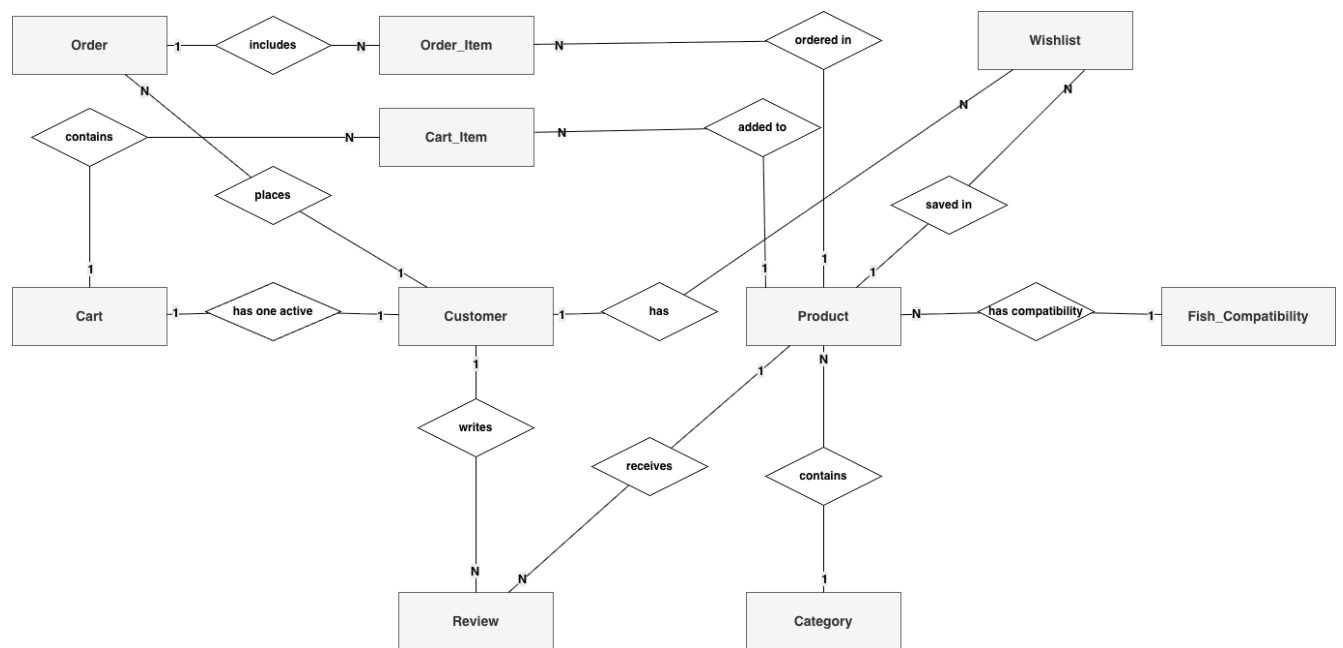


Figure 1: Entity Relationship Diagram (ERD)

The business rules that govern these relationships are listed below:

User Rules

- A Customer may register one account using a unique email address.
 - Each Customer must be uniquely identifiable within the system.
- (It is assumed that each customer registers with a valid and unique email address.)
- A Customer may log in and browse products without placing an order.
 - An Admin account is pre-created by the system and cannot be self-registered.
 - A Customer may update their profile information at any time.

Product & Category Rules

- An Admin may add, update, or delete Products at any time.
- Each Product must belong to exactly one Category.
- A Category may contain one or more Products.

- A Category may exist without any Products assigned to it.
- Each Product must have a name, price, stock quantity, and category.
- A Product with a stock quantity of zero is considered out-of-stock and cannot be added to the Cart.
- Products of type 'Plant' must store Water Level (Low/Medium/High) and Sunlight Requirement (Low/Medium/High) as care guide attributes.
- Products of type 'Fish' must have a compatibility mapping stored in the Fish_Compatibility table referencing compatible fish.

(It is assumed that care guide and compatibility data are statically defined by the Admin during product creation.)

Cart Rules

- A Customer may add one or more Products to their Cart.
- Each Cart must belong to exactly one Customer.
- A Customer may have only one active Cart at any given time.
- A Customer may update the quantity of any item in their Cart.
- A Customer may remove individual items from their Cart.
- Adding a Product already present in the Cart updates the existing quantity rather than inserting a duplicate row.
- Cart Items are cleared upon successful Order placement.

Order & Order Item Rules

- A Customer may place one or more Orders.
- Each Order must be placed by exactly one Customer.
- Each Order must contain at least one Order Item.

- An Order Item must reference exactly one Product and belong to exactly one Order.
- An Order must carry a status of Pending, Shipped, or Delivered.
- The total amount of an Order is the sum of across all its Order Items.
- The stock quantity of a Product must be decremented when an Order is confirmed.
- An Admin may view all Orders and update the status of any Order.
- A Customer may view only their own Order history.

Review Rules

- A Customer may submit one Review per Product.
- Each Review must be associated with exactly one Product and exactly one Customer.
- A Review must include a rating between 1 and 5.

(It is assumed that a Customer can only review Products that have been ordered by them.)

- An Admin may approve or remove Reviews.

Wishlist Rules

- A Customer may add one or more Products to their Wishlist.
- A Product may appear in the Wishlist of multiple Customers.
- A Customer may not add the same Product to their Wishlist more than once.

(It is assumed that a Wishlist entry is independent of Cart and Order operations.)

2.2 Normalization (up to 3NF) with Table Attributes

Normalization is an essential step in database design that helps in improving the efficiency, consistency, and accuracy of the database. This makes it easy to manage and maintain the database. Also, it ensures that the database is flexible to meet changing business needs. The main aim for normalizing the relations is to reduce anomalies, i.e., insertion, update, and deletion. Failure to reduce anomalies will result

in data redundancy, which may pose a threat to data integrity and create other problems in the future.

For Finora, normalization was applied step by step, starting from an Un-Normalized Form (UNF) and working through First, Second, and Third Normal Form. Each step made the structure cleaner and more reliable.

Un-Normalized Form (UNF)

It is the simplest database model, also known as non-first normal form (NF2). A UNF model suffers from data redundancy and lacks the efficiency of database normalization.

UNF: In this, all the entities and properties are gathered along with different repeating fields.

Customer (Customer_ID, Customer_Name, Email, Password, Phone_Number, Shipping_Address, Registration_Date, Is_Active { Category_ID, Category_Name, Description, { Product_ID, Product_Name, Description, Price, Stock_Quantity, Product_Type, Image_URL, Water_Level, Sunlight_Req, Is_Featured, {Compatible_With_ID, Compatibility_Note }, { Review_ID, Rating, Comment, Review_Date, Is_Approved } }, { Cart_ID, Created_At, Updated_At, Is_Active, { Cart_Item_ID, Product_ID, Quantity, Unit_Price, Added_At } }, { Order_ID, Order_Date, Total_Amount, Order_Status, Payment_Method, Payment_Status, Delivered_At, { Order_Item_ID, Product_ID, Quantity, Unit_Price, Subtotal } }, { Wishlist_ID, Product_ID, Added_Date, Note } })

In the Un-Normalized Form, all customer information, category and product data, fish compatibility mappings, cart contents, order records, order items, reviews, and wishlist entries are stored within a single nested structure. This design contains multiple repeating groups and nested attributes, violating atomicity. As a result, the UNF structure suffers from high data redundancy, insertion anomalies, update anomalies, and deletion anomalies, making it inefficient and difficult to maintain.

First Normal Form (1NF)

A relation is said to be in First Normal Form (1NF) if all its attributes are made up of atomic values and there are no repeating groups. To make a relation attain 1NF, a UNF table is split into different entities, with each attribute consisting of a single value.

1NF: In 1NF, the repeating groups are split, and the primary key is propagated to all entities.

Customer (Customer_ID, Customer_Name, Email, Password, Phone_Number, Shipping_Address, Registration_Date, Is_Active)

Customer_Category_Product (Customer_ID*, Category_ID*, Category_Name, Description, Product_ID*, Product_Name, Price, Stock_Quantity, Product_Type, Water_Level, Sunlight_Req, Compatible_With_ID, Compatibility_Note, Cart_ID*, Quantity, Unit_Price)

Customer_Order (Customer_ID*, Order_ID*, Order_Date, Total_Amount, Order_Status, Payment_Method, Payment_Status, Product_ID*, Order_Qty, Unit_Price, Subtotal)

Customer_Review (Customer_ID*, Product_ID*, Review_ID, Rating, Comment, Review_Date, Is_Approved)

Customer_Wishlist (Customer_ID*, Product_ID*, Wishlist_ID, Added_Date, Note)

The database is transformed into First Normal Form by removing repeating groups and ensuring that each attribute holds atomic values only. Data is divided into multiple relations Customer, Customer_Category_Product, Customer_Order, Customer_Review, and Customer_Wishlist. Each table now represents a specific relationship or entity, and primary keys are introduced to uniquely identify records. Although redundancy is reduced, partial and transitive dependencies may still exist at this stage.

Dependencies:

Customer_ID → Customer_Name, Email, Password, Phone_Number, Shipping_Address, Registration_Date, Is_Active

Category_ID → Category_Name, Description, Display_Order, Is_Active

Product_ID → Product_Name, Description, Price, Stock_Quantity, Product_Type, Water_Level, Sunlight_Req, Is_Featured

Cart_ID → Customer_ID, Created_At, Updated_At, Is_Active

Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status, Payment_Method, Payment_Status

Review_ID → Customer_ID, Product_ID, Rating, Comment, Review_Date, Is_Approved

Wishlist_ID → Customer_ID, Product_ID, Added_Date, Note

Second Normal Form (2NF)

A relation is said to be in Second Normal Form (2NF) if the relation is in First Normal Form (1NF) and the non-key attributes entirely depend on the primary key. This phase eliminates the occurrence of partial dependence. In other words, every table will now hold data that belongs to a single entity. This improves data consistency and minimizes redundancy.

2NF: Partial dependencies are removed by separating entities that were merged under composite keys.

Customer (Customer_ID, Customer_Name, Email, Password, Phone_Number, Shipping_Address, Registration_Date, Is_Active)

Category (Category_ID, Category_Name, Description, Display_Order, Is_Active)

Product (Product_ID, Product_Name, Description, Price, Stock_Quantity, Category_ID*, Product_Type, Image_URL, Water_Level, Sunlight_Req, Is_Featured, Is_Active)

Fish_Compatibility (Compatibility_ID, Fish_Product_ID*, Compatible_With_ID*, Compatibility_Note, Created_At)

Cart (Cart_ID, Customer_ID*, Created_At, Updated_At, Is_Active)

Cart_Item (Cart_Item_ID, Cart_ID*, Product_ID*, Quantity, Unit_Price, Added_At)

Order (Order_ID, Customer_ID*, Order_Date, Total_Amount, Order_Status, Payment_Method, Payment_Status, Delivered_At)

Order_Item (Order_Item_ID, Order_ID*, Product_ID*, Quantity, Unit_Price, Subtotal)

Review (Review_ID, Customer_ID*, Product_ID*, Rating, Comment, Review_Date, Is_Approved)

Wishlist (Wishlist_ID, Customer_ID*, Product_ID*, Added_Date, Note)

To achieve Second Normal Form, the relations are first ensured to be in 1NF, and then partial dependencies are eliminated. The merged relations (Customer_Category_Product, Customer_Order, Customer_Review, Customer_Wishlist) are decomposed into independent entities — Category, Product, Fish_Compatibility, Cart, Cart_Item, Order, Order_Item, Review, and Wishlist. Each table now holds data belonging to a single entity. This stage significantly improves data consistency and minimises redundancy caused by partial dependency.

Dependencies:

Customer_ID → Customer_Name, Email, Password, Phone_Number, Shipping_Address, Registration_Date, Is_Active

Category_ID → Category_Name, Description, Display_Order, Is_Active

Product_ID → Product_Name, Description, Price, Stock_Quantity, Category_ID, Product_Type, Image_URL, Water_Level, Sunlight_Req, Is_Featured, Is_Active

Compatibility_ID → Fish_Product_ID, Compatible_With_ID, Compatibility_Note, Created_At

Cart_ID → Customer_ID, Created_At, Updated_At, Is_Active

Cart_Item_ID → Cart_ID, Product_ID, Quantity, Unit_Price, Added_At

Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status, Payment_Method, Payment_Status, Delivered_At

Order_Item_ID → Order_ID, Product_ID, Quantity, Unit_Price, Subtotal

Review_ID → Customer_ID, Product_ID, Rating, Comment, Review_Date, Is_Approved

Wishlist_ID → Customer_ID, Product_ID, Added_Date, Note

Third Normal Form (3NF)

A relation is said to be in Third Normal Form (3NF) if it satisfies Second Normal Form (2NF) and has no transitive dependency. In Third Normal Form, the non-key attributes of a relation depend only on its primary key. Thus, redundancy is minimized.

3NF: Transitive dependencies are removed so that every non-key attribute depends directly and solely on the primary key.

Customer (Customer_ID, Customer_Name, Email, Password, Phone_Number, Shipping_Address, Registration_Date, Is_Active)

Category (Category_ID, Category_Name, Description, Display_Order, Is_Active)

Product (Product_ID, Product_Name, Description, Price, Stock_Quantity, Category_ID*, Product_Type, Image_URL, Water_Level, Sunlight_Req, Is_Featured, Is_Active)

Fish_Compatibility (Compatibility_ID, Fish_Product_ID*, Compatible_With_ID*, Compatibility_Note, Created_At)

Cart (Cart_ID, Customer_ID*, Created_At, Updated_At, Is_Active)

Cart_Item (Cart_Item_ID, Cart_ID*, Product_ID*, Quantity, Unit_Price, Added_At)

Order (Order_ID, Customer_ID*, Order_Date, Total_Amount, Order_Status, Payment_Method, Payment_Status, Delivered_At)

Order_Item (Order_Item_ID, Order_ID*, Product_ID*, Quantity, Unit_Price, Subtotal)

Review (Review_ID, Customer_ID*, Product_ID*, Rating, Comment, Review_Date, Is_Approved)

Wishlist (Wishlist_ID, Customer_ID*, Product_ID*, Added_Date, Note)

Third Normal Form is achieved by verifying and removing all transitive dependencies. The key transitive dependency examined was Product.Category_Name through Category_ID resolved because Category_Name is stored only in the Category table, not inside Product. Product stores only Category_ID. Order.Total_Amount is a stored computed value derived from Order_Items at the time of order placement and does not represent a live transitive dependency. Cart_Item.Unit_Price and Order_Item.Unit_Price are price snapshots captured at the time of cart addition or order placement they do not derive from another non-key attribute. Wishlist allows at most one entry per Customer-Product pair. The final 3NF structure is logically sound, efficient, and free from update anomalies.

Dependencies:

Customer_ID → Customer_Name, Email, Password, Phone_Number, Shipping_Address, Registration_Date, Is_Active

Category_ID → Category_Name, Description, Display_Order, Is_Active

Product_ID → Product_Name, Description, Price, Stock_Quantity, Category_ID, Product_Type, Image_URL, Water_Level, Sunlight_Req, Is_Featured, Is_Active

Compatibility_ID → Fish_Product_ID, Compatible_With_ID, Compatibility_Note, Created_At

Cart_ID → Customer_ID, Created_At, Updated_At, Is_Active

Cart_Item_ID → Cart_ID, Product_ID, Quantity, Unit_Price, Added_At

Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status, Payment_Method, Payment_Status, Delivered_At

Order_Item_ID → Order_ID, Product_ID, Quantity, Unit_Price, Subtotal

Review_ID → Customer_ID, Product_ID, Rating, Comment, Review_Date, Is_Approved

Wishlist_ID → Customer_ID, Product_ID, Added_Date, Note

Relational Model

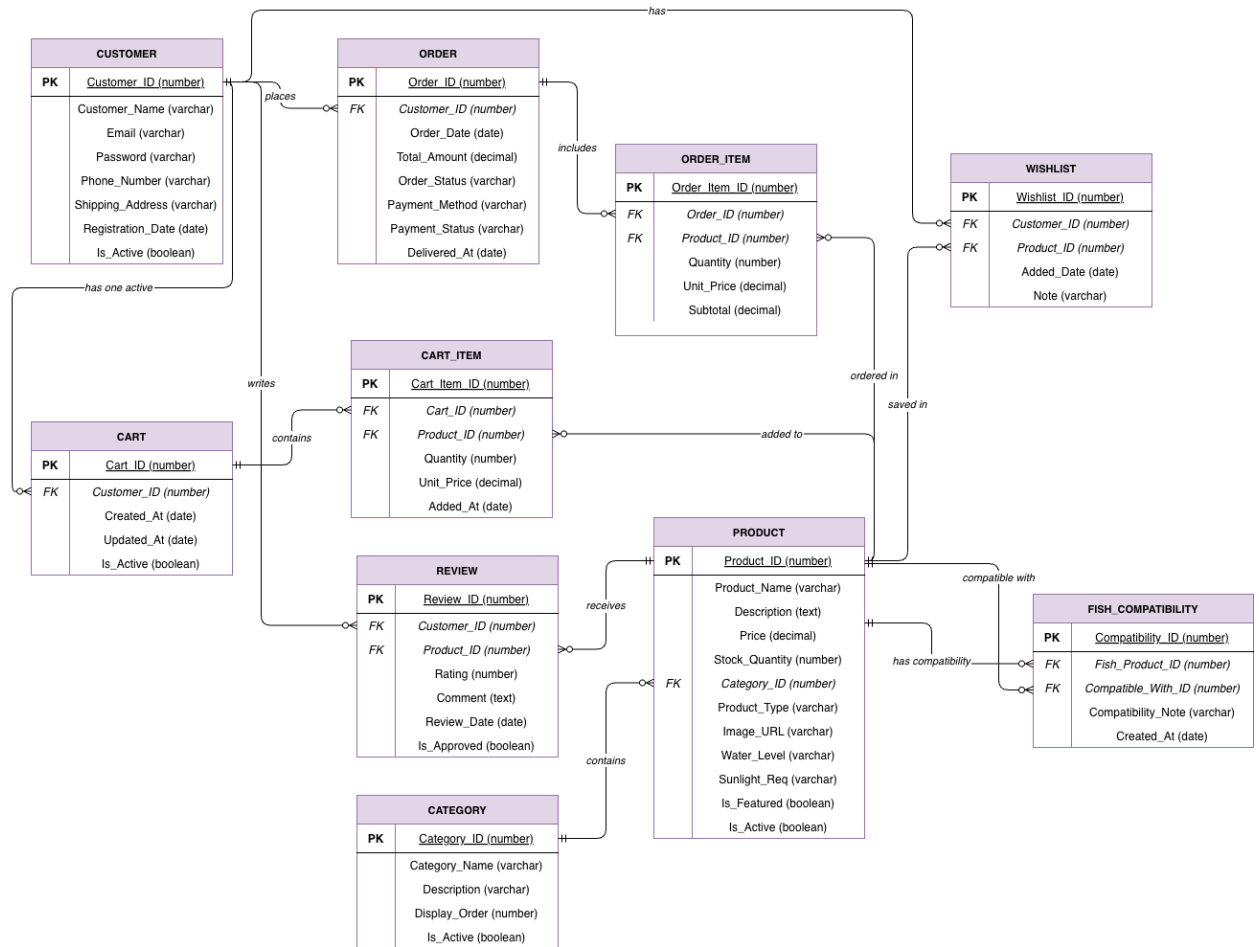


Figure 2: Relational Model

2.3 Database Development: Data Dictionary with Sample Data

The data dictionary is a detailed description of every table in the database. It lists each attribute, its data type, size, and any constraints applied to it. This is important because it serves as the blueprint for creating the actual database tables.

All ten tables of the Finora database are documented below, along with sample data showing what real records might look like in each table.

2.3.1 Table 1: User

Table 3: User Table

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
1	user_id (PK)	INT	—	PRIMARY KEY, NOT NULL, AUTO_INCREMENT	1
2	full_name	VARCHAR	100	NOT NULL	Rajan Shrestha
3	email	VARCHAR	100	NOT NULL, UNIQUE	rajan@email.com
4	password	VARCHAR	255	NOT NULL	hashed_password_1
5	phone_number	VARCHAR	15	NOT NULL	9841000001
6	address	VARCHAR	255	NOT NULL	Kathmandu, Nepal
7	role	VARCHAR	20	NOT NULL, DEFAULT 'Customer'	Customer
8	registration_date	DATE	—	NOT NULL, DEFAULT CURRENT_DATE	2024-01-10
9	account_status	VARCHAR	20	NOT NULL, DEFAULT 'Active'	Active
10	profile_image	VARCHAR	255	DEFAULT NULL	profile1.jpg

Screenshot of the Table:

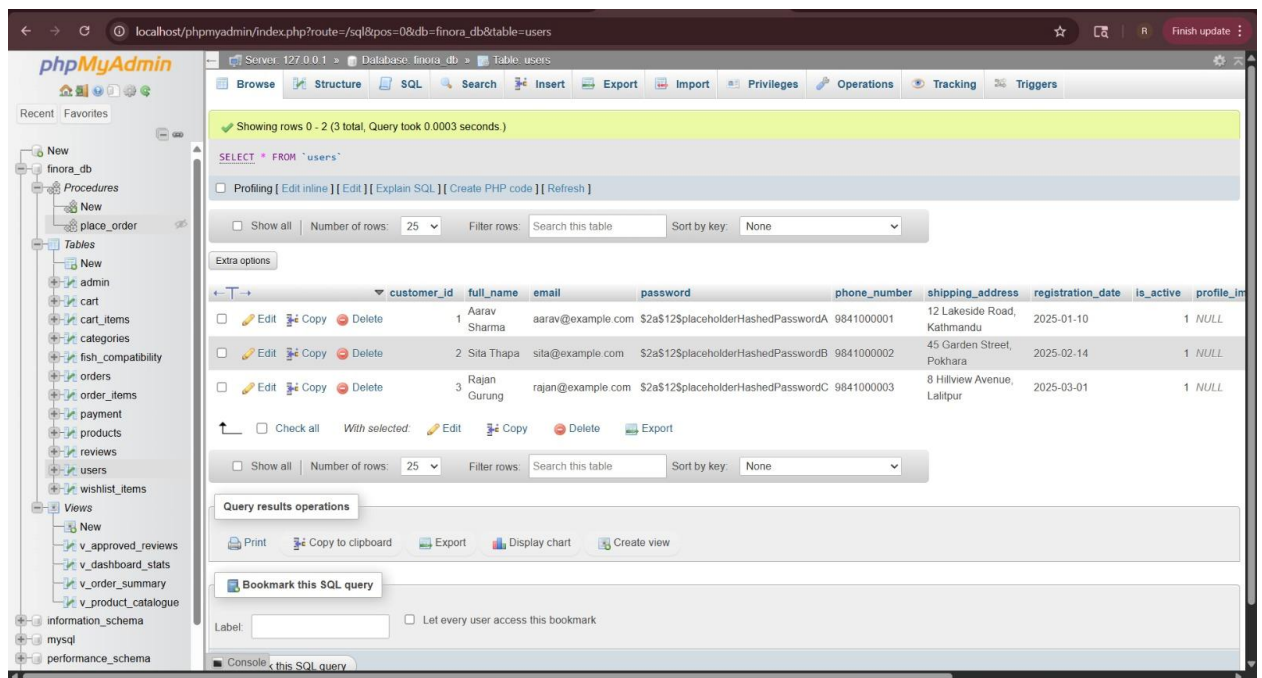


Figure 3: User Table

2.3.2 Table 2: Plant_Product

Table 4: Plant_Product Table

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
1	plant_id (PK)	INT	—	PRIMARY KEY, NOT NULL, AUTO_INCREMENT	1
2	name	VARCHAR	100	NOT NULL	Peace Lily
3	category	VARCHAR	50	NOT NULL	Indoor
4	price	DECIMAL	(10,2)	NOT NULL, CHECK (price > 0)	850.00
5	stock_quantity	INT	—	NOT NULL, CHECK (stock_quantity >= 0)	25

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
6	description	TEXT	—	NOT NULL	A low-light flowering plant.
7	light_requirement	VARCHAR	50	NOT NULL	Low
8	water_frequency	VARCHAR	50	NOT NULL	Weekly
9	care_level	VARCHAR	50	NOT NULL	Easy
10	image_url	VARCHAR	255	NOT NULL	peace_lily.jpg

Screenshot of the Table:

The screenshot shows the phpMyAdmin interface for the 'finora_db' database. The 'products' table is selected, and its structure and data are displayed. The table has 11 columns: product_id, product_name, description, price, stock_quantity, category_id, product_type, image_url, water_level, sunlight_req, and is_featured. The data is as follows:

product_id	product_name	description	price	stock_quantity	category_id	product_type	image_url	water_level	sunlight_req	is_featured
1	Peace Lily	A beautiful low-maintenance indoor plant.	12.99	50	1	Plant	/images/peace_lily.jpg	Low	Low	
2	Monstera Deliciosa	Iconic split-leaf tropical plant.	24.99	30	1	Plant	/images/monstera.jpg	Medium	Medium	
3	Snake Plant	Nearly indestructible beginner-friendly plant.	9.99	75	1	Plant	/images/snake_plant.jpg	Low	Low	
4	Rose Bush	Classic garden rose, available in red and pink.	18.99	40	2	Plant	/images/rose_bush.jpg	High	High	
5	Echeveria	Compact rosette-shaped succulent.	6.99	100	6	Plant	/images/echeveria.jpg	Low	High	
6	Guppy Fish	Colourful and hardy freshwater.	3.49	200	3	Aquarium	/images/guppy.jpg	NULL	NULL	

Figure 4: Plant_Product Table

2.3.3 Table 3: Aquarium_Product

Table 5: Aquarium_Product Table

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
1	aquarium_id (PK)	INT	—	PRIMARY KEY, NOT NULL, AUTO_INCREMENT	1
2	name	VARCHAR	100	NOT NULL	Crystal Clear Tank
3	type	VARCHAR	50	NOT NULL	Tank
4	price	DECIMAL	(10,2)	NOT NULL, CHECK (price > 0)	4500.00
5	stock_quantity	INT	—	NOT NULL, CHECK (stock_quantity >= 0)	10
6	size_capacity	VARCHAR	50	NOT NULL	50L
7	material	VARCHAR	50	NOT NULL	Glass
8	description	TEXT	—	NOT NULL	Crystal clear glass tank.
9	brand	VARCHAR	50	NOT NULL	AquaWorld
10	image_url	VARCHAR	255	NOT NULL	crystal_tank.jpg

Screenshot of the Table:

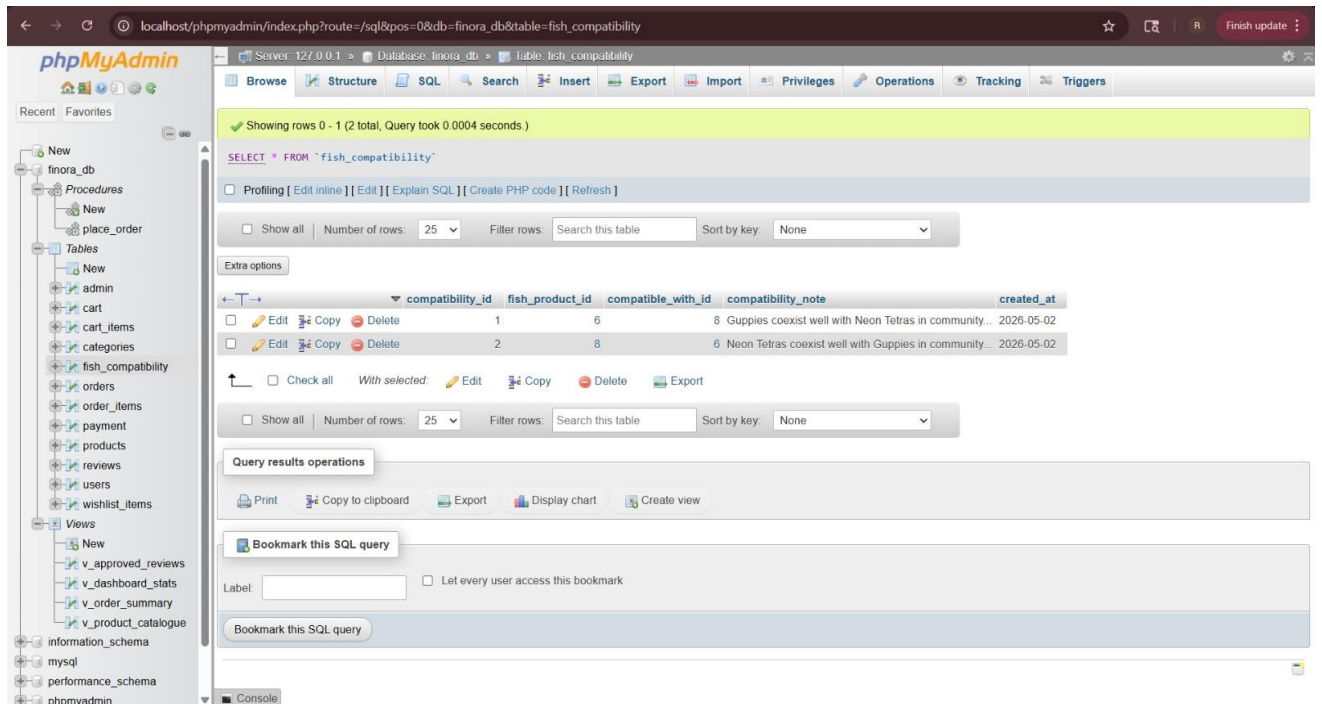


Figure 5: Aquarium_Product Table

2.3.4 Table 4: Order

Table 6: Oder Table

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
1	order_id (PK)	INT	—	PRIMARY KEY, NOT NULL, AUTO_INCREMENT	1
2	user_id (FK)	INT	—	FOREIGN KEY REFERENCES User(user_id), NOT NULL	1
3	order_date	DATE	—	NOT NULL, DEFAULT CURRENT_DATE	2024-03-01

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
4	total_amount	DECIMAL	(10,2)	NOT NULL, CHECK (total_amount >= 0)	1200.00
5	order_status	VARCHAR	20	NOT NULL, DEFAULT 'Pending'	Pending
6	payment_status	VARCHAR	20	NOT NULL, DEFAULT 'Unpaid'	Unpaid
7	shipping_address	VARCHAR	255	NOT NULL	Kathmandu, Nepal
8	contact_number	VARCHAR	15	NOT NULL	9841000001
9	delivery_date	DATE	—	NOT NULL	2024-03-05
10	notes	TEXT	—	DEFAULT NULL	Leave at door.

Screenshot of the Table:

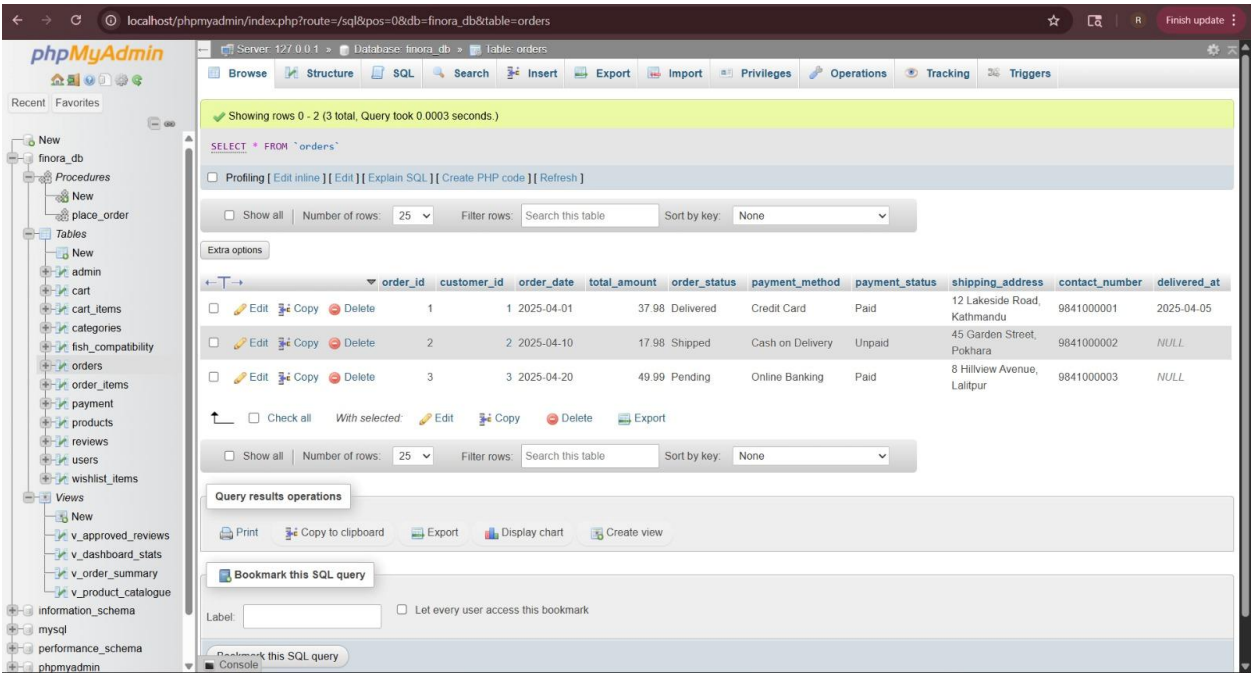


Figure 6: Oder Table

2.3.5 Table 5: Order_Item

Table 7: Order_Item Table

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
1	order_item_id (PK)	INT	—	PRIMARY KEY, NOT NULL, AUTO_INCREMENT	1
2	order_id (FK)	INT	—	FOREIGN KEY REFERENCES Order(order_id), NOT NULL	1
3	product_type	VARCHAR	20	NOT NULL, CHECK (product_type IN ('Plant','Aquarium'))	Plant

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
4	product_id	INT	—	NOT NULL	2
5	quantity	INT	—	NOT NULL, CHECK (quantity > 0)	2
6	unit_price	DECIMAL	(10,2)	NOT NULL, CHECK (unit_price > 0)	350.00
7	subtotal	DECIMAL	(10,2)	NOT NULL, CHECK (subtotal >= 0)	700.00
8	discount	DECIMAL	(5,2)	NOT NULL, DEFAULT 0.00	0.00
9	tax	DECIMAL	(5,2)	NOT NULL, DEFAULT 0.00	0.00
10	status	VARCHAR	20	NOT NULL, DEFAULT 'Active'	Active

Screenshot of the Table:

The screenshot shows the phpMyAdmin interface for the 'finora_db' database. The 'order_items' table is selected, and its structure and data are displayed. The table has columns: order_item_id, order_id, product_id, quantity, unit_price, and subtotal. The data shows 6 rows of order items.

order_item_id	order_id	product_id	quantity	unit_price	subtotal
1	1	1	2	12.99	25.98
2	1	5	1	6.99	6.99
3	1	8	2	2.99	5.98
4	2	6	3	3.49	10.47
5	2	10	1	4.99	4.99
6	3	9	1	49.99	49.99

Figure 7: Order_Item Table

2.3.6 Table 6: Review

Table 8: Review Table

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
1	review_id (PK)	INT	—	PRIMARY KEY, NOT NULL, AUTO_INCREMENT	1
2	user_id (FK)	INT	—	FOREIGN KEY REFERENCES User(user_id), NOT NULL	1
3	product_id	INT	—	NOT NULL	1

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
4	product_type	VARCHAR	20	NOT NULL, CHECK (product_type IN (Plant','Aquarium'))	Plant
5	rating	INT	—	NOT NULL, CHECK (rating BETWEEN 1 AND 5)	5
6	comment	TEXT	—	NOT NULL	Very healthy plant!
7	review_date	DATE	—	NOT NULL, DEFAULT CURRENT_DATE	2024-03-10
8	status	VARCHAR	20	NOT NULL, DEFAULT 'Pending'	Approved
9	admin_response	TEXT	—	DEFAULT NULL	Thank you for your review.
10	helpful_count	INT	—	NOT NULL, DEFAULT 0	3

Screenshot of the Table:

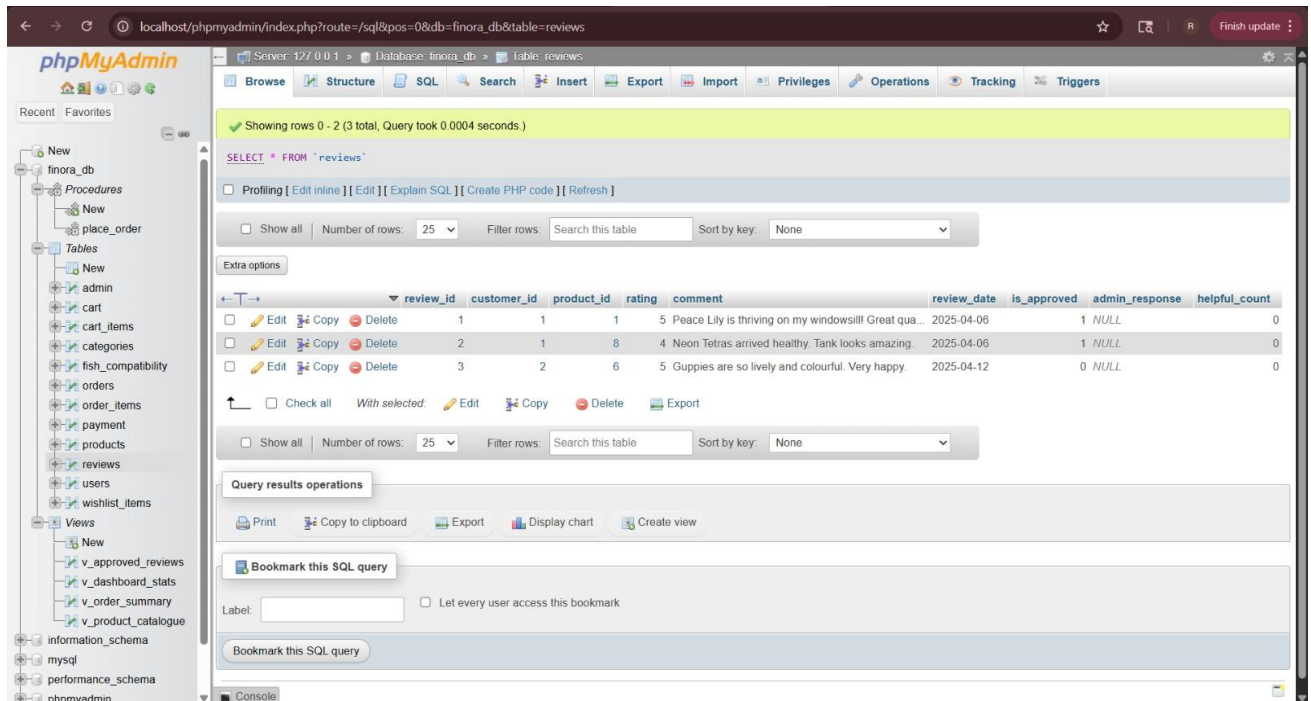


Figure 8: Review Table

2.3.7 Table 7: Wishlist

Table 9: Wishlist Table

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
1	wishlist_id (PK)	INT	—	PRIMARY KEY, NOT NULL, AUTO_INCREMENT	1
2	user_id (FK)	INT	—	FOREIGN KEY REFERENCES User(user_id), NOT NULL	1
3	product_id	INT	—	NOT NULL	3

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
4	product_type	VARCHAR	20	NOT NULL, CHECK (product_type IN ('Plant','Aquarium'))	Plant
5	added_date	DATE	—	NOT NULL, DEFAULT CURRENT_DATE	2024-02-15
6	priority	VARCHAR	20	NOT NULL, DEFAULT 'Medium'	High
7	notes	TEXT	—	DEFAULT NULL	Gift for mom.
8	status	VARCHAR	20	NOT NULL, DEFAULT 'Active'	Active
9	notification_flag	BOOLEAN	—	NOT NULL, DEFAULT FALSE	FALSE
10	last_updated	DATE	—	NOT NULL, DEFAULT CURRENT_DATE	2024-02-15

Screenshot of the Table:

The screenshot shows the phpMyAdmin interface for the 'finora_db' database. The 'wishlist_items' table is selected, and its structure and data are displayed. The table has the following columns: wishlist_id, customer_id, product_id, added_date, priority, notes, notification_flag, and last_updated. The data shows three rows of wishlist items.

wishlist_id	customer_id	product_id	added_date	priority	notes	notification_flag	last_updated
1	1	9	2026-05-02	High	Want to set up a fish tank soon.	1	2026-05-02
2	2	2	2026-05-02	Medium	Looking for a statement plant.	0	2026-05-02
3	3	7	2026-05-02	High	Betta for my desk at work.	1	2026-05-02

Figure 9: Wishlist Table

2.3.8 Table 8: Cart

Table 10: Cart Table

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
1	cart_id (PK)	INT	—	PRIMARY KEY, NOT NULL, AUTO_INCREMENT	1
2	user_id (FK)	INT	—	FOREIGN KEY REFERENCES User(user_id), NOT NULL	1
3	product_id	INT	—	NOT NULL	2

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
4	product_type	VARCHAR	20	NOT NULL, CHECK (product_type IN ('Plant','Aquarium'))	Plant
5	quantity	INT	—	NOT NULL, CHECK (quantity > 0)	3
6	price	DECIMAL	(10,2)	NOT NULL, CHECK (price > 0)	350.00
7	added_date	DATE	—	NOT NULL, DEFAULT CURRENT_DATE	2024-03-01
8	status	VARCHAR	20	NOT NULL, DEFAULT 'Active'	Active
9	total_price	DECIMAL	(10,2)	NOT NULL, CHECK (total_price >= 0)	1050.00
10	session_id	VARCHAR	100	NOT NULL	sess_abc123xyz

Screenshot of the Table:

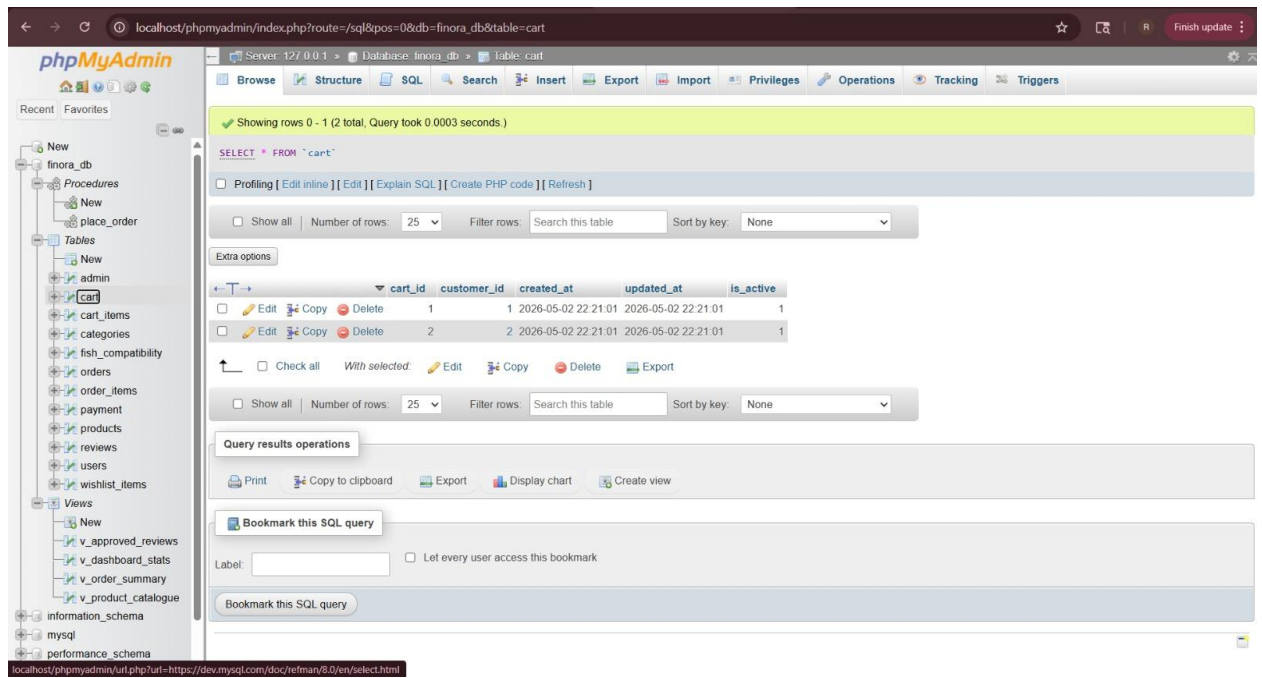


Figure 10: Cart Table

2.3.9 Table 9: Payment

Table 11: Payment Table

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
1	payment_id (PK)	INT	—	PRIMARY KEY, NOT NULL, AUTO_INCREMENT	1
2	order_id (FK)	INT	—	FOREIGN KEY REFERENCES Order(order_id), NOT NULL	2
3	payment_method	VARCHAR	50	NOT NULL	eSewa
4	transaction_id	VARCHAR	100	NOT NULL, UNIQUE	TXN20240301001

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
5	amount	DECIMAL	(10,2)	NOT NULL, CHECK (amount > 0)	4500.00
6	payment_date	DATE	—	NOT NULL, DEFAULT CURRENT_DATE	2024-03-01
7	payment_status	VARCHAR	20	NOT NULL, DEFAULT 'Pending'	Paid
8	currency	VARCHAR	10	NOT NULL, DEFAULT 'USD'	NPR
9	refund_status	VARCHAR	20	NOT NULL, DEFAULT 'None'	None
10	remarks	TEXT	—	DEFAULT NULL	Paid via eSewa app.

Screenshot of the Table:

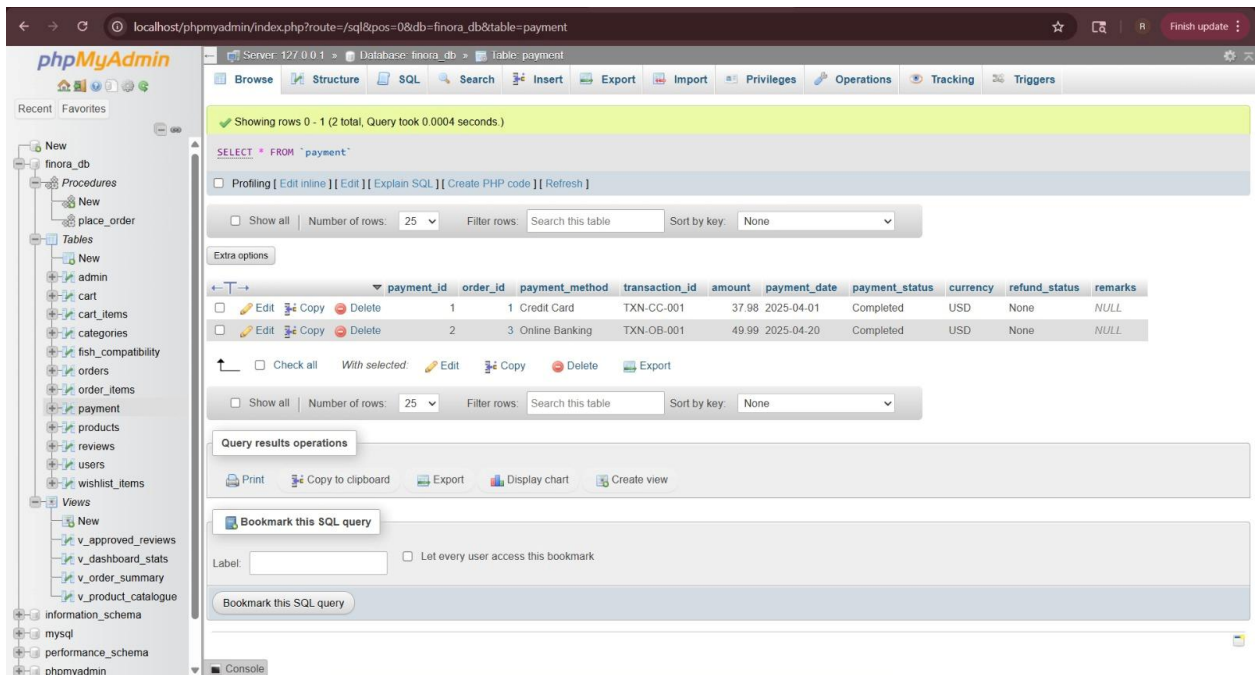


Figure 11: Payment Table

2.3.10 Table 10: Admin

Table 12: Admin Table

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
1	admin_id (PK)	INT	—	PRIMARY KEY, NOT NULL, AUTO_INCREMENT	1
2	full_name	VARCHAR	100	NOT NULL	Dibyant Bhatta
3	email	VARCHAR	100	NOT NULL, UNIQUE	admin@finora.com
4	password	VARCHAR	255	NOT NULL	hashed_admin_pass
5	role	VARCHAR	50	NOT NULL, DEFAULT 'Staff'	SuperAdmin
6	phone_number	VARCHAR	15	NOT NULL	9800000001

S.NO.	Attribute Name	Data Type	Size	Constraints	Sample Data
7	last_login	DATETIME	—	DEFAULT NULL	2024-03-01 09:00:00
8	account_status	VARCHAR	20	NOT NULL, DEFAULT 'Active'	Active
9	permissions	TEXT	—	NOT NULL	all
10	created_date	DATE	—	NOT NULL, DEFAULT CURRENT_DATE	2024-01-01

Screenshot of the Table:

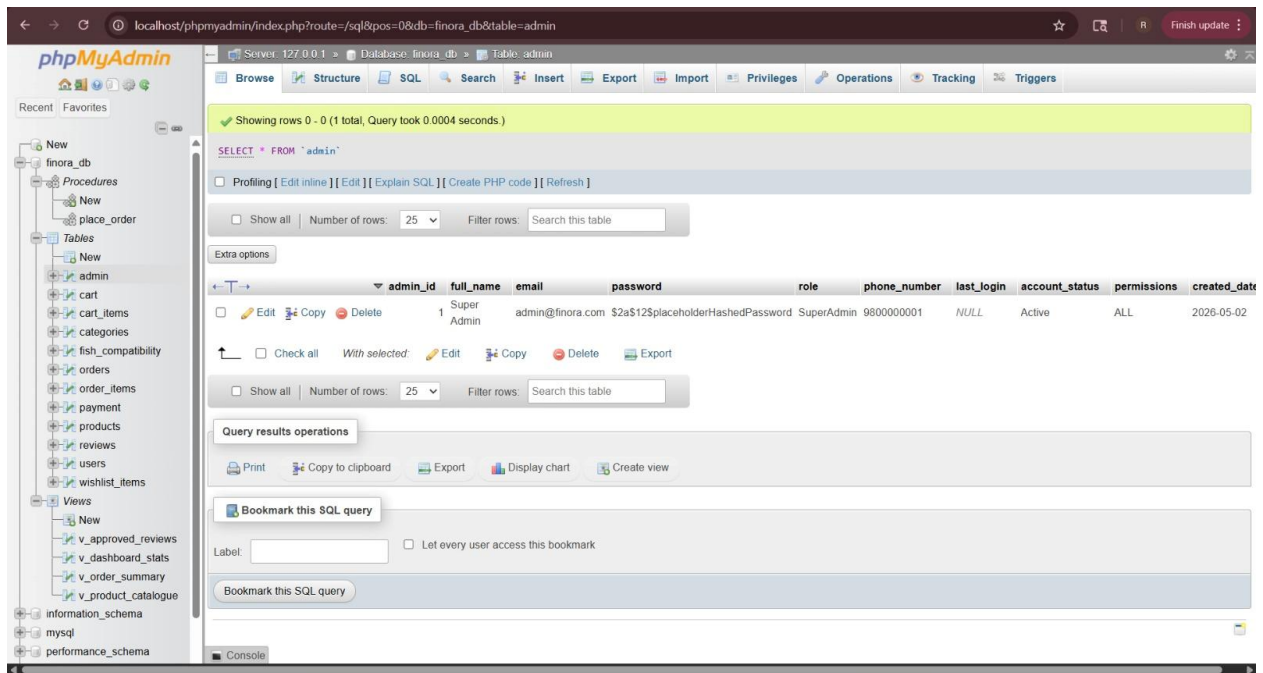


Figure 12: Admin Table

2.4 Relationship Diagram Screenshot (from XAMPP/phpMyAdmin)

The relationship diagram below was generated from phpMyAdmin (accessed through XAMPP) after importing the Finora database SQL file. It shows how all ten tables in the finora_db database are connected to each other through foreign key relationships. This is the actual database structure running locally on the development machine. The diagram visually confirms all the relationships described in the ERD section.

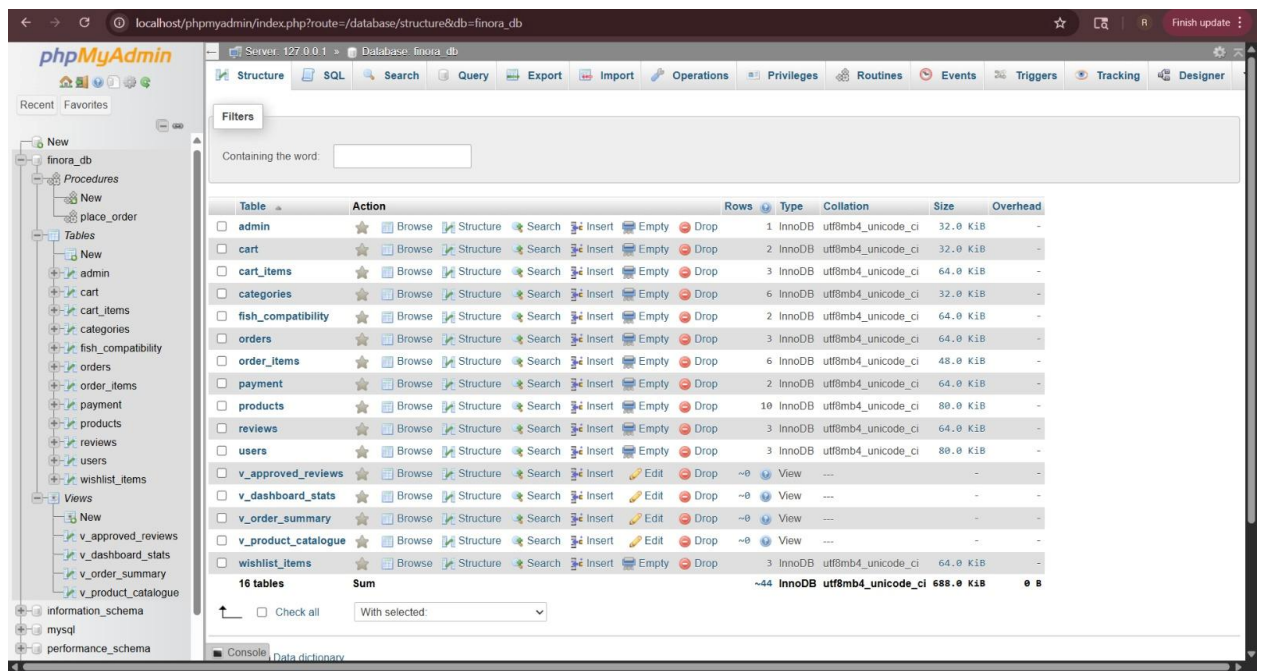


Figure 13: Relationship Diagram Screenshot

3. Wireframe and Design

This section presents the wireframe designs and actual design development for all major pages of the Finora platform. For each page, a research reference from an existing live website is provided first, followed by the wireframe created for Finora and then the developed design. The research phase helped identify key UI patterns and layout decisions that were then adapted and applied to fit Finora's brand and technical context.

3.1 Landing Page

The Landing Page is the very first screen a visitor sees when they open Finora. Its purpose is to immediately communicate what the platform is about which is a dual-category e-commerce shop for both plants and aquarium products. The page needs to draw users in and guide them toward either browsing plants or aquarium products. It also establishes the overall tone, colour scheme, and branding of the entire platform.

3.1.1 Research Reference

Bloomscape is a US-based online plant retailer that ships fully potted, ready-to-display houseplants directly to customers. Its landing page is one of the strongest real-world parallels to Finora's Landing Page.

The overall page structure including hero, category grid, featured products, care feature banner, newsletter and social proof maps directly onto the section-by-section layout of Finora's Landing Page, and the warm, nature-forward visual tone matches Finora's brand.

Website Name: Bloomscape (Bloomscape, 2026)

URL: <https://www.bloomscape.com>

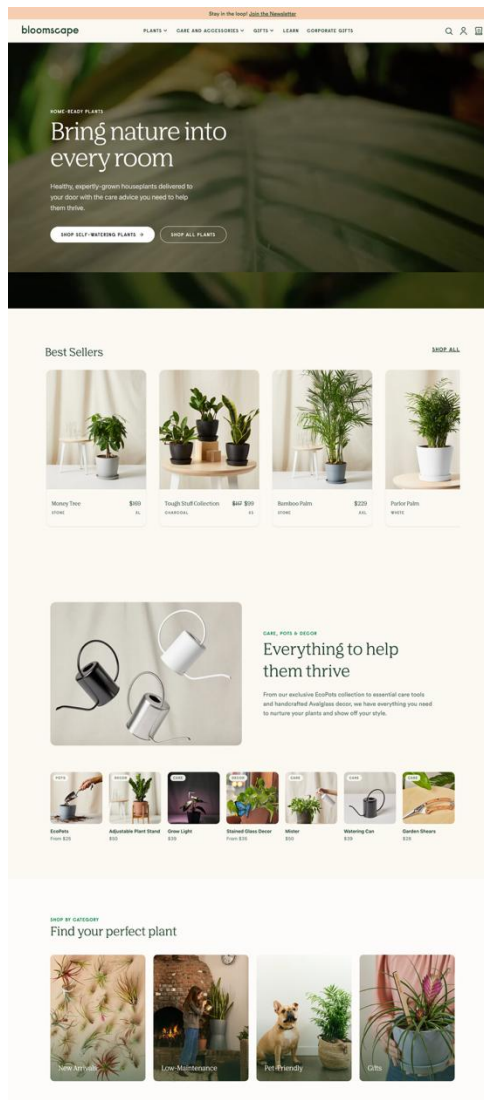


Figure 14: Landing

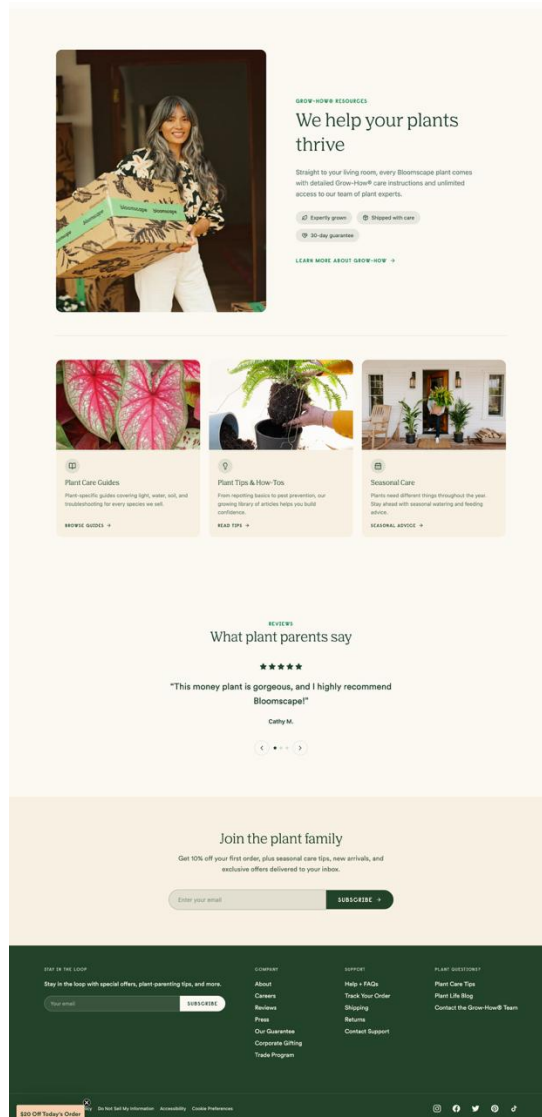


Figure 15: landing 2

Key points taken from this reference for the Finora page:

- A full-width hero section with a high-quality product photograph, a punchy headline, and a primary CTA button used in Finora's Landing Page hero.
- A category tile grid below the hero organising the catalogue into visually distinct product-type cards with names and product counts, matching Finora's category section on the Landing Page.
- A smart feature banner section promoting the plant care guide system is similar to banner section in Finora's page.
- A newsletter sign-up strip near the footer collecting email addresses matches with the bottom of Finora's Landing Page.

3.1.2 Wireframe Design



Figure 16: Wireframe

3.1.3 Actual Design

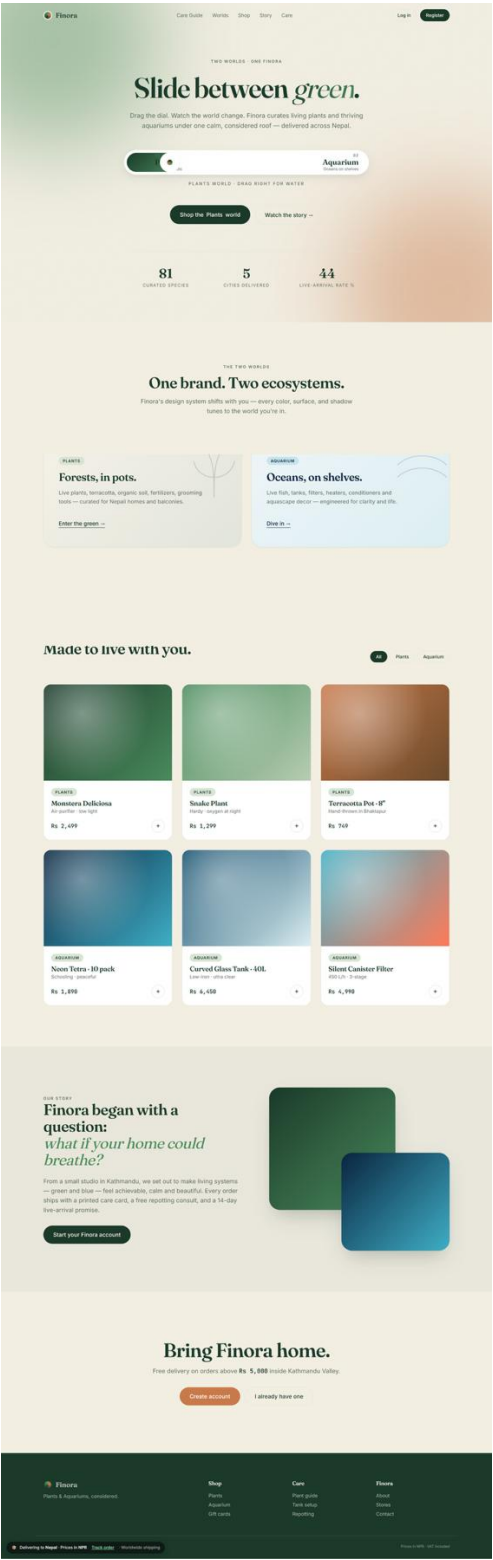


Figure 17: Actual

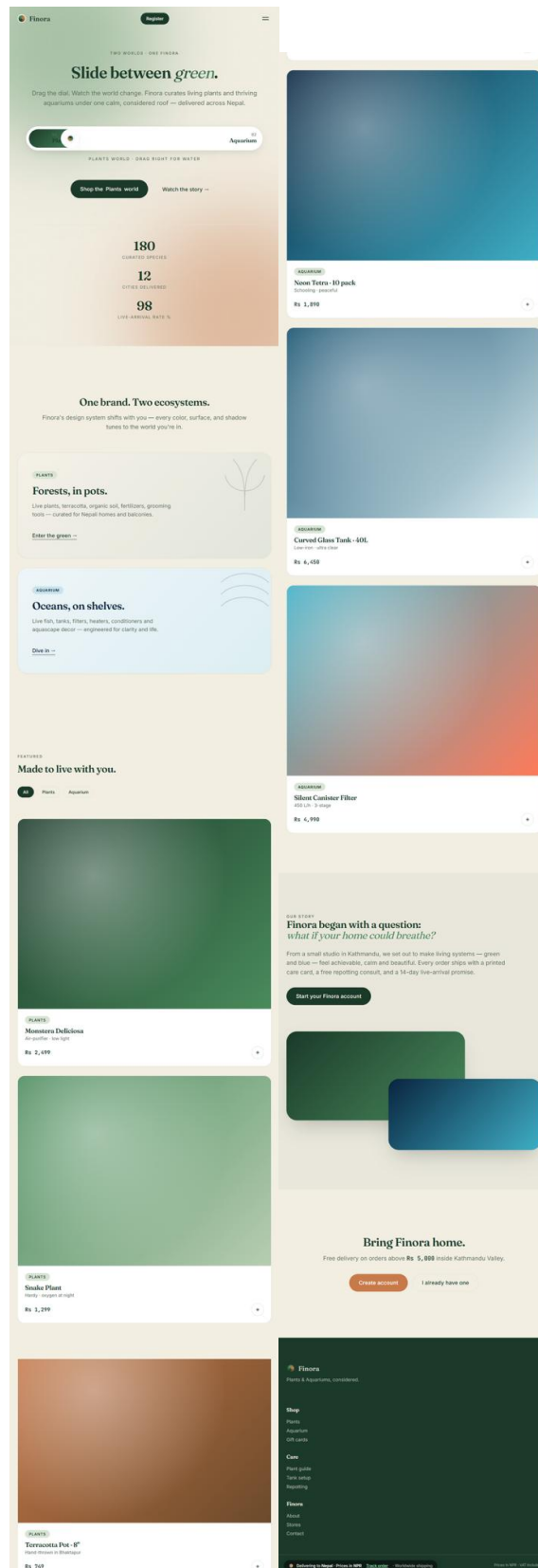


Figure 18:Actual 2

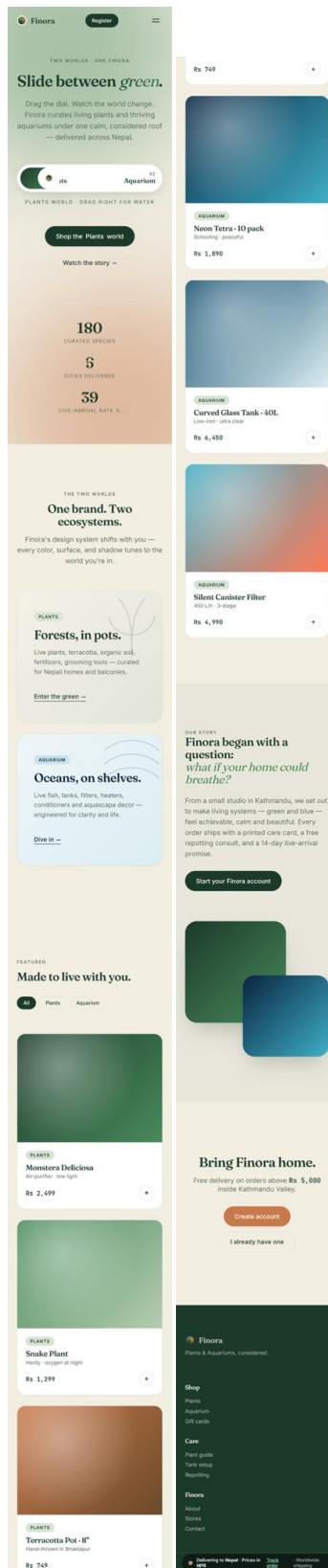


Figure 19: Actual 3

3.1.4 Design Development

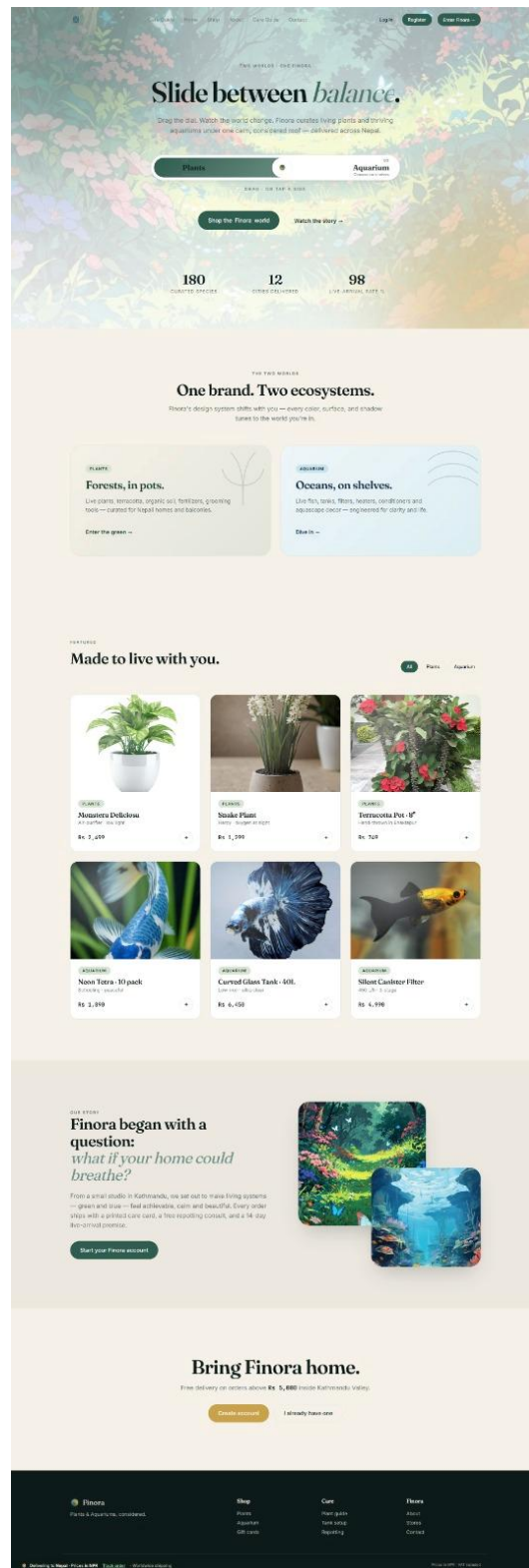


Figure 20: Final design

3.2 Login Page(s)

The Login Pages allow existing users and administrators to securely access their accounts. There are two separate login pages: one for regular customers and one for administrators. The customer login page presents a clean, minimal form where users enter their registered email address and password to sign in. The admin login page, found at a restricted URL, presents an identical form layout but is styled with a distinct label to make clear it is not for general use. Both pages apply server-side validation and session management upon successful authentication.

3.2.1 Research Reference

Aritzia is a Canadian women's fashion retailer widely cited as one of the best-designed direct-to-consumer websites. Its login and account creation pages use a split-panel layout that is the close match to Finora's Login Page structure.

The left side of the screen is occupied by a branded visual panel, a full-height image or coloured background carrying the platform's logo and a short tagline while the right side holds a clean, centred authentication form with email and password fields and a primary sign-in button. The website also maintains separate login routes for staff and admin, consistent with Finora's approach.

Website Name: Aritzia (Aritzia, 2025)

URL: <https://www.aritzia.com/intl/en/login>



Need Help?

Sign in or Create an Account to enjoy the benefits

Private Sales

Faster Checkout

List Curation

Email Address
stutiadhikari2@gmail.com

Password
***** [Show](#)

[Sign In](#)

[Forgot Password?](#) [Create An Account](#)

[Sign in with a Code](#)

Facebook

Figure 21: Reference

Key points taken from this reference for the Finora page:

- A split-screen two-panel layout with a branded visual or colour panel on the left and the login form on the right, exactly matching Finora's Login Page.
- The brand panel displays the platform logo or tagline and is styled in the platform's primary colour, matching Finora's left auth panel which shows the Finora logo, a brand quote and a back-to-Finora link.
- A minimalist form with two fields only which are the exact field set used in Finora's login form.

3.2.2 Wireframe Design

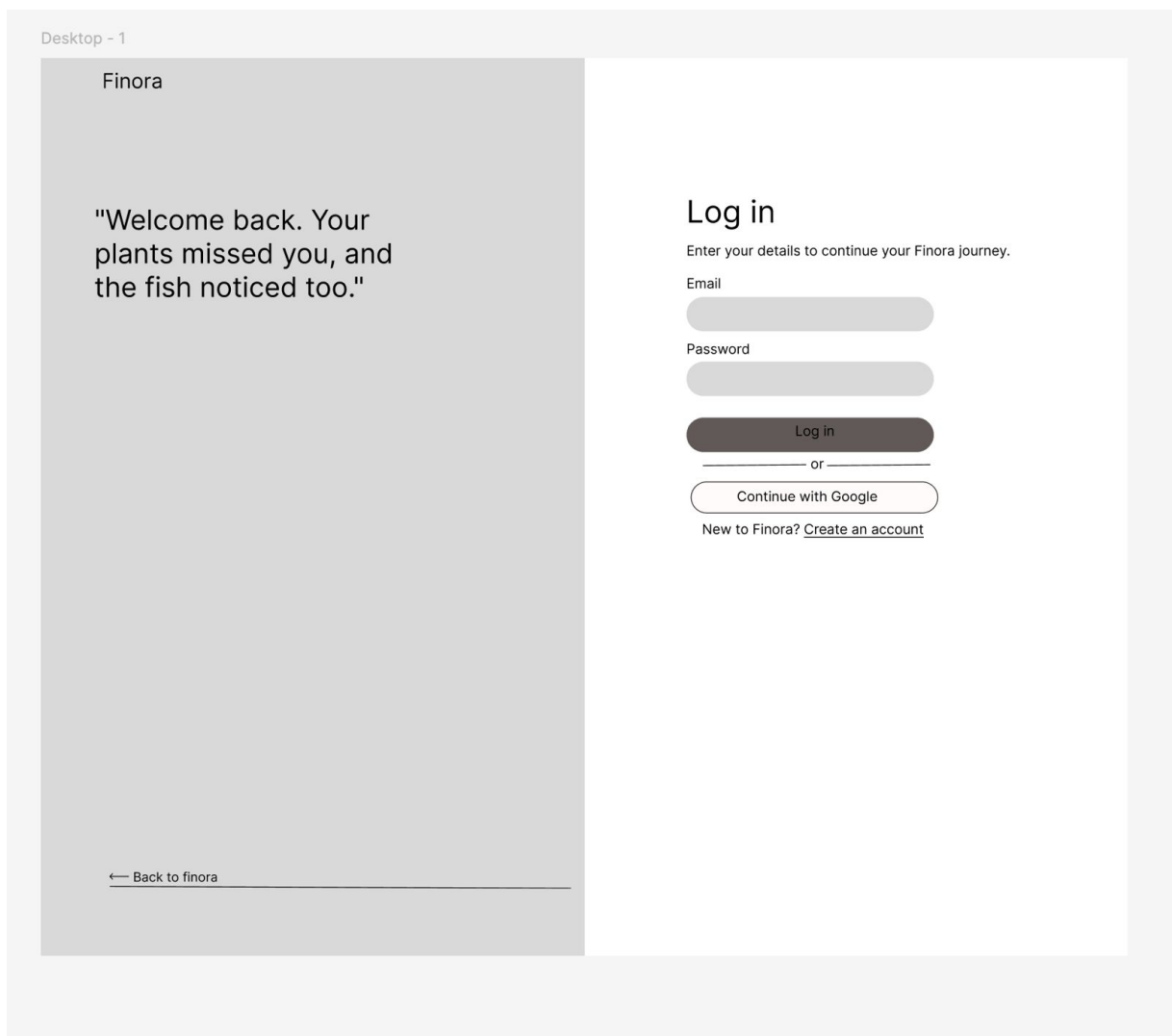
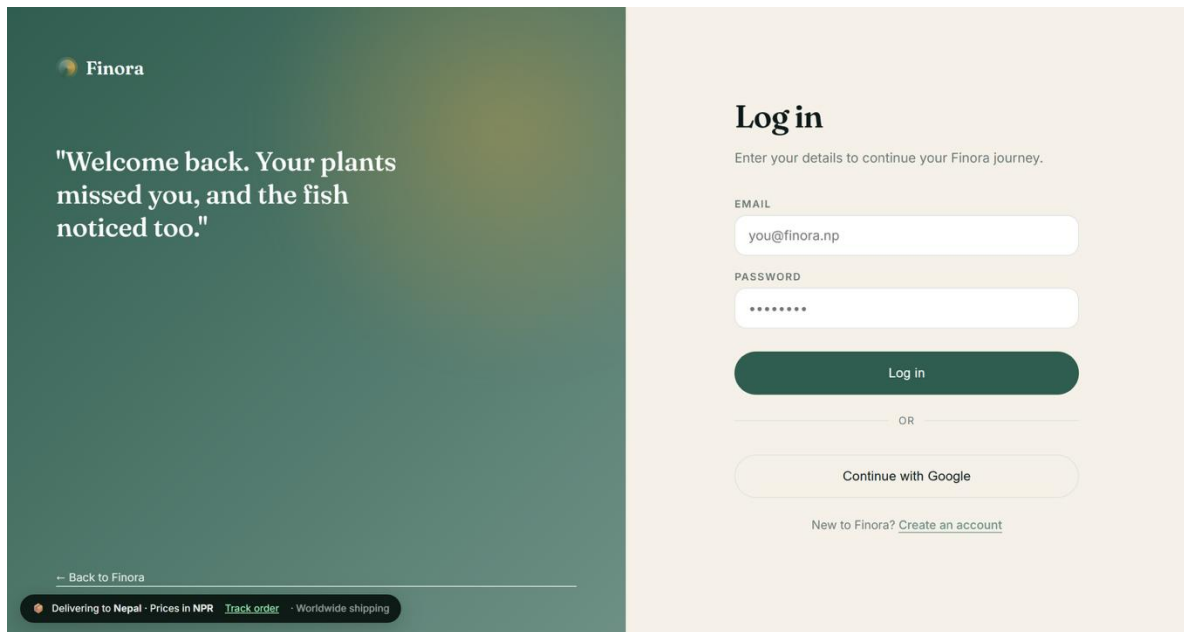


Figure 22

3.2.3 Actual Design



The image shows a login page for Finora, split into two main sections. The left section has a dark green background with a white Finora logo at the top left. Below the logo, a white quote reads: "Welcome back. Your plants missed you, and the fish noticed too." At the bottom left of this section, there is a link "Back to Finora" and a dark green button with white text that says "Delivering to Nepal · Prices in NPR · Track order · Worldwide shipping". The right section has a light beige background. It features a "Log in" heading, followed by the instruction "Enter your details to continue your Finora journey." Below this are two input fields: "EMAIL" with the value "you@finora.np" and "PASSWORD" with masked characters "*****". A dark green "Log in" button is positioned below the password field. Underneath the button is a horizontal line with the word "OR" in the center. Below the line is a light yellow button with the text "Continue with Google". At the bottom of the right section, there is a link: "New to Finora? [Create an account](#)".

Figure 23



"Welcome back. Your plants missed you, and the fish noticed too."

[← Back to Finora](#)

Log in

Enter your details to continue your Finora journey.

EMAIL

PASSWORD

Log in

OR

Continue with Google

New to Finora? [Create an account](#)

 Delivering to Nepal · Prices in NPR [Track order](#) · Worldwide shipping

Figure 24



"Welcome back. Your plants missed you, and the fish noticed too."

[← Back to Finora](#)

Log in

Enter your details to continue your Finora journey.

EMAIL

you@finora.np

PASSWORD

••••••••

Log in

OR

Continue with Google

New to Finora? [Create an account](#)



Delivering to **Nepal** · Prices in **NPR**

[Track order](#)

· Worldwide shipping

Figure 25

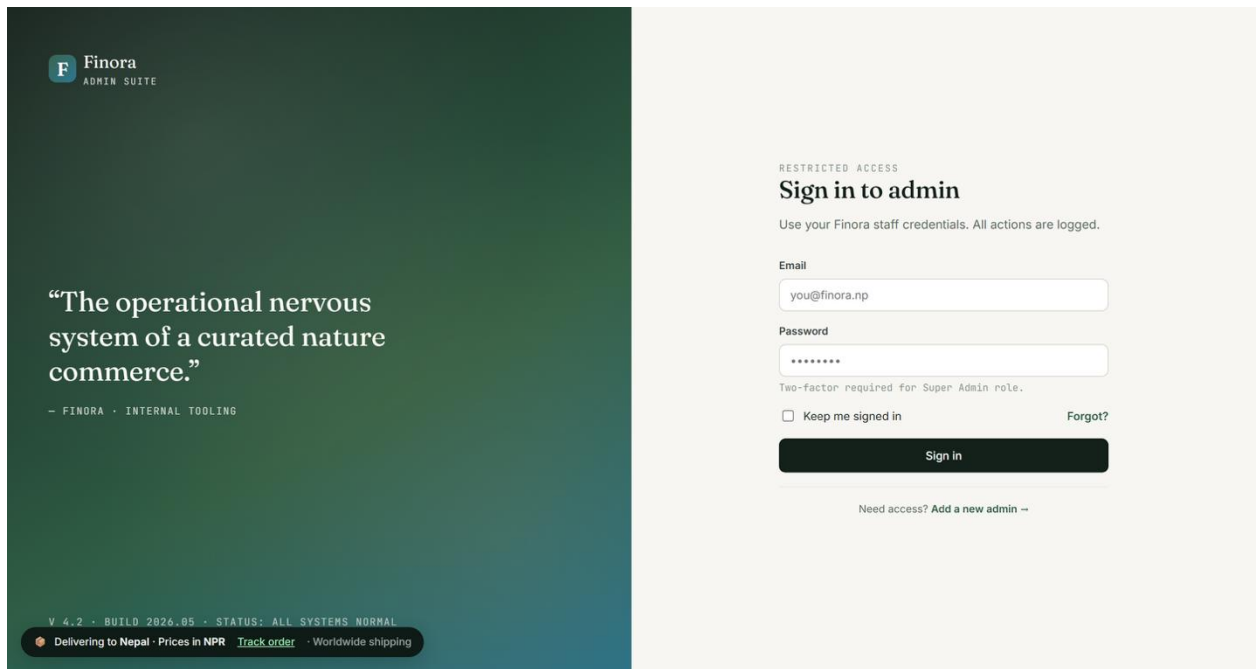


Figure 26

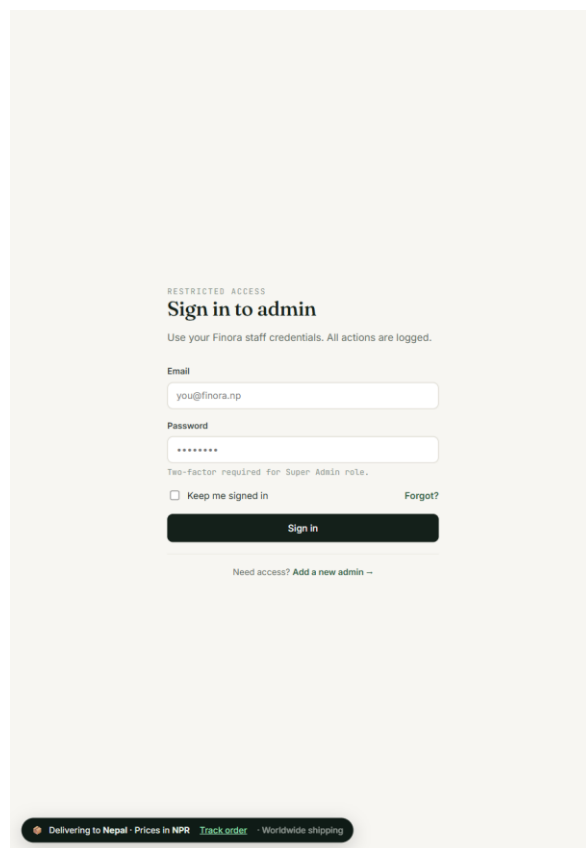


Figure 27

RESTRICTED ACCESS

Sign in to admin

Use your Finora staff credentials. All actions are logged.

Email

you@finora.np

Password

.....

Two-factor required for Super Admin role.

☐ Keep me signed in

Forgot?

Sign in

Need access? [Add a new admin](#) →

 Delivering to **Nepal** · Prices in **NPR**

[Track order](#)

· Worldwide shipping

Figure 28

54

3.2.4 Design Development

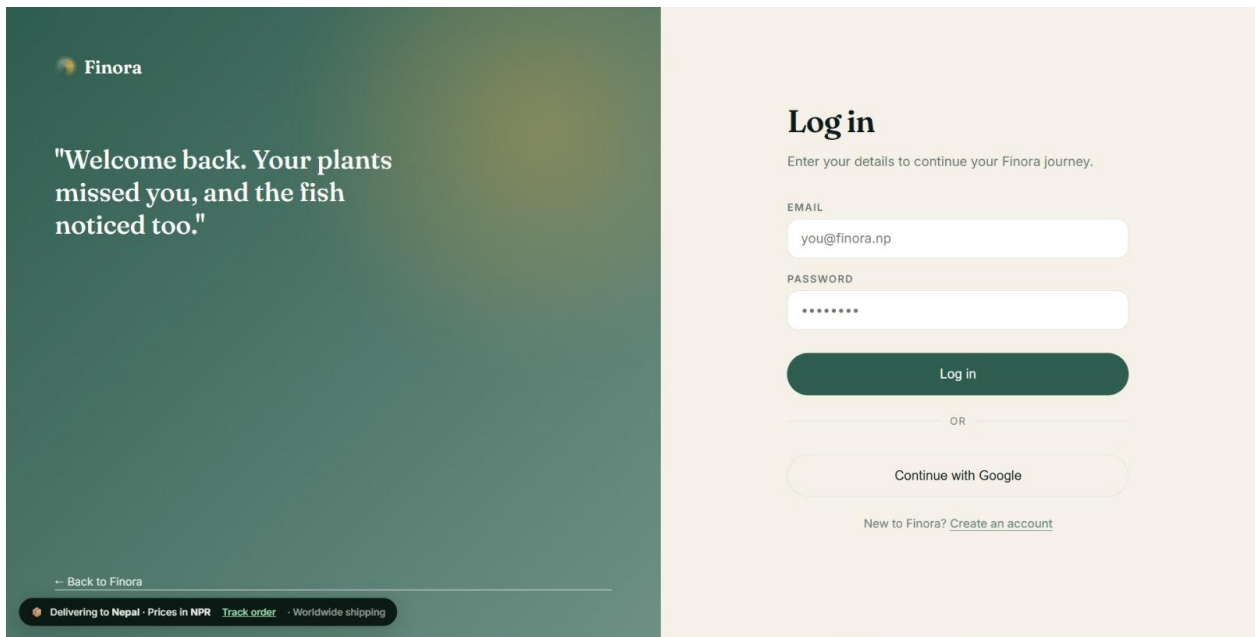


Figure 29

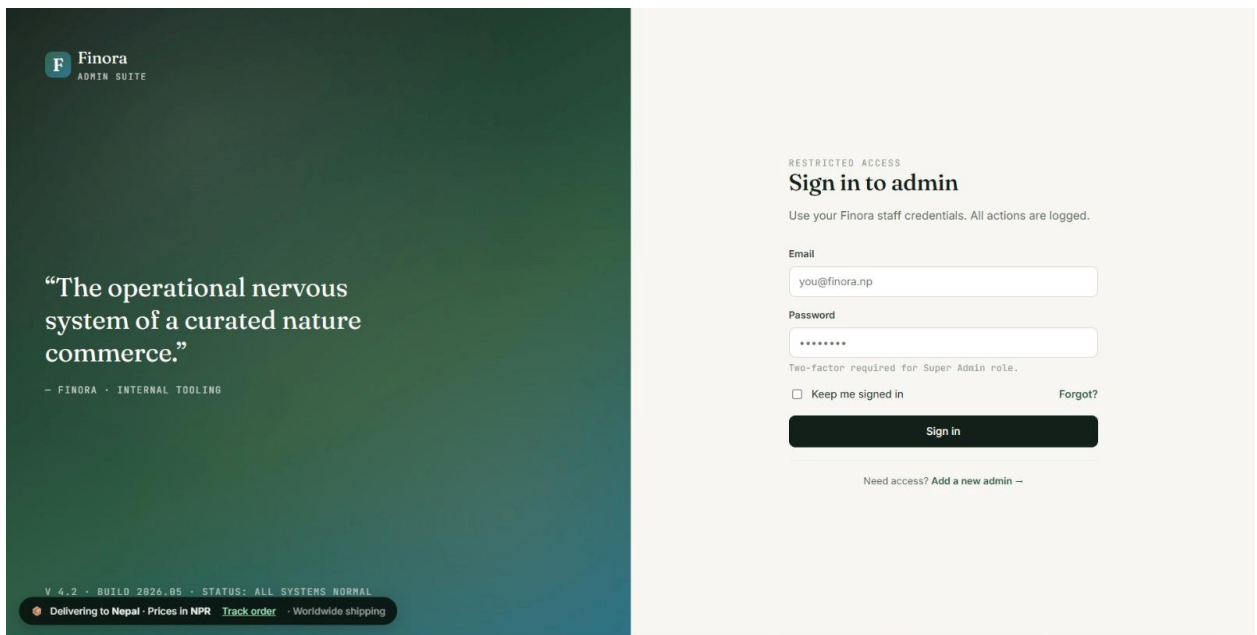


Figure 30

3.3 Register Page(s)

The Register Pages handle the creation of new accounts on the platform. The customer registration form collects the user's full name, email address, password, phone number, and address. Basic client-side validation ensures all required fields are filled and that the password meets the minimum-security criteria before the form is submitted. Upon successful registration, the customer is redirected to their dashboard.

The second page under registration is the Add Admin page, accessible only to logged-in super-administrators. It allows a super-admin to provision a new staff or admin account by supplying the new admin's name, email, role, phone number, and initial permissions. This keeps administrative access tightly controlled and prevents self-registration on the admin side.

3.3.1 Research Reference

Aritzia is a Canadian women's fashion retailer widely cited as one of the best-designed direct-to-consumer websites. Along with its login page, the registration page was also taken as a reference. Its account registration page is a structural match for Finora's Register Page. Like Finora, it uses a split-panel layout for its authentication screens. The form collects first name, last name, email address, and password as the minimum required fields, identical to the core field set in Finora's Register Page.

Website Name: Aritzia

URL: <https://www.aritzia.com/intl/en/my-account/create-account>



[Need Help?](#)

It's better on the inside. Join us

Have an account? [Sign In](#)

First Name	
Last Name	
Email Address	
Password	Show

By clicking "Create Account" or "Sign Up With Facebook", I agree that I have read and accept the [Terms of Use](#).

Create Account

or Sign Up with

 Facebook
--

This site is protected by reCAPTCHA and the Google [Privacy Policy](#) and [Terms of Service](#) apply.

Figure 31

Key points taken from this reference for the Finora page:

- A split-panel layout matching the login page. It also mirrors Finora's Register Page which uses the same auth two-column layout as the Login Page, styled with the aqua world class.
- A two-column name row collecting first name and last name in side-by-side fields, matching Finora's Register Page.
- A phone number field as a required registration input alongside email and password, matching Finora's register form.

3.3.2 Wireframe Design

Desktop - 2

Finora

"Two worlds — one home. Begin where you breathe easiest."

Create account

Join Finora and bring living things home.

First name

Last name

Email

Phone

Password

Create my Finora account

Already with us? [Log in](#)

Figure 32

3.3.3 Actual Design

Finora

"Two worlds — one home.
Begin where you breathe
easiest."

Free delivery on orders above Rs 5,000 inside Kathmandu Valley.

Delivering to Nepal · Prices in NPR [Track order](#) · Worldwide shipping

Create account

Join Finora and bring living things home.

FIRST NAME LAST NAME

Aanya Shrestha

EMAIL

you@finora.np

PHONE

+977 98XXXXXXX

PASSWORD

At least 8 characters

Create my Finora account

Already with us? [Log in](#)

Figure 33



"Two worlds — one home.
Begin where you breathe
easiest."

Free delivery on orders above Rs 5,000 inside Kathmandu Valley.

Create account

Join Finora and bring living things home.

FIRST NAME

Aanya

LAST NAME

Shrestha

EMAIL

you@finora.np

PHONE

+977 98XXXXXXX

PASSWORD

At least 8 characters

Create my Finora account

Already with us? [Log in](#)

Delivering to Nepal · Prices in NPR [Track order](#) · Worldwide shipping

Figure 34



"Two worlds — one home.
Begin where you breathe
easiest."

Free delivery on orders above Rs 5,000 inside Ka
Valley.

Create account

Join Finora and bring living things home.

FIRST NAME

Aanya

LAST NAME

Shresth

EMAIL

you@finora.np

PHONE

+977 98XXXXXXX

PASSWORD

At least 8 characters

Create my Finora account

Already with us? [Log in](#)

Figure 35

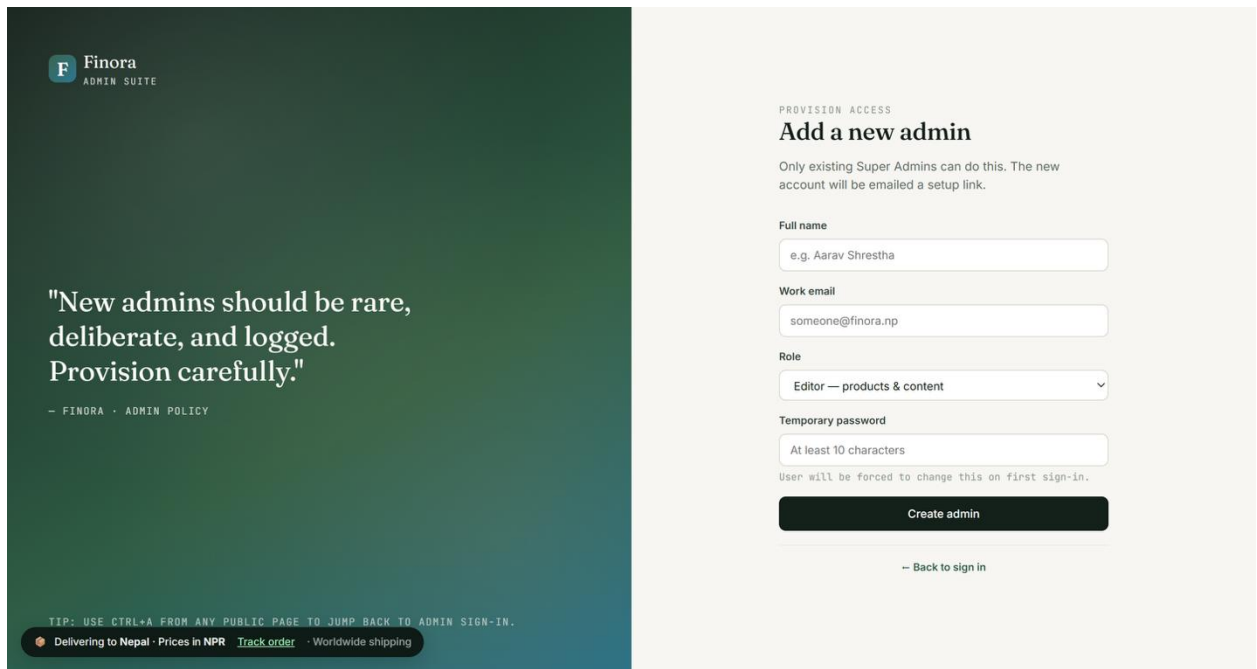


Figure 36

PROVISION ACCESS

Add a new admin

Only existing Super Admins can do this. The new account will be emailed a setup link.

Full name

e.g. Aarav Shrestha

Work email

someone@finora.np

Role

Editor — products & content

Temporary password

At least 10 characters

User will be forced to change this on first sign-in.

Create admin

[← Back to sign in](#)

 Delivering to Nepal · Prices in NPR [Track order](#) · Worldwide shipping

Figure 37

PROVISION ACCESS

Add a new admin

Only existing Super Admins can do this. The new account will be emailed a setup link.

Full name

e.g. Aarav Shrestha

Work email

someone@finora.np

Role

Editor — products & content

Temporary password

At least 10 characters

User will be forced to change this on first sign-in.

Create admin

← Back to sign in



Delivering to Nepal · Prices in NPR

[Track order](#)

· Worldwide shipping

Figure 38

3.3.4 Design Development

Finora

"Two worlds — one home.
Begin where you breathe
easiest."

Free delivery on orders above Rs 5,000 inside Kathmandu Valley.

Delivering to Nepal · Prices in NPR · Track order · Worldwide shipping

Create account

Join Finora and bring living things home.

FIRST NAME LAST NAME

Aanya Shrestha

EMAIL

you@finora.np

PHONE

+977 98XXXXXXX

PASSWORD

At least 8 characters

Create my Finora account

Already with us? [Log in](#)

Figure 39

Finora
ADMIN SUITE

"New admins should be rare,
deliberate, and logged.
Provision carefully."

FINORA · ADMIN POLICY

TIP: USE CTRL+A FROM ANY PUBLIC PAGE TO JUMP BACK TO ADMIN SIGN-IN.

Delivering to Nepal · Prices in NPR · Track order · Worldwide shipping

Add a new admin

PROVISION ACCESS

Only existing Super Admins can do this. The new account will be emailed a setup link.

Full name

e.g. Aarav Shrestha

Work email

someone@finora.np

Role

Editor — products & content

Temporary password

At least 10 characters

User will be forced to change this on first sign-in.

Create admin

Back to sign in

Figure 40

3.4 Home Page

The Home Page is the main discovery hub for logged-in customers and returning visitors. It features a dual-world toggle in the navigation bar that switches the entire page context between the Plants world and the Aquarium world, updating all content, imagery, and calls-to-action accordingly. The page opens with a large hero section showcasing a featured product and headline copy tailored to the active world, followed by a category grid that organises all product types into visual tiles. Below the categories, a curated Top Picks section presents four handpicked products selected by Finora's botanists or aquarists. A smart feature banner highlights either the Plant Care Guide system or the Fish Compatibility Finder depending on the active world. The page closes with a Community Favourites section displaying bestselling products ranked by sales, rating, or recency, and a newsletter subscription form.

3.4.1 Research Reference

PLNTS.com is a European online plant retailer whose home page is the closest structural match to Finora's Home Page. The page opens with a hero banner featuring a featured product and a direct CTA, followed by a visual category grid covering all product types. The full-page sequence including hero, world toggle, category grid, featured products, care feature and newsletter matches Finora's Home Page section by section. PLNTS.com also features a dual product-world concept, making it the strongest single reference for the overall home page experience.

Website Name: PLNTS.com (PLNTS, 2026)

URL: <https://plnts.com/en-us>

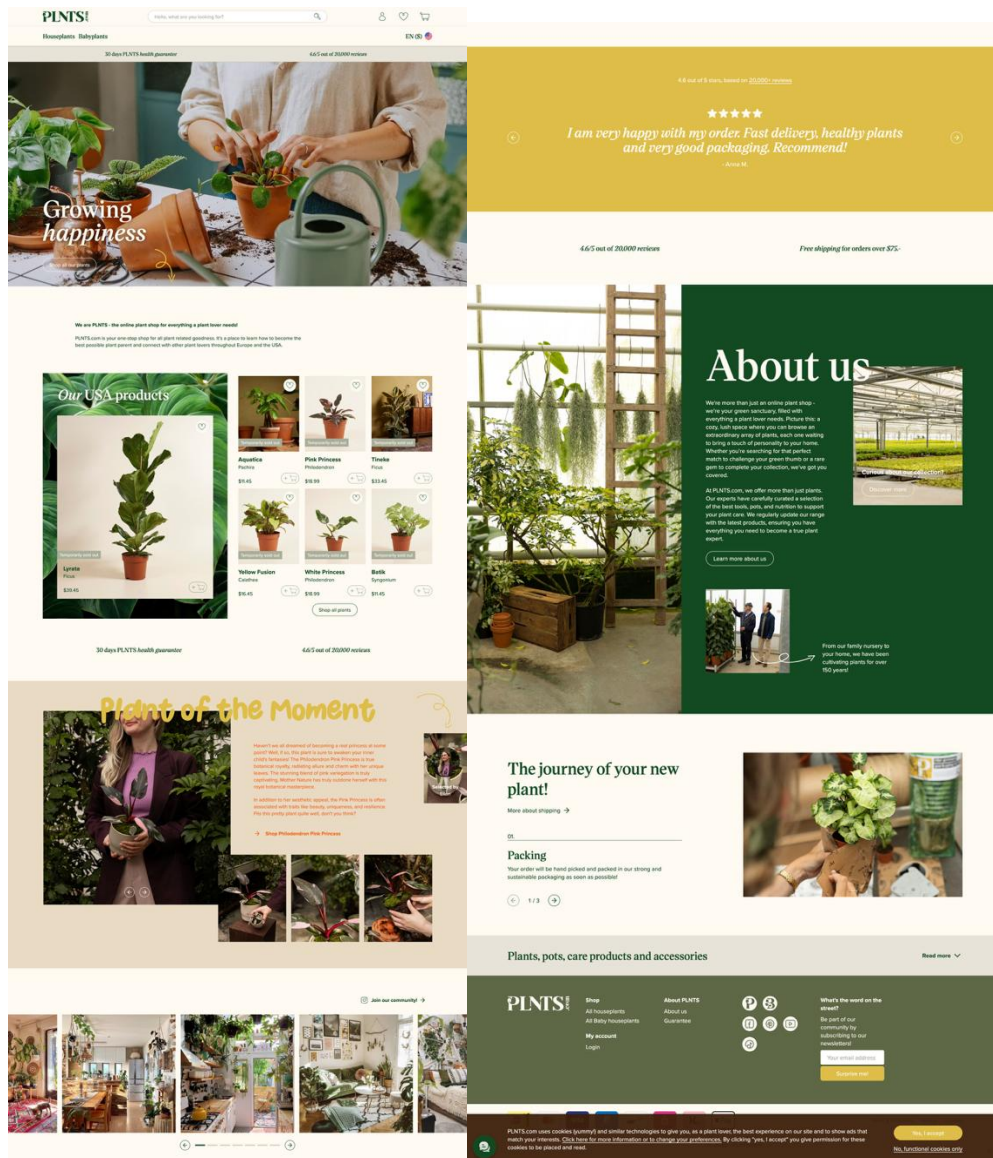


Figure 41

Key points taken from this reference for the Finora page:

- A hero section with a featured product image, headline copy, and a primary CTA button, with a floating product card overlaid on the image showing product name, variant, price and Add to Cart, matching Finora's hero float-card on the Home Page.
- A world or mode toggle in the site navigation that switches all visible content between two product contexts (Plants and Aquarium).
- A curated Top Picks product grid showing four handpicked products with category label, product name, care chips, star rating, price, Add to Cart

button, and a wish list heart icon which is identical to Finora’s “Curated Selection” section on the Home Page.

3.4.2 Wireframe Design

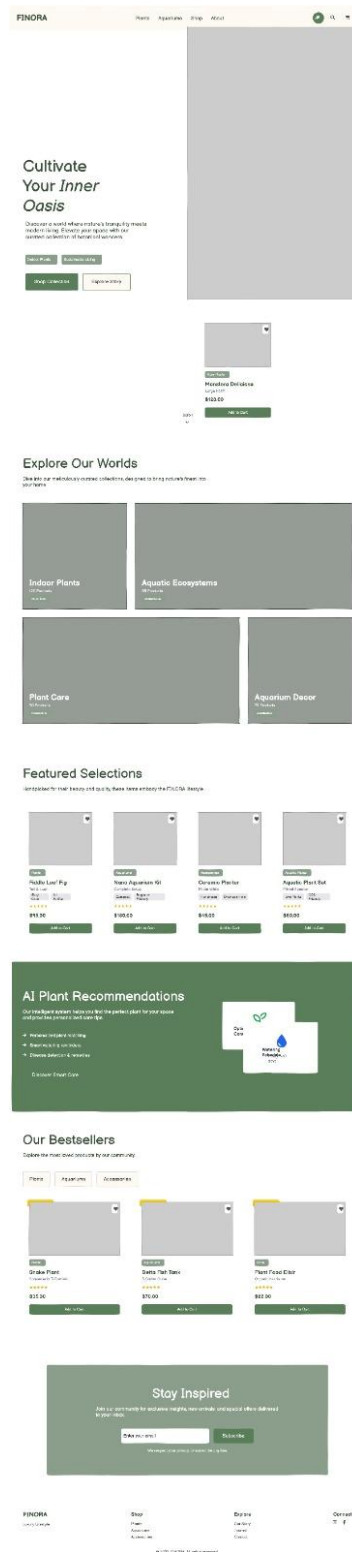


Figure 42

3.4.3 Actual Design

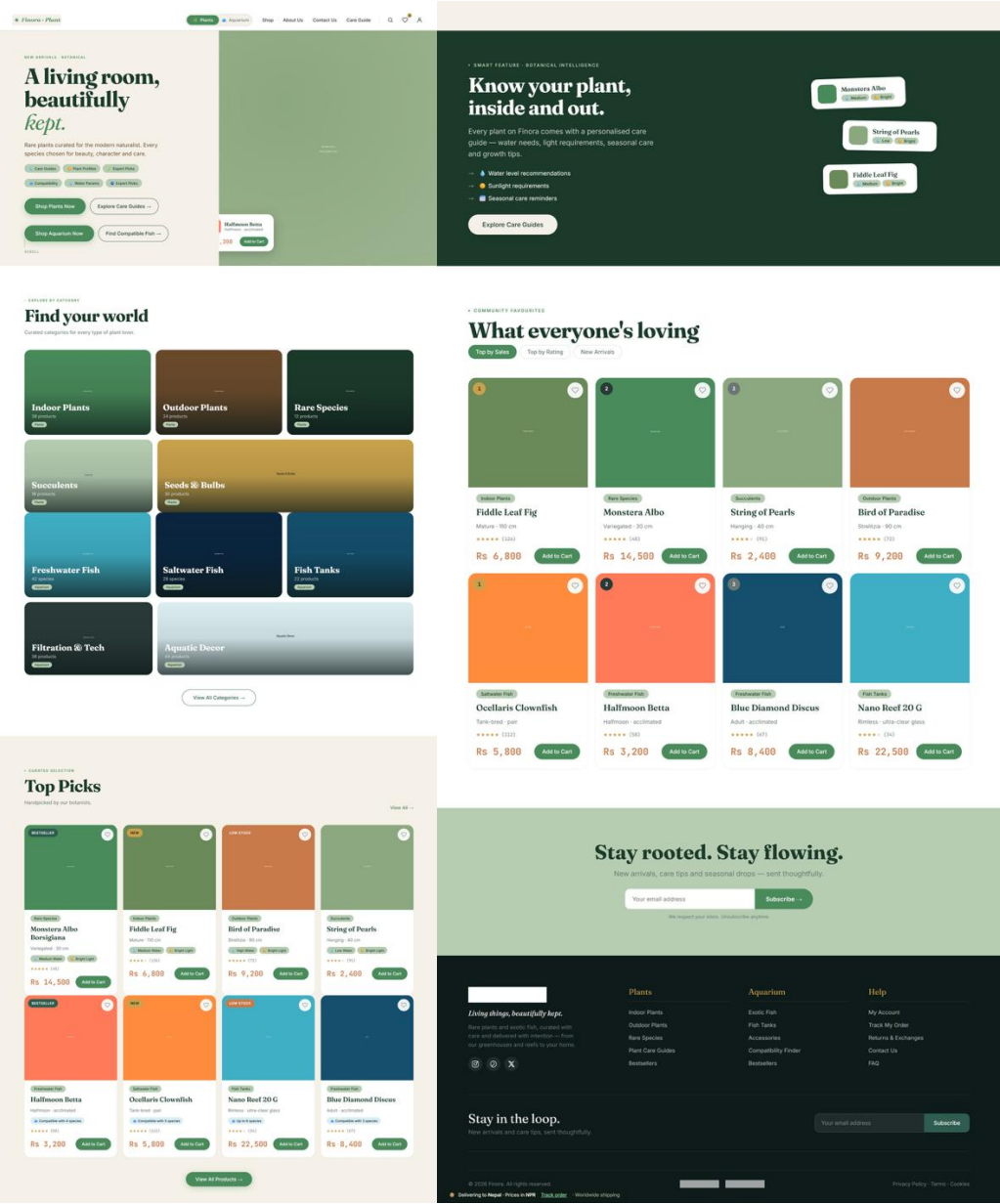


Figure 43

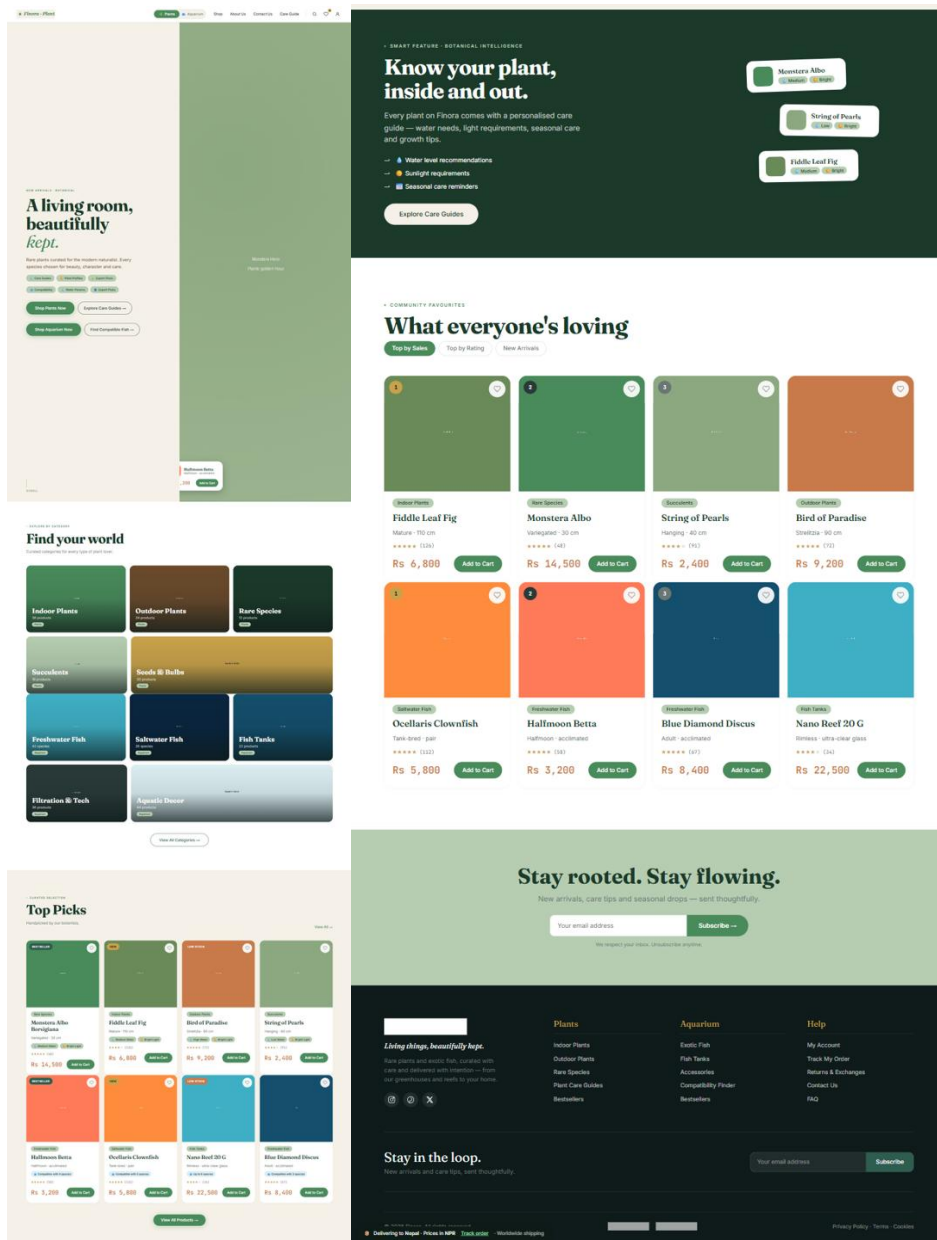


Figure 44

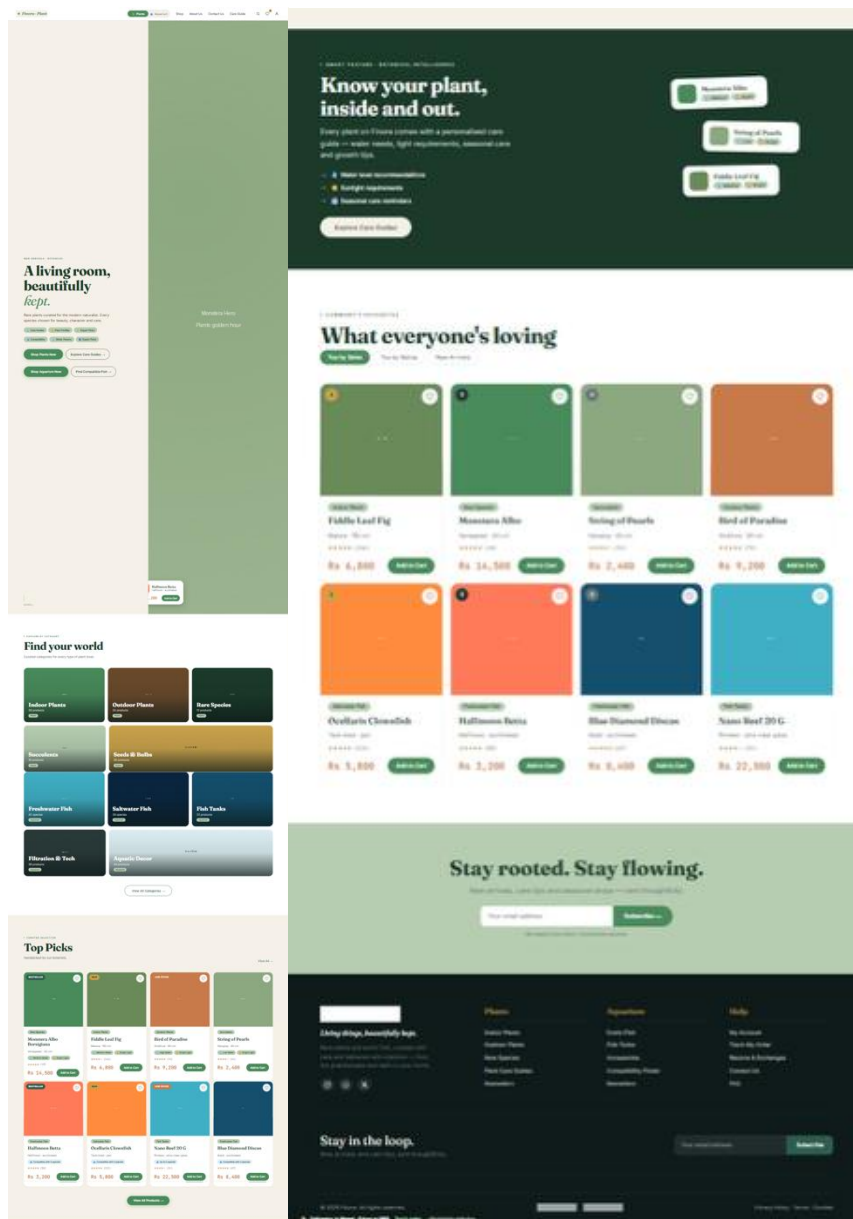


Figure 45

3.4.4 Design Development

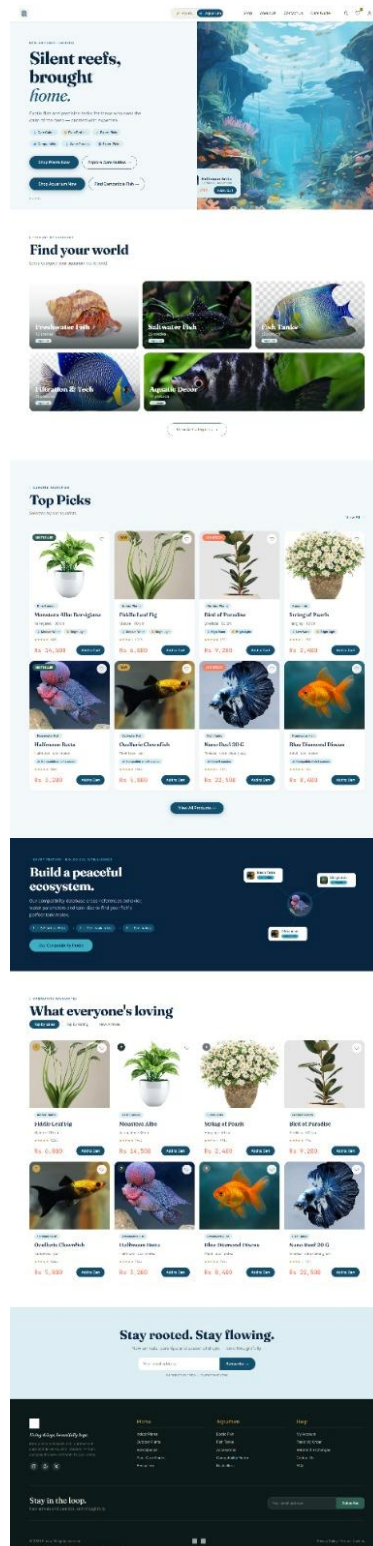


Figure 46

3.5 Shop Page

The Shop Page is the primary product browsing and purchasing interface. It presents the full Finora catalogue under the heading "The Living Shop" and supports both the

Plants and Aquarium worlds through the same toggle mechanism found site wide. The page includes a filter and sort sidebar or toolbar that allows customers to narrow results by category, price range, stock status, and product-specific attributes such as care level or fish type. Each product is displayed as a card showing the product image, category label, name, variant information, smart care or compatibility chips, star rating, price, and an Add to Cart button. A wish list icon on each card lets users save items without leaving the shop. A contextual help banner appears as the user scrolls, offering guidance for customers unsure of which product suits their needs.

3.5.1 Research Reference

LiveAquaria is the largest online aquarium and fish shop in the United States. Its shop browsing pages provide the strongest real-world reference for Finora's Shop Page, particularly for the aquarium world view. Products are displayed in a card grid showing image, name, short description, price, stock status, and an Add to Cart button. A left-hand filter sidebar allows narrowing by category, price, availability, and species type. The page also switches between plant and aquarium product sets via the site's mode system, and a help banner appears as the user scrolls for undecided shoppers.

Website Name: LiveAquaria (LiveAquaria, 2025)

URL: <https://www.liveaquaria.com/collections>

- A mode toggle in the navigation switching the entire shop between Plants and Aquarium product worlds, updating the category strip, product grid, and sidebar filter options directly matching Finora's Plants/Aquarium toggle on the Shop Page.
- A horizontal scrollable category strip above the product grid allowing instant category switching without using the sidebar.
- Product cards displaying image, category label, product name, one-line description, star rating with review count, price, Add to Cart button, and a wish list heart icon that matches the exact card structure rendered by Finora's shop product grid.
- Status badges on product cards (New, Bestseller, Low Stock) giving immediate visual cues.

3.5.2 Wireframe Design

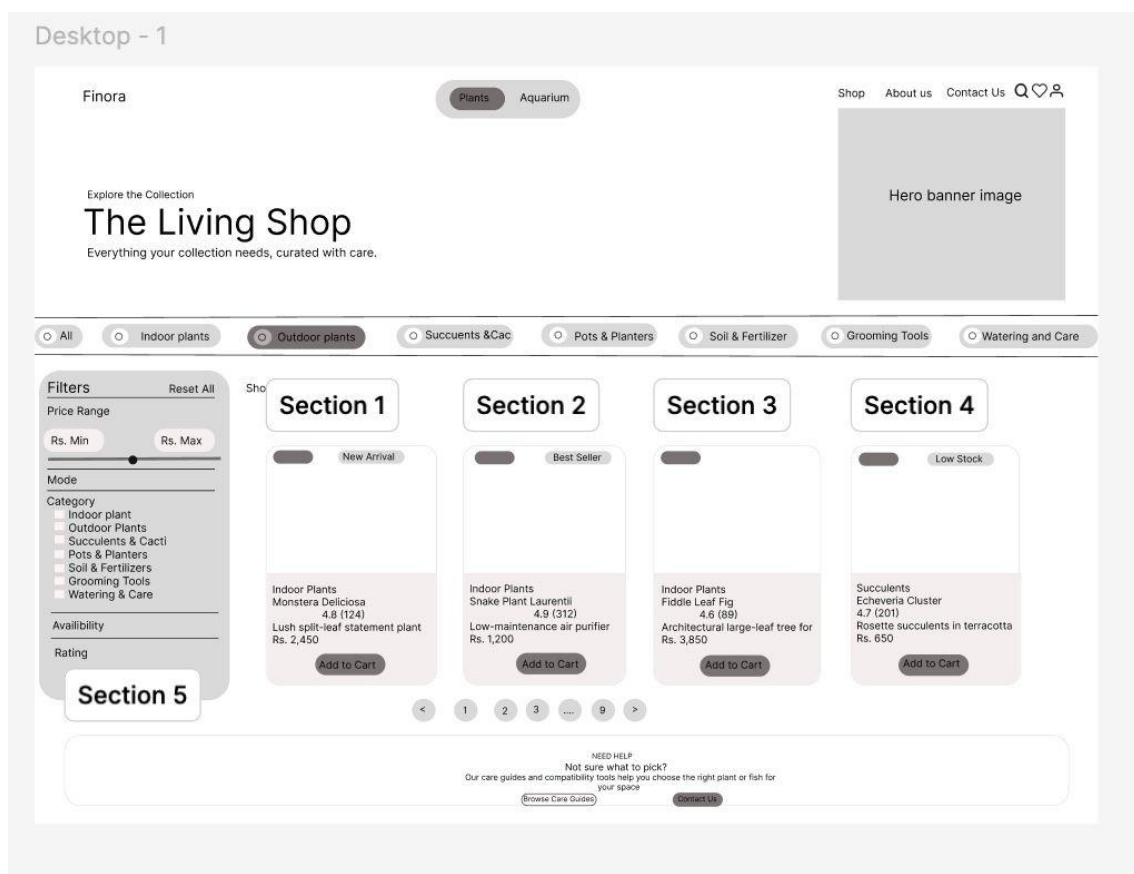


Figure 48

3.5.3 Actual Design

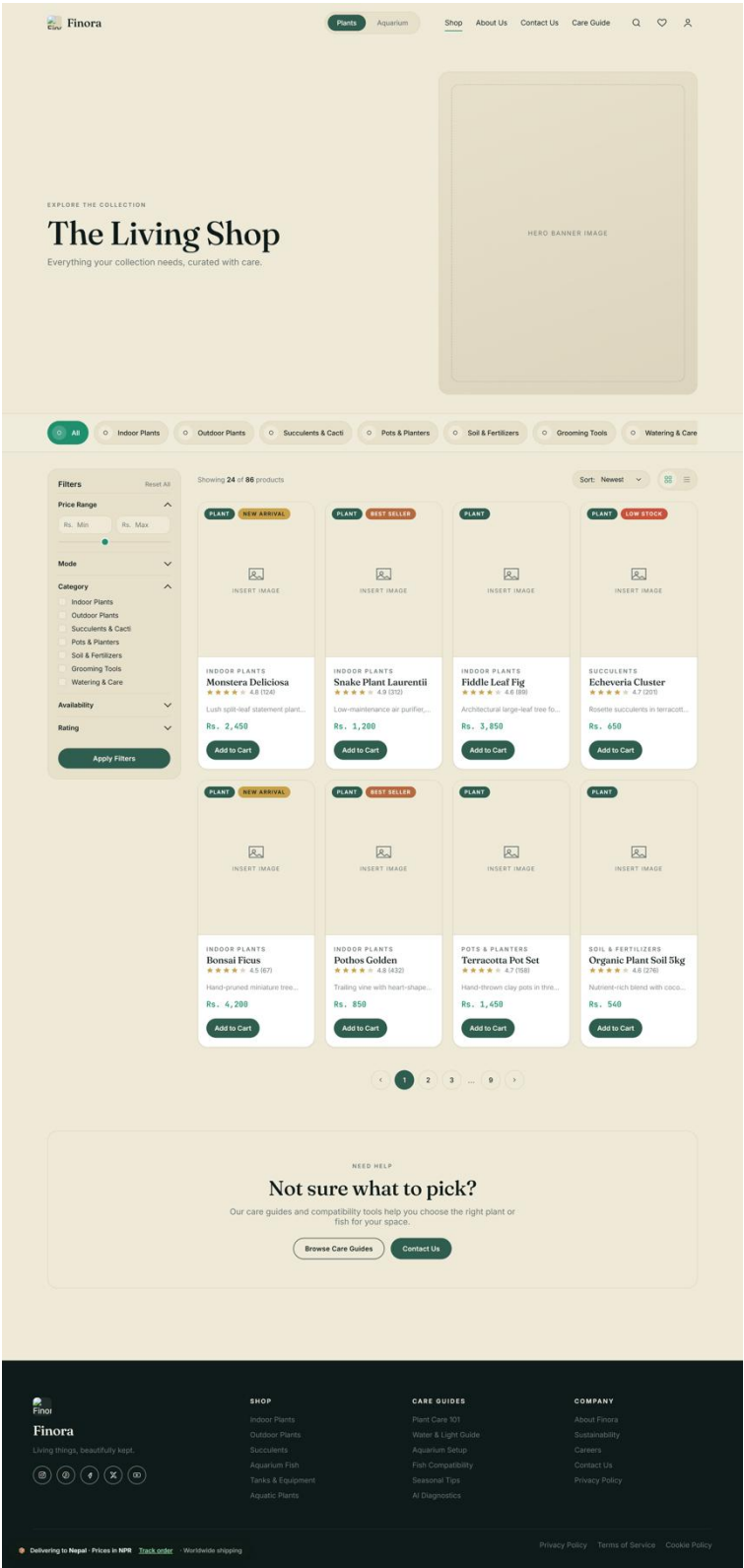


Figure 49

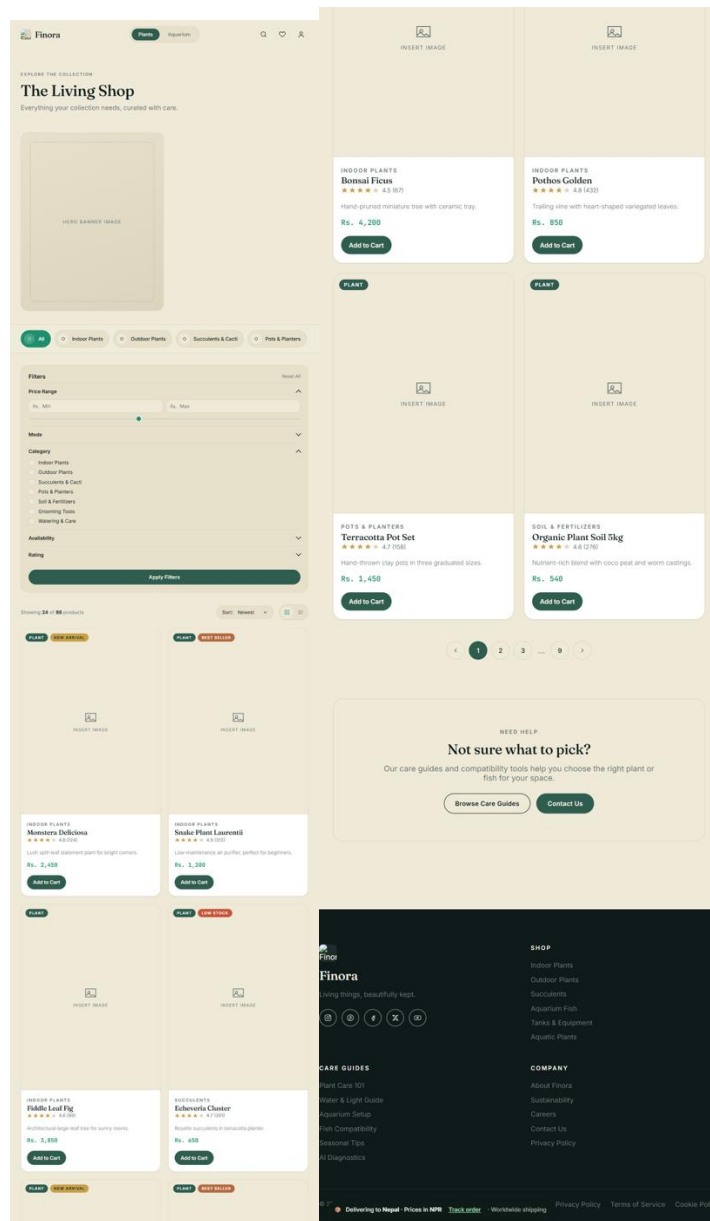


Figure 50

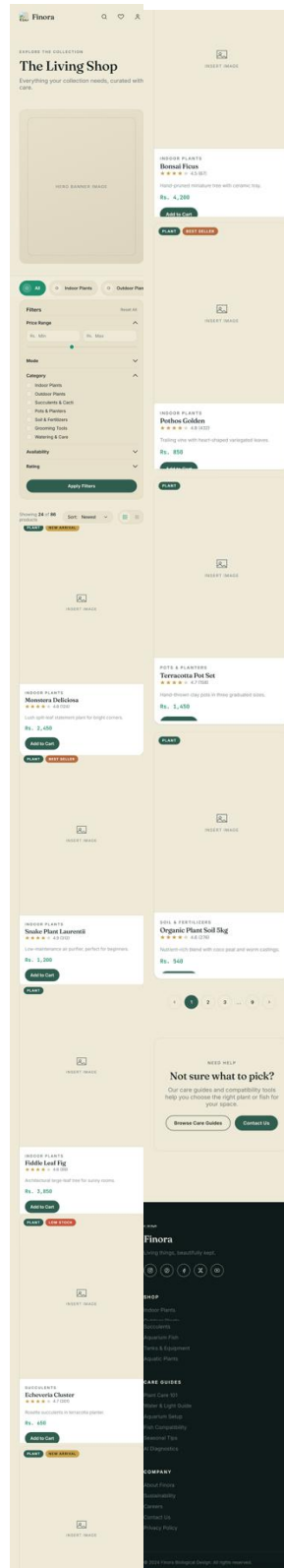


Figure 51

3.5.4 Design Development

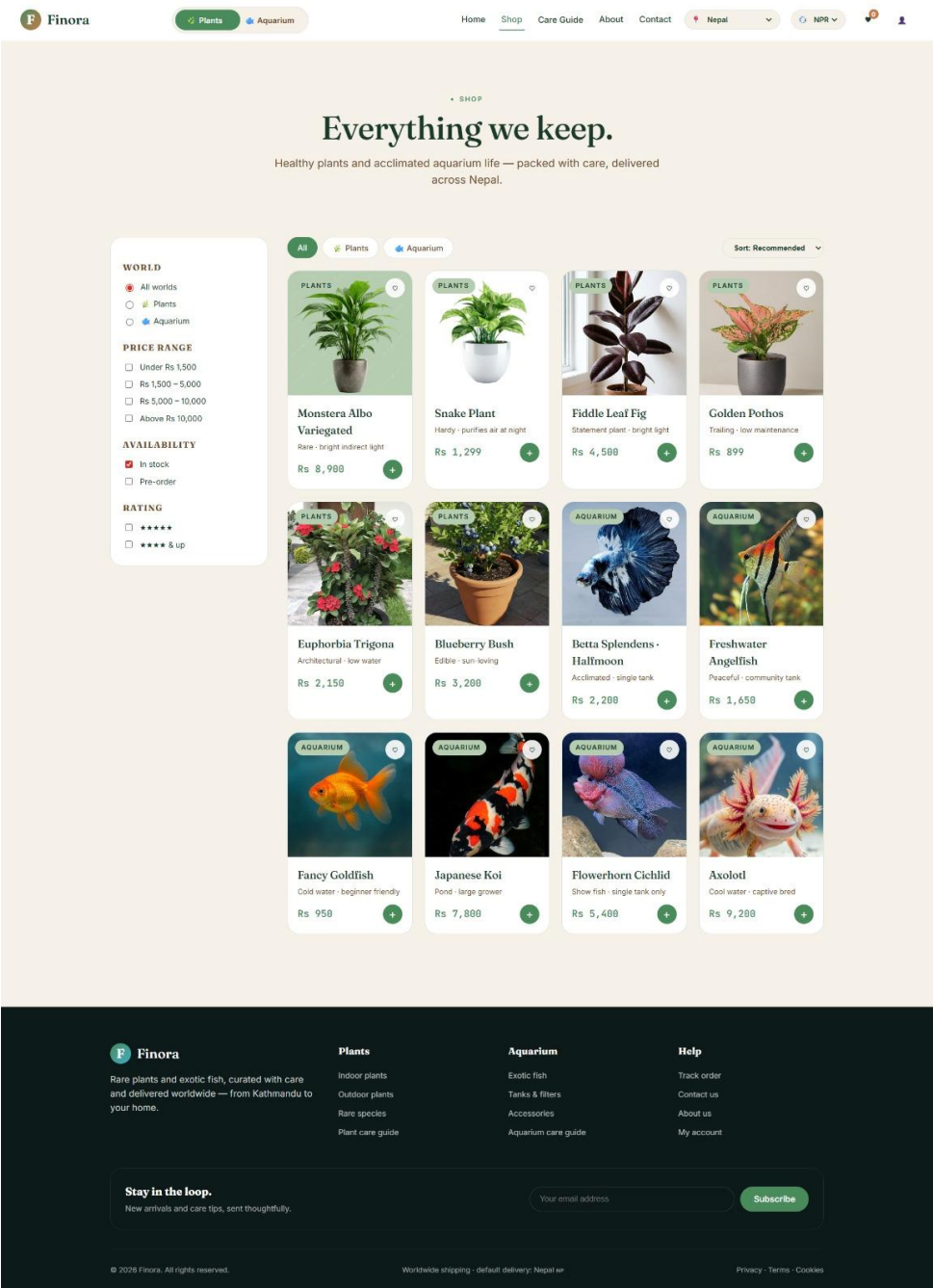


Figure 52

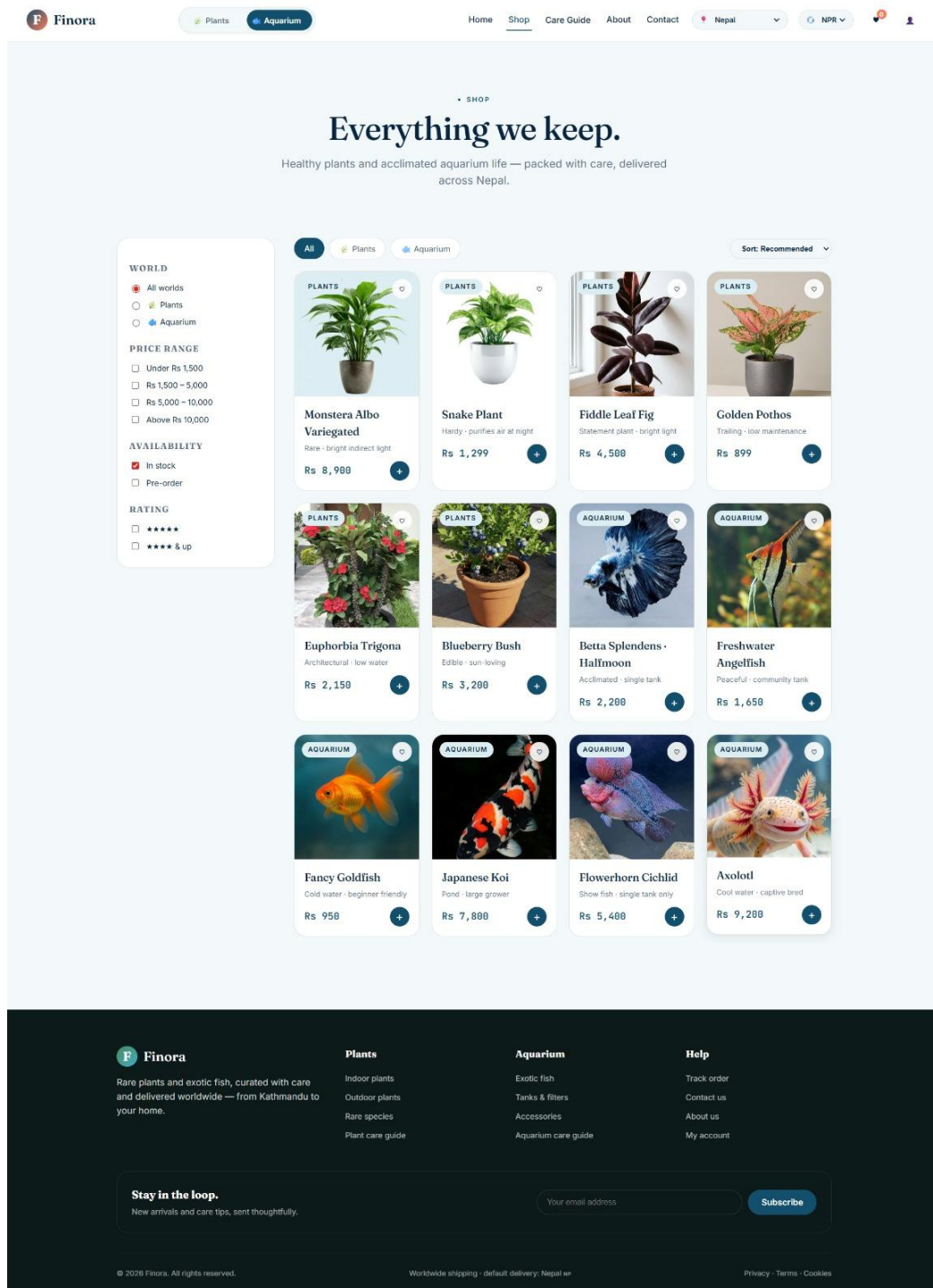


Figure 53

3.6 Track-Order Page

The Track-Order Page allows customers to check the status of any placed order without needing to navigate through their full account dashboard. The page presents a simple

form where the customer enters their order ID and registered email address to look up the order. Once submitted, the page displays the order's current status such as Pending, Confirmed, Shipped, or Delivered, along with the estimated delivery date and a visual progress indicator showing where the order is in the fulfilment pipeline. This page is accessible to both logged-in users and guests who placed an order without an account, making it a convenient self-service tool that reduces the need to contact customer support for routine status checks.

3.6.1 Research Reference

Nike's order tracking and guest order lookup page is the best real-world reference for Finora's Track-Order Page because it is specifically designed to serve both logged-in members and non-account guests with an identical, frictionless lookup experience. The page presents a simple input form where the user enters their order number and clicks a track button to retrieve their order's status. The result is displayed as a visual step-by-step progress bar showing the order's journey from Placed through Packed, Dispatched, Out for Delivery, and Delivered.

Website Name: Nike-Guest Order Lookup

URL: <https://www.nike.com/orders/details/>

Find a Store | Help | Join Us | Sign In

Find a StoreAbout Us

View or Manage Your Order

To check the status of your order, or to start a return, please enter your order number and email address.

Order Number*

Email address*

Submit

Already a member?Sign in

Resources

Find a Store

Help

Get Help

Returns

Contact Us

Company

About Nike

News

Careers

Investors

Sustainability

Egypt

© 2026 Nike, Inc. All rights reserved

Terms of Use

Privacy & Cookie Policy

Figure 54

Key points taken from this reference for the Finora page:

- A guest-accessible order lookup form with a single Order Number input field and a track button, requiring no login which matches Finora’s Track-Order Page which uses an order ID input and lookup button without forcing authentication.
- A five-stage visual progress bar (Placed, Packed, Dispatched, Out for Delivery, Delivered) with dot markers and a filled progress line showing how far the order has advanced, directly matching Finora’s stages.
- An order summary result card shown after lookup displaying the order number, placement date, shipping destination, and items ordered which matches Finora’s result card showing order metadata alongside the tracker.

3.6.2 Actual Design

[← Back to Finora](#)

Live tracking · Nepal default, worldwide shipping

TRACK AN ORDER

Where's my Finora order?

Enter your order number (any string works for the demo) — we'll show a live progress timeline that updates every few seconds.

Track

Order

Placed

Ship to

Items

Placed

09:42

Packed

10:15

Dispatched

11:30

Out for delivery

~14:00

Delivered

~15:30

FN-2026-08421

Today, 09:42

Kathmandu, Nepal

Domestic · 1-3 days

3 plants · 1 ceramic pot

[08:46 AM] Packed at our Kathmandu hub

[08:46 AM] Order received

Updates every few seconds. We notify you by email and SMS at each stage.

📍 Delivering to Nepal · Prices in NPR

[Track order](#)

Worldwide shipping

Figure 55

TRACK AN ORDER

Where's my Finora order?

Enter your order number (any string works for the demo) — we'll show a live progress timeline that updates every few seconds.

FN-2026-08421

Track

Order
Placed
Ship to
Items

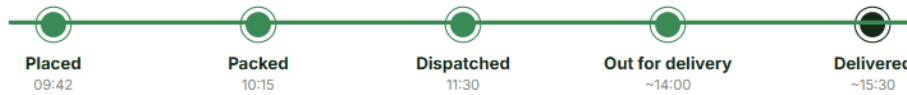
FN-2026-08421

Today, 09:42

Kathmandu, Nepal

Domestic · 1–3 days

3 plants · 1 ceramic pot



[11:02 AM] Delivered — thank you!

[11:02 AM] Out for delivery in your area

[11:01 AM] Handed to courier

[11:01 AM] Packed at our Kathmandu hub

[11:01 AM] Order received

Updates every few seconds. We notify you by email and SMS at each stage.

Figure 56

← **Back to
Finora**

Live tracking · Nepal default, worldwide
shipping

TRACK AN ORDER

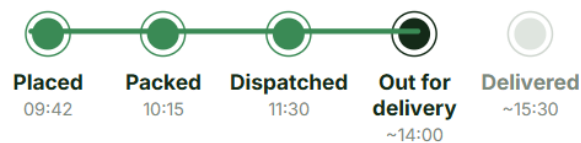
Where's my Finora order?

Enter your order number (any string works for the
demo) — we'll show a live progress timeline that
updates every few seconds.

FN-2026-08421

Track

Order FN-2026-08421
Placed Today, 09:42
Ship to Kathmandu, Nepal **Domestic · 1-3 days**
Items 3 plants · 1 ceramic pot



[11:32 AM] Out for delivery in your area

[11:32 AM] Handed to courier

[11:32 AM] Packed at our Kathmandu hub

[11:32 AM] Order received

Updates every few seconds. We notify you by email
and SMS at each stage.



**Delivering to Nepal · Prices in
NPR**

[Track
order](#)

· **Worldwide
shipping**

Figure 57

3.6.3 Design Development

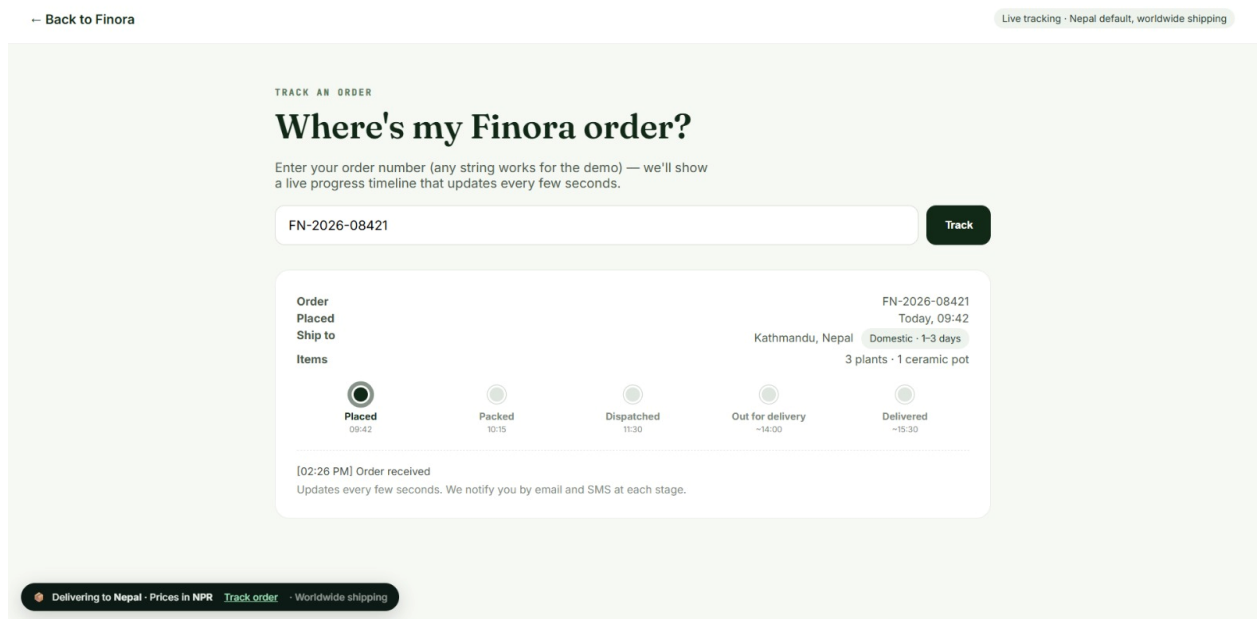


Figure 58

3.7 Wishlist Page

The Wishlist Page is a public-facing saved items page that allows customers to view and manage all the products they have marked for later. The page header displays the total count of wish listed items under the label "Your Collection" and provides a short description encouraging the user to bring saved items home. Products are displayed in the same card format used on the Shop Page, allowing users to add any item directly to the cart or remove it from the wish list. A recommendations section beneath the saved items suggests related products based on the contents of the wish list. The wish list persists across sessions, so saved items remain available the next time the customer logs in. This page is separate from the User Dashboard wish list, which offers additional management features like sorting and filtering.

3.7.1 Research Reference

Aritzia has one of the most polished wish list ("My List") implementations among lifestyle e-commerce brands. Its My List page is the closest real-world structural match to Finora's Wishlist Page. The page opens with a header showing the list title and the total item count, then displays all saved products in a card grid. Each card

shows the product image, name, price, and availability status, with buttons to move the item to the cart or remove it from the list.

Website Name: Aritzia

URL: <https://www.aritzia.com/us/en/mylist>

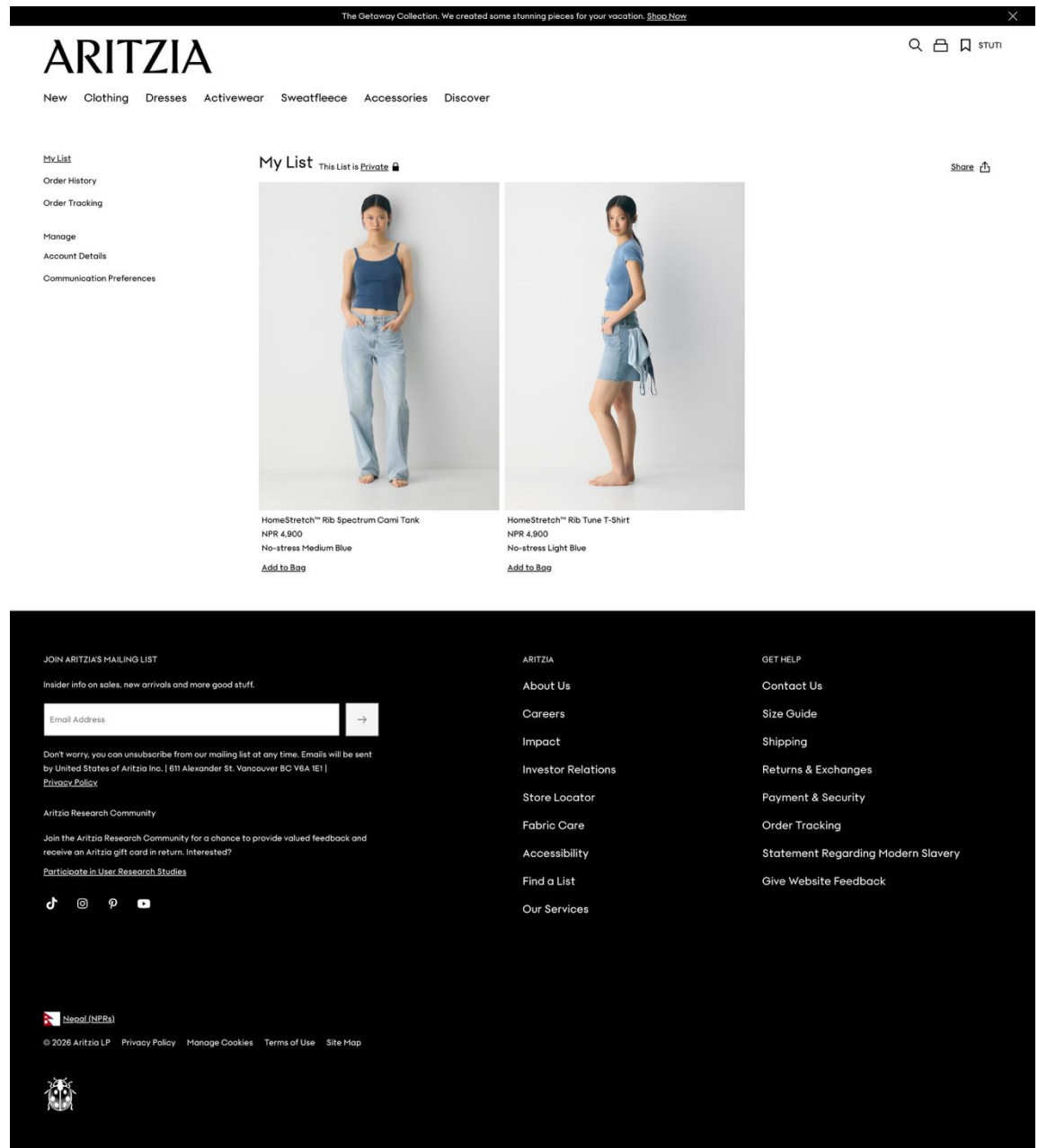


Figure 59

Key points taken from this reference for the Finora page:

- Product cards in the wish list grid showing image, name, price, stock status, and both an Add to Cart and a Remove action which matches the wish list card layout in Finora where each card has Add to Cart and a remove button.
- Session-persistent wish list storage that keeps saved items after logout and restores them on next login, consistent with Finora's wish list which persists across sessions as stated in the platform feature list.

3.7.2 Wireframe Design

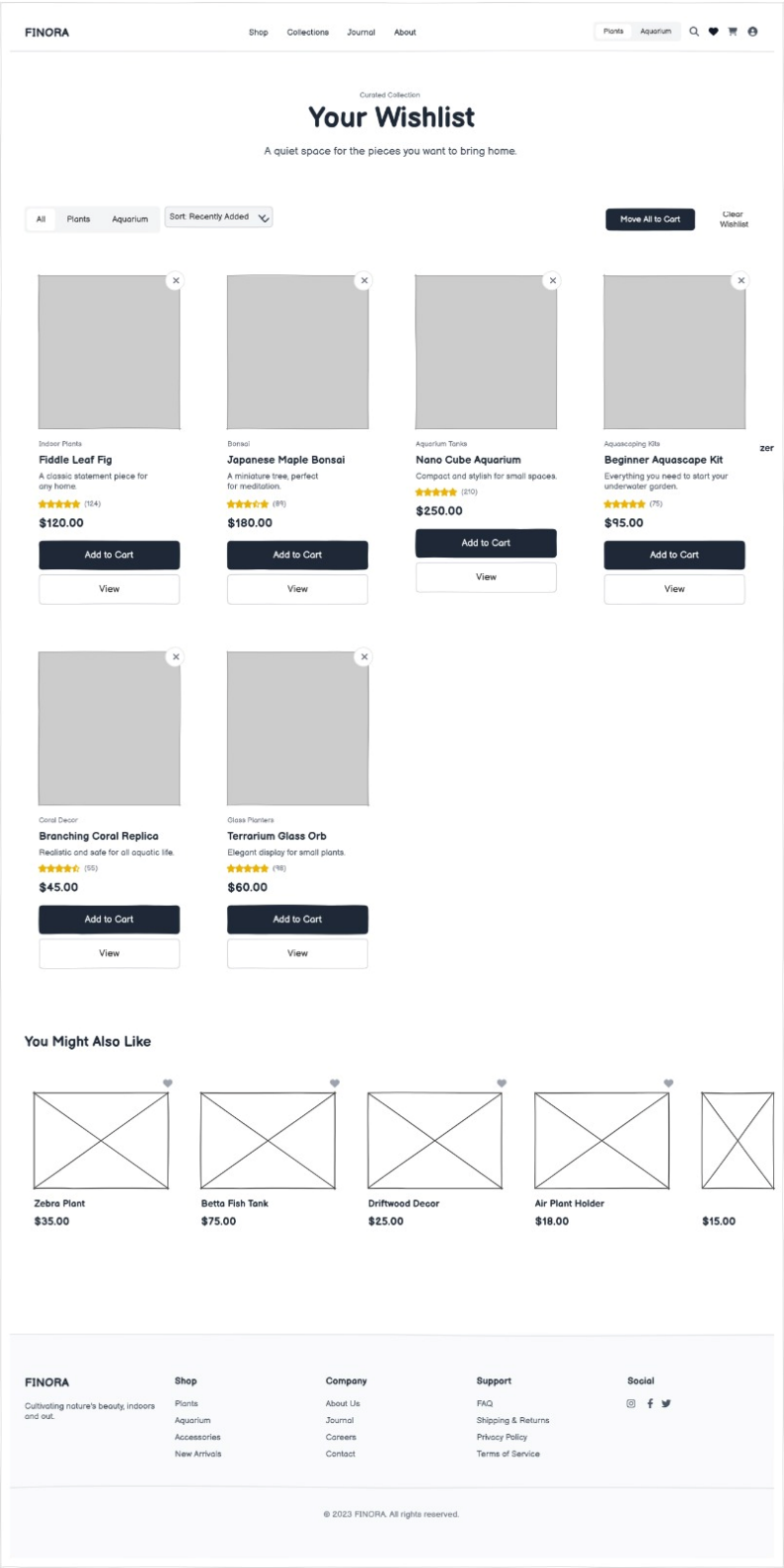


Figure 60

3.7.3 Actual Design



Figure 61

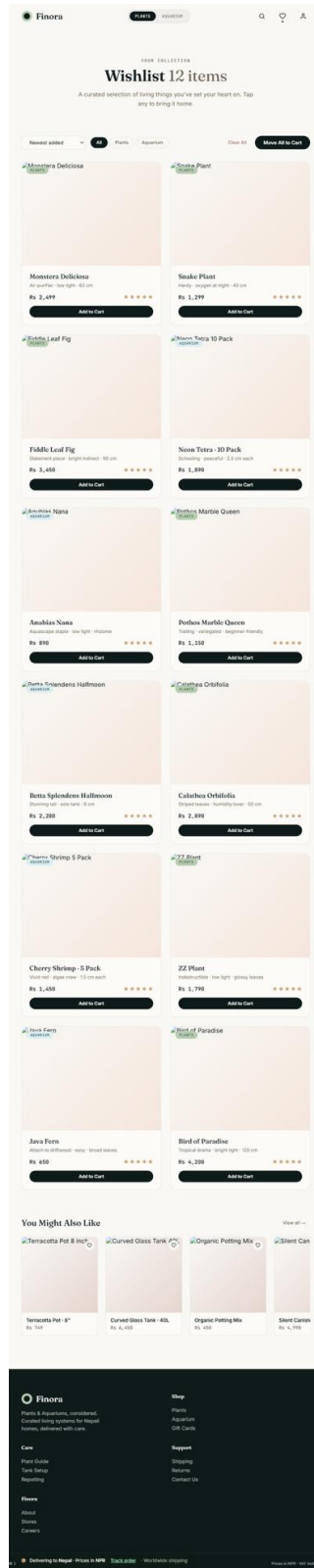


Figure 62

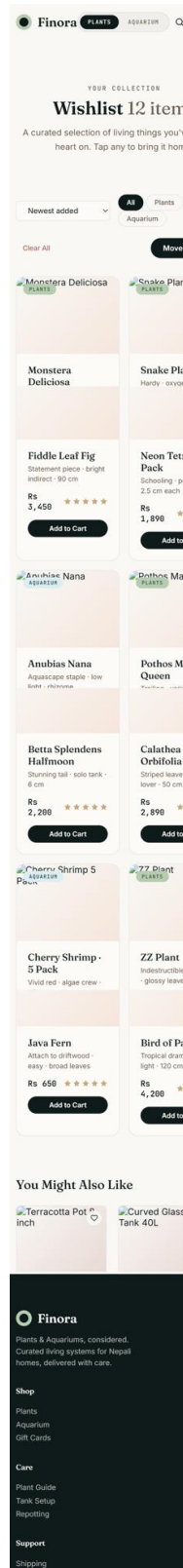


Figure 63

3.8 Contact Page

The Contact Page provides customers with multiple ways to reach the Finora support team. The page opens with a welcoming hero section under the headline "Let's care for your world together" and includes direct contact details such as email address, phone number, and operational hours. Below the hero, a structured message form allows visitors to select a topic, enter their name, email, and optionally an order ID for order-related queries, and type a detailed message. The form includes an anti-spam CAPTCHA element and confirms submission with a success message. A support section beneath the form organises frequently raised topics into expandable cards to help customers resolve common issues without contacting staff. The page concludes with a community section encouraging customers to follow Finora on social media.

3.8.1 Research Reference

Allbirds is a US sustainable footwear brand whose Contact page is consistently cited by Shopify as one of the best-designed help and contact pages in e-commerce. The page opens with the heading "How Can We Help?" and immediately presents multiple contact channel options such as live chat, phone, email, and text.

Website Name: Allbirds-How Can We Help?

URL: <https://www.allbirds.com/pages/help-center>

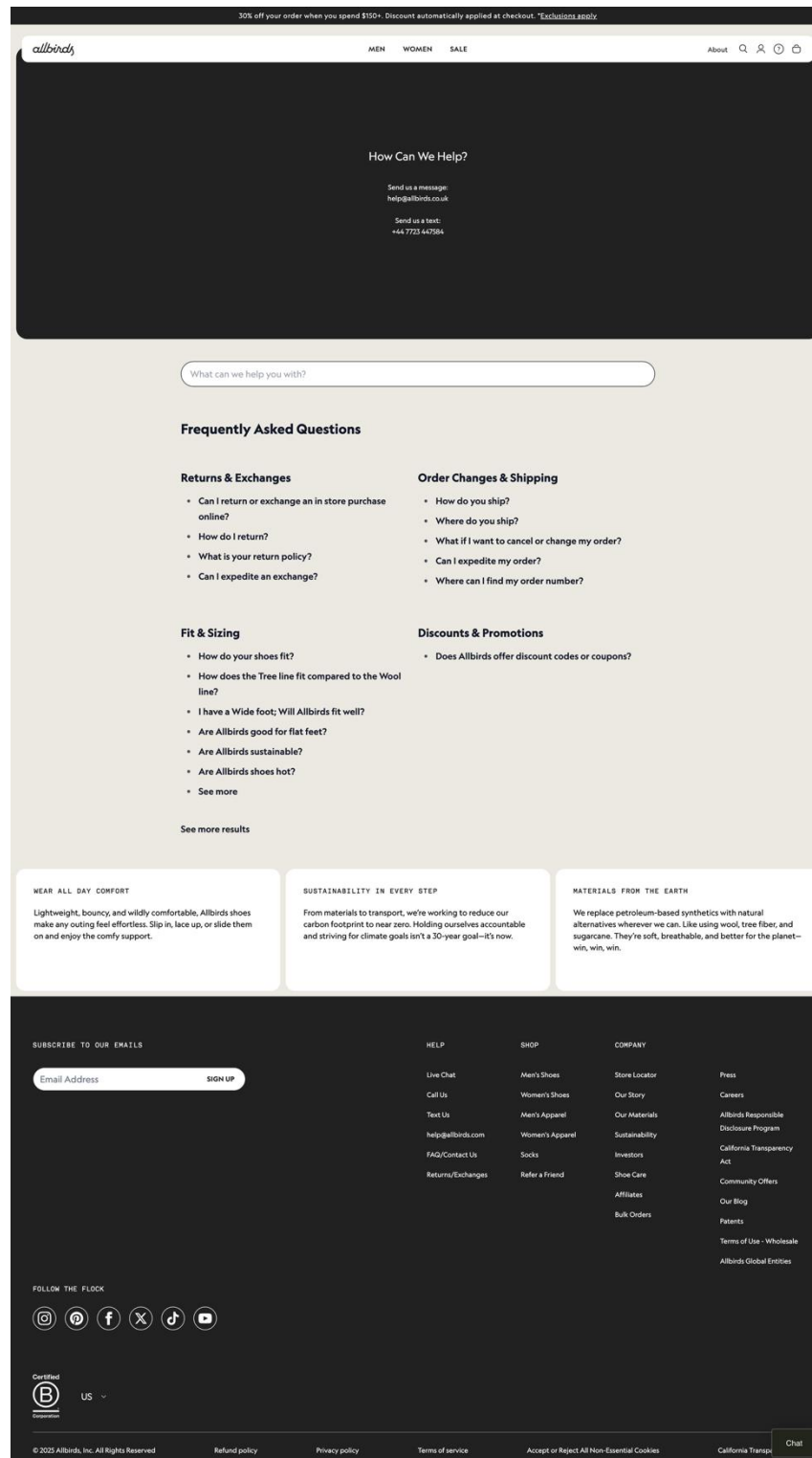


Figure 64

Key points taken from this reference for the Finora page:

- A hero section with a welcoming headline (“How Can We Help?”) and a short paragraph describing the scope of support offered, matching Finora’s Contact Page hero which opens with “Let’s care for your world together.” and lists what the team helps with.
- A row of contact channel cards each showing the channel name, contact detail, and operating hours, matching Finora’s four info-cards (Call Us with phone and hours, Email Support with address and response time, Instagram, and Studio location).
- A structured contact message form with fields for name, email, a topic dropdown, and a free-text message area, matching Finora’s contact form with name, email, phone, subject select, and text area fields.

3.8.2 Wireframe Design

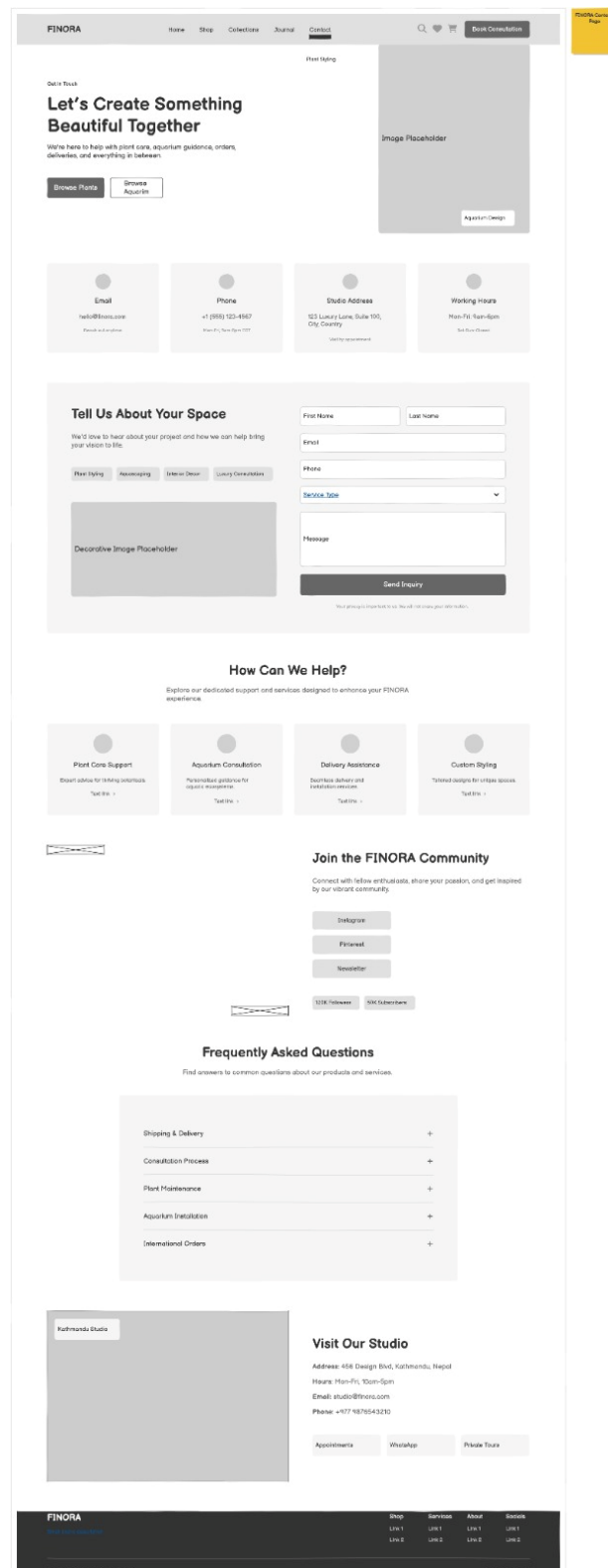


Figure 65

3.8.3 Actual Design

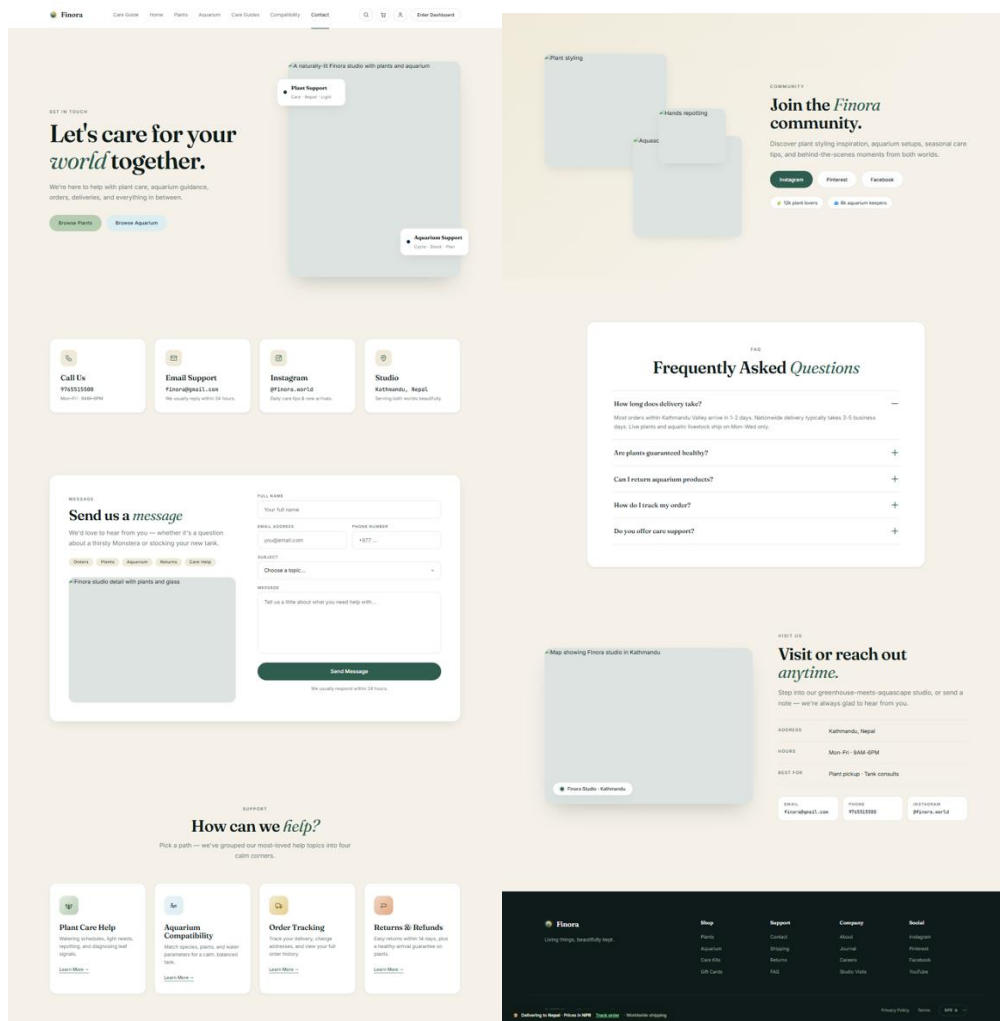


Figure 66

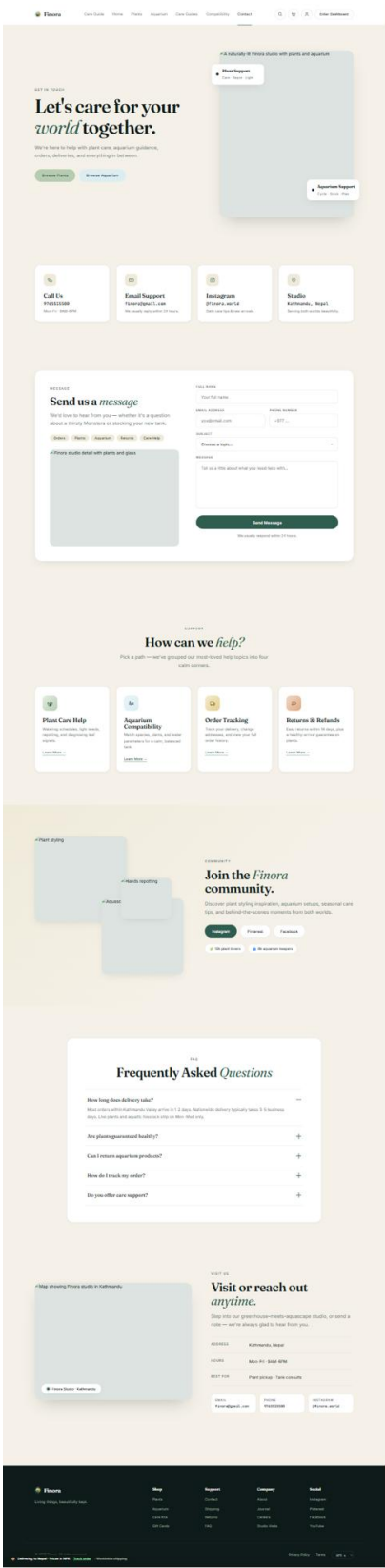


Figure 67

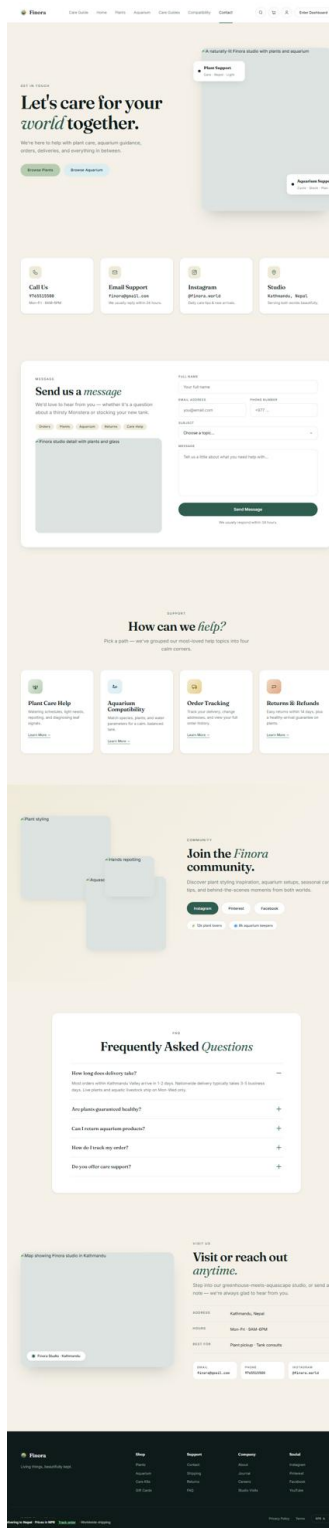


Figure 68

3.8.4 Design Development

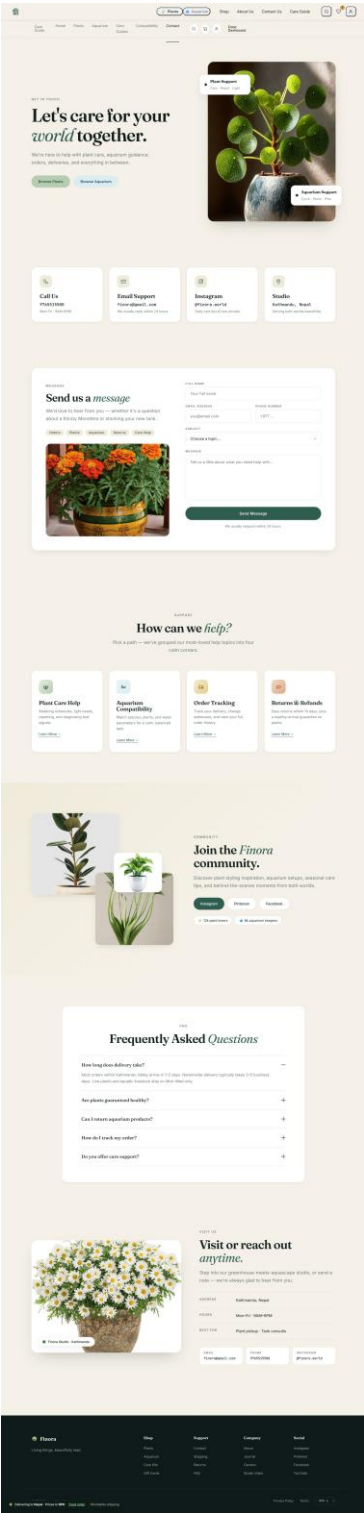


Figure 69

3.9 Care Guide Page

The Care Guide Page serves as Finora's botanical knowledge library. Accessible under the label "Care Library", it is designed to help plant owners understand how to keep their purchased plants healthy and thriving. The page lists all plants in the catalogue, and each entry can be expanded or navigated to for detailed care information including watering frequency (low, medium, or high), sunlight requirements (low, medium, bright indirect), soil type recommendations, seasonal care tips, and common issues to watch for. The page also serves as an educational resource for customers who are browsing before purchasing, enabling them to assess whether a particular plant suits their home environment and care capacity. Content on this page is managed by administrators through the product management system, where plant-type products carry dedicated care attribute fields.

3.9.1 Research Reference

Lively Root is a US online plant retailer with one of the most comprehensive and well-structured plant care guide libraries available on any live e-commerce site. Its care guide directory page lists all plants in the catalogue with individual links, and each guide page provides structured per-attribute care information covering light requirement, watering frequency, soil type, humidity, temperature range, fertilisation schedule, and seasonal notes.

Website Name: Lively Root-Plant Care Directory

URL: <https://www.livelyroot.com/pages/care-guide-directory>

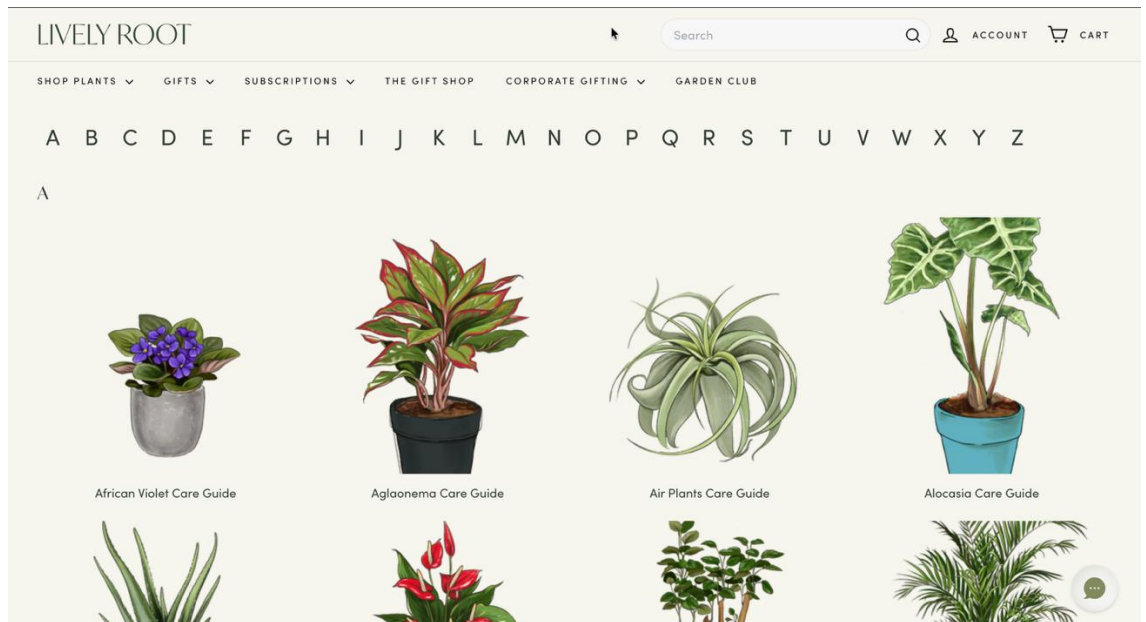


Figure 70

Key points taken from this reference for the Finora page:

- A care library hero section with an eyebrow label (“Care Library” in Finora) and a headline framing the page as an educational resource, matching the care-hero section at the top of Finora’s Care Guide Page.
- A world toggle that switches the visible guide set between plant care content and aquarium care content.
- Care guide cards with a topic chip tag is the exact card structure of Finora’s care-card component rendered by its JavaScript grid.
- A care guide page that functions as both a post-purchase support resource and a pre-purchase decision aid, helping customers assess suitability before buying has the same dual purpose stated for Finora’s Care Guide Page in the feature list.

3.9.2 Actual Design

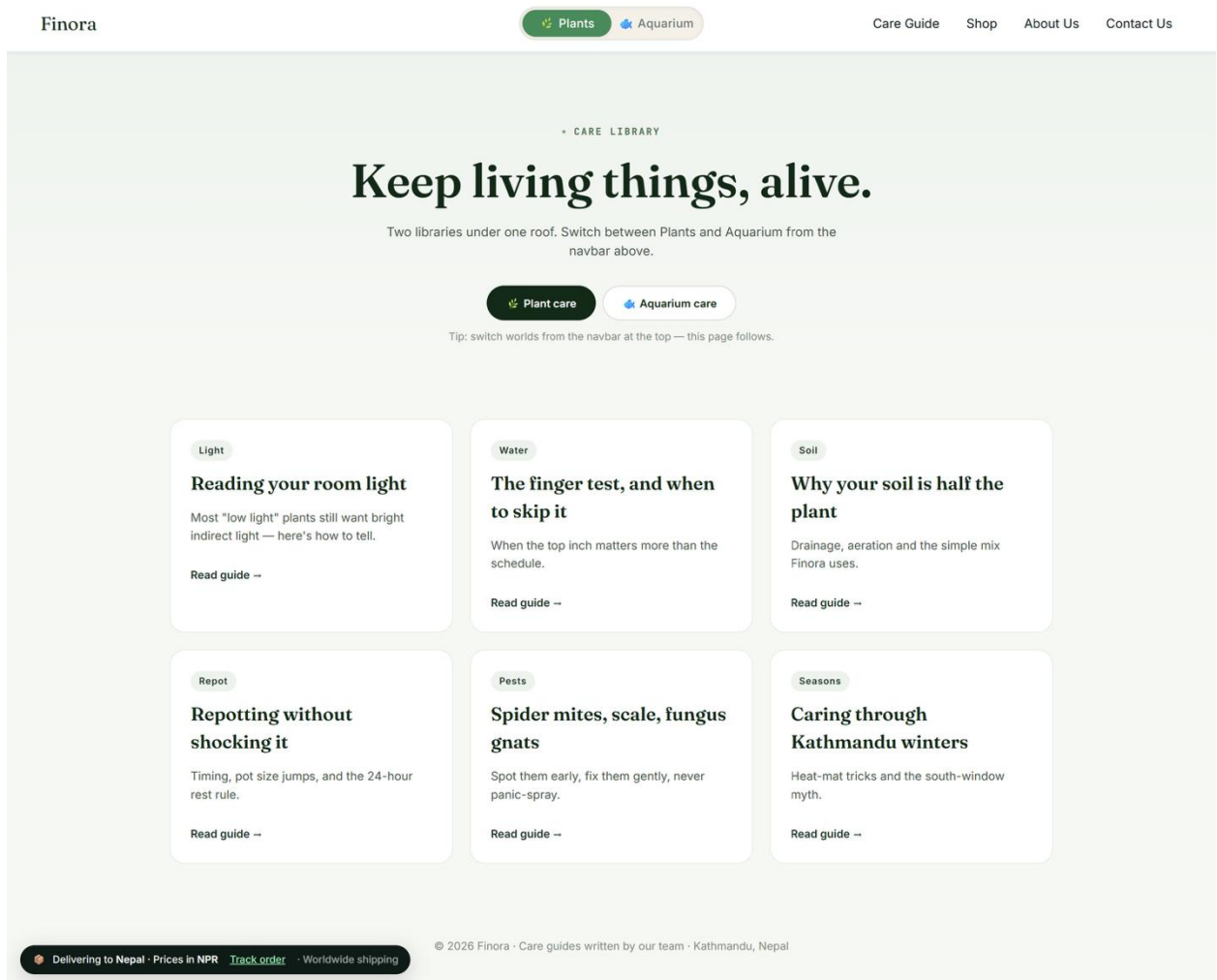


Figure 71

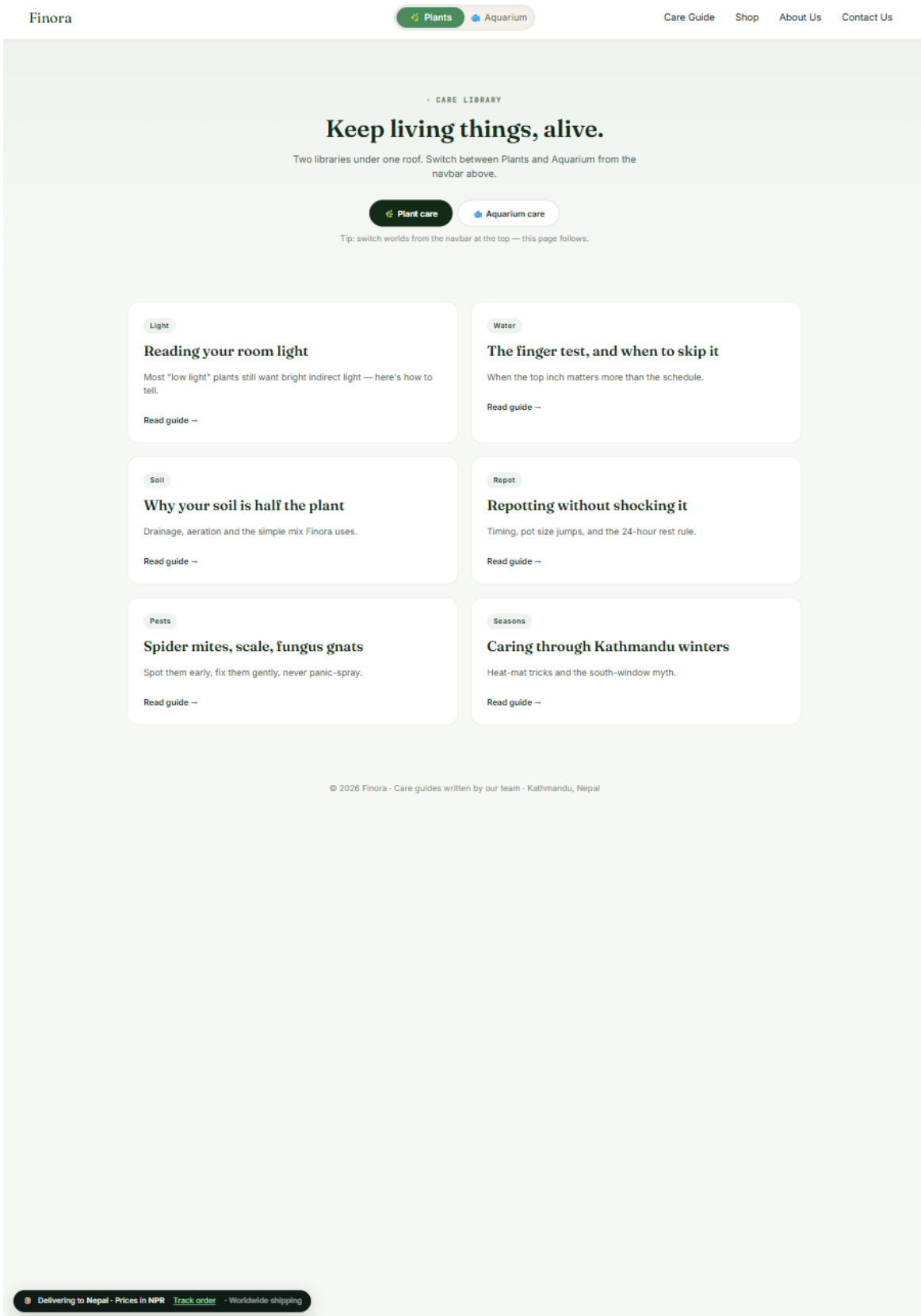


Figure 72



Keep liv

Two libraries under one ro



Plan

Tip: switch worlds f

Light

Reading your room light

Most "low light" plants still want bright indirect light — here's ho

[Read guide →](#)

Water

The finger test, and when to skip it

When the top inch matters more than the schedule.

[Read guide →](#)

Soil

Why your soil is half the plant

Drainage, aeration and the simple mix Finora uses.

[Read guide →](#)

Repot

Repotting without shocking it

Timing, pot size jumps, and the 24-hour rest rule.

[Read guide →](#)

Figure 73

3.9.3 Design Development

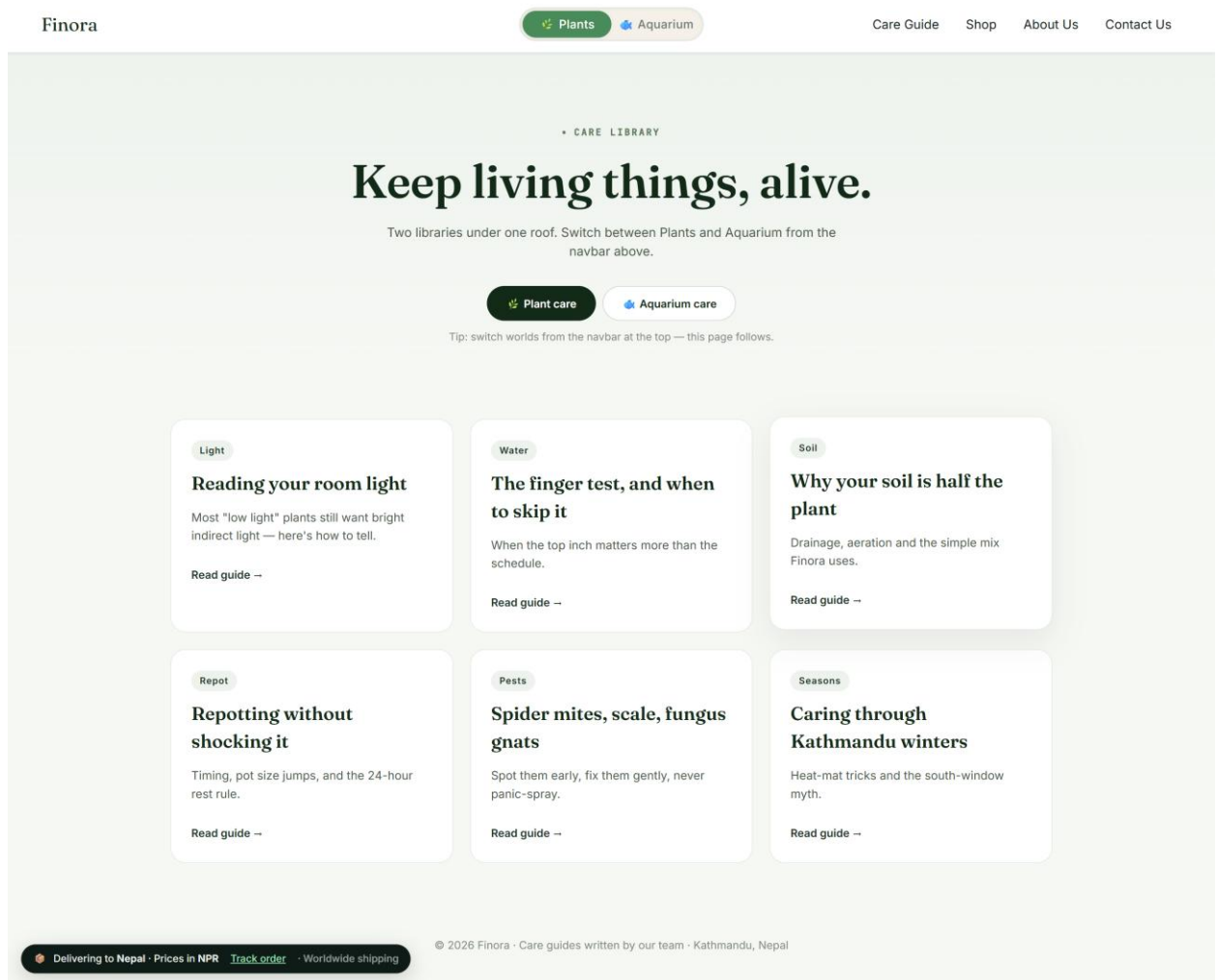


Figure 74

3.10 About Page

The About Page introduces the story, mission, and team behind Finora. The hero section opens with the headline "Bringing Nature and Aquatic Beauty Together" and a short paragraph describing the origin and philosophy of the platform. A dedicated "Our Story" section narrates how Finora was created to solve the problem of scattered, unspecialised plant and aquarium shopping experiences. A values section outlines the core beliefs that guide the platform, such as curation quality, care-first product information, and respect for hobbyist expertise. A team section introduces the people behind the project. The page concludes with a journey or timeline section tracing Finora's development. The About Page establishes trust and brand identity for new visitors who want to understand the platform before making their first purchase.

3.10.1 Research Reference

Warby Parker is a US eyewear brand widely considered to have one of the best-written and best-structured About pages in e-commerce. It's About page is the strongest single reference for Finora's About Page in terms of section structure, narrative tone, and content depth.

Website Name: Warby Parker: About Us

URL: <https://www.warbyparker.com/history>



Figure 75

Key points taken from this reference for the Finora page:

- An Our Story section with a prominent pull-quote (“Nature transforms spaces. We simply help people bring it home.” in Finora) and a narrative paragraph about the founding motivation that matches Finora’s story-section with its story-card, story-quote, and story-body components.

3.10.2 Actual Design

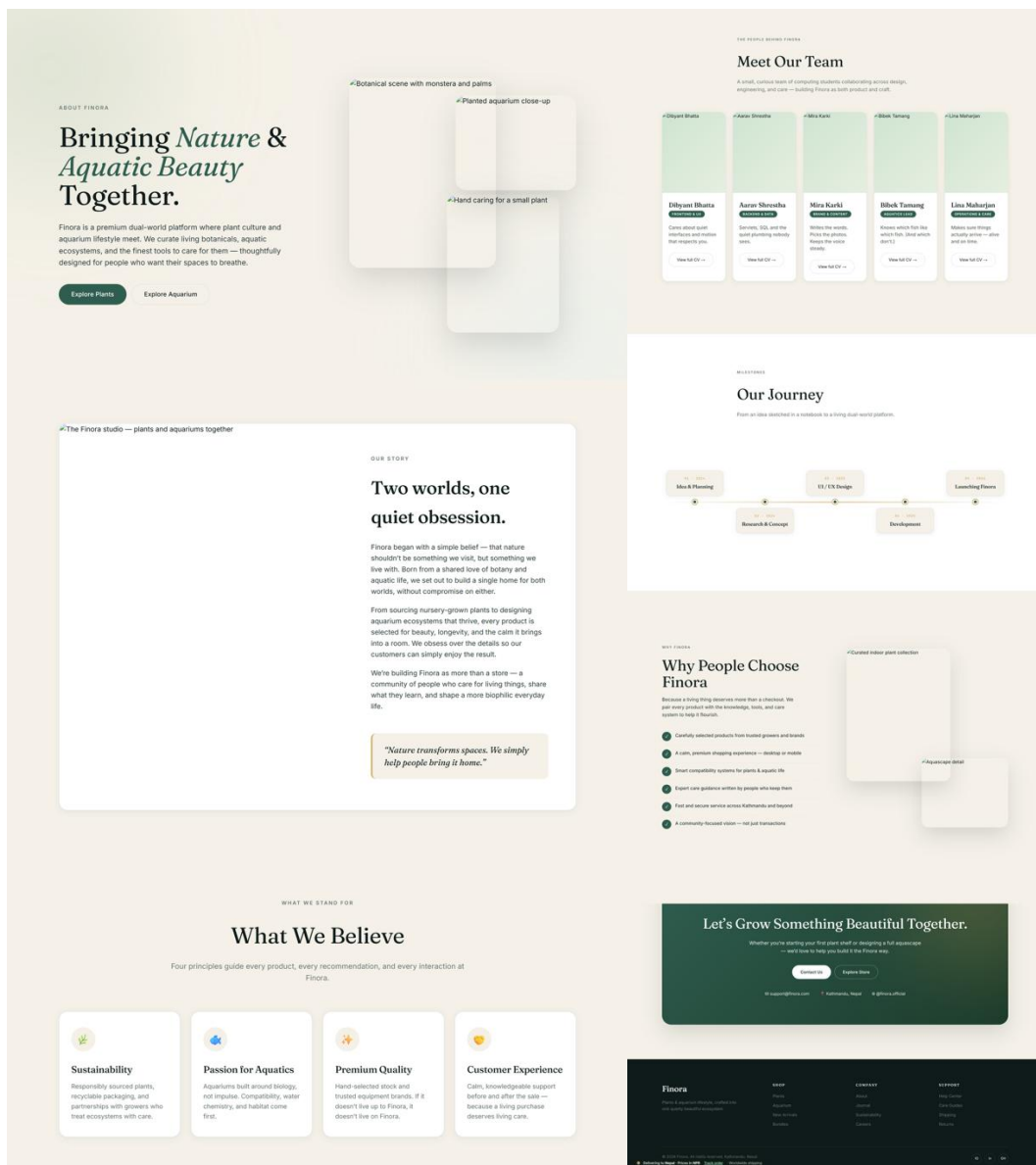


Figure 76

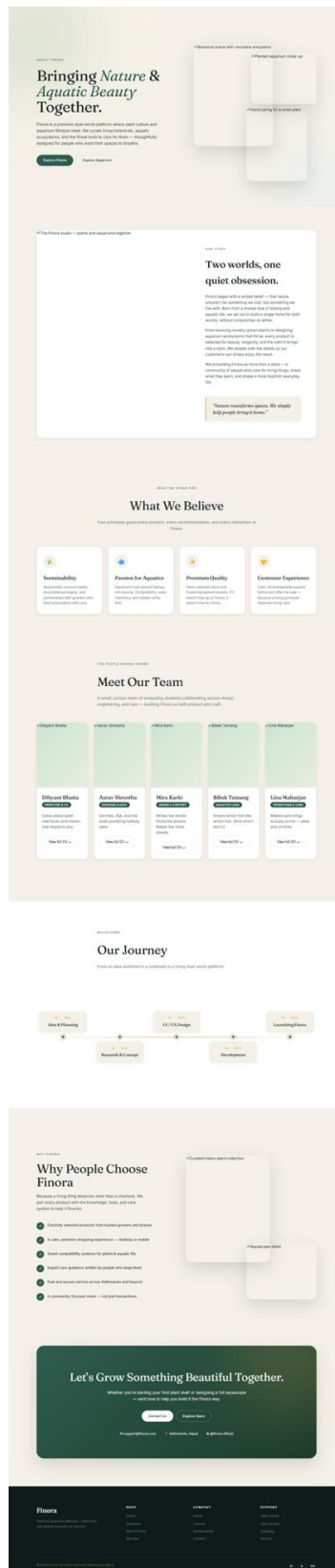


Figure 77

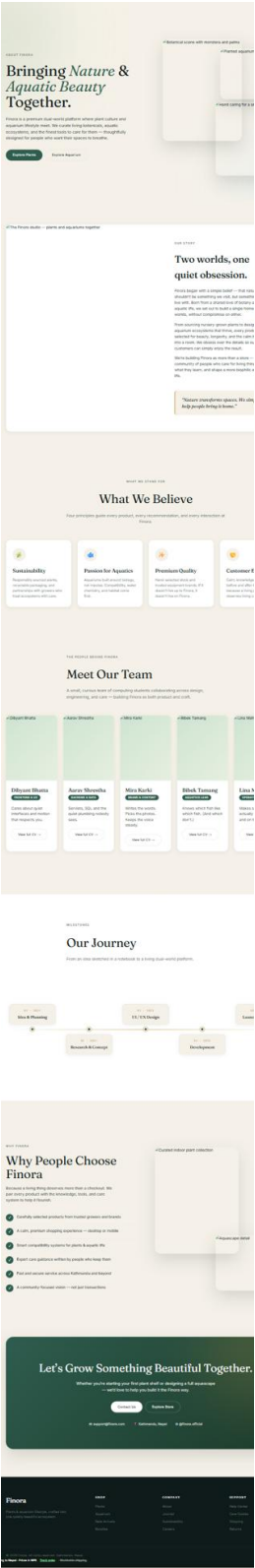


Figure 78

3.10.3 Design Development

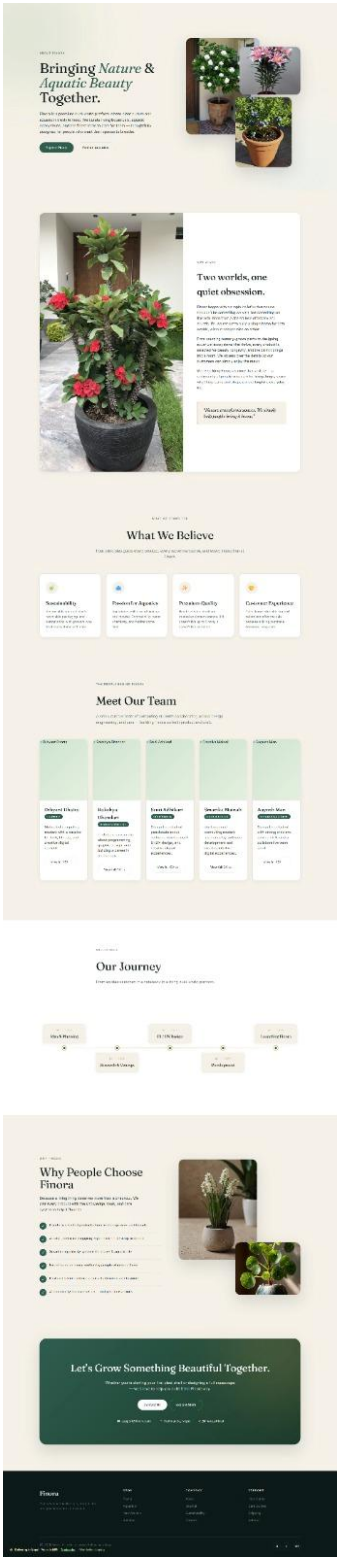


Figure 79

3.11 User Dashboard

The User Dashboard is the main landing page within the logged-in customer account area. It greets the customer by name and provides a personalised summary of account activity. A row

of stat cards displays key figures at a glance, such as total orders placed, items currently in the wish list, reviews submitted, and reward points or account standing. Below the stats, the dashboard surfaces quick links to the most used sections: orders, wish list, reviews, profile, notifications, and settings. A sidebar navigation panel is visible throughout the entire dashboard area and allows the customer to move between all account pages without returning to the main site. The dashboard is designed to reduce the number of clicks needed for routine account management tasks.

3.11.1 Wireframe Design

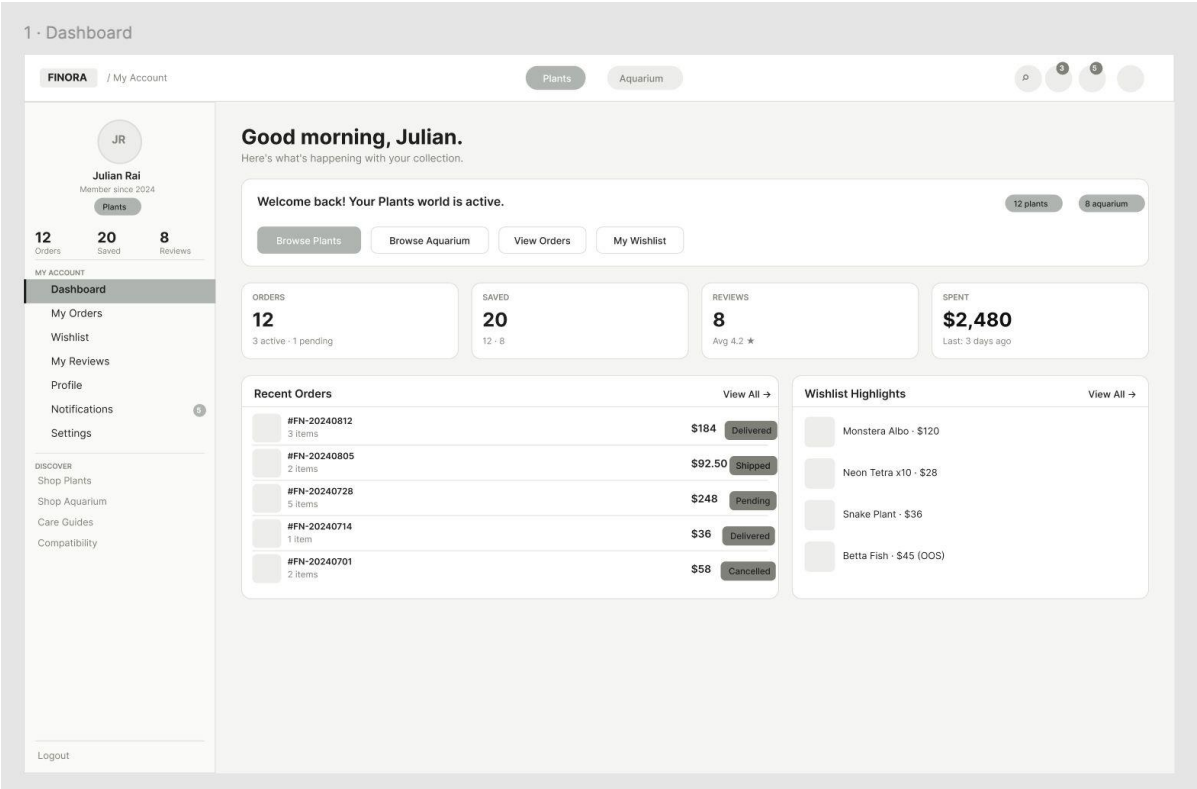


Figure 80

3.11.2 Actual Design

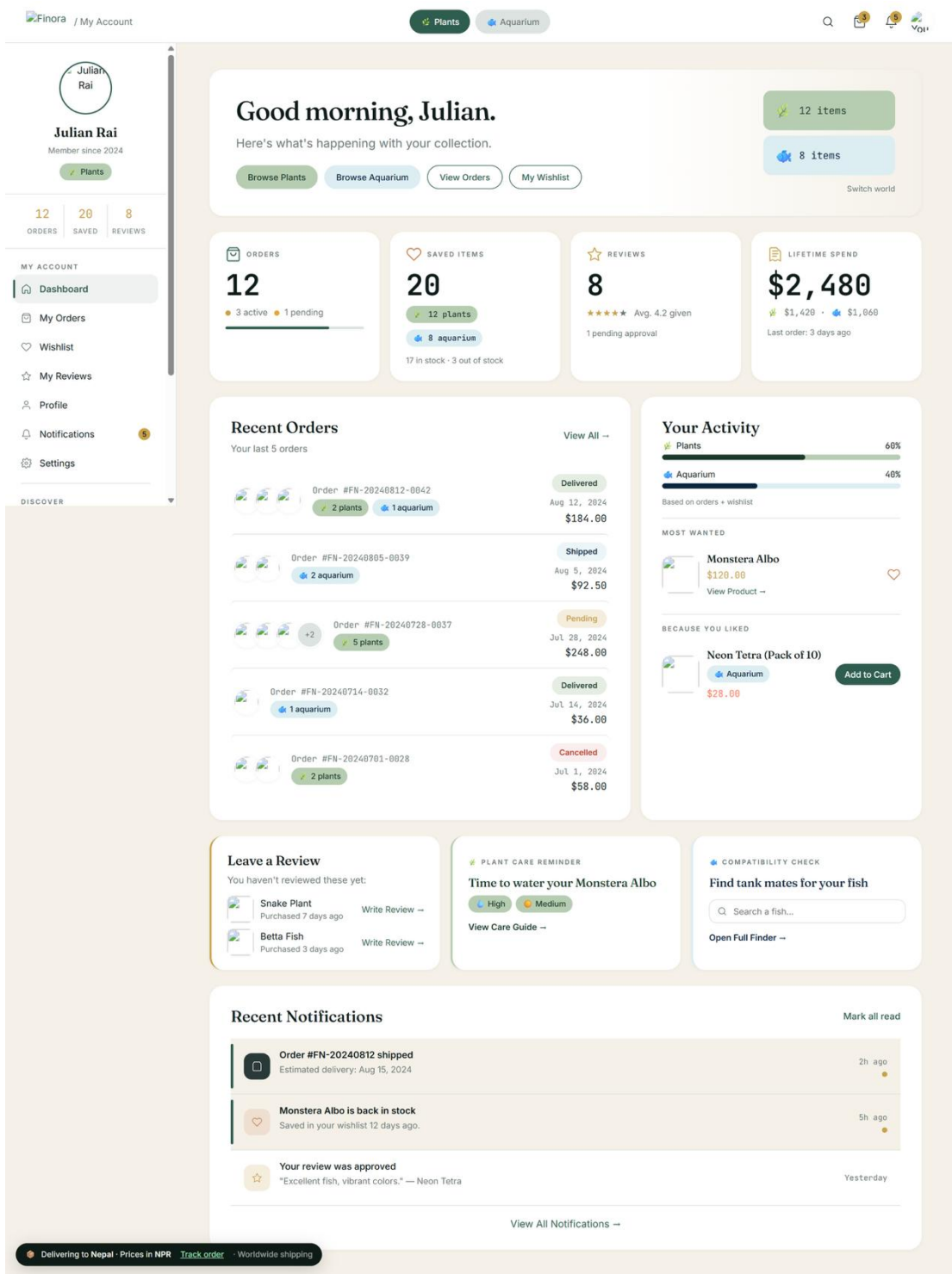


Figure 81

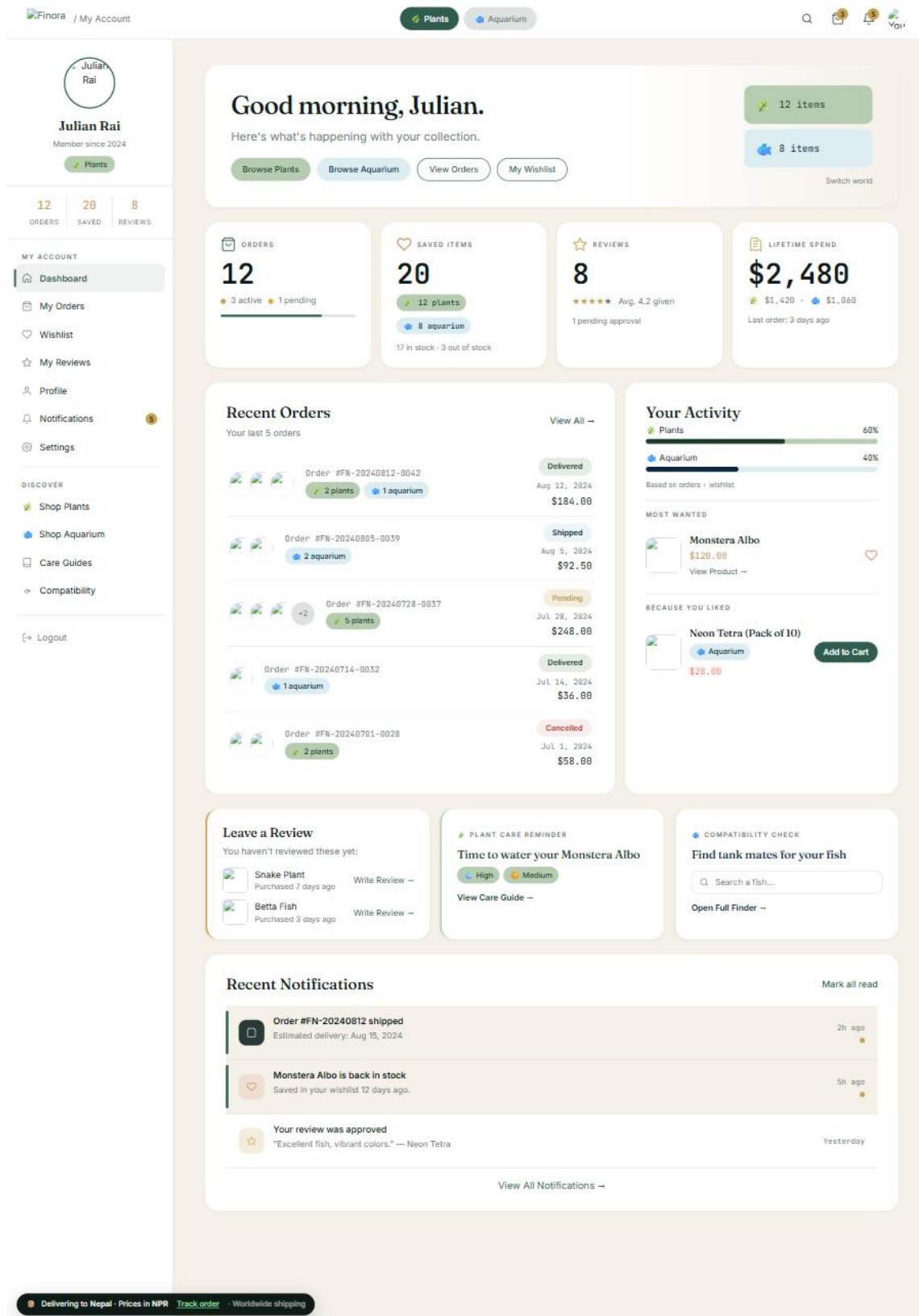


Figure 82

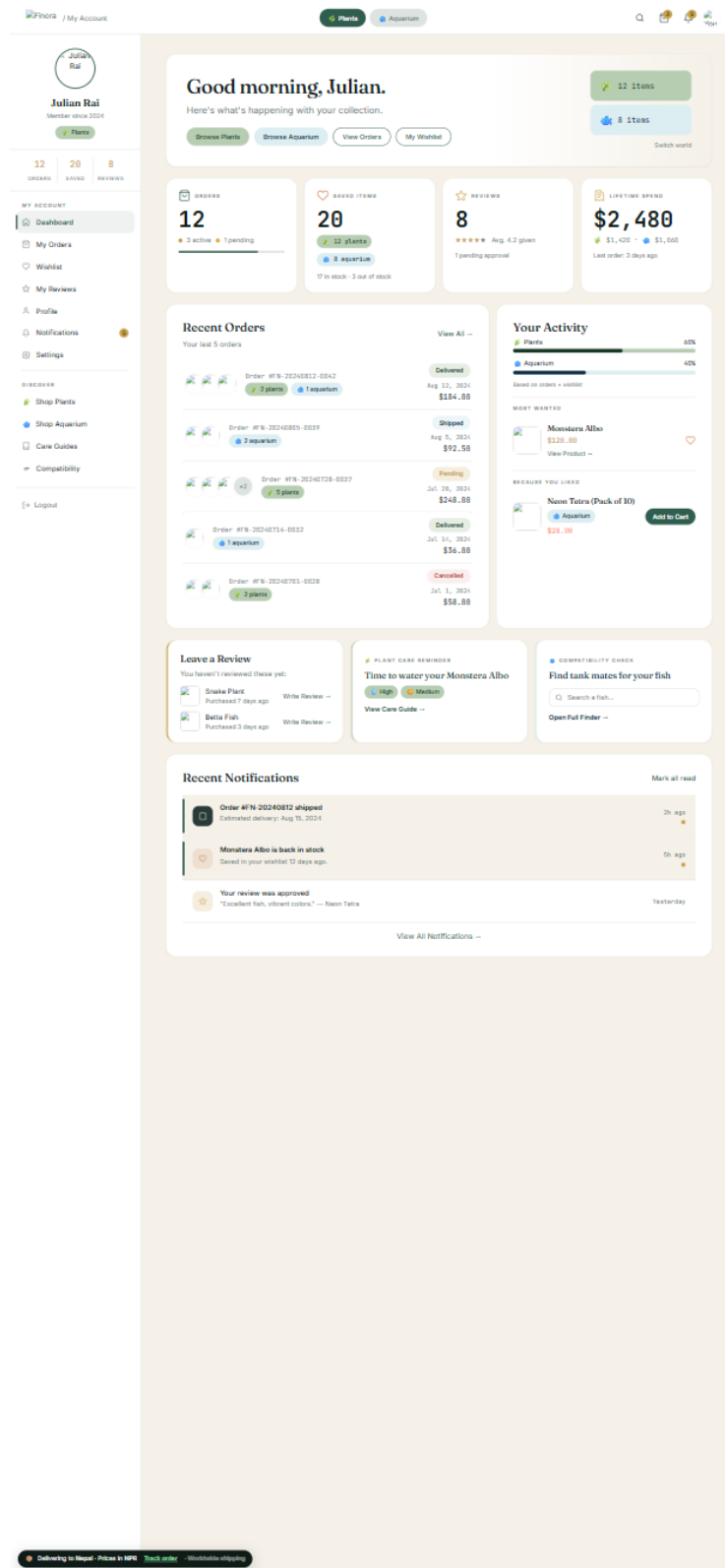


Figure 83

3.11.3 Design Development

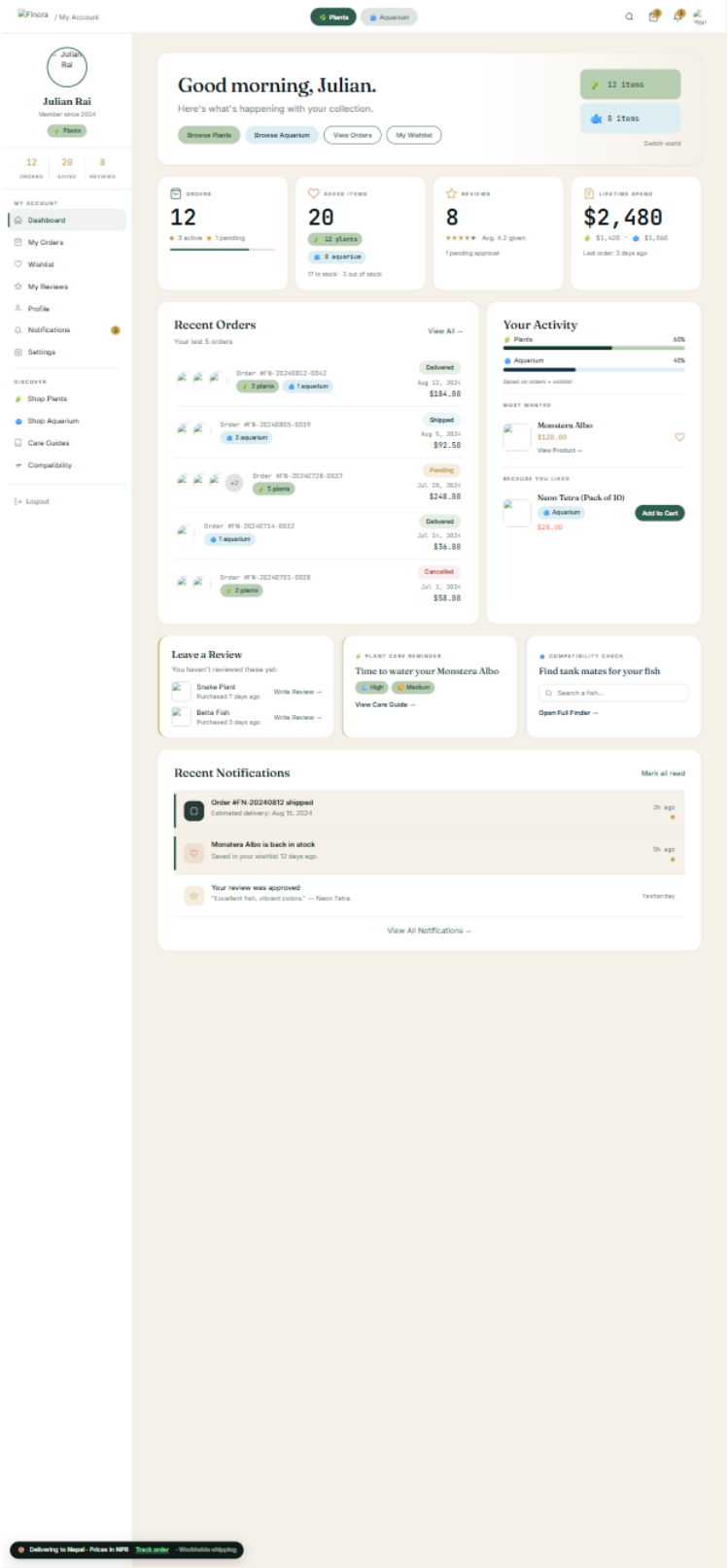


Figure 84

3.12 User Orders

The User Orders page provides a complete history of all orders placed by the logged-in customer. Orders are listed in reverse chronological order and each entry shows the order ID, date placed, number of items, total amount, payment status, and current order status such as Pending, Confirmed, Shipped, or Delivered. Customers can filter orders by status or search by order ID. Each order row can be expanded or clicked to reveal a detailed view showing individual line items, the unit price at the time of purchase, the shipping address used, and the payment method. For active orders, a link to the Track-Order Page is provided. For delivered orders, a prompt is shown encouraging the customer to leave a review for purchased products.

3.12.1 Wireframe Design

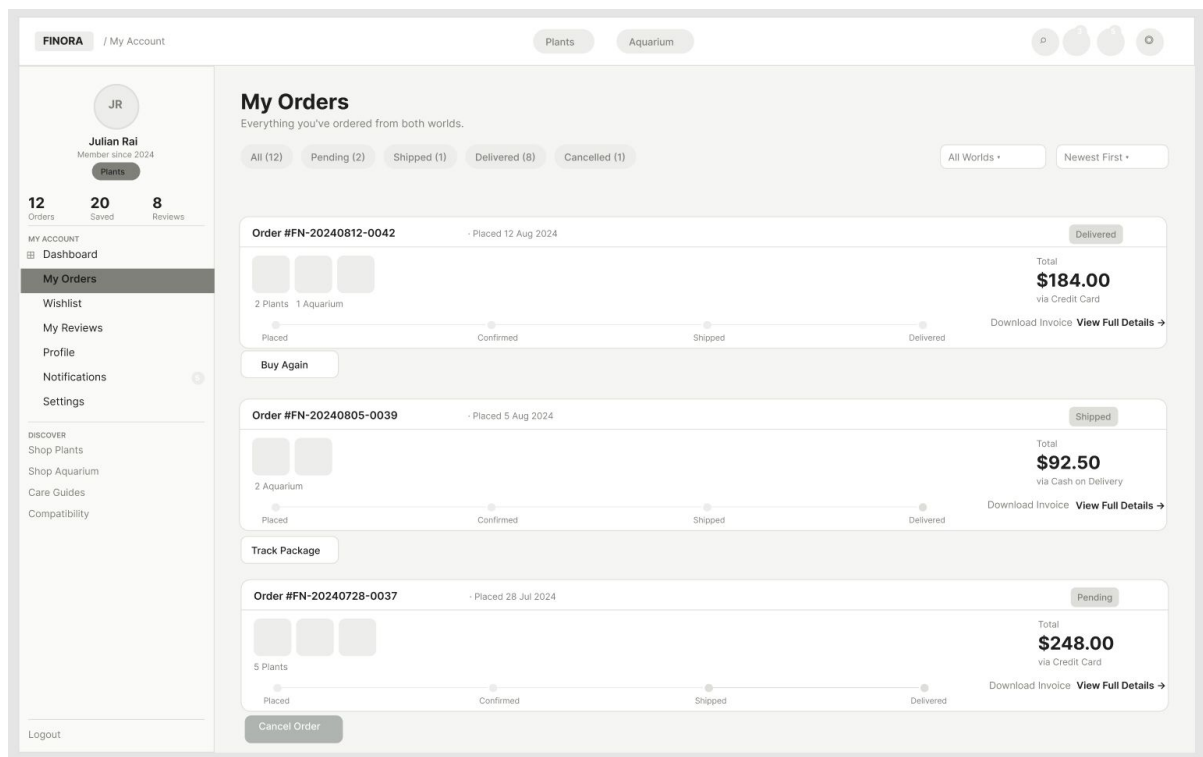


Figure 85\

3.12.2 Actual Design

Finora / My Account

Plants

Aquarium

Q

🛒

👤

🌐

Julian Rai

Member since 2024

Plants

12

ORDERS

20

SAVED

8

REVIEWS

MY ACCOUNT

Dashboard

My Orders

Wishlist

My Reviews

Profile

Notifications

Settings

DISCOVER

My Orders

Everything you've ordered from both worlds.

All (12)

Pending (2)

Shipped (1)

Delivered (8)

Cancelled (1)

All Worlds

Newest First

Order #FN-20240812-0042

Placed on 12 Aug 2024

✓ Delivered

2 Plants

1 Aquarium

3 items total

Total

\$184.00

Paid via Credit Card

Paid

Placed

Confirmed

Shipped

Delivered

Buy Again

Download Invoice

View Full Details --

Order #FN-20240805-0039

Placed on 5 Aug 2024

-- Shipped

2 Aquarium

2 items total

Total

\$92.50

Paid via Cash on Delivery

Pending

Placed

Confirmed

Shipped

Delivered

Track Package

Download Invoice

View Full Details --

Order #FN-20240728-0037

Placed on 28 Jul 2024

⌚ Pending

+1

5 Plants

5 items total

Total

\$248.00

Paid via Credit Card

Paid

Placed

Confirmed

Shipped

Delivered

Cancel Order

Download Invoice

View Full Details --

Delivering to Nepal · Prices in NPR

Track order

Worldwide shipping

Figure 86\

Finora / My Account

Plants

Aquarium

Q

🛒 3

👤

🌱

Julian Rai

Julian Rai

Member since 2024

Plants

12

20

8

ORDERS

SAVED

REVIEWS

MY ACCOUNT

Dashboard

My Orders

Wishlist

My Reviews

Profile

Notifications

Settings

DISCOVER

Shop Plants

Shop Aquarium

Care Guides

Compatibility

Logout

My Orders

Everything you've ordered from both worlds.

All (12)

Pending (2)

Shipped (1)

Delivered (8)

Cancelled (1)

All Worlds

Newest First

Order #FN-20240812-0042

Placed on 12 Aug 2024

Delivered

2 Plants

1 Aquarium

3 items total

Total

\$184.00

Paid via Credit Card

Paid

Placed

Confirmed

Shipped

Delivered

Buy Again

Download Invoice

View Full Details

Order #FN-20240805-0039

Placed on 5 Aug 2024

Shipped

2 Aquarium

2 items total

Total

\$92.50

Paid via Cash on Delivery

Pending

Placed

Confirmed

Shipped

Delivered

Track Package

Download Invoice

View Full Details

Order #FN-20240728-0037

Placed on 28 Jul 2024

Pending

+1

5 Plants

5 items total

Total

\$248.00

Paid via Credit Card

Paid

Placed

Confirmed

Shipped

Delivered

Cancel Order

Download Invoice

View Full Details

Delivering to Nepal - Prices in NPR

Track order

Worldwide shipping

Figure 87

119

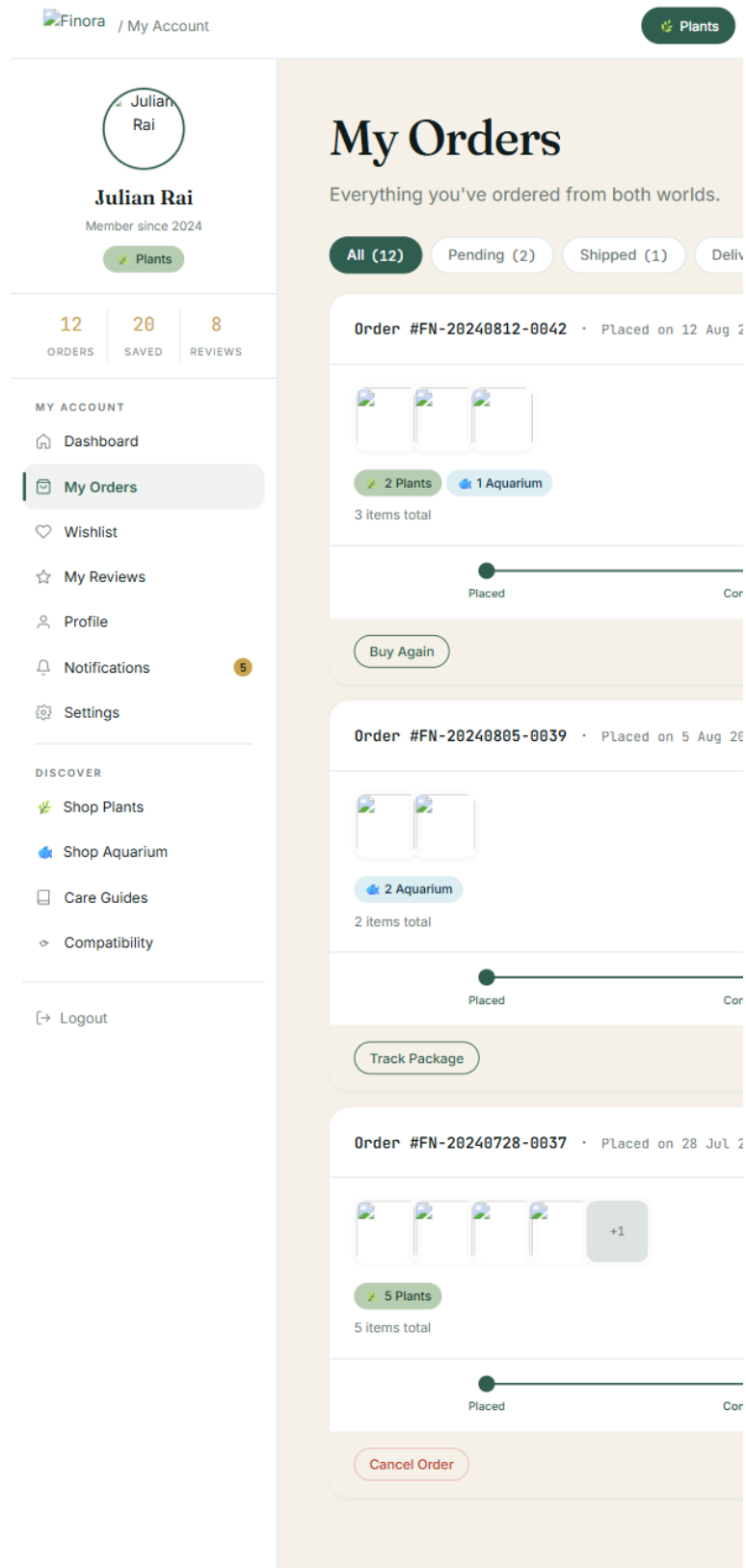


Figure 88

3.12.3 Design Development

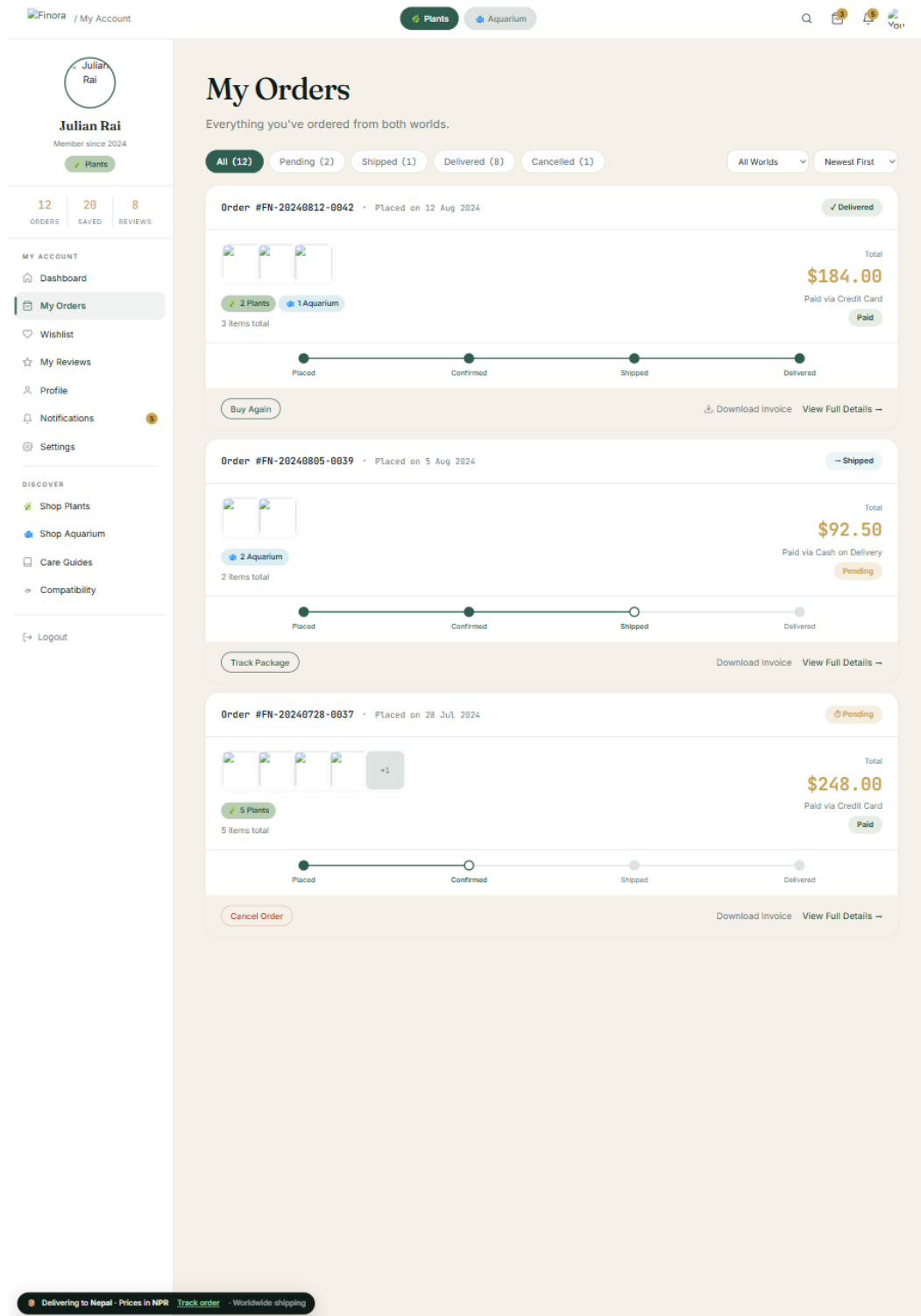


Figure 89

3.13 User Wishlist

The User Wishlist page within the dashboard is a more feature-rich version of the public Wishlist Page. It displays all saved products for the logged-in customer, organised and filterable by product world (Plants or Aquarium). Each product card shows the product

image, name, current price, stock status, and care or compatibility chips. From this page, the customer can move items to the cart, remove items from the wish list, or navigate directly to the product detail view on the shop. If a wish listed item has dropped in price or returned to stock since it was saved, a notification badge highlights the change. The page also allows customers to add personal notes to individual wish list entries for their own reference.

3.13.1 Wireframe Design

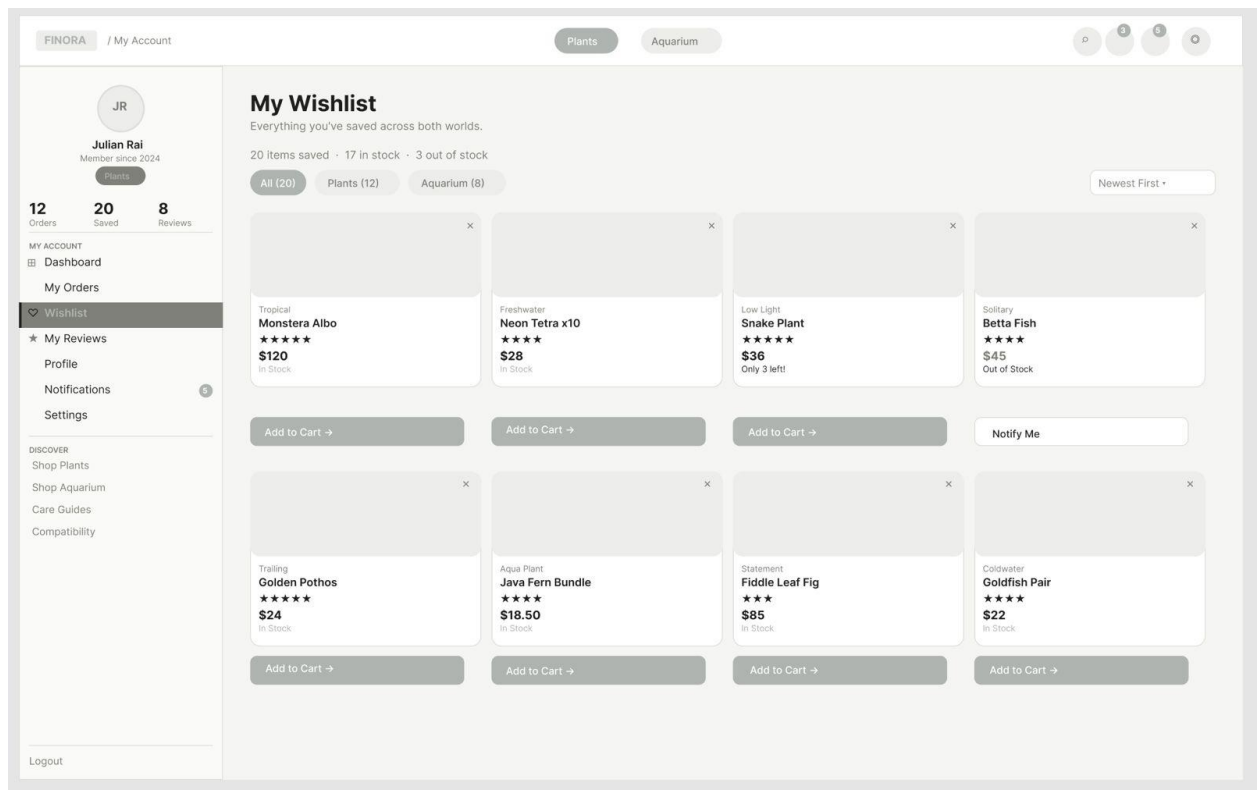


Figure 90

3.13.2 Actual Design

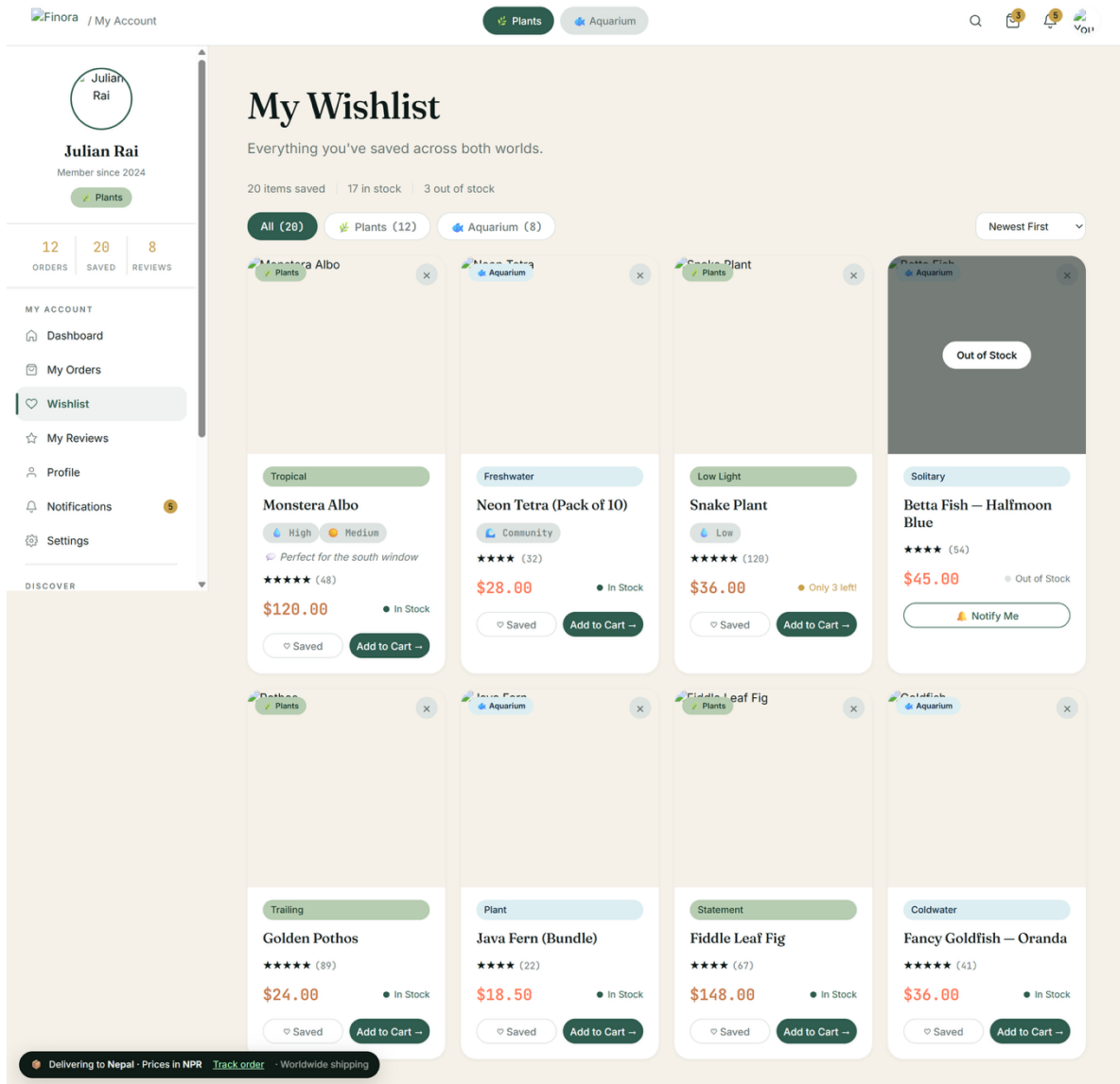


Figure 91

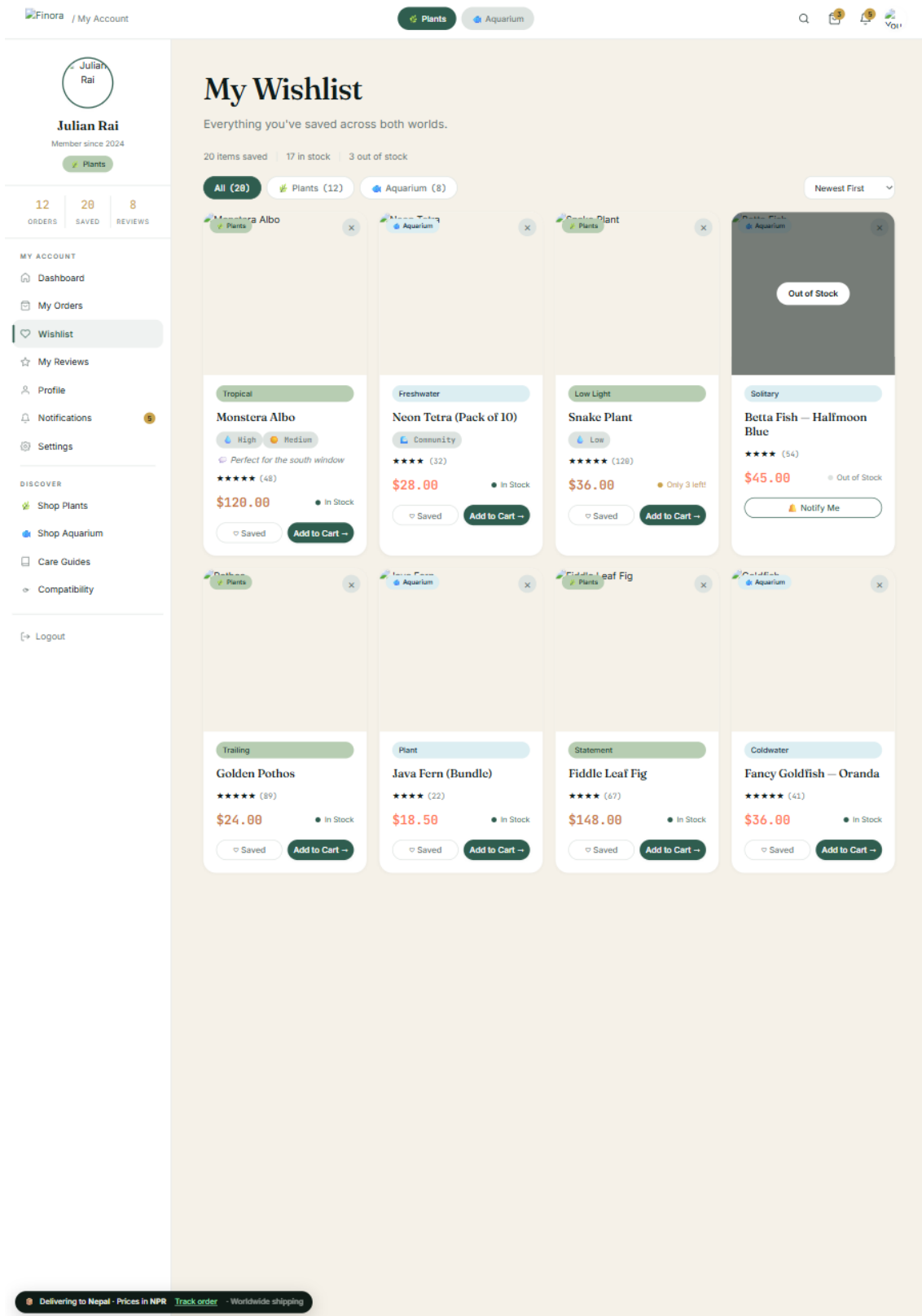


Figure 92

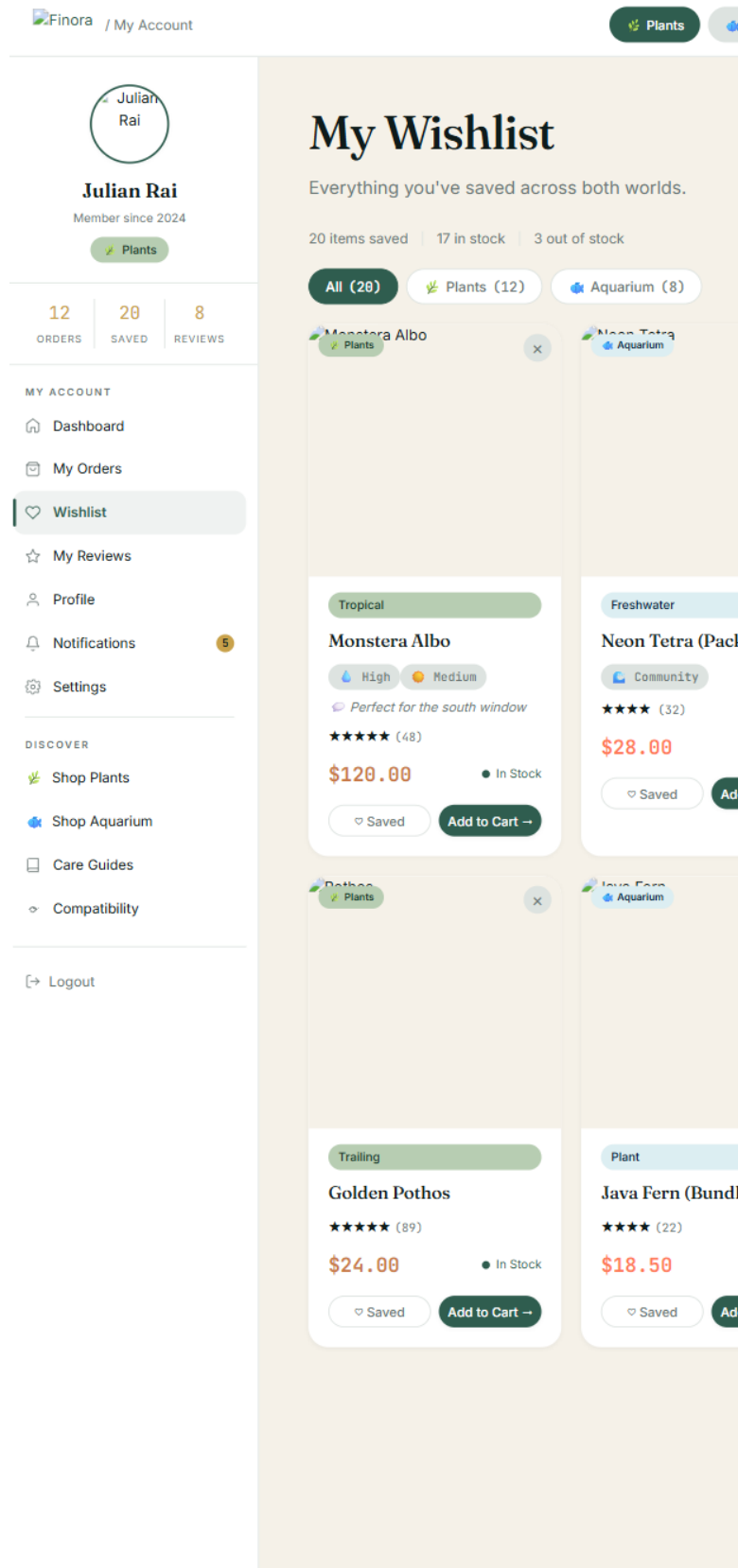


Figure 93

3.14 User Reviews

The User Reviews page displays all product reviews that the logged-in customer has submitted across both the Plants and Aquarium catalogues. Each review entry shows the product name and image, the star rating given (1 to 5), the written comment, the date of submission, and the current moderation status which is either Pending or Approved. Customers can edit a review that is still awaiting approval or delete a review entirely. For products that have been purchased but not yet reviewed, a prompt appears at the top of the page encouraging the customer to share their experience. If an administrator has responded publicly to a review, that response is displayed beneath the customer's comment. This page gives customers full visibility and control over their contribution to the platform's review system.

3.14.1 Wireframe Design

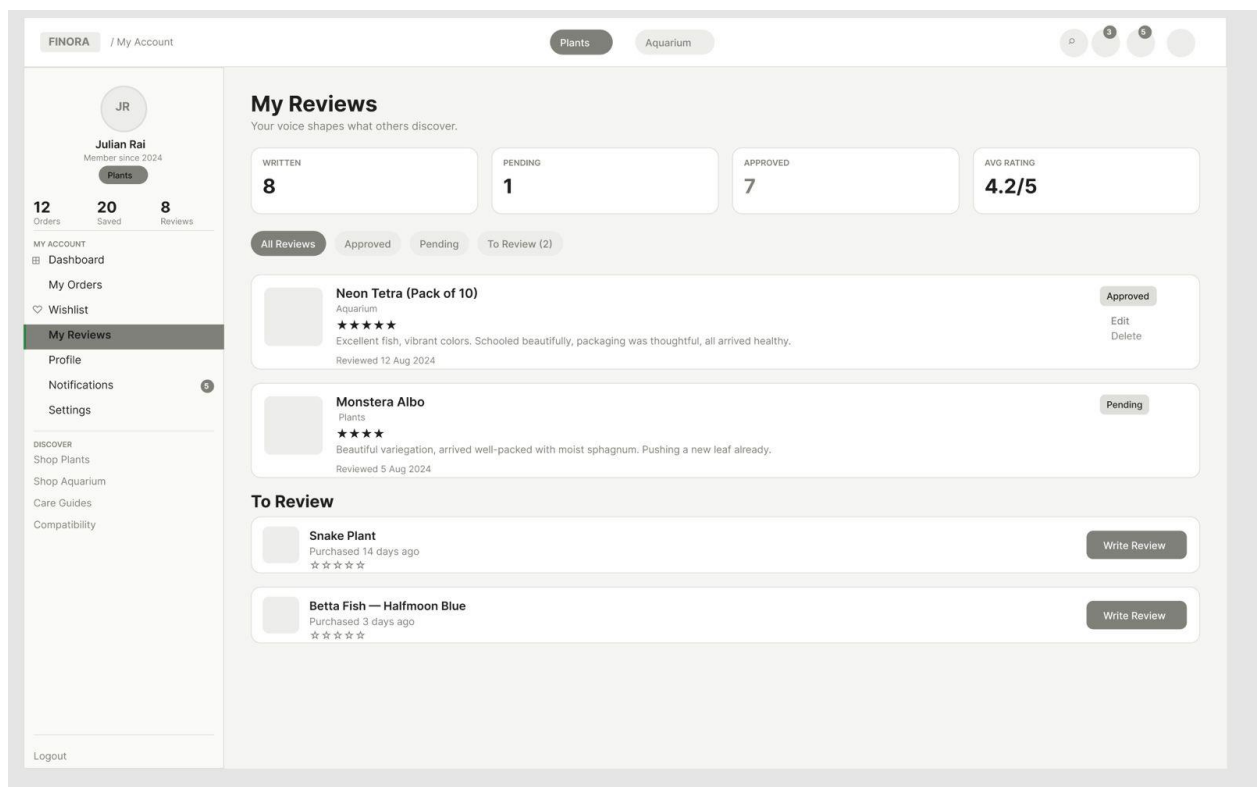


Figure 94

3.14.2 Actual Design

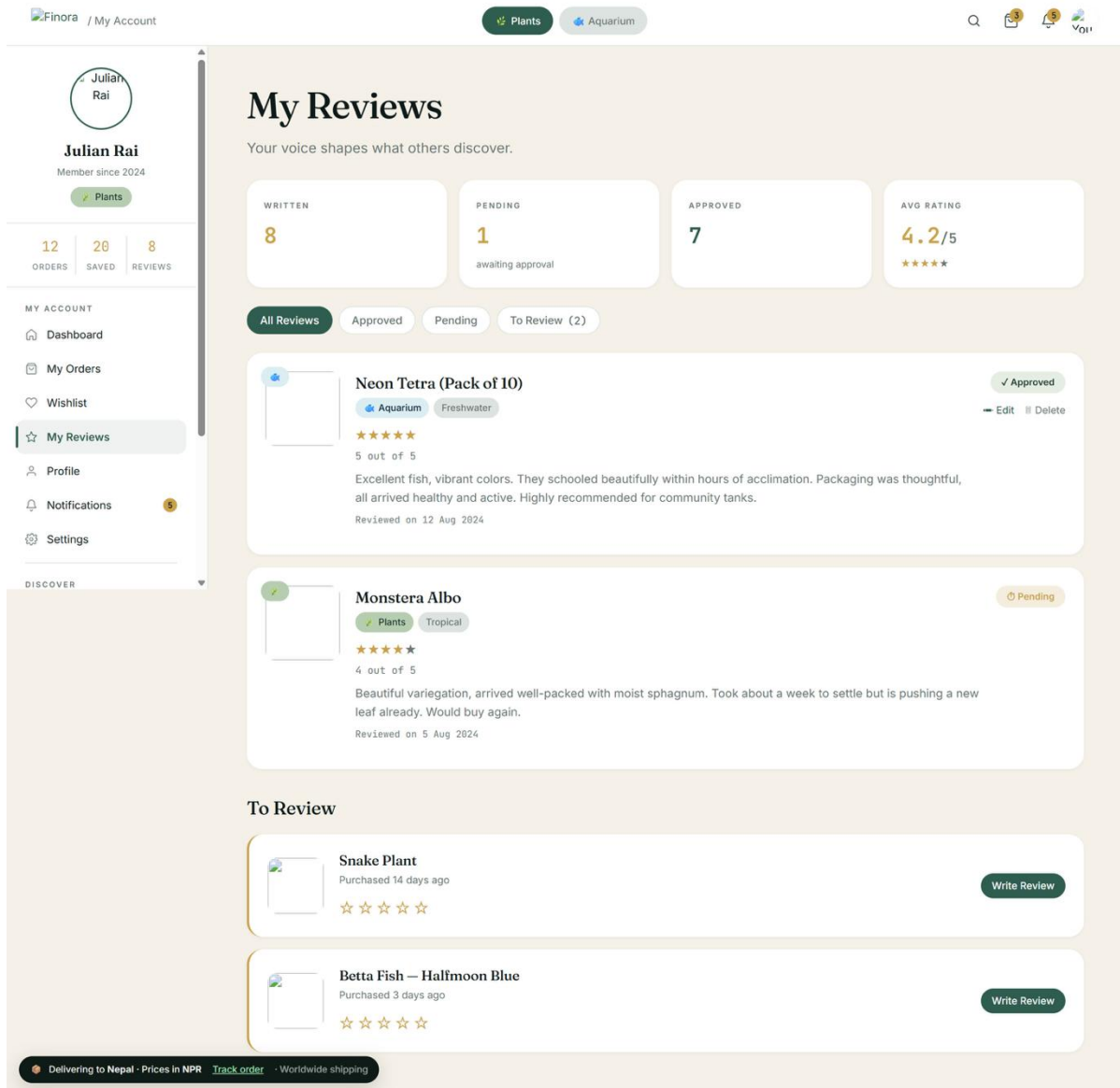


Figure 95

Finora / My Account

Plants

Aquarium

Q

Julian Rai

Julian Rai

Member since 2024

Plants

12

ORDERS

20

SAVED

8

REVIEWS

MY ACCOUNT

Dashboard

My Orders

Wishlist

My Reviews

Profile

Notifications

Settings

DISCOVER

Shop Plants

Shop Aquarium

Care Guides

Compatibility

Logout

My Reviews

Your voice shapes what others discover.

WRITTEN

8

PENDING

1

awaiting approval

APPROVED

7

AVG RATING

4.2/5

★★★★★

All Reviews

Approved

Pending

To Review (2)

Neon Tetra (Pack of 10)

Aquarium

Freshwater

★★★★★

5 out of 5

Excellent fish, vibrant colors. They schooled beautifully within hours of acclimation. Packaging was thoughtful, all arrived healthy and active. Highly recommended for community tanks.

Reviewed on 12 Aug 2024

Approved

Edit

Delete

Monstera Albo

Plants

Tropical

★★★★★

4 out of 5

Beautiful variegation, arrived well-packed with moist sphagnum. Took about a week to settle but is pushing a new leaf already. Would buy again.

Reviewed on 5 Aug 2024

Pending

To Review

Snake Plant

Purchased 14 days ago

★★★★★

Write Review

Betta Fish — Halfmoon Blue

Purchased 3 days ago

★★★★★

Write Review

Delivering to Nepal - Prices in NPR

Track order

Worldwide shipping

Figure 96

128

Julian Rai

Julian Rai

Member since 2024

Plants

12

ORDERS

20

SAVED

8

REVIEWS

MY ACCOUNT

Dashboard

My Orders

Wishlist

My Reviews

Profile

Notifications5

Settings

DISCOVER

Shop Plants

Shop Aquarium

Care Guides

Compatibility

Logout

My Reviews

Your voice shapes what others c

WRITTEN

8

All Reviews

Approved

Penc

Neon Tetra (Pa

Aquarium

Fres

★★★★★

5 out of 5

Excellent fish, vibr

all arrived healthy

Reviewed on 12 Aug

Monstera Albc

Plants

Tropica

★★★★★

4 out of 5

Beautiful variegatic

leaf already. Woulc

Reviewed on 5 Aug 2

To Review

Snake Plant

Purchased 14 days ago

☆☆☆☆☆

Figure 97

129

3.14.3 Design Development

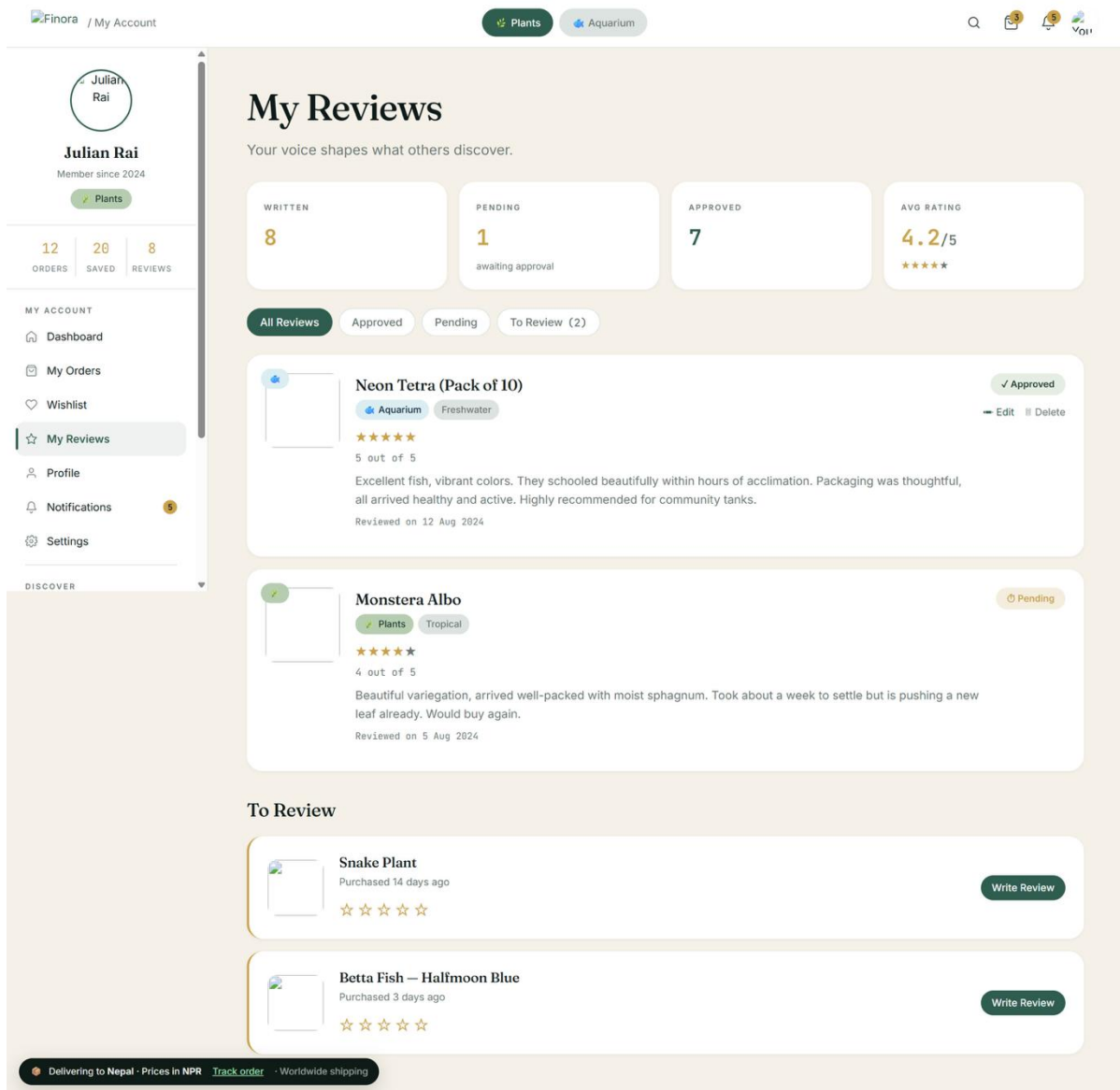


Figure 98

3.15 User Profile

The User Profile page allows the logged-in customer to view and update their personal account information. The page is divided into sections covering basic information such as full name and email address, contact details including phone number, and the default

shipping address used at checkout. Each section has an edit mode that the customer can activate by clicking an edit button, making changes, and saving. A separate password change section requires the customer to enter their current password before setting a new one, preventing unauthorised changes. The page also includes an option to upload or change a profile picture. At the bottom of the page, a danger zone section provides the option to permanently delete the account, which requires an additional confirmation step to prevent accidental removal.

3.15.1 Wireframe Design

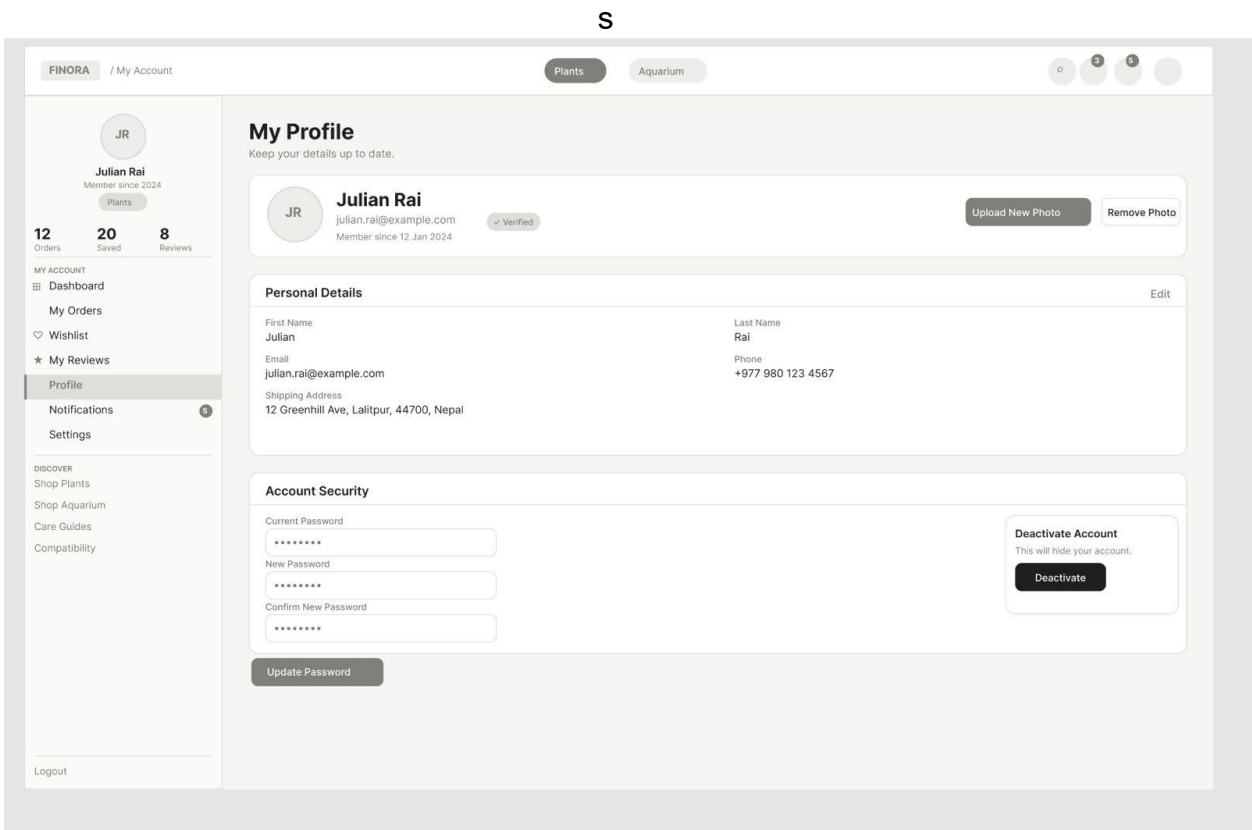


Figure 99

3.15.2 Actual Design

Finora / My Account

Plants

Aquarium

Q

\$

\$

011

Julian Rai

Julian Rai

Member since 2024

Plants

12

ORDERS

20

SAVED

8

REVIEWS

MY ACCOUNT

Dashboard

My Orders

Wishlist

My Reviews

Profile

Notifications

Settings

DISCOVER

My Profile

Keep your details up to date.

Julian Rai

Julian Rai

Member since 12 Jan 2024

Upload New Photo

Remove Photo

JPG, PNG or WEBP - Max 2MB

Personal Details

Edit

FIRST NAME

Julian

LAST NAME

Rai

EMAIL ADDRESS

julian.ra@example.com

Verified

PHONE NUMBER

+977 980 123 4567

SHIPPING ADDRESS

12 Greenhill Avenue

Lalitpur, 44700

Nepal

Account Security

CURRENT PASSWORD

NEW PASSWORD

CONFIRM NEW PASSWORD

Update Password

Deactivate Account

This will hide your account and cancel pending orders.

Deactivate

Delivering to Nepal - Prices in NPR

Track order

Worldwide shipping

Figure 100

Finora / My Account

PlantsAquarium

Julian Rai

Julian Rai

Member since 2024

Plants

12

ORDERS

28

SAVED

8

REVIEWS

MY ACCOUNT

Dashboard

My Orders

Wishlist

My Reviews

Profile

Notifications

Settings

DISCOVER

Shop Plants

Shop Aquarium

Care Guides

Compatibility

Logout

My Profile

Keep your details up to date.

Julian Rai

Julian Rai

julian.ra@example.com

Member since 12 Jan 2024

Upload New Photo

Remove Photo

JPG, PNG or WEBP · Max 2MB

Personal Details

Edit

FIRST NAME

Julian

LAST NAME

Rai

EMAIL ADDRESS

julian.ra@example.com

PHONE NUMBER

+977 980 123 4567

SHIPPING ADDRESS

12 Greenhill Avenue
Lalitpur, 44700
Nepal

Account Security

CURRENT PASSWORD

NEW PASSWORD

Use 8+ characters with a number and uppercase letter.

CONFIRM NEW PASSWORD

Update Password

Deactivate Account

This will hide your account and cancel pending orders.

Deactivate

Delivering to Nepal · Prices in NPR

Track order

Worldwide shipping

Figure 101

Julian Rai

Julian Rai

Member since 2024

Plants

12

ORDERS

20

SAVED

8

REVIEWS

MY ACCOUNT

Dashboard

My Orders

Wishlist

My Reviews

Profile

Notifications5

Settings

DISCOVER

Shop Plants

Shop Aquarium

Care Guides

Compatibility

Logout

My Profile

Keep your details up to date.

Julian Rai

Julian Ra

julian.ra@example.com

Member since 12

Personal Details

FIRST NAME

Julian

EMAIL ADDRESS

julian.ra@example.com

Verified

PHONE NUMBER

+977 980 123 4567

SHIPPING ADDRESS

12 Greenhill Avenue

Lalitpur, 44700

Nepal

Account Security

CURRENT PASSWORD

NEW PASSWORD

Use 8+ characters with a number and uppercase

CONFIRM NEW PASSWORD

134

3.15.3 Design Development

Finora / My Account

PlantsAquarium

SearchCartWishlistYou

Julian Rai

Julian Rai

Member since 2024

Plants

12

ORDERS

20

SAVED

8

REVIEWS

MY ACCOUNT

Dashboard

My Orders

Wishlist

My Reviews

Profile

Notifications

Settings

DISCOVER

My Profile

Keep your details up to date.

Julian Rai

Upload New Photo

Remove Photo

JPG, PNG or WEBP · Max 2MB

julian.ra@example.com

Verified

Member since 12 Jan 2024

Personal Details

Edit

FIRST NAME

Julian

LAST NAME

Rai

EMAIL ADDRESS

julian.ra@example.com

Verified

PHONE NUMBER

+977 980 123 4567

SHIPPING ADDRESS

12 Greenhill Avenue

Lalitpur, 44700

Nepal

Account Security

CURRENT PASSWORD

NEW PASSWORD

Use 8+ characters with a number and uppercase letter.

CONFIRM NEW PASSWORD

Update Password

Deactivate Account

This will hide your account and cancel pending orders.

Deactivate

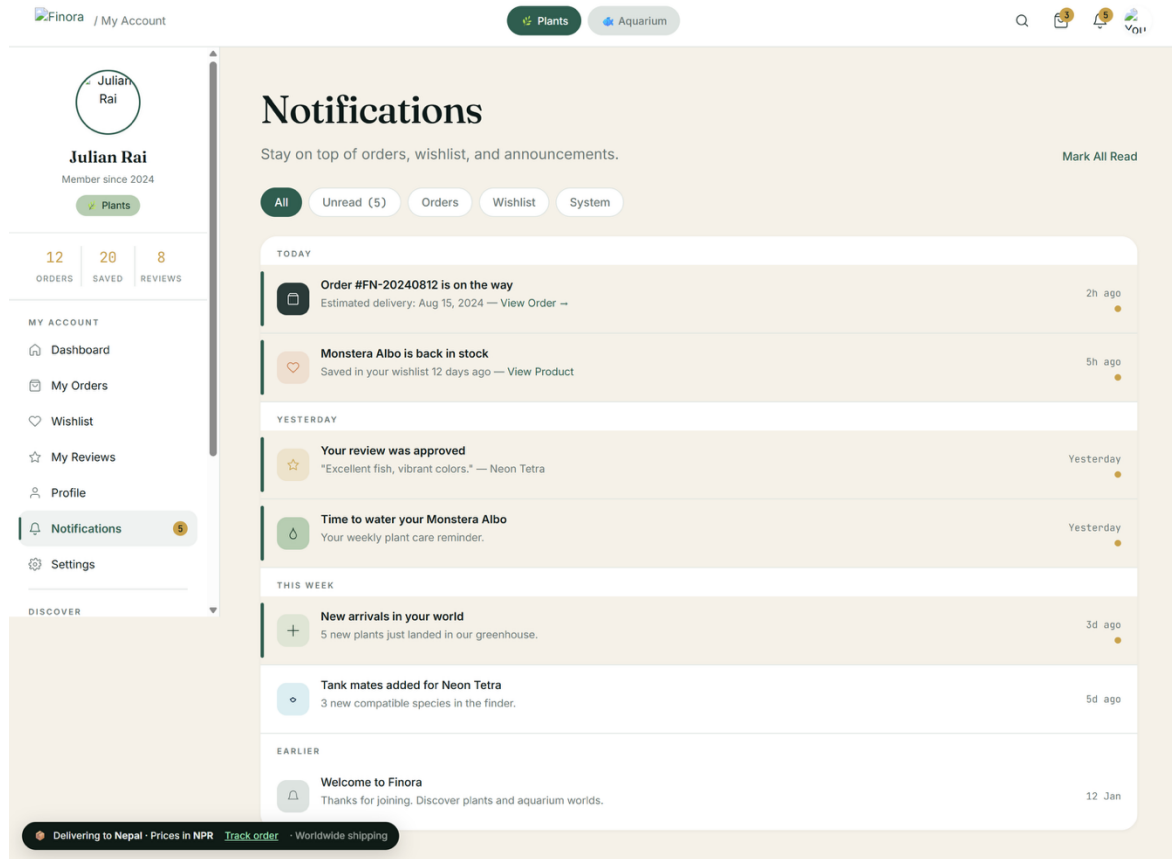
Delivering to Nepal · Prices in NPR · Track order · Worldwide shipping

3.16 User Notification

The User Notification page collects all system-generated and activity-triggered alerts for the logged-in customer in one place. Notifications are displayed in a feed layout with the most recent at the top, and each entry shows a short message, an associated icon or category label, and a timestamp. Types of notifications include order status updates such as when an order has been confirmed, shipped, or delivered, price drop alerts for products saved to the wishlist, restock notifications for previously out-of-stock items the customer expressed interest in, and review approval confirmations when a submitted review goes live. Customers can mark individual notifications as read or

use a Mark All as Read action to clear the badge count. The page also links directly to the related order or product where applicable.

3.16.1 Actual Design



Finora / My Account

Plants

Aquarium

Julian Rai

Julian Rai

Member since 2024

Plants

12

ORDERS

28

SAVED

8

REVIEWS

MY ACCOUNT

Dashboard

My Orders

Wishlist

My Reviews

Profile

Notifications

Settings

DISCOVER

Shop Plants

Shop Aquarium

Care Guides

Compatibility

Logout

Notifications

Stay on top of orders, wishlist, and announcements.

Mark All Read

All

Unread (5)

Orders

Wishlist

System

TODAY

Order #FN-20240812 is on the way

Estimated delivery: Aug 15, 2024 — View Order →

2h ago

Monstera Albo is back in stock

Saved in your wishlist 12 days ago — View Product

5h ago

YESTERDAY

Your review was approved

"Excellent fish, vibrant colors." — Neon Tetra

Yesterday

Time to water your Monstera Albo

Your weekly plant care reminder.

Yesterday

THIS WEEK

New arrivals in your world

5 new plants just landed in our greenhouse.

3d ago

Tank mates added for Neon Tetra

3 new compatible species in the finder.

5d ago

EARLIER

Welcome to Finora

Thanks for joining. Discover plants and aquarium worlds.

12 Jan

Delivering to Nepal - Prices in NPR

Track order

Worldwide shipping

137

Finora / My Account

PlantsAquarium

Julian Rai

Julian Rai

Member since 2024

Plants

12ORDERS28WISHLIST8REVIEWS

MY ACCOUNT

- Dashboard
- My Orders
- Wishlist
- My Reviews
- Profile
- Notifications
- Settings

DISCOVER

- Shop Plants
- Shop Aquarium
- Care Guides
- Compatibility

Logout

Notifications

Stay on top of orders, wishlist, and announcements.

Mark All Read

AllUnread (5)OrdersWishlistSystem

TODAY

- Order #N-20240812 is on the way

Estimated delivery: Aug 15, 2024 — View Order →

2h ago
- Monstera Albo is back in stock

Saved to your wishlist 10 days ago — View Product

5h ago

YESTERDAY

- Your review was approved

"Excellent fish, vibrant colors." — Neon Tetra

Yesterday
- Time to water your Monstera Albo

Your weekly plant care reminder.

Yesterday

THIS WEEK

- New arrivals in your world

5 new plants just landed in our greenhouse.

3d ago
- Tank mates added for Neon Tetra

3 new compatible species in the finder.

5d ago

EARLIER

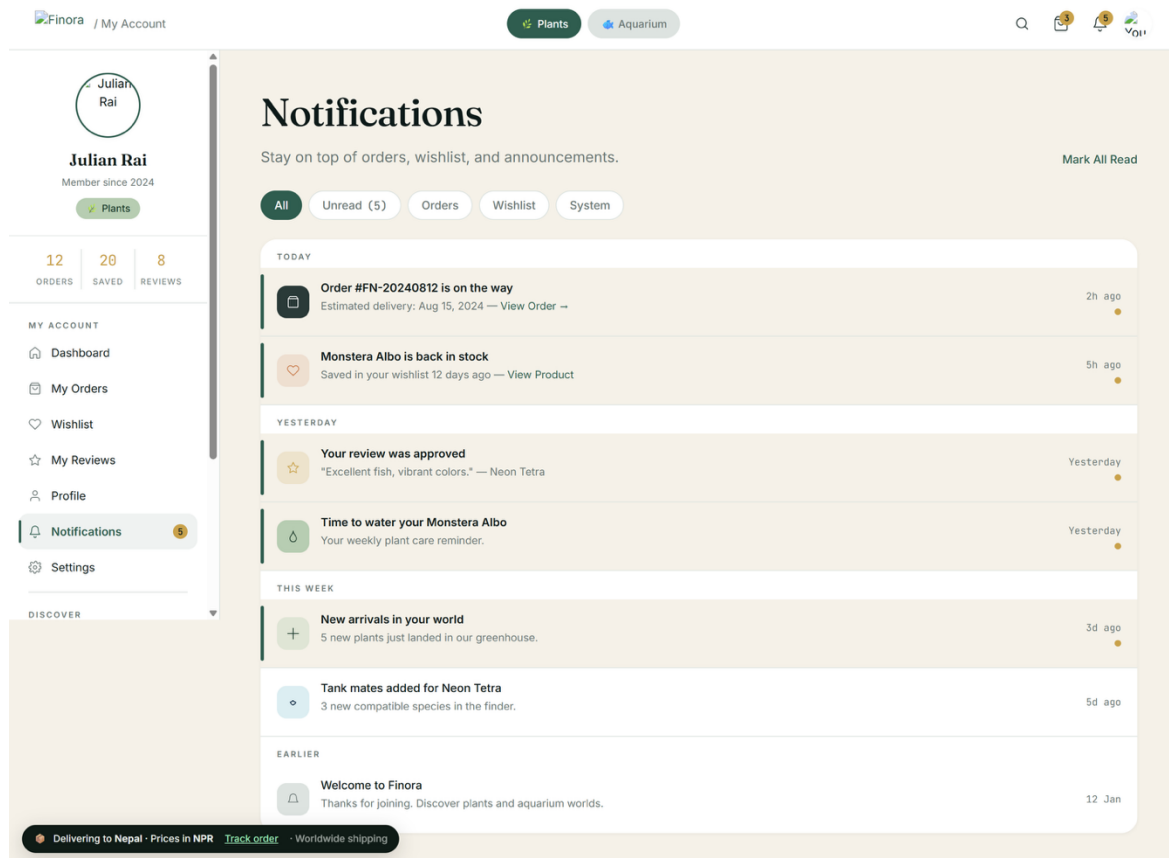
- Welcome to Finora

Thanks for joining. Discover plants and aquarium worlds.

12 Jan

Delivering to Madrid · Prices in NPS · Track order · Withdraw shipping

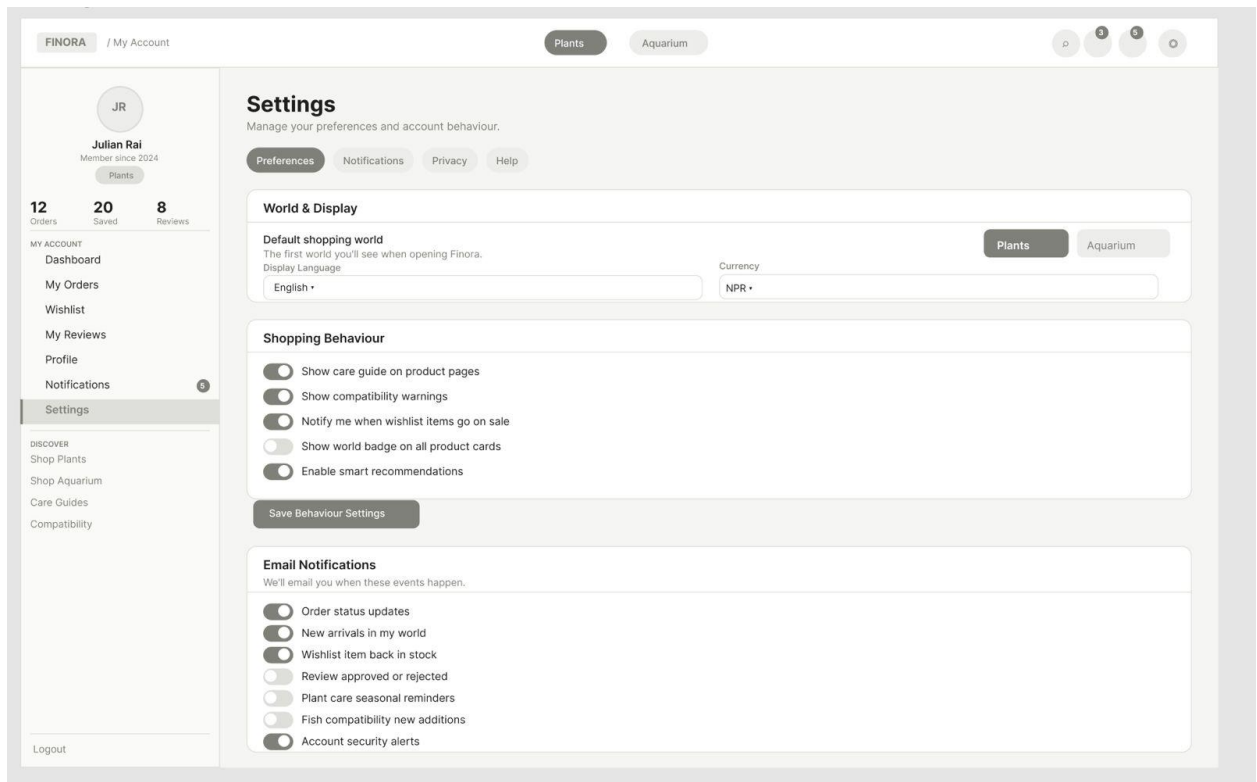
3.16.2 Design Development



3.17 User Settings

The User Settings page gives customers control over their account preferences and platform behaviour. It is organised into sections covering email preferences, where the customer can toggle promotional emails separately from transactional ones such as order confirmations and shipping updates. A privacy section allows the customer to control whether their email address is visible on any public-facing profile elements. A session preferences section lets the customer configure how long they remain logged in and whether a persistent cookie should be set for automatic sign-in on their device. Any changes made on this page are saved independently per section with a dedicated save button, ensuring partial changes are not lost if the customer navigates away before completing all sections.

3.17.1 Wireframe Design



3.17.2 Actual Design

Finora / My Account

Plants

Aquarium

Q

Julian Rai

Julian Rai

Member since 2024

Plants

12

28

8

ORDERS

SAVED

REVIEWS

MY ACCOUNT

Dashboard

My Orders

Wishlist

My Reviews

Profile

Notifications

Settings

DISCOVER

Settings

Manage your preferences and account behaviour.

Preferences

Notifications

Privacy

Help

World & Display

Your default shopping world

This is the first world you'll see when opening Finora.

Plants

Aquarium

DISPLAY LANGUAGE

English

CURRENCY

NPR

Save Preferences

Shopping Behaviour

Show care guide on product pages

Show compatibility warnings

Notify me when wishlist items go on sale

Show world badge on all product cards

Enable smart recommendations

Save Behaviour Settings

Email Notifications

We'll email you when these events happen.

Order status updates

Recommended

New arrivals in my world

Wishlist item back in stock

Review approved or rejected

Plant care seasonal reminders

Fish compatibility new additions

Account security alerts (always on for safety)

Your Data

WHAT WE STORE

Your orders and order history

Wishlist items and collections

Product reviews you've written

Profile information

World preference

Download My Data

We'll email you a download link within 24 hours.

Download My Data

Delete All Data

Permanently removes all your data. Your account will be deactivated.

Delete All Data

Cookie Preferences

Essential Cookies

Required for site to function.

Analytics Cookies

Marketing Cookies

Save Cookie Preferences

Common Questions

How do I track my order?

Can I change my shipping address after ordering?

How does the compatibility finder work?

Are your plants guaranteed to arrive healthy?

How do I return a product?

What payment methods are accepted?

How do I cancel my account?

Still need help?

Email support@finora.com — typical response within 24 hours on weekdays.

Go to Contact Page

Delivering to Nepal. Prices in NPR. ~~3883.0000~~ **3883.0000** Worldwide shipping

141

Finora / My Account

Plants

Aquarium

Q

Julian Rai

Julian Rai

Member since 2024

Plants

12

28

8

ORDERS

SAVED

REVIEWS

MY ACCOUNT

Dashboard

My Orders

Wishlist

My Reviews

Profile

Notifications

Settings

DISCOVER

Shop Plants

Shop Aquarium

Care Guides

Compatibility

Logout

Settings

Manage your preferences and account behaviour.

Preferences

Notifications

Privacy

Help

World & Display

Your default shopping world

This is the first world you'll see when opening Finora.

Plants

Aquarium

DISPLAY LANGUAGE

CURRENCY

English

NPR

Save Preferences

Shopping Behaviour

Show care guide on product pages

Show compatibility warnings

Notify me when wishlist items go on sale

Show world badge on all product cards

Enable smart recommendations

Save Behaviour Settings

Email Notifications

We'll email you when these events happen.

Order status updates

New arrivals in my world

Wishlist item back in stock

Review approved or rejected

Plant care seasonal reminders

Fish compatibility new additions

Account security alerts

Your Data

WHAT WE STORE

Your orders and order history

Wishlist items and collections

Product reviews you've written

Profile information

World preference

Download My Data

We'll email you a download link within 24 hours.

Delete All Data

Permanently removes all your data. Your account will be deactivated.

Cookie Preferences

Essential Cookies

Analytics Cookies

Marketing Cookies

Save Cookie Preferences

Common Questions

How do I track my order?

Can I change my shipping address after ordering?

How does the compatibility finder work?

Are your plants guaranteed to arrive healthy?

How do I return a product?

What payment methods are accepted?

How do I cancel my account?

Still need help?

Email support@finora.com — typical response within 24 hours on weekdays.

Go to Contact Page

Delivering to Nepal - Prices in NPR

Track orders

Worldwide shipping

142

Finora / My Account

Plants

Julian Rai

Member since 2024

Plants

12

ORDERS

20

SAVED

8

REVIEWS

MY ACCOUNT

Dashboard

My Orders

Wishlist

My Reviews

Profile

Notifications

Settings

DISCOVER

Shop Plants

Shop Aquarium

Care Guides

Compatibility

Logout

Manage your preferences and account behaviour

Preferences

Notifications

Privacy

Help

World & Display

Your default shopping world

This is the first world you'll see when opening Finora.

DISPLAY LANGUAGE

English

Save Preferences

Shopping Behaviour

Show care guide on product pages

Show compatibility warnings

Notify me when wishlist items go on sale

Show world badge on all product cards

Enable smart recommendations

Save Behaviour Settings

Email Notifications

We'll email you when these events happen.

Order status updates

Recommended

New arrivals in my world

Wishlist item back in stock

Review approved or rejected

Plant care seasonal reminders

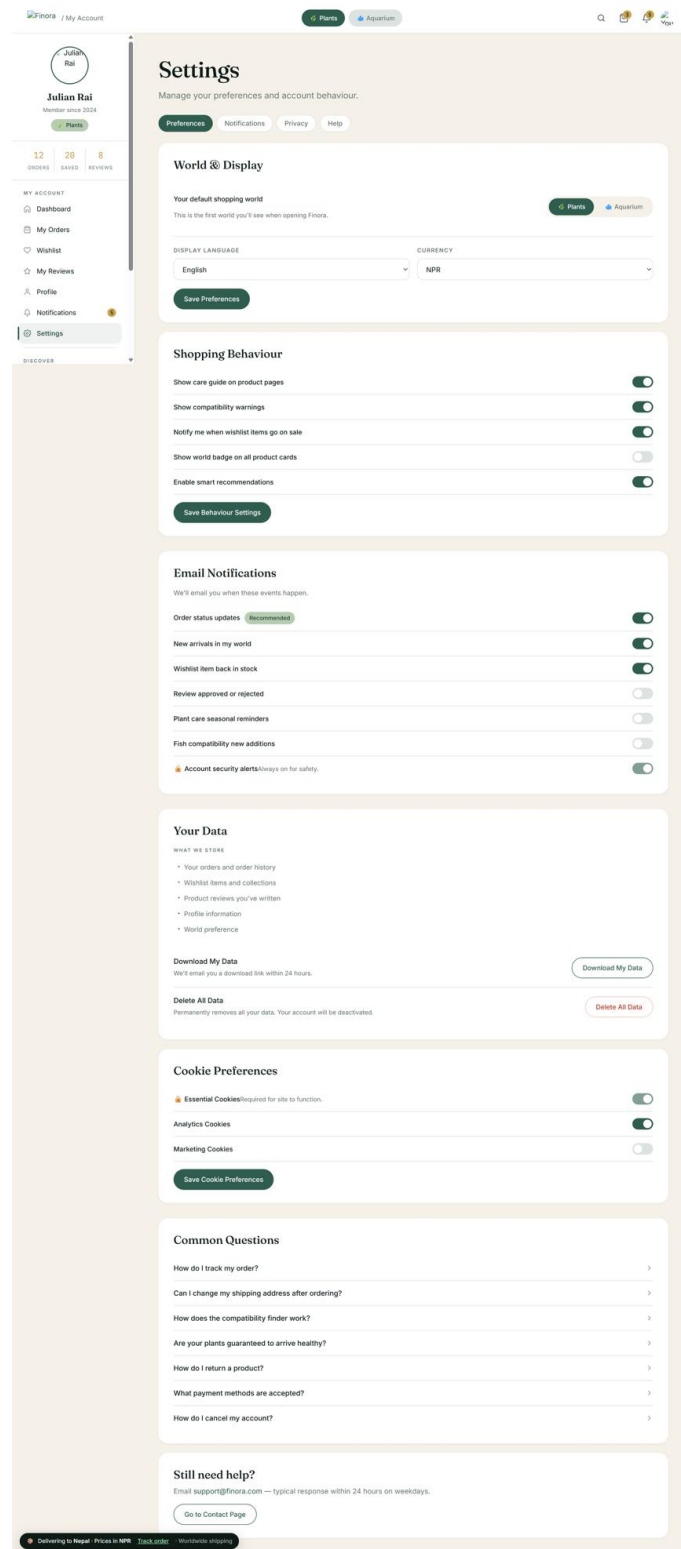
Fish compatibility new additions

Account security alerts

Always on for safety.

143

3.17.3 Design Development

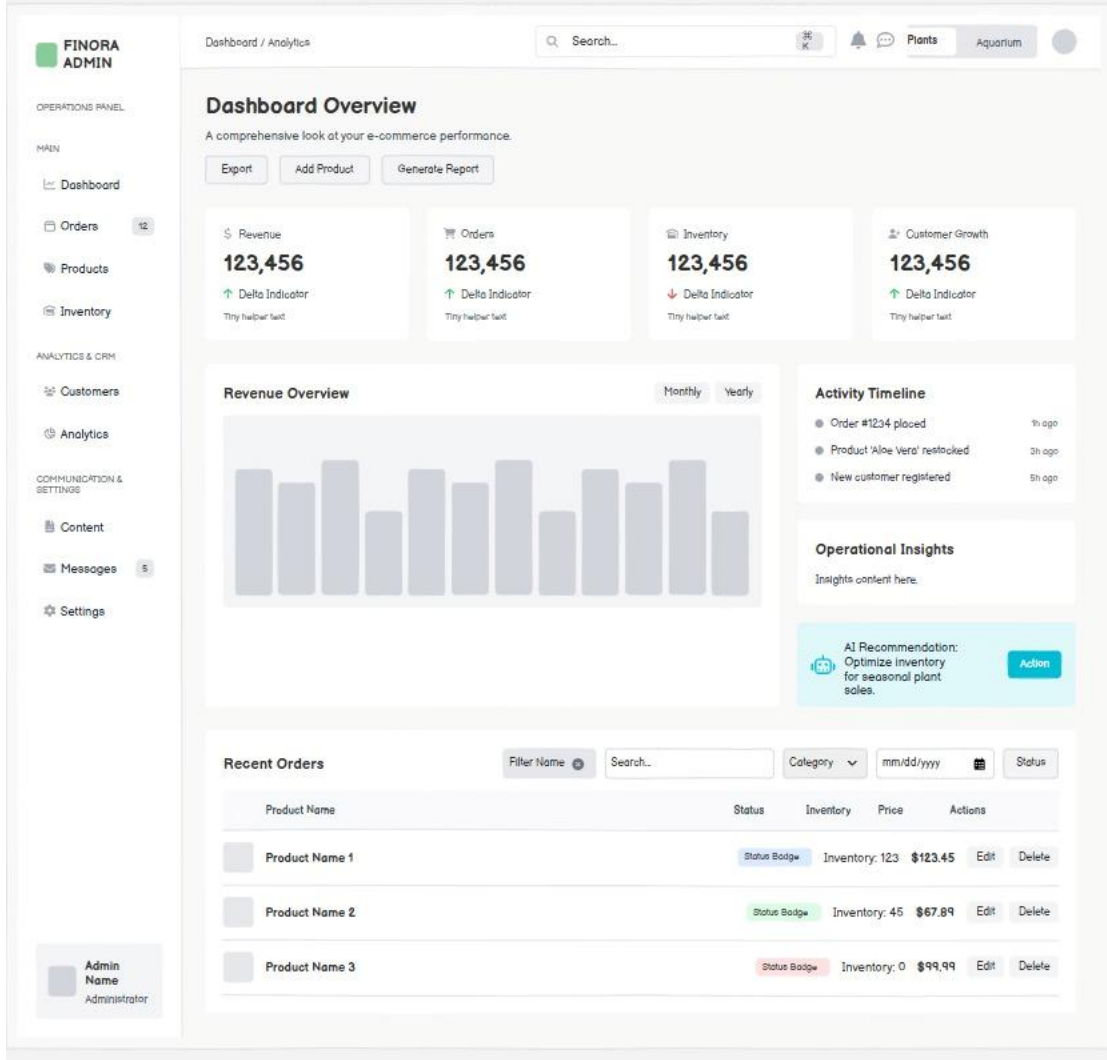


3.18 Admin Dashboard

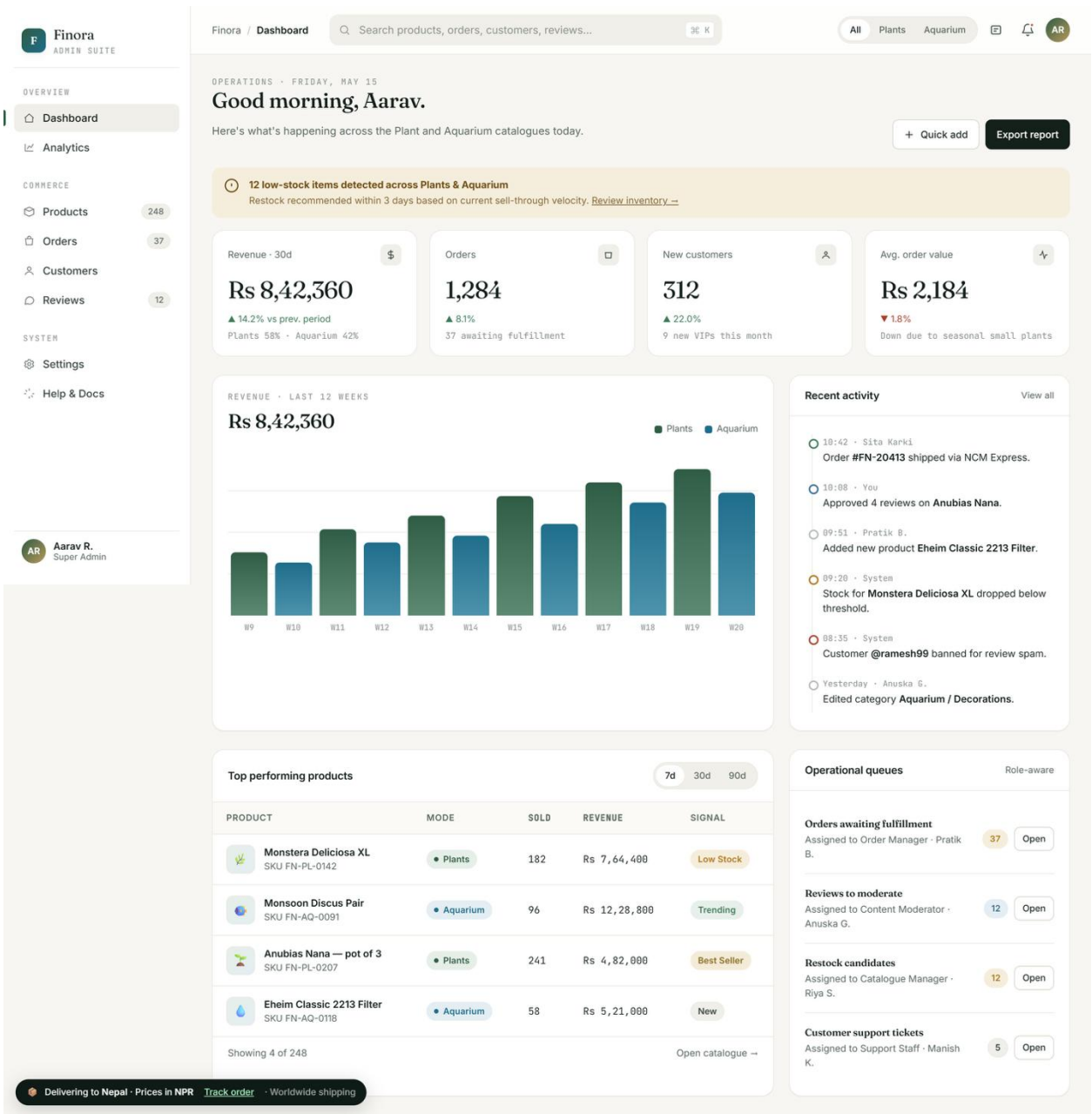
The Admin Dashboard is the first page an administrator sees upon logging in and serves as the operational command centre for the entire platform. A greeting header shows the

current date and a personalised welcome message. Below the header, a row of key performance stat cards displays today's total revenue, number of new orders, active customer count, and a platform-wide health metric. A revenue chart beneath the stats plots earnings over the last twelve weeks, giving the administrator a quick visual read of sales trends. Additional panels highlight the most recent orders awaiting fulfilment, low-stock product alerts requiring attention, and recent customer registrations. Quick action buttons allow the administrator to jump directly to the most frequent tasks such as adding a product, viewing pending orders, or processing a refund without navigating through multiple menus.

3.18.1 Wireframe Design



3.18.2 Actual Design



F

Home

248

37

12

Finora / Dashboard

Search products, 0 36 K

AllPlantsAquarium

AR

OPERATIONS - FRIDAY, MAY 15

Good morning, Aarav.

Here's what's happening across the Plant and Aquarium catalogues today.

+ Quick add

Export report

12 low-stock items detected across Plants & Aquarium

Restock recommended within 3 days based on current sell-through velocity. [Review inventory...](#)

Revenue - 30d

\$

Rs 8,42,360

▲ 14.2% vs prev. period

Plants 58% · Aquarium 42%

Orders

1,284

▲ 8.1%

37 awaiting fulfillment

New customers

312

▲ 22.0%

9 new VIPs this month

Avg. order value

Rs 2,184

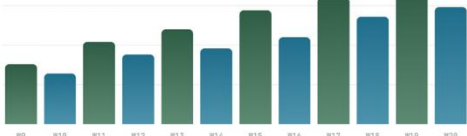
▼ 1.8%

Down due to seasonal small plants

REVENUE - LAST 12 WEEKS

Rs 8,42,360

PlantsAquarium



Recent activity

View all

18:42 - Sita Kovid

Order #FN-20413 shipped via NCM Express.

18:08 - You

Approved 4 reviews on Anubias Nana.

09:51 - Pratik B.

Added new product Eheim Classic 2213 Filter.

09:28 - System

Stock for Monstera Deliciosa XL dropped below threshold.

08:35 - System

Customer @ramesh99 banned for review spam.

Yesterday - Anuska G.

Edited category Aquarium / Decorations.

Top performing products

7d30d90d

PRODUCT	MODE	SOLD	REVENUE	SIGNAL
 Monstera Deliciosa XL SKU FN-PL-0142	Plants	182	Rs 7,64,400	Low Stock
 Monsoon Discus Pair SKU FN-AQ-0091	Aquarium	96	Rs 12,28,800	Trending
 Anubias Nana — pot of 3 SKU FN-PL-0207	Plants	241	Rs 4,82,000	Best Seller
 Eheim Classic 2213 Filter SKU FN-AQ-0118	Aquarium	58	Rs 5,21,000	New

Showing 4 of 248

Open catalogue →

Operational queues

Role-aware

Orders awaiting fulfillment

Assigned to Order Manager - Pratik B.

37Open

Reviews to moderate

Assigned to Content Moderator - Anuska G.

12Open

Restock candidates

Assigned to Catalogue Manager - Riya S.

12Open

Customer support tickets

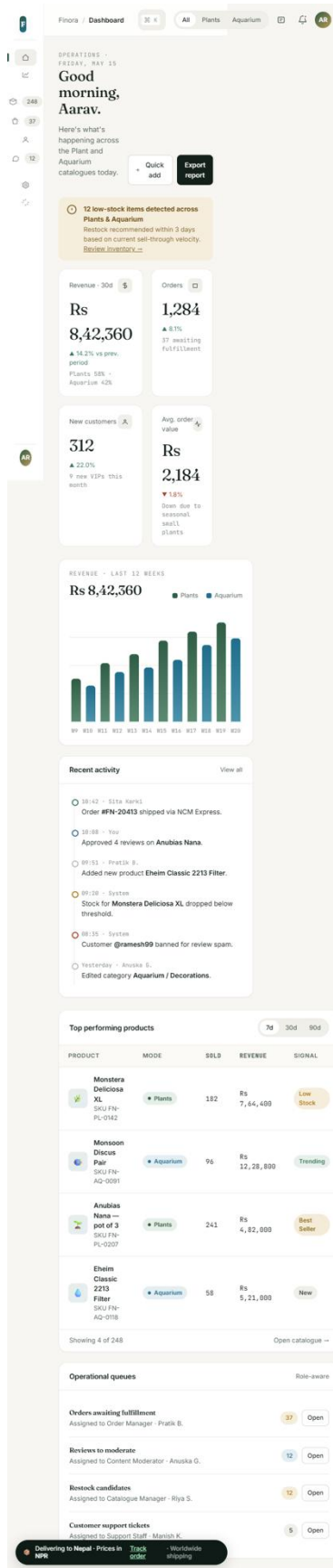
Assigned to Support Staff - Manish K.

5Open

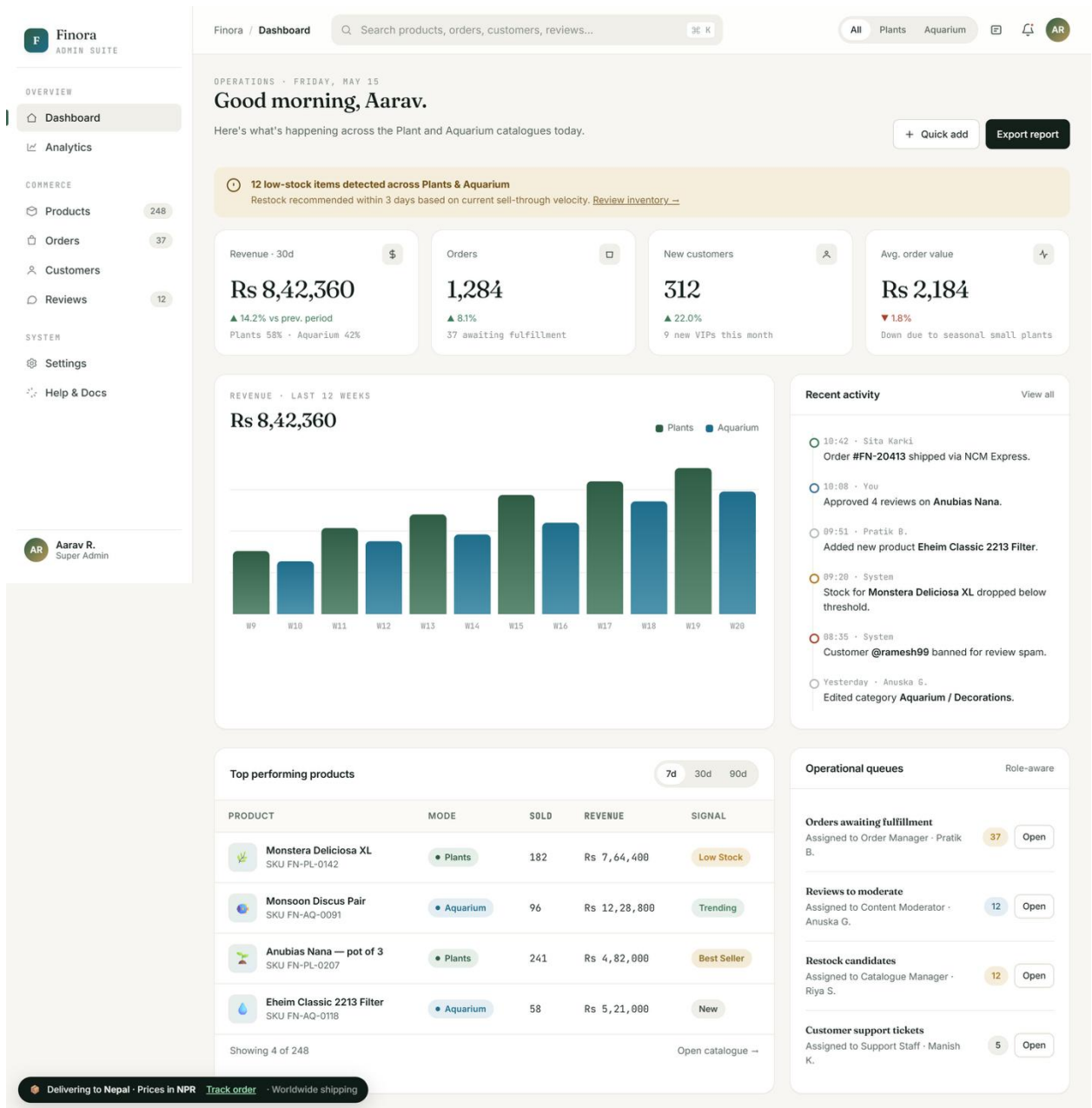
Delivering to Nepal · Prices in NPR

Track order

Worldwide shipping



3.18.3 Design Development

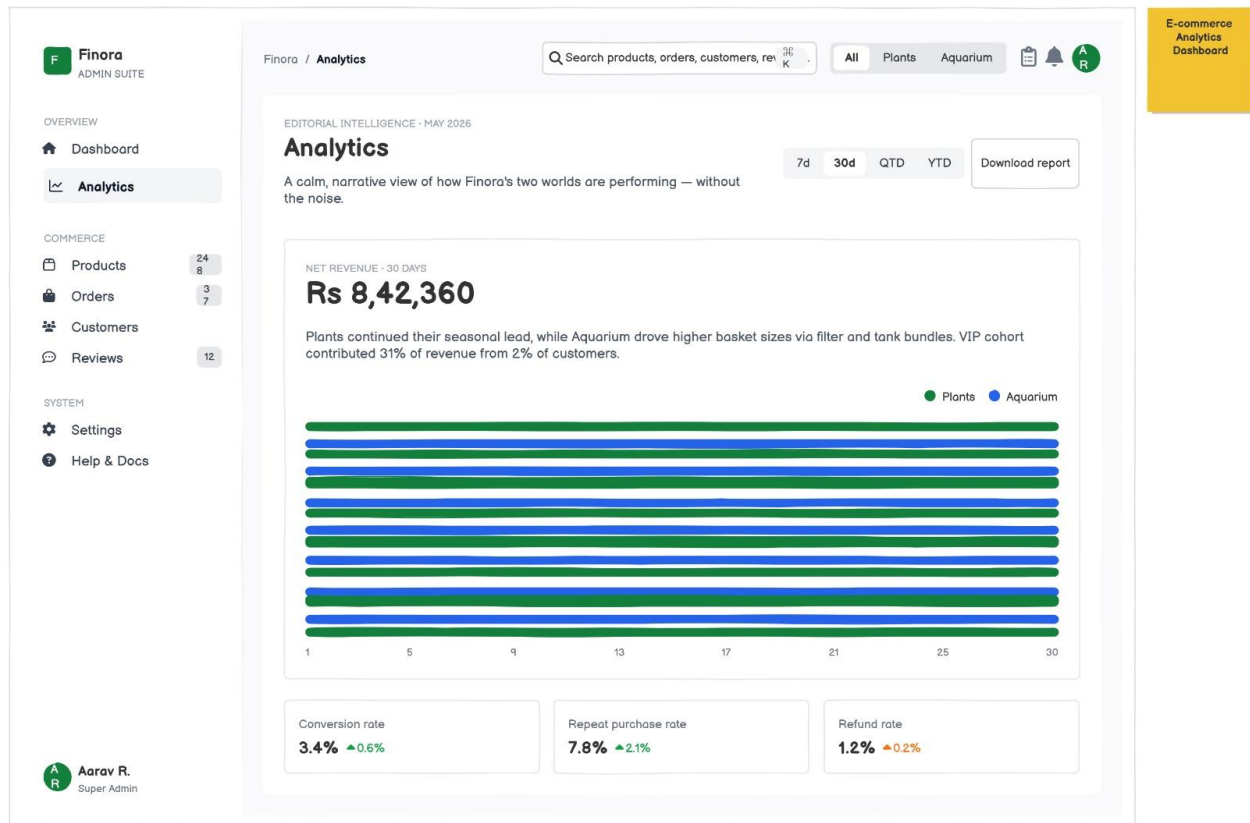


3.19 Admin Analytics

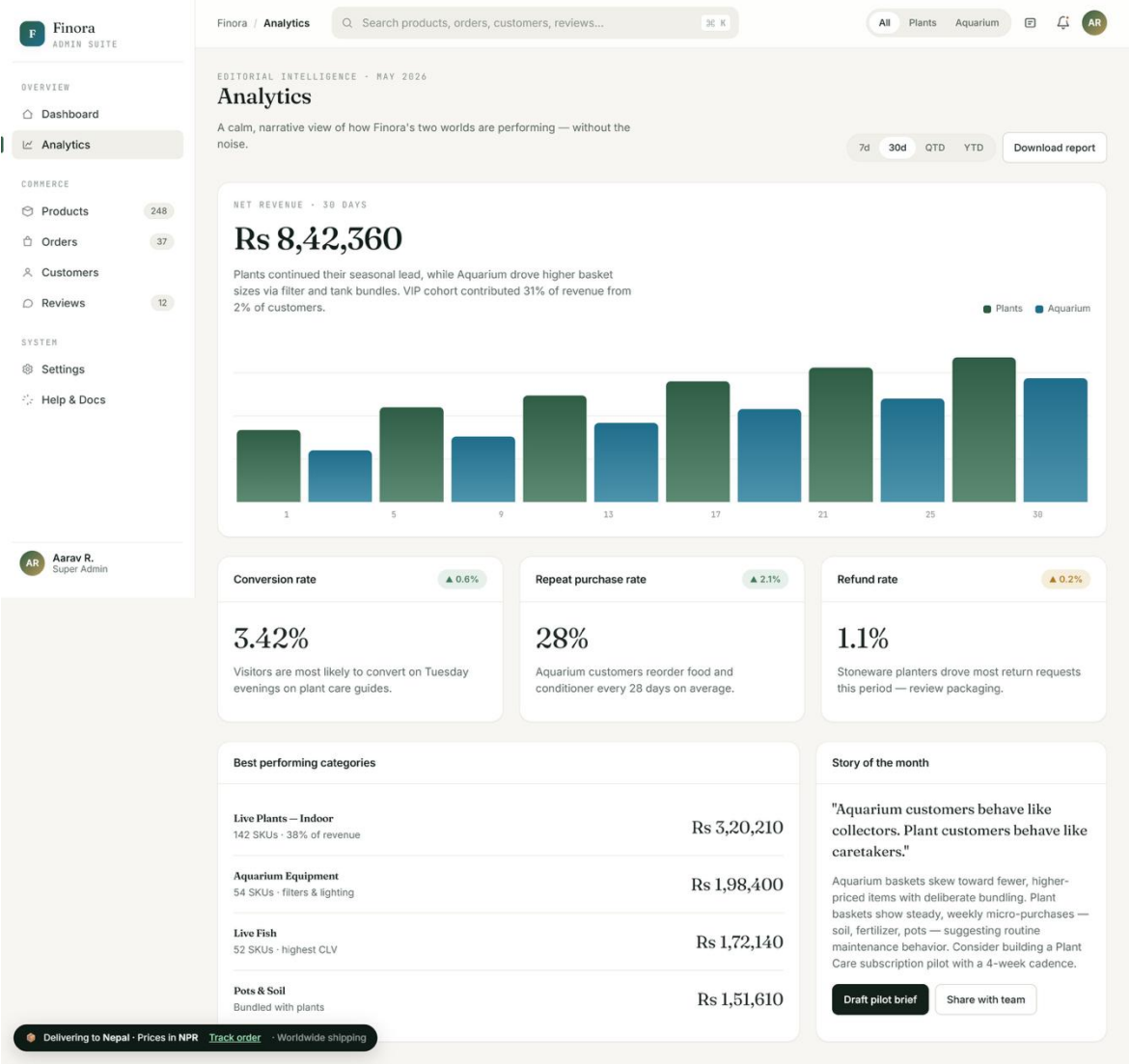
The Admin Analytics page provides in-depth reporting and business intelligence data for administrators monitoring the performance of the Finora platform. The page opens with a summary of net revenue over the last thirty days alongside key rate metrics including conversion rate, repeat purchase rate, and refund rate, each displayed with a directional change indicator showing week-on-week movement. Below the summary, detailed charts and tables break down revenue by product category, top-selling individual products by quantity and by revenue, new versus returning customer ratios,

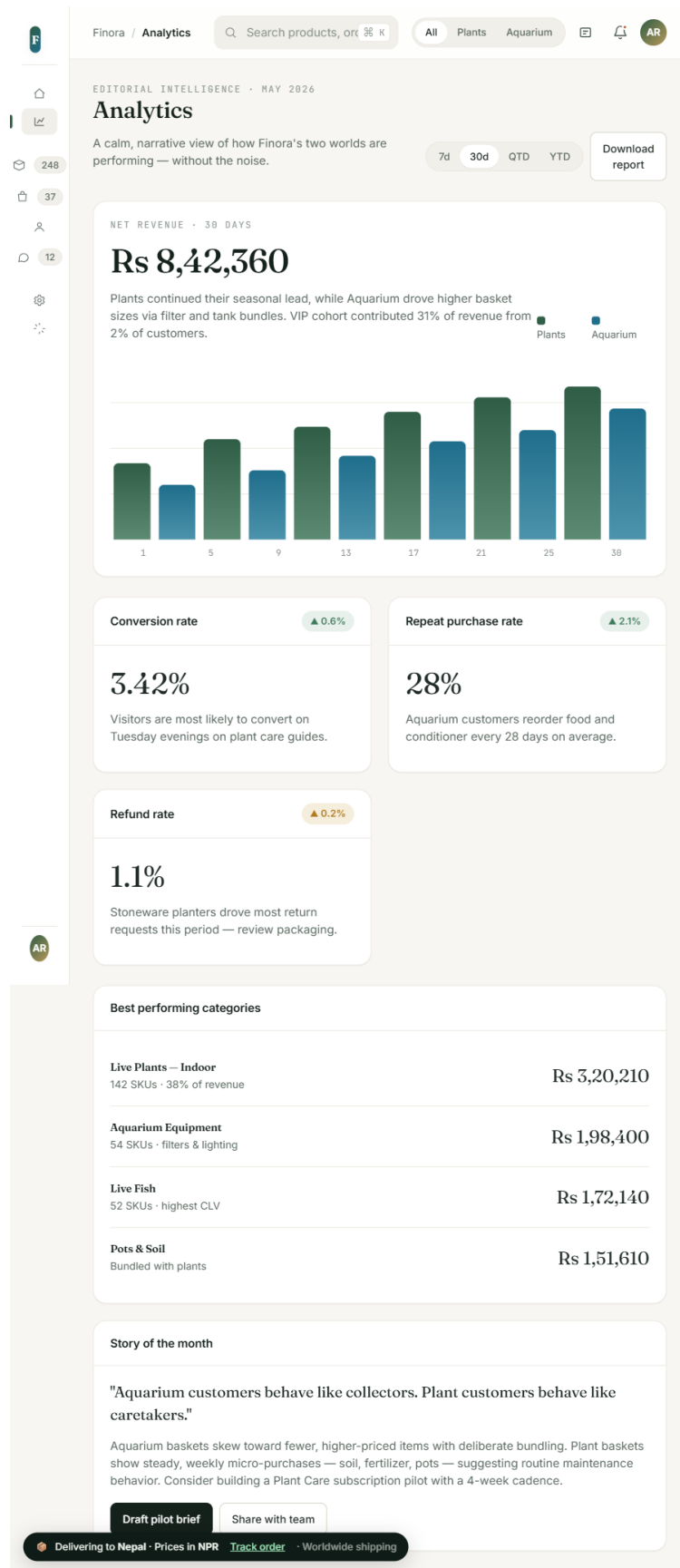
and stock turnover rates. Administrators can filter the data by date range such as weekly, monthly, or yearly, and can export the current view as a CSV or PDF file for external reporting. The analytics page is read-only and does not allow any direct edits to platform data.

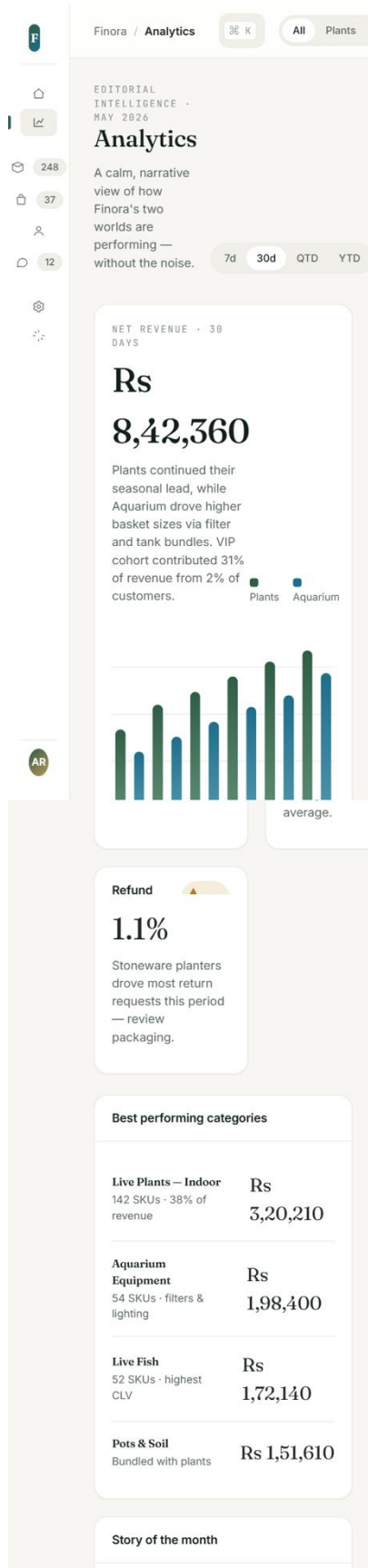
3.19.1 Wireframe Design



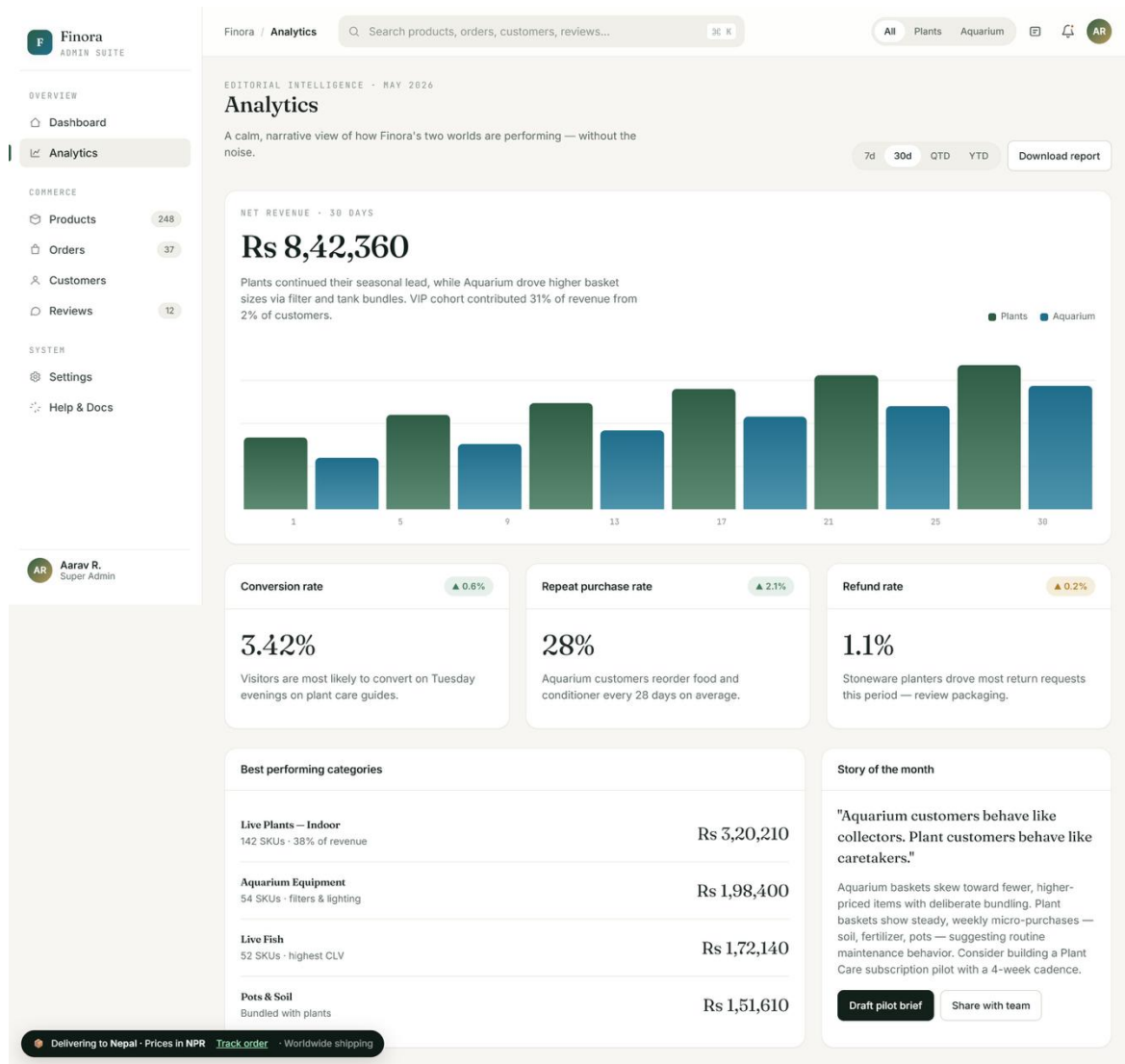
3.19.2 Actual Design







3.19.3 Design Development



3.20 Admin Product Management

The Admin Product Management page is where administrators create, update, and remove products from the Finora catalogue. The page header shows the total count of active SKUs and provides a search bar and filter controls for narrowing the product list by world, category, stock status, and product type. The main table lists all products with columns for product name, world mode (Plants or Aquarium), category, price, current stock quantity, active status, and any system signals such as a low-stock warning. Each row includes action buttons to edit or soft-delete the product. The Add Product button opens a form that dynamically adjusts its fields based on whether the product type is set to Plant or Aquarium, showing care attribute fields for plants and compatibility and

equipment specification fields for aquarium products. Administrators can also upload multiple product images and set featured status from this page.

3.20.1 Wireframe Design

Finora

ADMIN SUITE

OVERVIEW

Dashboard

Analytics

COMMERCE

Products248

Orders37

Customers

Reviews12

SYSTEM

Settings

Help & Docs

Aarav R.

Super Admin

Finora / Products

Q Search products, orders, customers, re

AllPlantsAquarium

AR

CATALOGUE · 248 ACTIVE SKUS

Products

Manage Plants and Aquarium SKUs, Inventory, badges, and pricing in Nepalese Rupees.

Import CSV

New product

Smart suggestion · 3 fish species missing compatibility data

Adding tank-mate guidance helps Aquarium customers complete their first build. Open the affected SKUs to fill in the data.

All 248Plants 142Aquarium 106Low stock 12Trending 9Drafts 4

CategorySort: Best selling

Catalogue

3 selectedTagExportArchive

	PRODUCT	MODE	CATEGORY	PRICE (RS)	STOCK	STATUS	SIGNALS
☒	\$(product.name) Monstera Deliciosa XL FN-PL-0142	Plants	Live Plants	4,200	4	Active	Low StockBest Seller
☐	Monsoon Discus Pair FN-AQ-0091	Aquarium	Live Fish	12,800	18	Active	TrendingFrequently Wishlisted
☐	Anubias Nana — pot of 3 FN-PL-0207	Plants	Live Plants	2,000	86	Active	Best Seller
☐	Eheim Classic 2213 Filter FN-AQ-0010	Aquarium	Filters	8,900	22	Active	New

155

3.20.2 Actual Design

Finora

ADMIN SUITE

OVERVIEW

Dashboard

Analytics

COMMERCE

Products248

Orders37

Customers

Reviews12

SYSTEM

Settings

Help & Docs

AR

Aarav R.

Super Admin

Finora / Products

Q Search products, orders, customers, reviews...

AllPlantsAquarium

AR

CATALOGUE · 248 ACTIVE SKUS

Products

Manage Plants and Aquarium SKUs, inventory, badges, and pricing in Nepalese Rupees.

Import CSV

+ New product

Smart suggestion · 3 fish species missing compatibility data

Adding tank-mate guidance helps Aquarium customers complete their first build. Open the affected SKUs to fill in the data.

All 248Plants 142Aquarium 106Low stock 12Trending 9Drafts 4

CategorySort: Best selling

Catalogue

3 selectedTagExportArchive

	PRODUCT	MODE	CATEGORY	PRICE (RS)	STOCK	STATUS	SIGNALS		
☒	 \$(product.name) — Monstera Deliciosa XL FN-PL-0142	Plants	Live Plants	4,200	4	Active	Low Stock Best Seller	Edit	Delete
☐	 Monsoon Discus Pair FN-AQ-0891	Aquarium	Live Fish	12,800	18	Active	Trending Frequently Wishlisted	Edit	Delete
☐	 Anubias Nana — pot of 3 FN-PL-0207	Plants	Live Plants	2,000	86	Active	Best Seller	Edit	Delete
☐	 Eheim Classic 2213 Filter FN-AQ-0118	Aquarium	Filters	8,990	22	Active	New	Edit	Delete
☐	 Stoneware Planter — Sand FN-PL-0298	Plants	Pots & Soil	1,450	5	Low	Low Stock High Returns	Edit	Delete
☐	 Neon Tetra — group of 10 FN-AQ-0044	Aquarium	Live Fish	1,200	240	Active	Compatibility data missing	Edit	Delete
☐	 Organic Compost — 5kg FN-PL-0312	Plants	Soil	650	312	Draft	—	Edit	Delete

Showing 7 of 248 products · low-stock threshold: 6

<

1

2

3

...

36

>

Delivering to Nepal · Prices in NPR

Track order

Worldwide shipping

F

248

37

12

AR

Finora / Products

Search products, orders, or SKUs

AllPlantsAquarium

AR

CATALOGUE · 248 ACTIVE SKUS

Products

Manage Plants and Aquarium SKUs, inventory, badges, and pricing in Nepalese Rupees.

Import CSV

+New product

Smart suggestion · 3 fish species missing compatibility data

Adding tank-mate guidance helps Aquarium customers complete their first build. Open the affected SKUs to fill in the data.

All 248Plants 142Aquarium 106Low stock 12Trending 9Drafts 4

CategorySort: Best selling

Catalogue3 selectedTagExportArchive

	PRODUCT	MODE	CATEGORY	PRICE (RS)	STOCK	STATUS
☒	 Monstera Deliciosa XL FN-PL-0142	Plants	Live Plants	4,200	4	Active
☐	 Monsoon Discus Pair FN-AQ-0091	Aquarium	Live Fish	12,800	18	Active
☐	 Anubias Nana — pot of 3 FN-PL-0207	Plants	Live Plants	2,000	86	Active
☐	 Eheim Classic 2213 Filter FN-AQ-0118	Aquarium	Filters	8,990	22	Active
☐	 Stoneware Planter — Sand FN-PL-0298	Plants	Pots & Soil	1,450	5	Low
☐	 Neon Tetra — group of 10 FN-AQ-0044	Aquarium	Live Fish	1,200	240	Active
☐	 Organic Compost — 5kg FN-PL-0312	Plants	Soil	650	312	Draft

Showing 7 of 248 products · low-stock threshold: 6

<123...36>

Delivering to Nepal · Prices in NPRTrack order · Worldwide shipping

157



3.20.3 Design Development

Finora / Products

Search products, orders, customers, reviews...

Overview

- Dashboard
- Analytics

Commerce

- Products 248
- Orders 37
- Customers
- Reviews 12

System

- Settings
- Help & Docs

Aarav R. Super Admin

CATALOGUE · 248 ACTIVE SKUS

Products

Manage Plants and Aquarium SKUs, inventory, badges, and pricing in Nepalese Rupees.

Import CSV + New product

Smart suggestion · 3 fish species missing compatibility data
Adding tank-mate guidance helps Aquarium customers complete their first build. Open the affected SKUs to fill in the data.

All 248 Plants 142 Aquarium 106 Low stock 12 Trending 9 Drafts 4

Category Sort: Best selling

Catalogue 3 selected Tag Export Archive

	PRODUCT	MODE	CATEGORY	PRICE (RS)	STOCK	STATUS	SIGNALS		
☒	Monstera Deliciosa XL FN-PL-0142	Plants	Live Plants	4,200	4	Active	Low Stock Best Seller	Edit	Delete
☐	Monsoon Discus Pair FN-AQ-0891	Aquarium	Live Fish	12,800	18	Active	Trending Frequently Wishlisted	Edit	Delete
☐	Anubias Nana — pot of 3 FN-PL-0207	Plants	Live Plants	2,000	86	Active	Best Seller	Edit	Delete
☐	Eheim Classic 2213 Filter FN-AQ-0118	Aquarium	Filters	8,990	22	Active	New	Edit	Delete
☐	Stoneware Planter — Sand FN-PL-0298	Plants	Pots & Soil	1,450	5	Low	Low Stock High Returns	Edit	Delete
☐	Neon Tetra — group of 10 FN-AQ-0044	Aquarium	Live Fish	1,200	240	Active	Compatibility data missing	Edit	Delete
☐	Organic Compost — 5kg FN-PL-0312	Plants	Soil	650	312	Draft	—	Edit	Delete

Showing 7 of 248 products · low-stock threshold: 6

Delivering to Nepal · Prices in NPR Track order · Worldwide shipping

3.21 Admin Order Management

The Admin Order Management page gives administrators full visibility and control over all customer orders across the platform. A stats row at the top of the page displays the number of orders awaiting fulfilment, payments currently pending, open refund requests, and failed payment attempts in the last twenty-four hours. The main order table lists all orders with columns for order ID, customer name, number of items, total amount, payment method, payment status, and current order status. Administrators can filter orders by status such as Pending, Processing, Shipped, Delivered, or Cancelled, and can search by customer name or order ID. Selecting one or more orders enables bulk status updates. Individual order rows can be expanded to show line items, shipping details, and an internal notes field where administrators can log fulfilment actions or customer communications.

3.21.1 Actual Design

F

Finora

ADMIN SUITE

OVERVIEW

Dashboard

Analytics

COMMERCE

Products248

Orders37

Customers

Reviews12

SYSTEM

Settings

Help & Docs

AR

Aarav R.

Super Admin

Finora / Orders

Search products, orders, customers, reviews...

36 K

AllPlantsAquarium

AR

FULFILLMENT - 37 AWAITING ACTION

Orders

Track payments, partial fulfillments, COD, refunds and shipment progress across both modes.

Print picking listExport orders

Awaiting fulfillment37

Payment pending8

Refunds open2

Failed payments - 24h4

AllNewProcessingPartially shippedShippedDeliveredRefundedFailed

Search by order ID, customer, SKU...

ORDER	CUSTOMER	ITEMS	TOTAL (RS)	PAYMENT	FULFILLMENT	PRIORITY	
#{order.id} - #FN-28413 May 15 - 10:42	Sita Karki Kathmandu - \$(customer.email)	3 items - 2 plants, 1 pot	6,540	Paid - eSewa	Paid Packed Shipped Delivered	Standard	Open
#FN-28412 May 15 - 09:18	Pratik Bhandari Pokhara - pratikb@mail.np	1 aquarium kit	28,400	COD pending	Placed Confirm Ship Deliver	High value	Open
#FN-28411 May 15 - 08:05	Anuska Gurung Lalitpur - VIP	5 items - partial	14,720	Paid - Khalti	Paid Packed Shipped Out for delivery	Vip	Open
#FN-28409 May 14 - 22:11	Ramesh Shah Biratnagar	2 fish, 1 filter	19,180	Failed - Gateway	Awaiting payment	Fraud check	Cancel
#FN-28405 May 14 - 17:32	Manish Khatri Bhaktapur	1 plant kit	3,250	Refund issued	- - Returned Refunded	Standard	Open

Showing 5 of 1284 - Last sync: a moment ago

<

1

2

3

>

Delivering to Nepal - Prices in NPR

Track order

Worldwide shipping



3.21.2 Design Development

The screenshot displays the 'Orders' section of the Finora Admin Suite. The sidebar on the left includes navigation links for Overview, Dashboard, Analytics, Commerce (Products, Orders, Customers, Reviews), and System (Settings, Help & Docs). The main content area features a search bar, a status filter (All, Plants, Aquarium), and a summary of order statistics: 37 Awaiting fulfillment, 8 Payment pending, 2 Refunds open, and 4 Failed payments. Below this is a table of orders with columns for Order ID, Customer, Items, Total (RS), Payment, Fulfillment, and Priority. The table lists five orders, including details like customer names, item counts, total values, payment statuses, and fulfillment progress. A footer bar indicates delivery to Nepal and provides links for tracking orders and worldwide shipping.

ORDER	CUSTOMER	ITEMS	TOTAL (RS)	PAYMENT	FULLFILLMENT	PRIORITY
\$[order.id] - #FN-28413 May 15 - 10:42	Sita Karki Kathmandu - \$(customer.email)	3 items - 2 plants, 1 pot	6,540	Paid - eSewa	Paid Packed Shipped Delivered	Standard
#FN-28412 May 15 - 09:18	Pratik Bhandari Pokhara - pratikb@mail.np	1 aquarium kit	28,400	COD pending	Placed Confirm Ship Deliver	High value
#FN-28411 May 15 - 08:05	Anuska Gurung Lalitpur - VIP	5 items - partial	14,720	Paid - Khalti	PaidPackedShippedOut for delivery	VIP
#FN-28409 May 14 - 22:11	Ramesh Shah Biratnagar	2 fish, 1 filter	19,180	Failed - Gateway	Awaiting payment	Fraud check
#FN-28405 May 14 - 17:32	Manish Khatri Bhaktapur	1 plant kit	3,250	Refund Issued	- - Returned Refunded	Standard

3.22 Admin Customer Management

The Admin Customer Management page provides administrators with a comprehensive view of all registered customer accounts on the platform. A stats row surfaces high-level customer metrics including the number of active customers in the last thirty days, the count of VIP customers defined by a minimum lifetime value threshold, the platform-wide average customer lifetime value, and an overall engagement score based on wish list activity, review submissions, and repeat purchases. The main customer table lists all accounts with columns for customer name, account tier, total lifetime value, average order value, and top product category. Administrators can search by name or email, view an individual customer's full order history, reset a customer's password by triggering a reset email, and suspend or permanently ban accounts that violate platform policies.

3.22.1 Wireframe Design

F

Finora

Admin Suite

OVERVIEW

Dashboard

Analytics

COMMERCE

Products248

Orders37

Customers

Reviews12

SYSTEM

Settings

Help & Docs

Finora / Customers

Search products, orders, customers, reviews...

AllPlantsAquarium

CUSTOMER INTELLIGENCE 9,482 PROFILES

Customers

Segment builderExport CSV

Lifetime value, engagement and behavior across the Finora ecosystem.

Active customers · 30d3,2066.4%

VIP customers184CLV ≥ Rs 50,000

Avg. lifetime valueRs 18,420Plants 1.4x Aquarium

Engagement score7.2 / 10Wishlist + reviews + reorder

AllVIPNew · 30dAt riskReviewers

Search by name, email, phone...

CUSTOMER	TIER	CLV (RS)	AOV	TOP CATEGORY	ENGAGEMENT
S K \$(customer.name) — Sita Karki \$(customer.email) 2 min ago Open	VIP	1,28,400	3,210	Plants	9.1 / 10
P B Pratik Bhandari pratikb@mail.np 3 hr ago Open	Active	62,180	5,650	Aquarium	7.8 / 10
A G Anuska Gurung anuska.g@mail.np Today Open	VIP	96,720	4,030	Plants	8.6 / 10
M K Manish Khatri manish.k@mail.np 42 days ago Open Re-engage	At risk	8,420	2,105	Aquarium	3.2 / 10

A R

Aarav R.

Super Admin

3.22.2 Actual Design

Finora

ADMIN SUITE

OVERVIEW

Dashboard

Analytics

CORNERCE

Products248

Orders37

Customers

Reviews12

SYSTEM

Settings

Help & Docs

AR

Aarav R.

Super Admin

Finora / Customers

Q Search products, orders, customers, reviews...

ALLPlantsAquarium

9,482 PROFILES

Customers

Lifetime value, engagement and behavior across the Finora ecosystem.

Segment builderExport CSV

Active customers · 30d

3,206

▲ 6.4%

VIP customers

184

CLV ≥ Rs 50,000

Avg. lifetime value

Rs 18,420

Plants 1.4 · Aquarium

Engagement score

7.2 / 10

WishList + reviews + reorder

AllVIPNew · 30dAt riskReviewers

Search by name, email, phone...

CUSTOMER	TIER	CLV (RS)	AOV	TOP CATEGORY	ENGAGEMENT	LAST ACTIVE	
SK \$(customer.name) — Sita Karki \$(customer.email)	VIP	1,28,400	3,210	Plants	9.1 / 10	2 min ago	Open
PB Pratik Bhandari pratikb@mail.np	Active	62,180	5,650	Aquarium	7.8 / 10	3 hr ago	Open
AG Anuska Gurung anuska.g@mail.np	VIP	96,720	4,030	Plants	8.6 / 10	Today	Open
MK Manish Khatri manish.k@mail.np	At risk	8,420	2,105	Aquarium	3.2 / 10	42 days ago	OpenRe-engage
RS Ramesh Shah @ramesh99 · suspended	Banned	1,240	620	—	0.4 / 10	4 hr ago	Restore

VIP logic: spend ≥ Rs 50k · 4+ orders · active reviewer

1

2

3

Delivering to Nepal · Prices in NPR

Track order

Worldwide shipping

165

F

Home

Analytics

248

37

1

12

Settings

Help

Finora / Customers

Search products, o

K

All

Plants

Aquarium

AR

CUSTOMER INTELLIGENCE · 9,482 PROFILES

Customers

Lifetime value, engagement and behavior across the Finora ecosystem.

Segment builder

Export CSV

Active customers · 30d

3,206

▲ 6.4%

VIP customers

184

CLV ≥ Rs 50,000

Avg. lifetime value

Rs 18,420

Plants 1.4× Aquarium

Engagement score

7.2 / 10

Wishlist + reviews + reorder

All

VIP

New · 30d

At risk

Reviewers

Search by name, email, phone...

CUSTOMER	TIER	CLV (RS)	A0V	TOP CATEGORY	ENGAGEMENT
SK \$(customer.name) — Sita Karki \$(customer.email)	VIP	1,28,400	3,210	Plants	9.1 / 10
PB Pratik Bhandari pratikb@mail.np	Active	62,180	5,650	Aquarium	7.8 / 10
AG Anuska Gurung anuska.g@mail.np	VIP	96,720	4,030	Plants	8.6 / 10
MK Manish Khatri manish.k@mail.np	At risk	8,420	2,105	Aquarium	3.2 / 10
RS Ramesh Shah @ramesh99 · suspended	Banned	1,240	620	—	0.4 / 10

VIP logic: spend ≥ Rs 50k · 4+ orders · active reviewer

<

1

2

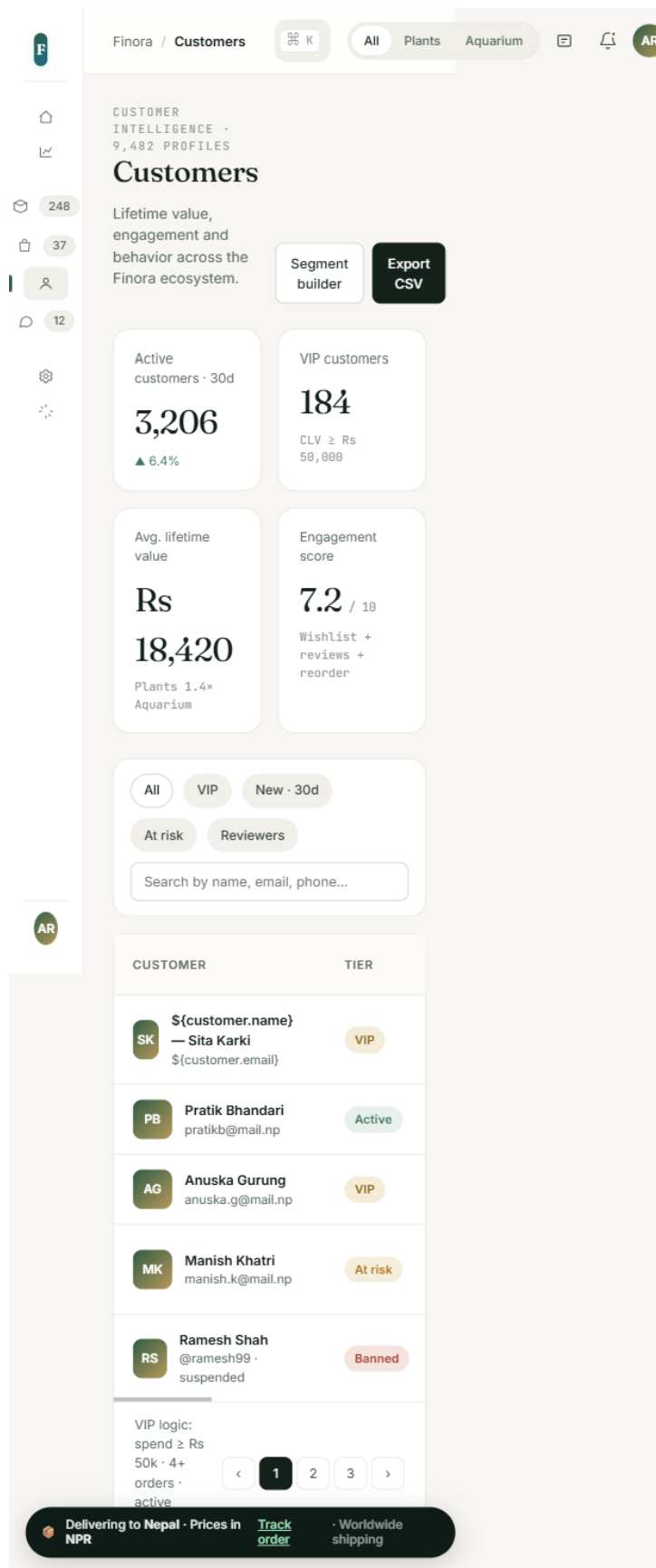
3

>

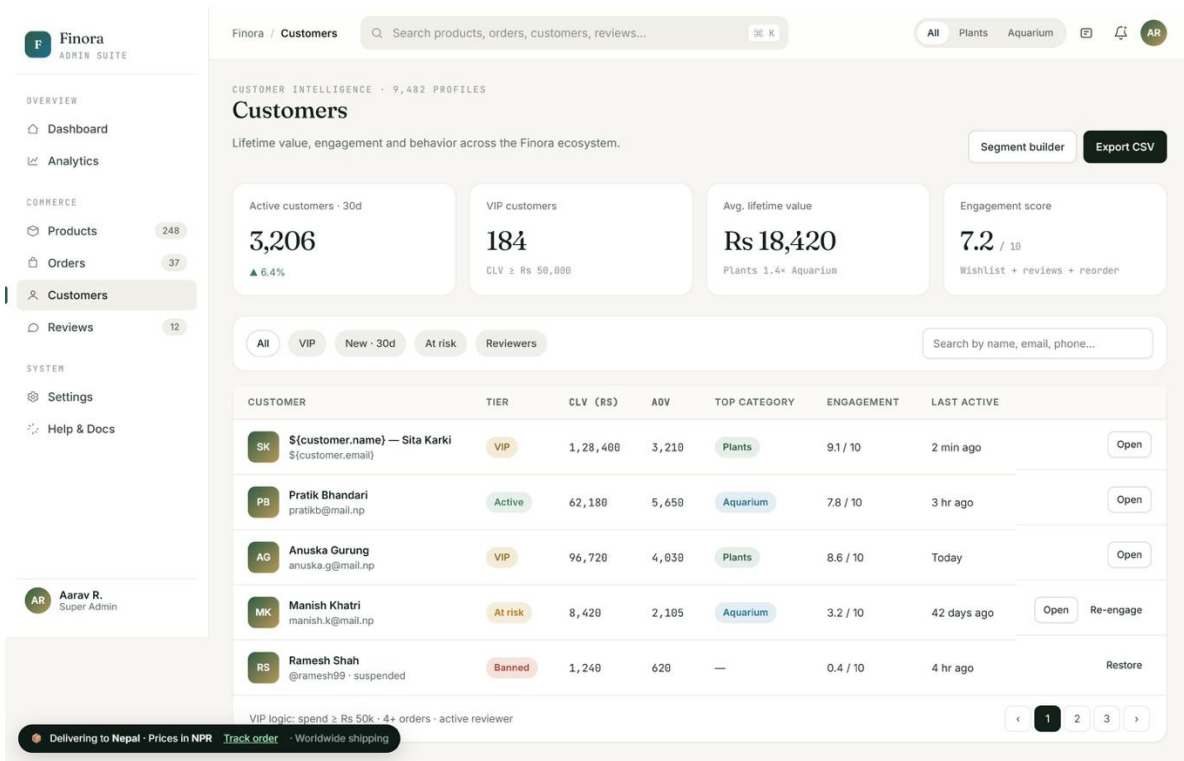
Delivering to Nepal · Prices in NPR

Track order

Worldwide shipping



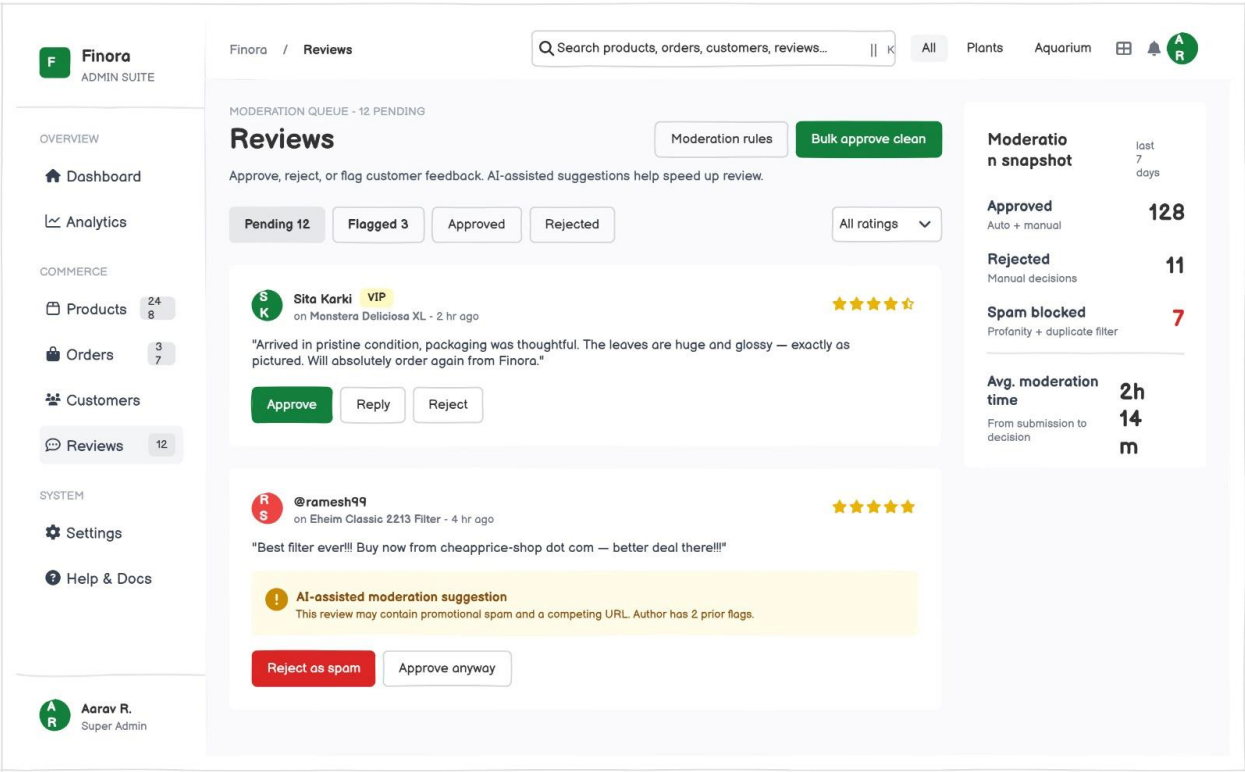
3.22.3 Design Development



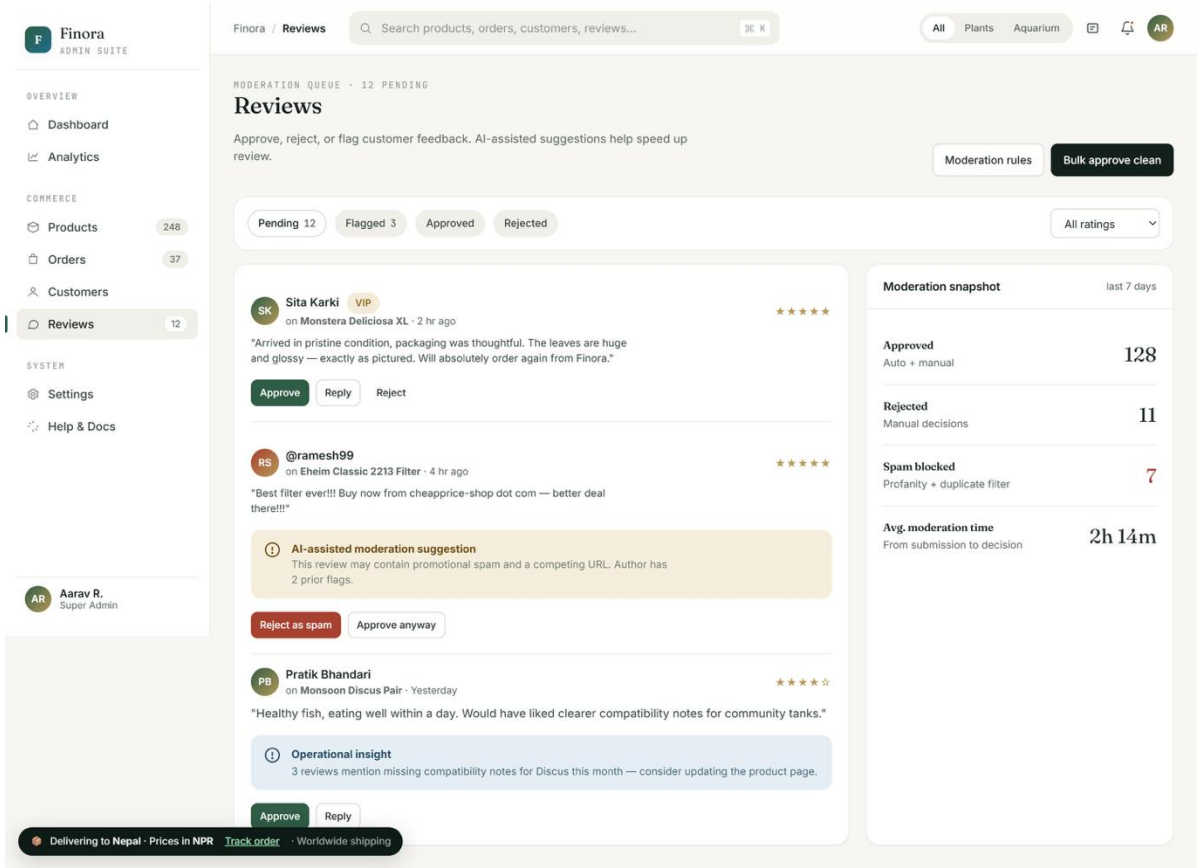
3.23 Admin Review Moderation

The Admin Review Moderation page is where administrators oversee all customer-submitted product reviews before they appear publicly on the platform. The page header displays the number of reviews currently in the moderation queue awaiting a decision. A moderation snapshot panel shows the total reviews approved, rejected, and responded to in the last seven days. The main review queue lists each pending review with the customer's name, the product being reviewed, the star rating, the written comment, and the submission date. For each review, the administrator can approve it to make it visible on the product page, reject and delete it with an optional internal reason, or edit the content if it contains minor formatting issues while keeping the sentiment intact. Administrators can also post a public response to any approved review, which appears beneath the customer's comment on the product page.

3.23.1 Wireframe Design



3.23.2 Actual Design



F

Home

Reviews

248

37

12

Settings

AR

Finora / Reviews

Search products, orders, or reviews

AllPlantsAquarium

MODERATION QUEUE · 12 PENDING

Reviews

Approve, reject, or flag customer feedback. AI-assisted suggestions help speed up review.

Moderation rulesBulk approve clean

Pending 12Flagged 3ApprovedRejected

All ratings

SK

Sita KarkiVIP

on Monstera Deliciosa XL · 2 hr ago

★★★★★

"Arrived in pristine condition, packaging was thoughtful. The leaves are huge and glossy — exactly as pictured. Will absolutely order again from Finora."

ApproveReplyReject

RS

@ramesh99

on Eheim Classic 2213 Filter · 4 hr ago

★★★★★

"Best filter ever!!! Buy now from cheapprice-shop dot com — better deal there!!!"

AI-assisted moderation suggestion

This review may contain promotional spam and a competing URL. Author has 2 prior flags.

Reject as spamApprove anyway

PB

Pratik Bhandari

on Monsoon Discus Pair · Yesterday

★★★★★

"Healthy fish, eating well within a day. Would have liked clearer compatibility notes for community tanks."

Operational insight

3 reviews mention missing compatibility notes for Discus this month — consider updating the product page.

ApproveReply

AR

Delivering to Nepal · Prices in NPRTrack order · Worldwide shipping

Moderation snapshot

last 7 days

Approved

Auto + manual

128

Rejected

Manual decisions

11

Spam blocked

Profanity + duplicate filter

7

Avg. moderation time

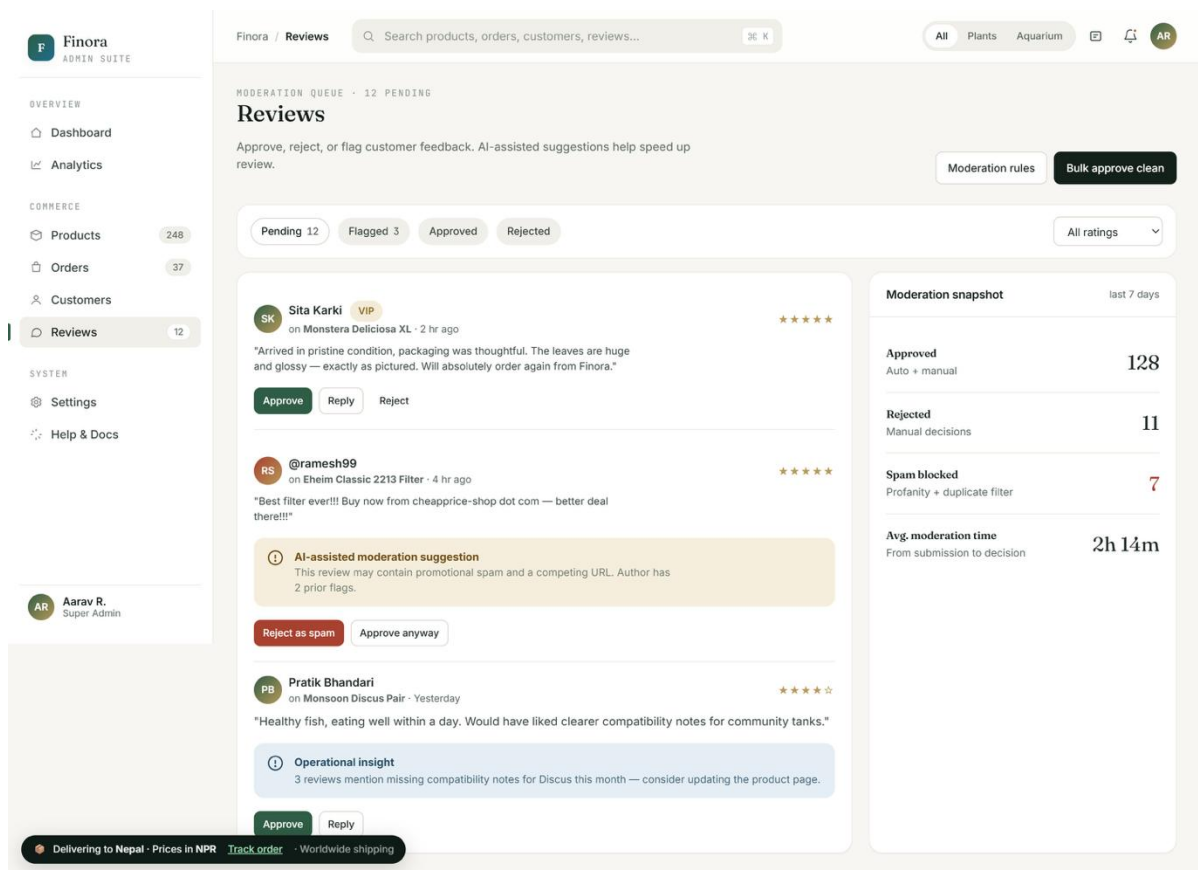
From submission to decision

2h 14m

170



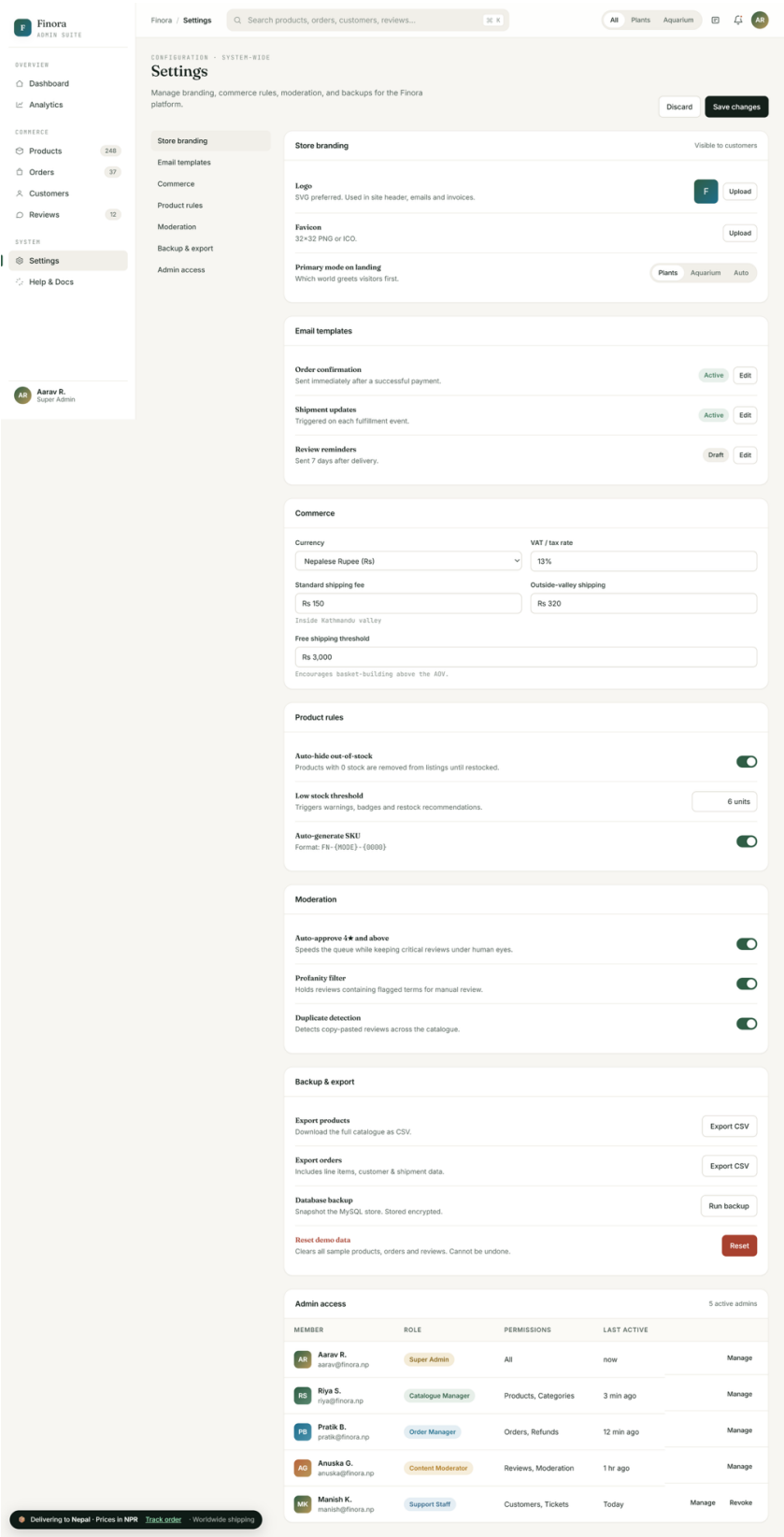
3.23.3 Design Development



3.24 Admin Settings

The Admin Settings page allows administrators to configure system-wide platform behaviour and operational parameters. It is divided into clearly labelled sections. The Store Branding section controls the platform name, logo, tagline, and colour scheme visible to customers. The Email Templates section lets administrators customise the content of transactional emails sent for order confirmations, shipping updates, and password resets. The Commerce section covers tax rate configuration, shipping options including flat-rate and free-above-threshold rules, and toggle controls for enabling or disabling promotions. The Product Rules section defines global thresholds such as the minimum stock level that triggers a low-stock alert. A Maintenance Mode toggle allows administrators to take the customer-facing site offline with a custom message while still retaining access to the admin panel. Changes in each section are saved independently to prevent accidental overrides across unrelated configurations.

3.24.1 Actual Design



248

37

12

Finora / Settings

Search products, or by R +

AllPlantsAquarium

Settings

Manage branding, commerce rules, moderation, and backups for the Finora platform.

DiscardSave changes

Store branding

Visible to customers

Logo
SVG preferred. Used in site header, emails and invoices.

Upload

Favicon
32x32 PNG or ICO.

Upload

Primary mode on landing
Which world greets visitors first.

PlantsAquariumAuto

Email templates

Order confirmation
Sent immediately after a successful payment.

ActiveEdit

Shipment updates
Triggered on each fulfillment event.

ActiveEdit

Review reminders
Sent 7 days after delivery.

DraftEdit

Commerce

Currency
Nepalese Rupee (Rs)

VAT / tax rate
13%

Standard shipping fee
Rs 150

Outside-valley shipping
Rs 320

Free shipping threshold
Rs 3,000

Product rules

Auto-hide out-of-stock
Products with 0 stock are removed from listings until restocked.

Low stock threshold
Triggers warnings, badges and restock recommendations.

6 units

Auto-generate SKU
Format: PR- (9999) - (9999)

Moderation

Auto-approve 1★ and above
Speeds the queue while keeping critical reviews under human eyes.

Profanity filter
Holds reviews containing flagged terms for manual review.

Duplicate detection
Detects copy-pasted reviews across the catalogue.

Backup & export

Export products
Download the full catalogue as CSV.

Export CSV

Export orders
Includes line items, customer & shipment data.

Export CSV

Database backup
Snapshot the MySQL store. Stored encrypted.

Run backup

Reset demo data
Clears all sample products, orders and reviews. Cannot be undone.

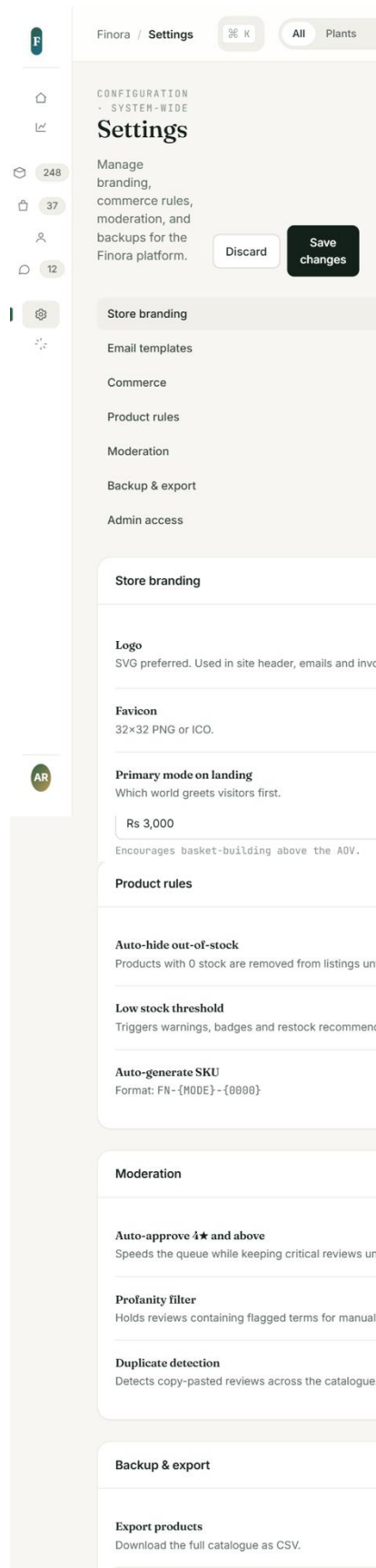
Reset

Admin access

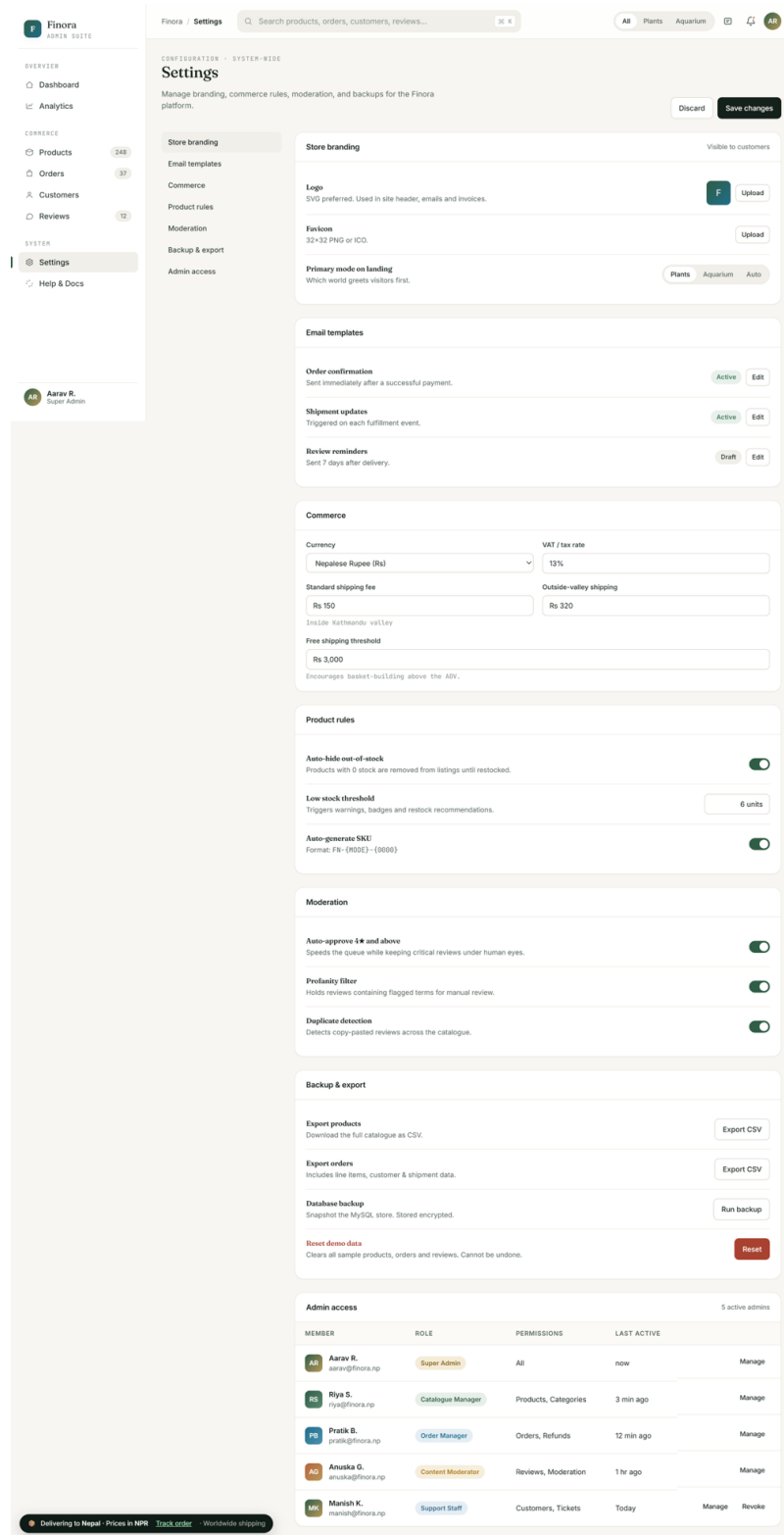
5 active admins

MEMBER	ROLE	PERMISSIONS	LAST ACTIVE	
Anav R. anav@finora.np	Super Admin	All	now	Manage
Riya S. riya@finora.np	Catalogue Manager	Products, Categories	3 min ago	Manage
Pratik B. pratik@finora.np	Order Manager	Orders, Refunds	12 min ago	Manage
Anuska G. anuska@finora.np	Content Moderator	Reviews, Moderation	1 hr ago	Manage
Manish K. manish@finora.np	Support Staff	Customers, Tickets	Today	Manage Revoke

Delivering to Nepal · Prices in NPR · Track order · Worldwide shipping



3.24.2 Design Development



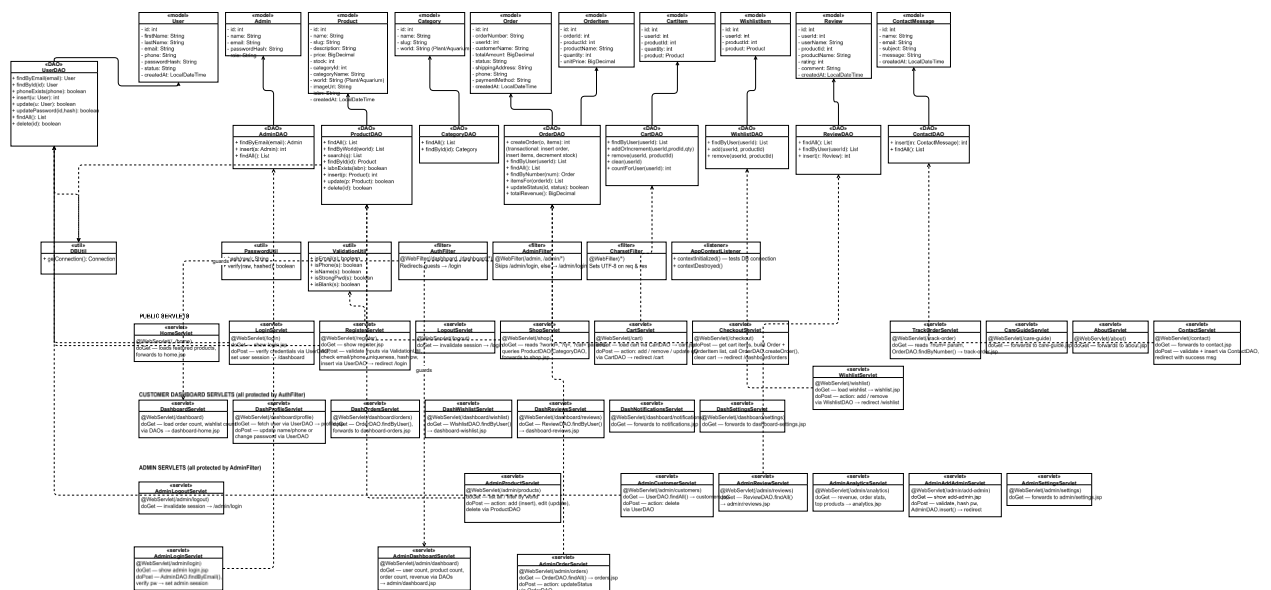
4. Java Classes and Methods

This section provides a structured walkthrough of every Java class implemented in the Finora Maven web application. The codebase is organised into five logical layers:

utility classes, data access objects (DAOs), filter classes, servlet controllers, and a dashboard sub-system. Each layer is described below with method-level detail, supported by code summaries and screenshot markers.

4.1 Overall Class Diagram

The diagram below illustrates the top-level package structure and the relationships between the major components of the Finora application. The architecture follows a clean MVC (Model-View-Controller) separation with a dedicated DAO layer sitting between the controllers and the database.



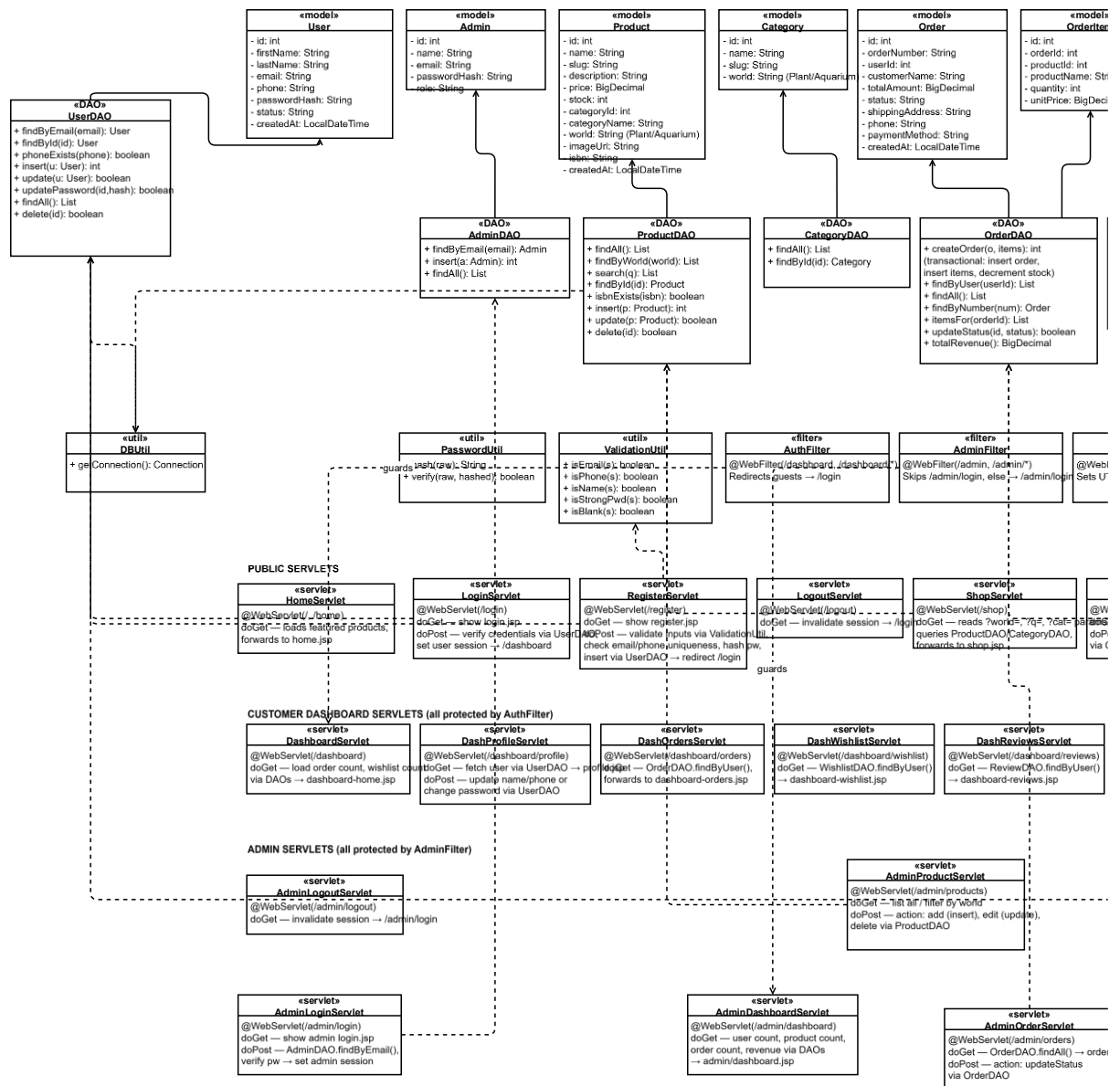


Table 13

Package	Classes	Responsibility
com.finora.model	9	Data transfer objects: User, Admin, Product, Order, OrderItem, CartItem, Category, Review, WishListItem, ContactMessage
com.finora.dao	9	JDBC data access — one DAO per domain entity
com.finora.util	3	DBUtil, PasswordUtil, ValidationUtil
com.finora.filter	3	AuthFilter, AdminFilter, CharsetFilter
com.finora.listener	1	AppContextListener
com.finora.controller	12	Public-facing page and action servlets
com.finora.controller.admin	8	Admin panel servlets
com.finora.controller.dashboard	7	Authenticated user dashboard servlets

4.2 DAO Methods

The DAO layer is responsible for all interaction with the MySQL database. Every DAO class opens and closes its own JDBC connection through `DBUtil.getConnection()`, uses `PreparedStatement` to prevent SQL injection, and maps `ResultSet` rows into the corresponding model object via a private `map()` helper method.

4.2.1 UserDAO

UserDAO handles all customer account operations. It is used by `LoginServlet`, `RegisterServlet`, and the various dashboard controllers.

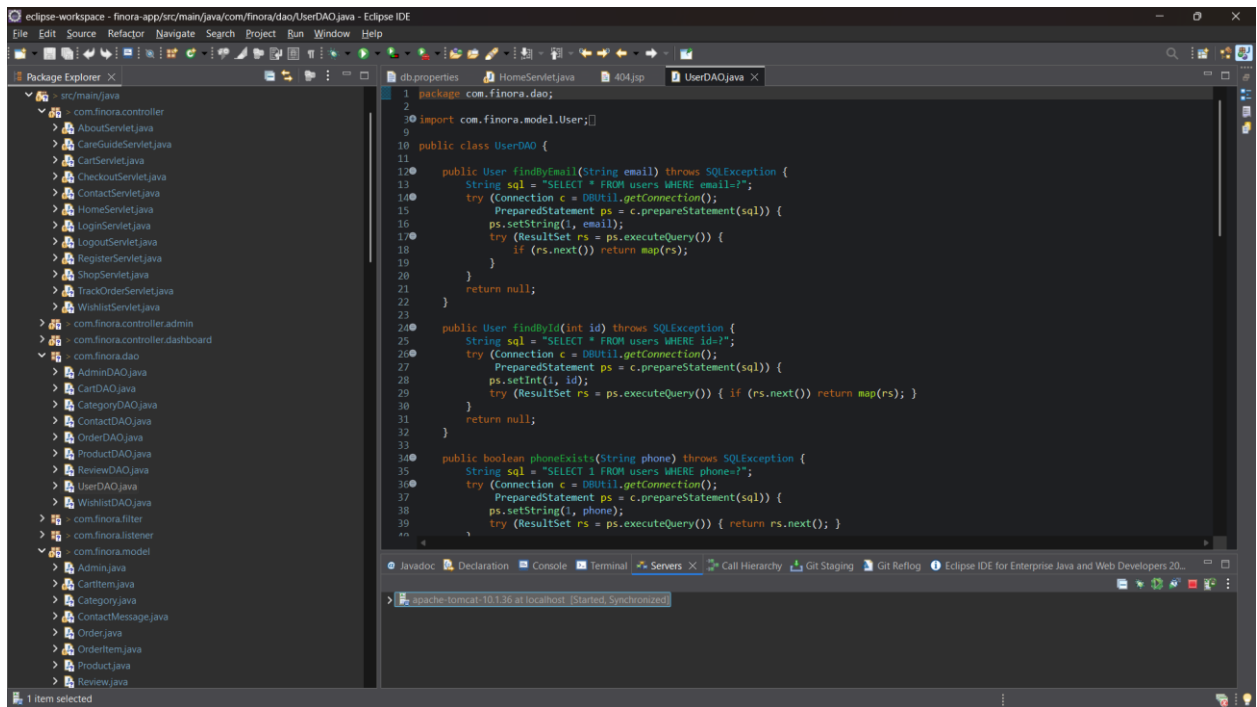


Table 14

Method	Return Type	Parameters	Description
findByEmail()	User	String email	Queries the users table by email address. Returns the mapped User object or null if not found. Used by LoginServlet during authentication.
findById()	User	int id	Retrieves a single user record by primary key. Used by dashboard controllers to reload fresh user state.
phoneExists()	boolean	String phone	Checks whether a phone number already exists in the table. Prevents duplicate registration with the same mobile number.
insert()	int	User u	Inserts a new user row with hashed password and status='active'. Returns the auto-generated primary key. Called by RegisterServlet.
update()	boolean	User u	Updates first_name, last_name, and phone for an existing user. Returns

Method	Return Type	Parameters	Description
			true if one or more rows were affected.
updatePassword()	boolean	int userId, String hash	Sets a new bcrypt hash for the given user ID. Used from the profile settings change-password flow.
findAll()	List<User>	—	Returns every user row ordered by id DESC. Consumed by AdminCustomerServlet to populate the customer management table.
delete()	boolean	int id	Hard-deletes the user record by primary key. Restricted to admin operations.
map() (private)	User	ResultSet rs	Internal helper that reads column values from the current ResultSet row and returns a fully populated User bean.

4.2.2 AdminDAO

AdminDAO manages the admins table. It is significantly simpler than UserDao because admin accounts are only created by a super-admin and are never self-registered.

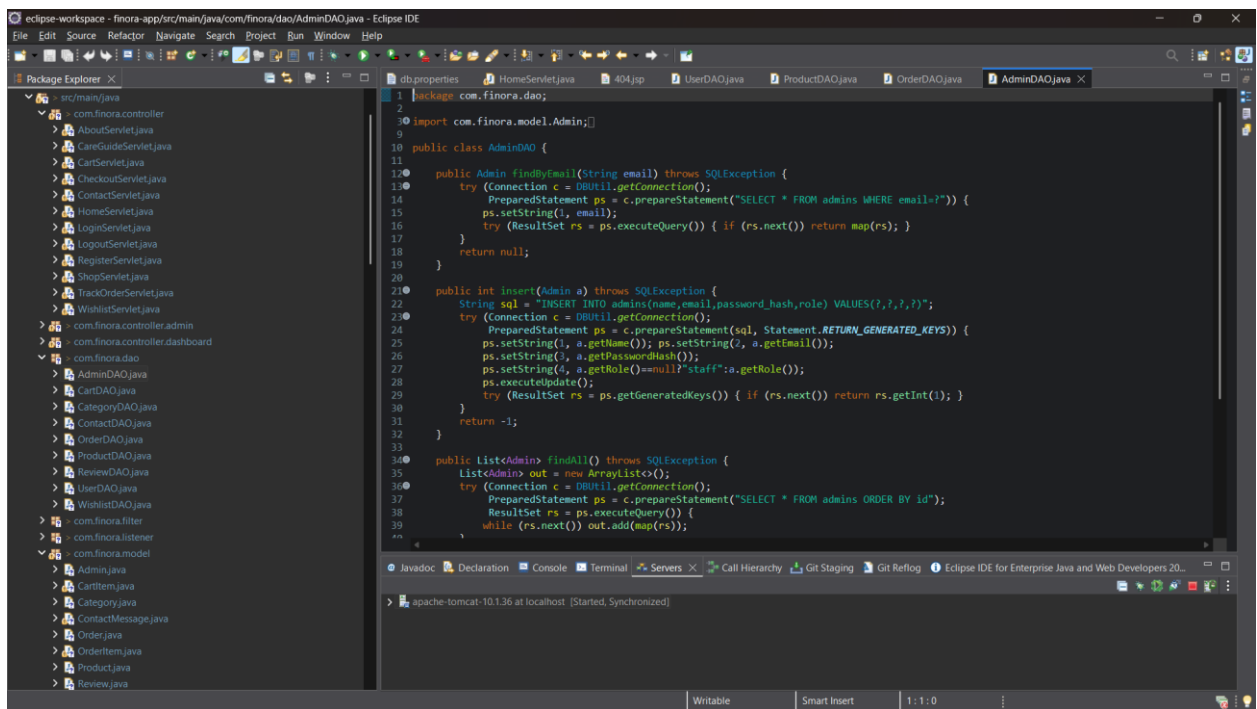


Table 15

Method	Return Type	Description
findByEmail()	Admin	Looks up an admin by email. Called by AdminLoginServlet to verify credentials.
insert()	int	Creates a new admin with a hashed password and role (super or staff). Called by AdminAddAdminServlet.
findAll()	List<Admin>	Retrieves all admin records ordered by id. Used in the add-admin management view.
map() (private)	Admin	Maps the current ResultSet row to an Admin bean.

4.2.3 ProductDAO

ProductDAO is the most query-heavy DAO in the system. It always JOINS the categories table to include category names in the result set, avoiding extra round-trips from the view layer.

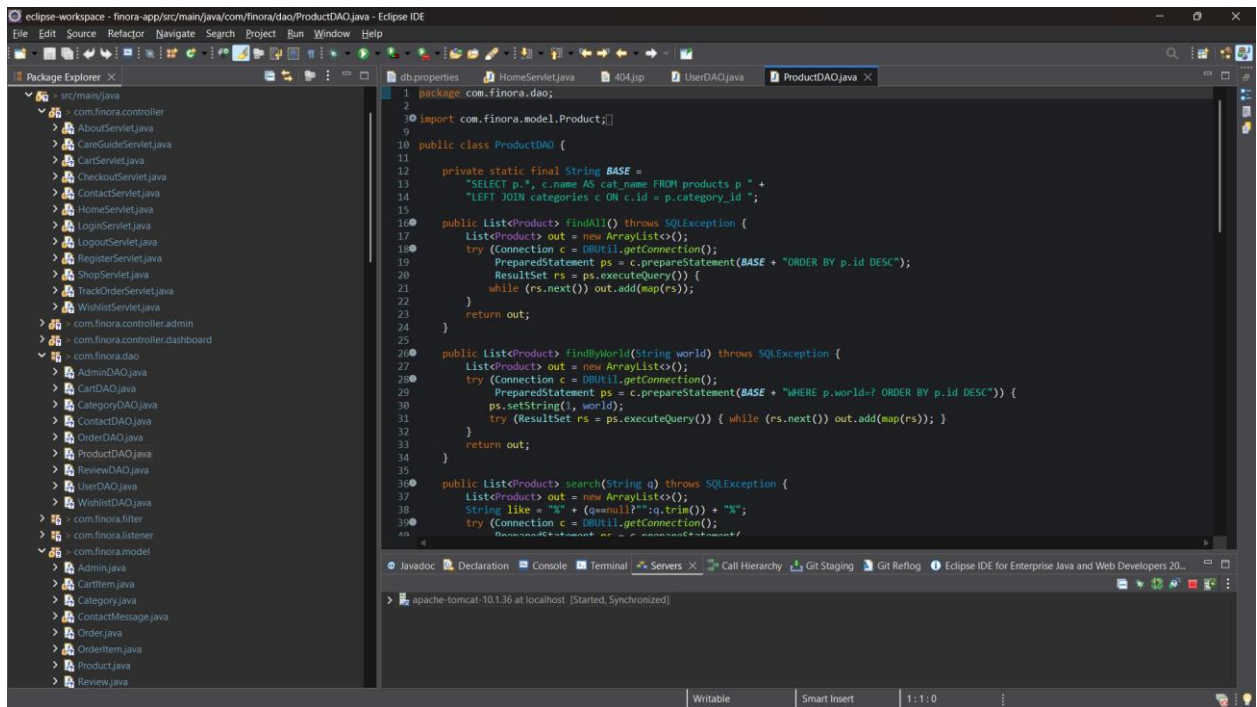


Table 16

Method	Return Type	Description
findAll()	List<Product>	Returns all products joined with categories, ordered newest-first. Used by HomeServlet and AdminProductServlet.
findByWorld()	List<Product>	Filters products by world column ('plants' or 'aquarium'). Supports the dual-mode shop feature.
search()	List<Product>	Full-text LIKE search across product name, isbn, and category name. Drives the shop search bar.
findById()	Product	Retrieves a single product with its category. Used when editing a product in admin.
isbnExists()	boolean	Prevents duplicate ISBNs when adding new products.
insert()	int	Persists a new product and returns the generated ID.
update()	boolean	Updates all mutable product fields in-place.
delete()	boolean	Hard-deletes a product by ID. Admin-only.

4.2.4 CategoryDAO

CategoryDAO is a lightweight, read-mostly DAO. Categories are pre-seeded in the database and are only ever displayed, not created at runtime through the application.

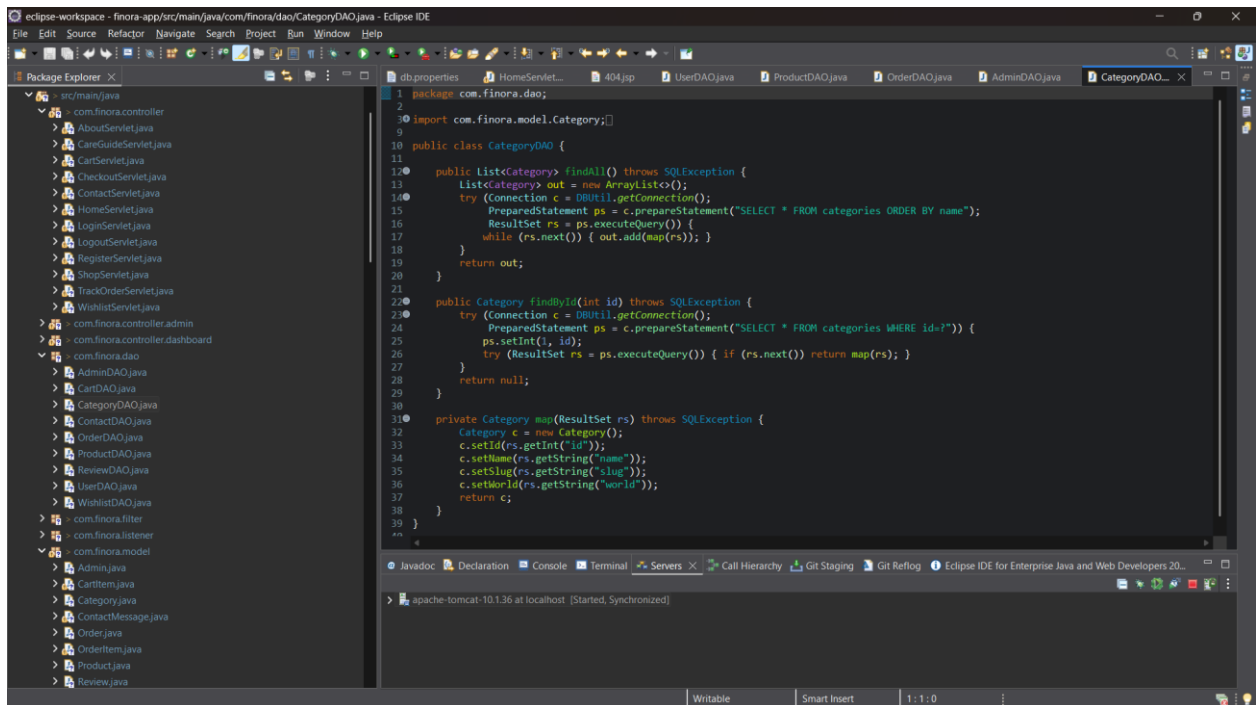


Table 17

Method	Return Type	Description
findAll()	List<Category>	Returns all category rows ordered alphabetically. Used in ShopServlet and AdminProductServlet.
findById()	Category	Fetches a single category by primary key. Useful when displaying a category name on a product detail page.

4.2.5 CartDAO

CartDAO manages the cart_items table. It uses MySQL's ON DUPLICATE KEY UPDATE syntax to handle the common case of a user adding a product that is already in the cart, incrementing the quantity rather than inserting a duplicate row.

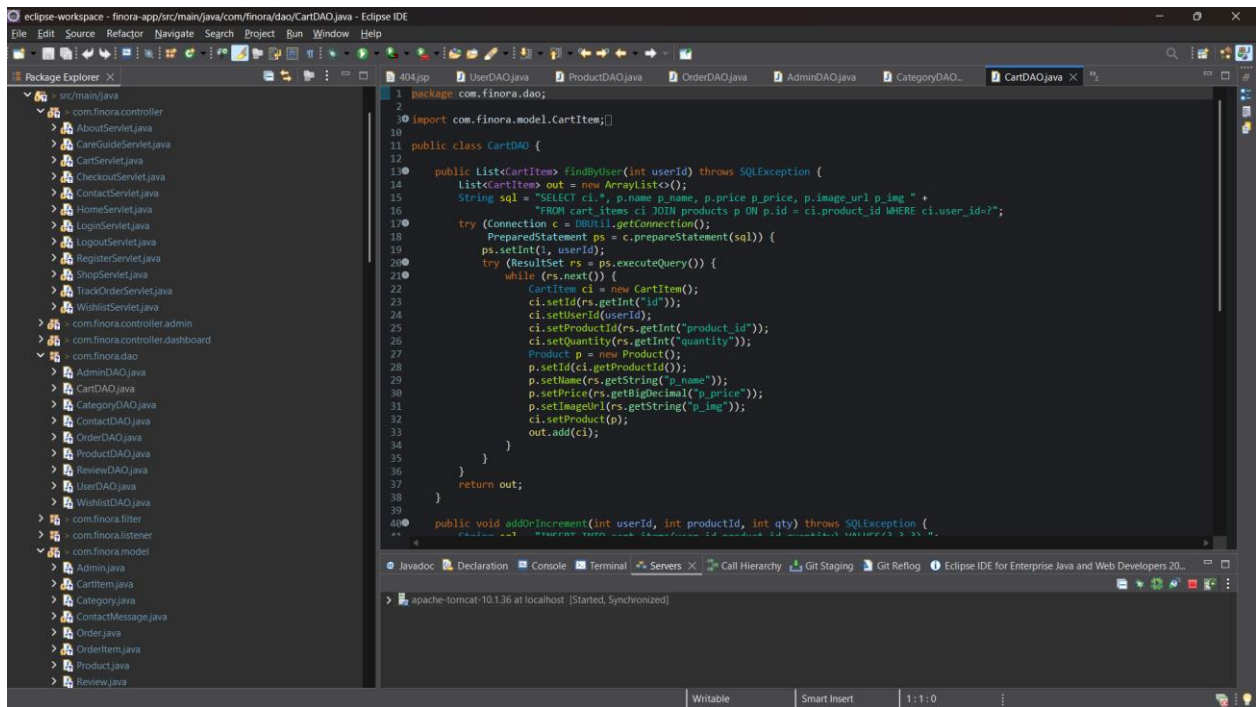


Table 18

Method	Return Type	Description
findByUser()	List<CartItem>	Retrieves all cart rows for a user, JOINing product name, price, and image so the view needs no additional queries.
addOrIncrement()	void	Inserts a new cart_items row, or increments quantity if (user_id, product_id) already exists — handled atomically with ON DUPLICATE KEY UPDATE.
remove()	void	Deletes the cart row matching a specific user and product.
clear()	void	Deletes all cart rows for a user. Called by CheckoutServlet after a successful order is placed.
countForUser()	int	Returns the total item quantity in the cart using SUM(quantity). Used to display the cart badge count in the navbar.

4.2.6 OrderDAO

OrderDAO is the most complex DAO in the system. Its createOrder() method is wrapped in a JDBC transaction — if any step fails (order header insert, order items batch insert, or stock decrement), the entire operation is rolled back to maintain data integrity.

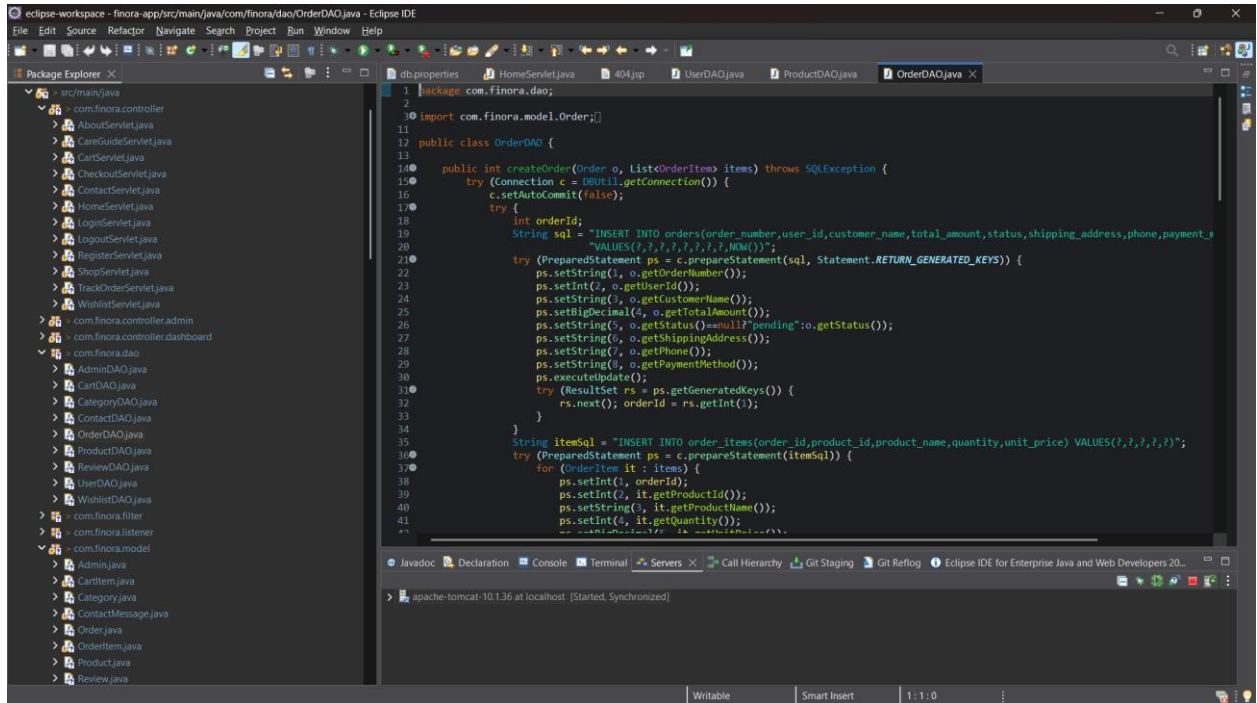


Table 19

Method	Return Type	Description
createOrder()	int	Transactional method: inserts the order header, batches all order_items rows, and decrements stock using GREATEST(stock-qty, 0). Rolls back on any exception.
findByUser()	List<Order>	Fetches all orders for a customer ordered newest-first. Used in the user dashboard orders page.
findAll()	List<Order>	Returns every order in the system. Used by AdminOrderServlet and AdminDashboardServlet.
findByNumber()	Order	Looks up an order by its human-readable order number (e.g. FIN-1716000000000). Used in the track-order feature.
itemsFor()	List<OrderItem>	Retrieves all line items belonging to a given order ID.

Method	Return Type	Description
updateStatus()	boolean	Changes an order's status field. Called by AdminOrderServlet when the admin updates an order (e.g. pending → shipped).
totalRevenue()	BigDecimal	Aggregates total_amount for all non-cancelled orders. Feeds the revenue stat card on the admin dashboard.

4.2.7 ReviewDAO

ReviewDAO handles customer product reviews. All queries JOIN users and products to avoid extra lookups in the view layer.

```

1 package com.finora.dao;
2
3 import com.finora.model.Review;
4
5 public class ReviewDAO {
6
7     public List<Review> findAll() throws SQLException {
8         List<Review> out = new ArrayList<>();
9         String sql = "SELECT r.*, u.first_name fn, u.last_name ln, p.name pn FROM reviews r " +
10             "LEFT JOIN users u ON u.id=r.user_id LEFT JOIN products p ON p.id=r.product_id ORDER BY r.id DESC";
11         try (Connection c = DBUtil.getConnection();
12             PreparedStatement ps = c.prepareStatement(sql)) {
13             ResultSet rs = ps.executeQuery();
14             while (rs.next()) out.add(map(rs));
15         }
16         return out;
17     }
18
19     public List<Review> findByUser(int userId) throws SQLException {
20         List<Review> out = new ArrayList<>();
21         String sql = "SELECT r.*, u.first_name fn, u.last_name ln, p.name pn FROM reviews r " +
22             "LEFT JOIN users u ON u.id=r.user_id LEFT JOIN products p ON p.id=r.product_id WHERE r.user_id=? ORDER BY r.id DESC";
23         try (Connection c = DBUtil.getConnection();
24             PreparedStatement ps = c.prepareStatement(sql)) {
25             ps.setInt(1, userId);
26             ResultSet rs = ps.executeQuery();
27             while (rs.next()) out.add(map(rs));
28         }
29         return out;
30     }
31
32     public int insert(Review r) throws SQLException {
33         String sql = "INSERT INTO reviews(user_id,product_id,rating,comment,created_at) VALUES(?,?,,?,NOW())";
34         try (Connection c = DBUtil.getConnection();
35             PreparedStatement ps = c.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
36             ps.setInt(1, r.user_id);
37             ps.setInt(2, r.product_id);
38             ps.setFloat(3, r.rating);
39             ps.setString(4, r.comment);
40             ps.executeUpdate();
41             ResultSet rs = ps.getGeneratedKeys();
42             if (rs.next()) return rs.getInt(1);
43         }
44         return -1;
45     }
46 }

```

Table 20

Method	Return Type	Description
findAll()	List<Review>	Returns all reviews joined with user full name and product name, ordered newest-first. Used by AdminReviewServlet.
findByUser()	List<Review>	Returns reviews posted by a specific user. Shown on the user dashboard reviews tab.

Method	Return Type	Description
insert()	int	Persists a new review with rating and comment, timestamped at insertion.

4.2.8 WishlistDAO

WishlistDAO manages the wishlist_items table. The add() method uses INSERT IGNORE to silently handle duplicate additions rather than throwing a constraint violation.

```

1 package com.finora.dao;
2
3 import com.finora.model.Product;
4
5 public class WishlistDAO {
6
7     public List<WishlistItem> findByUser(int userId) throws SQLException {
8         List<WishlistItem> out = new ArrayList<>();
9         String sql = "SELECT w.*, p.name p_name, p.price p_price, p.image_url p_img " +
10             "FROM wishlist_items w JOIN products p ON p.id=w.product_id WHERE w.user_id=?";
11         try (Connection c = DBUtil.getConnection();
12             PreparedStatement ps = c.prepareStatement(sql)) {
13             ps.setInt(1, userId);
14             try (ResultSet rs = ps.executeQuery()) {
15                 while (rs.next()) {
16                     WishlistItem w = new WishlistItem();
17                     w.setId(rs.getInt("id"));
18                     w.setUserid(userId);
19                     w.setProductId(rs.getInt("product_id"));
20                     Product p = new Product();
21                     p.setId(w.getProductId());
22                     p.setName(rs.getString("p_name"));
23                     p.setPrice(rs.getBigDecimal("p_price"));
24                     p.setImageUrl(rs.getString("p_img"));
25                     w.setProduct(p);
26                     out.add(w);
27                 }
28             }
29         }
30         return out;
31     }
32
33     public void add(int userId, int productId) throws SQLException {
34         try (Connection c = DBUtil.getConnection();
35             PreparedStatement ps = c.prepareStatement("INSERT IGNORE INTO wishlist_items (user_id, product_id) VALUES (?, ?)")) {
36             ps.setInt(1, userId);
37             ps.setInt(2, productId);
38             ps.executeUpdate();
39         }
40     }
41 }

```

Table 21

Method	Return Type	Description
findByUser()	List<WishlistItem>	Fetches all wishlist rows for a user, joined with product details. Used in WishlistServlet and all dashboard controllers.
add()	void	Inserts a wishlist row using INSERT IGNORE — if the (user_id, product_id) pair already exists the statement does nothing.
remove()	void	Deletes the wishlist row matching user and product. Called by WishlistServlet on remove action.

4.2.9 ContactDAO

ContactDAO persists messages submitted through the Contact Us form into the `contact_messages` table. It is intentionally simple messages are stored for admin review and no automated email is triggered.

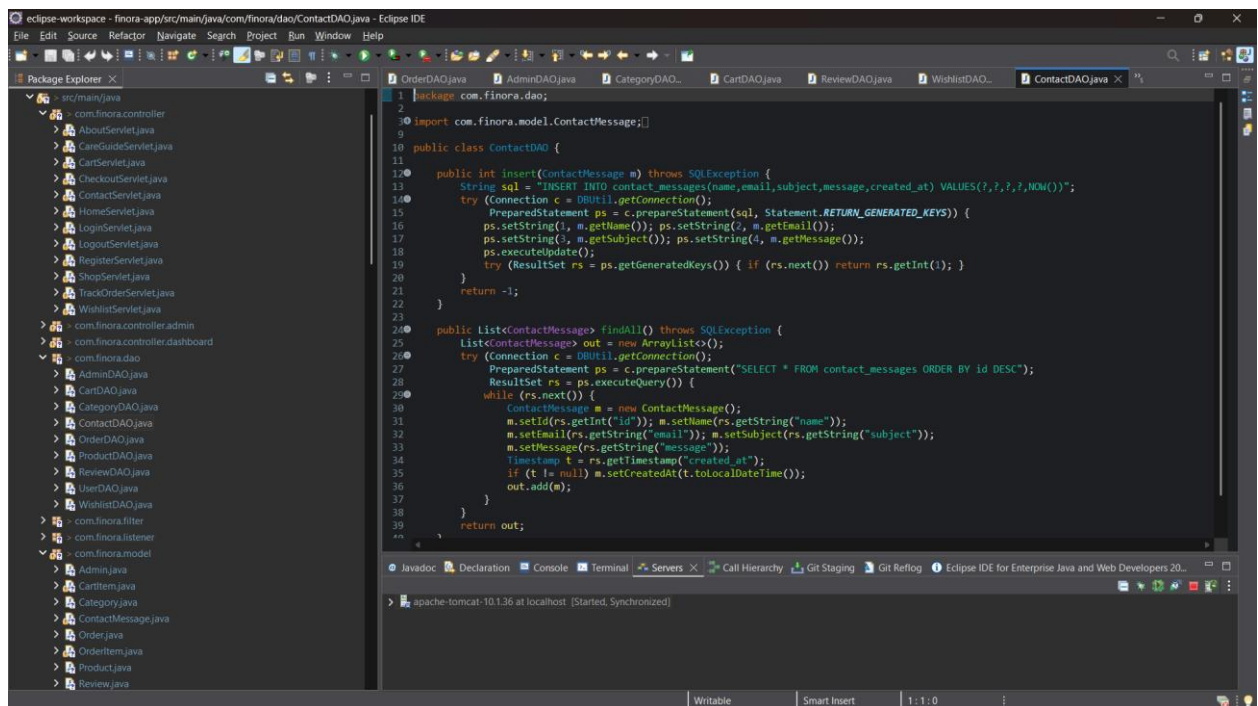


Table 22

Method	Return Type	Description
<code>insert()</code>	<code>int</code>	Saves a new contact message with name, email, subject, message, and current timestamp. Returns the generated ID.
<code>findAll()</code>	<code>List<ContactMessage></code>	Returns all contact messages newest-first. Available for future admin message-inbox view.

4.3 Utility Classes

Three utility classes provide shared infrastructure used across the entire application. All three are declared `final` with private constructors to prevent instantiation — they expose only static methods.

4.3.1 PasswordUtil

PasswordUtil wraps the jBCrypt library to provide consistent, one-way password hashing throughout the application. Passwords are never stored in plain text; only the bcrypt hash is persisted in the database.

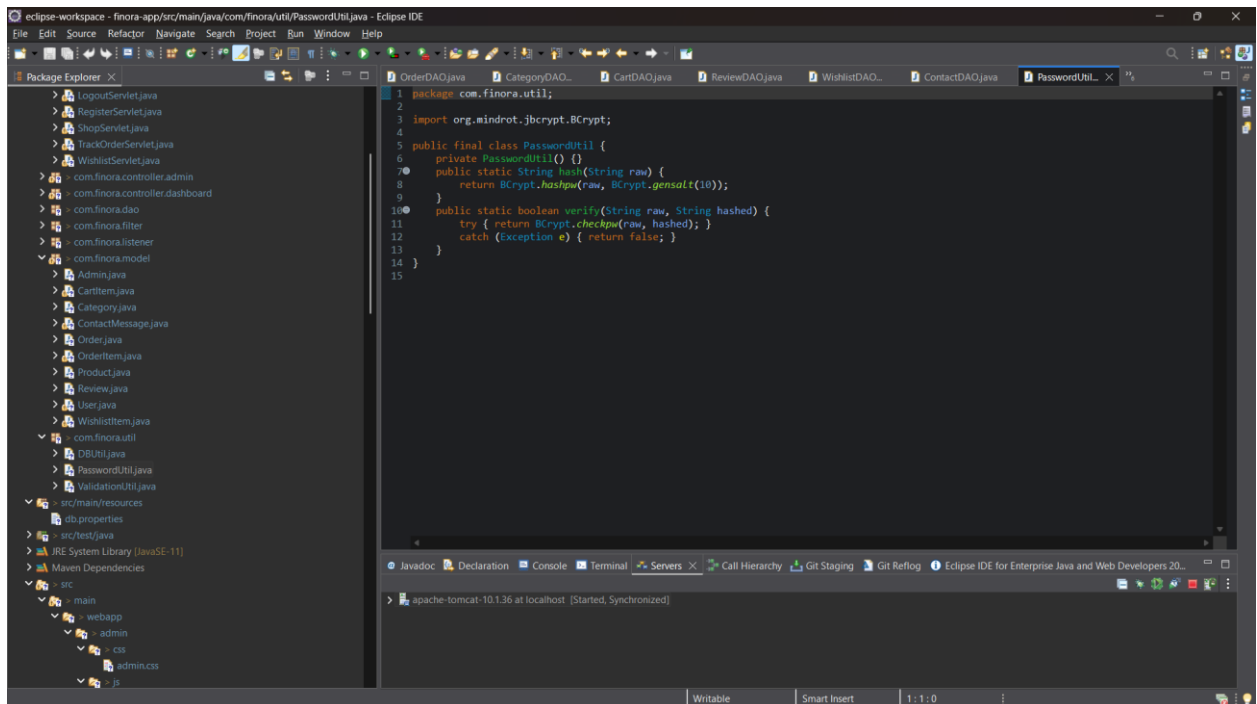


Table 23

Method	Signature	Description
hash()	static String hash(String raw)	Hashes a plain-text password using BCrypt.hashpw() with a cost factor of 10. The salt is automatically generated and embedded in the returned hash string.
verify()	static boolean verify(String raw, String hashed)	Compares a plain-text password against a stored bcrypt hash using BCrypt.checkpw(). Returns false on any exception, preventing error leakage.

4.3.2 DBUtil

DBUtil acts as a centralised JDBC connection factory. Rather than hardcoding database credentials in each DAO, the application reads them from a properties file (src/main/resources/db.properties) that is loaded once in a static initialiser block.

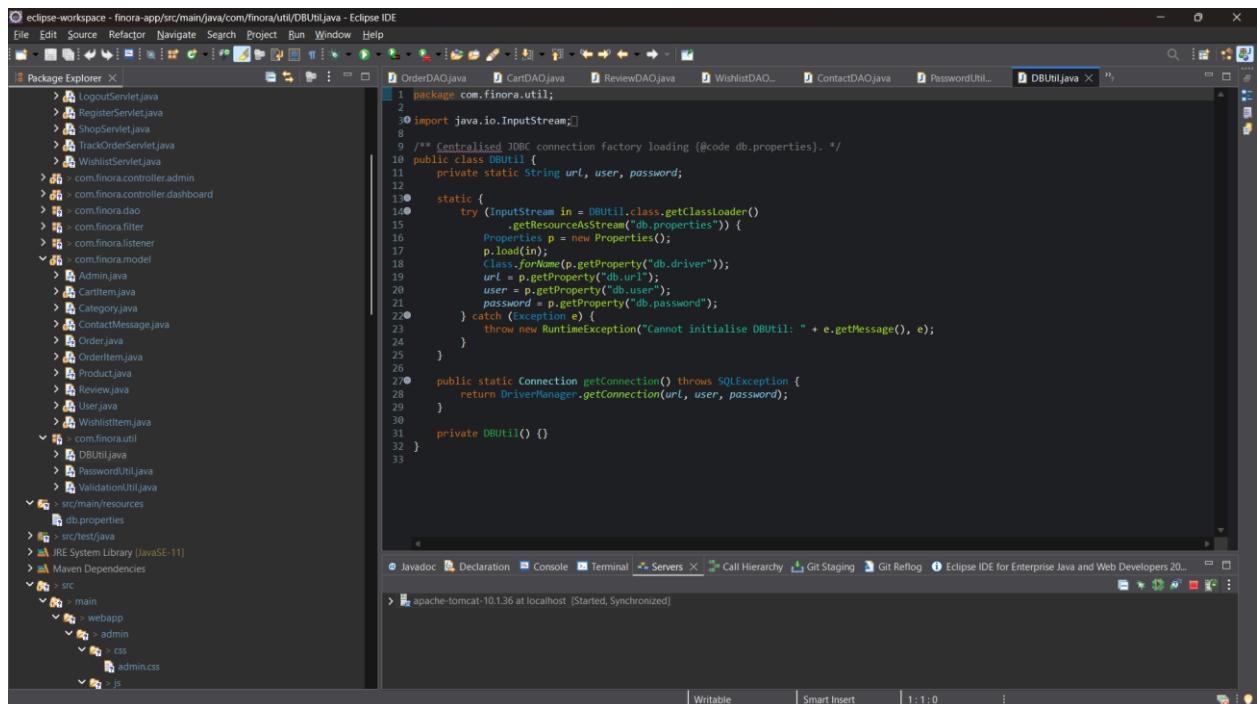


Table 24

Method / Block	Description
static { } initialiser	Runs once when the class is first loaded. Reads db.properties from the classpath, registers the JDBC driver via Class.forName(), and stores the URL, username, and password as private static fields. Throws RuntimeException on failure so the application fails fast at startup rather than at query time.
getConnection()	Opens and returns a new java.sql.Connection using the pre-loaded credentials. Each DAO is responsible for closing the connection in a try-with-resources block.

4.3.3 ValidationUtil

ValidationUtil provides lightweight, regex-based input validators used by RegisterServlet and ContactServlet. Centralising validation here keeps individual servlets concise and ensures consistent rules across the application.

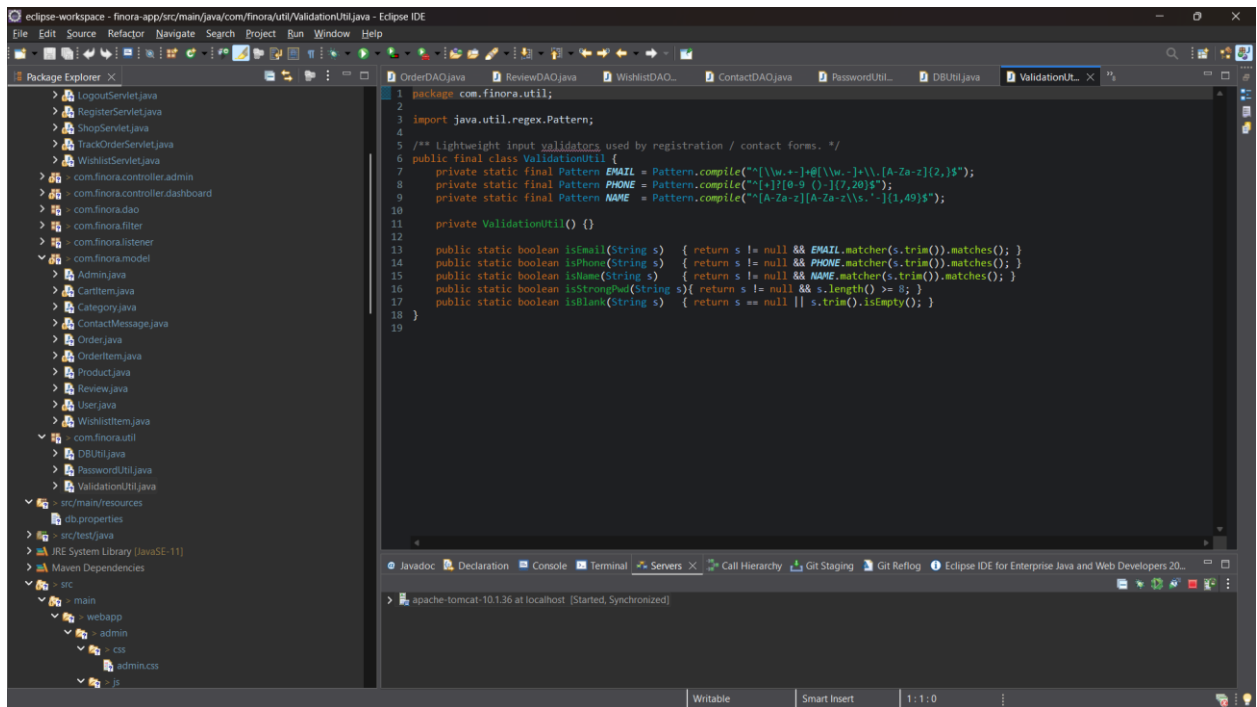


Table 25

Method	Regex / Rule	Used By
isEmail()	<code>^[\\w.+]+@[\\w.-]+\\. [A-Za-z]{2,}\$</code>	RegisterServlet, LoginServlet, ContactServlet, AdminLoginServlet, AdminAddAdminServlet
isPhone()	<code>^[+]?[0-9 ()-]{7,20}\$</code>	RegisterServlet
isName()	<code>^[A-Za-z][A-Za-z\\s.-]{1,49}\$</code>	RegisterServlet
isStrongPwd()	<code>length >= 8</code>	RegisterServlet, AdminAddAdminServlet
isBlank()	<code>null or empty after trim()</code>	ContactServlet, AdminProductServlet

4.4 Controller Classes (Servlets)

The controller layer consists of 27 HttpServlet subclasses divided across three sub-packages. Every servlet is annotated with `@WebServlet` and declares its URL pattern, so no servlet

mappings are required in web.xml. All servlets follow the same two-method convention described below.

Table 26

Sub-package	Count	URL Pattern	Scope
com.finora.controller	12	/login, /register, /logout, /home, /shop, /cart, /checkout, /wishlist, /track- order, /about, /contact, /care-guide	Public-facing pages and actions
com.finora.controller.admin	8	/admin/login, /admin/logout, /admin/dashboard, /admin/products, /admin/orders, /admin/customers, /admin/reviews, /admin/analytics, /admin/add-admin, /admin/settings	Admin panel — protected by AdminFilter
com.finora.controller.dashboard	7	/dashboard, /dashboard/profile, /dashboard/orders, /dashboard/wishlist, /dashboard/reviews, /dashboard/notifications, /dashboard/settings	Authenticated user area — protected by AuthFilter

4.4.1 doGet: Handling Page Load Requests

The doGet() method handles HTTP GET requests — typically when a user navigates to a page. The general pattern used across the application is:

```

13 @WebServlet("/login")
14 public class LoginServlet extends HttpServlet {
15     private final UserDao dao = new UserDao();
16
17     @Override protected void doGet(HttpServletRequest req, HttpServletResponse res)
18         throws ServletException, IOException {
19         HttpSession s = req.getSession();
20         Object ok = s.getAttribute("success");
21         if (ok != null) { req.setAttribute("success", ok); s.removeAttribute("success"); }
22         req.getRequestDispatcher("/WEB-INF/views/login.jsp").forward(req, res);
23     }
24
25     @Override protected void doPost(HttpServletRequest req, HttpServletResponse res)
26         throws ServletException, IOException {
27         String email = req.getParameter("email");
28         String pwd = req.getParameter("password");
29         if (!ValidationUtil.isValid(email) || ValidationUtil.isBlank(pwd)) {
30             req.setAttribute("error", "Please enter a valid email and password.");
31             req.getRequestDispatcher("/WEB-INF/views/login.jsp").forward(req, res); return;
32         }

```

- Load any data required by the view from the DAO layer.
- Set the data as request attributes using req.setAttribute().
- Forward the request to the appropriate JSP view via req.getRequestDispatcher(...).forward().

For servlets where no database query is needed (AboutServlet, CareGuideServlet, TrackOrderServlet), doGet() simply forwards directly to the JSP with no intermediate logic. The table below documents every servlet's doGet() behaviour:

Table 27

Servlet	URL	doGet() Behaviour
LoginServlet	/login	Checks session for a flash success message (set by RegisterServlet after account creation), copies it to request scope, then forwards to login.jsp.
RegisterServlet	/register	No data loading needed. Forwards directly to register.jsp.
LogoutServlet	/logout	Invalidates the session and redirects to /home. No view rendered.
HomeServlet	/, /home	Calls ProductDAO.findAll() and sets result as featuredProducts. Forwards to home.jsp.
ShopServlet	/shop	Reads optional q (search) and world (filter) query parameters. Delegates to ProductDAO.search(), findByWorld(), or findAll() accordingly. Also loads all categories. Forwards to shop.jsp.

Servlet	URL	doGet() Behaviour
CartServlet	/cart	Redirects to /login if no session user. Otherwise loads CartDAO.findByUser() and forwards to the dashboard cart view.
WishlistServlet	/wishlist	Loads WishlistDAO.findByUser() if a user is in session. Forwards to wishlist.jsp. Gracefully shows an empty wishlist for guests.
TrackOrderServlet	/track-order	Forwards directly to track-order.jsp. Order lookup is not yet implemented server-side.
AboutServlet	/about	Forwards to about.jsp. Static content only.
ContactServlet	/contact	Forwards to contact.jsp. Static form, no preloaded data.
CareGuideServlet	/care-guide	Forwards to care-guide.jsp. Static content.
CheckoutServlet	/checkout	No doGet() — checkout only accepts POST.

4.4.2 doPost: Handling Form Submissions

doPost() handles HTTP POST requests — form submissions, add-to-cart actions, checkout, and admin data mutations. The general pattern is:

```

24
25
26 @Override protected void doPost(HttpServletRequest req, HttpServletResponse res)
27     throws ServletException, IOException {
28     String email = req.getParameter("email");
29     String pwd = req.getParameter("password");
30     if (!ValidationUtil.isEmail(email) || ValidationUtil.isBlank(pwd)) {
31         req.setAttribute("error", "Please enter a valid email and password.");
32         req.getRequestDispatcher("/WEB-INF/views/login.jsp").forward(req, res); return;
33     }
34     try {
35         User u = dao.findByEmail(email.trim().toLowerCase());
36         if (u == null || !PasswordUtil.verify(pwd, u.getPasswordHash())) {
37             req.setAttribute("error", "Invalid email or password.");
38             req.getRequestDispatcher("/WEB-INF/views/login.jsp").forward(req, res); return;
39         }
40         HttpSession s = req.getSession(true);
41         s.setAttribute("user", u);
42         res.sendRedirect(req.getContextPath() + "/dashboard");
43     } catch (Exception e) {
44         req.setAttribute("error", "Login error: " + e.getMessage());
45         req.getRequestDispatcher("/WEB-INF/views/login.jsp").forward(req, res);
46     }
47 }

```

1. Read and trim form parameters from req.getParameter().
2. Validate inputs either with ValidationUtil or inline logic.

3. On validation failure: set an error attribute and re-render the form via forward().
4. On success: call the relevant DAO method and redirect (Post/Redirect/Get pattern) to prevent duplicate submissions.

Table 28

Servlet	URL	doPost() Behaviour
LoginServlet	/login	Validates email format and non-blank password. Calls UserDAO.findByEmail() then PasswordUtil.verify(). On success sets user in session and redirects to /dashboard. On failure re-renders login.jsp with an error message.
RegisterServlet	/register	Validates name, email, phone, and password via ValidationUtil. Checks for duplicate email and phone via DAO. On success hashes the password, calls UserDAO.insert(), stores a flash success in session, then redirects to /login.
CartServlet	/cart	Reads action ('remove' or implicit add) and productId. Calls CartDAO.remove() or CartDAO.addOrIncrement() accordingly. Redirects back to /cart after each operation.
WishlistServlet	/wishlist	Reads action and productId. Calls WishlistDAO.remove() or WishlistDAO.add(). Redirects back to /wishlist.
CheckoutServlet	/checkout	Retrieves cart items, calculates total, constructs an Order with a timestamped order number (FIN-{millis}), calls OrderDAO.createOrder() in a transaction, then clears the cart and redirects to /dashboard/orders.
ContactServlet	/contact	Validates name, email, and message. On success calls ContactDAO.insert() and sets a success attribute. Re-renders the contact form with either a success message or error.

4.4.3 Admin Servlet Methods

Admin servlets follow the same doGet/doPost pattern but are protected by AdminFilter. The table below covers both HTTP methods for each admin controller.

Table 29

Servlet	URL	doGet()	doPost()
AdminLoginServlet	/admin/login	Forwards to admin/login.jsp.	Verifies credentials via AdminDAO and PasswordUtil. Sets admin in session. Redirects to /admin/dashboard.
AdminLogoutServlet	/admin/logout	Removes admin from session. Redirects to /admin/login.	N/A
AdminDashboardServlet	/admin/dashboard	Loads productCount, orderCount, userCount, totalRevenue, and recentOrders. Forwards to admin/dashboard.jsp.	N/A
AdminProductServlet	/admin/products	Loads all products and categories. Forwards to admin/products.jsp.	Action='delete': calls ProductDAO.delete(). Action='update': calls ProductDAO.update(). Default (add): validates, checks ISBN uniqueness, calls ProductDAO.insert(). Redirects to /admin/products.
AdminOrderServlet	/admin/orders	Loads all orders via OrderDAO.findAll(). Forwards to admin/orders.jsp.	Reads id and status. Calls OrderDAO.updateStatus(). Redirects to /admin/orders.
AdminCustomerServlet	/admin/customers	Loads all users via UserDAO.findAll(). Forwards to admin/customers.jsp.	Action='delete': calls UserDAO.delete(). Redirects to /admin/customers.

Servlet	URL	doGet()	doPost()
AdminReviewServlet	/admin/reviews	Loads all reviews via ReviewDAO.findAll(). Forwards to admin/reviews.jsp.	N/A — review management is read-only in current implementation.
AdminAnalyticsServlet	/admin/analytics	Forwards to admin/analytics.jsp. Chart data is rendered from session stats.	N/A
AdminAddAdminServlet	/admin/add-admin	Forwards to admin/add-admin.jsp.	Validates email and password. Checks for duplicate email. Creates and inserts new Admin via AdminDAO.insert(). Sets success or error attribute and re-renders the form.
AdminSettingsServlet	/admin/settings	Forwards to admin/settings.jsp.	N/A — static settings page.

```

1 package com.finora.controller.admin;
2
3 import com.finora.dao.AdminDAO;
4
5 @WebServlet("/admin/add-admin")
6 public class AdminAddAdminServlet extends HttpServlet {
7     private final AdminDAO dao = new AdminDAO();
8
9     @Override protected void doGet(HttpServletRequest req, HttpServletResponse res)
10         throws ServletException, IOException {
11         req.getRequestDispatcher("/WEB-INF/views/admin/add-admin.jsp").forward(req, res);
12     }
13
14     @Override protected void doPost(HttpServletRequest req, HttpServletResponse res)
15         throws ServletException, IOException {
16         String name = req.getParameter("name");
17         String email = req.getParameter("email");
18         String pwd = req.getParameter("password");
19         String role = req.getParameter("role");
20
21         try {
22             if (!ValidationUtil.isEmail(email) || !ValidationUtil.isStrongPwd(pwd)) {
23                 req.setAttribute("error", "Invalid email or password too short.");
24             } else if (dao.findByEmail(email) != null) {
25                 req.setAttribute("error", "Admin already exists.");
26             } else {
27                 Admin a = new Admin();
28                 a.setName(name);
29                 a.setEmail(email.toLowerCase());
30                 a.setPasswordHash(PasswordUtil.hash(pwd));
31                 a.setRole(role == null ? "staff" : role);
32                 dao.insert(a);
33                 req.setAttribute("success", "Admin created.");
34             }
35         } catch (Exception e) {
36             req.setAttribute("error", e.getMessage());
37         }
38         req.getRequestDispatcher("/WEB-INF/views/admin/add-admin.jsp").forward(req, res);
39     }
40 }

```

4.4.4 Dashboard Servlet Methods

All dashboard servlets are protected by AuthFilter and follow an identical structural pattern: retrieve the User object from the session, call the three core DAOs (OrderDAO, WishlistDAO, ReviewDAO) to populate sidebar counts and page content, then forward to the relevant JSP.

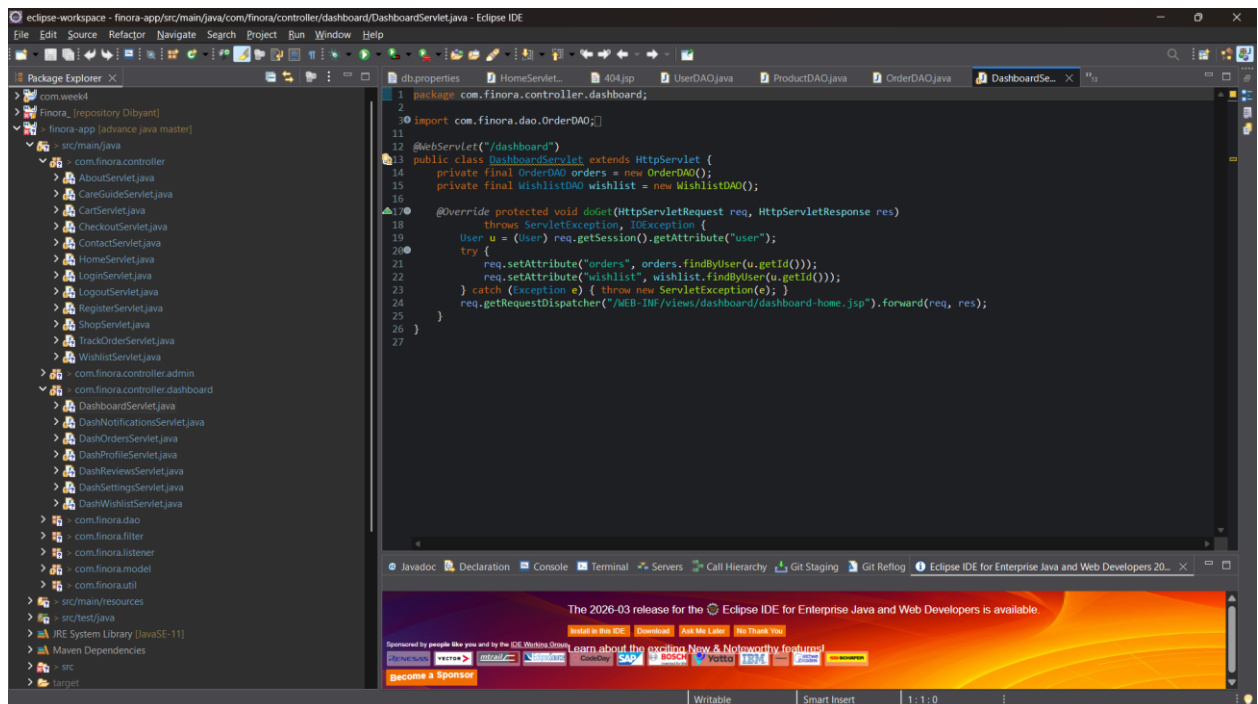


Table 30

Servlet	URL	View Forwarded To	Unique Data Loaded
DashboardServlet	/dashboard	dashboard-home.jsp	orders + wishlist serves as the landing overview page
DashProfileServlet	/dashboard/profile	dashboard-profile.jsp	orders + wishlist + reviews all three for sidebar and profile stats
DashOrdersServlet	/dashboard/orders	dashboard-orders.jsp	orders primary content is the order history table

Servlet	URL	View Forwarded To	Unique Data Loaded
DashWishlistServlet	/dashboard/wishlist	dashboard-wishlist.jsp	wishlist primary content is the saved items grid
DashReviewsServlet	/dashboard/reviews	dashboard-reviews.jsp	reviews primary content is the posted reviews list
DashNotificationsServlet	/dashboard/notifications	dashboard-notifications.jsp	orders notification items derived from recent orders
DashSettingsServlet	/dashboard/settings	dashboard-settings.jsp	orders + wishlist + reviews standard sidebar population

4.4.5 Filter Classes

Three servlet filters are applied to incoming requests before they reach any controller.

Table 31

Filter Class	URL Pattern	doFilter() Logic
CharsetFilter	/*	Sets character encoding to UTF-8 on both request and response for every request. Applied globally to handle international characters correctly.
AuthFilter	/dashboard, /dashboard/*	Checks for a non-null user attribute in the HttpSession. If absent (guest), redirects to /login. Otherwise passes the request down the filter chain.
AdminFilter	/admin, /admin/*	Checks for a non-null admin attribute in the session. Allows the /admin/login path through unconditionally so the login form itself is

Filter Class	URL Pattern	doFilter() Logic
		accessible. Redirects all other unauthenticated admin requests to /admin/login.

```

1 package com.finora.filter;
2
3 import jakarta.servlet.*;
4
5 /** Protects /admin/* (except login). */
6 @WebFilter(urlPatterns = {"/admin", "/admin/*"})
7 public class AdminFilter implements Filter {
8     @Override public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
9         throws IOException, ServletException {
10         HttpServletRequest r = (HttpServletRequest) req;
11         String path = r.getRequestURL().substring(r.getContextPath().length());
12         if (path.startsWith("/admin/login")) { chain.doFilter(req, res); return; }
13         HttpSession s = r.getSession(false);
14         if (s == null || s.getAttribute("admin") == null) {
15             ((HttpServletResponse) res).sendRedirect(r.getContextPath() + "/admin/login");
16             return;
17         }
18         chain.doFilter(req, res);
19     }
20 }

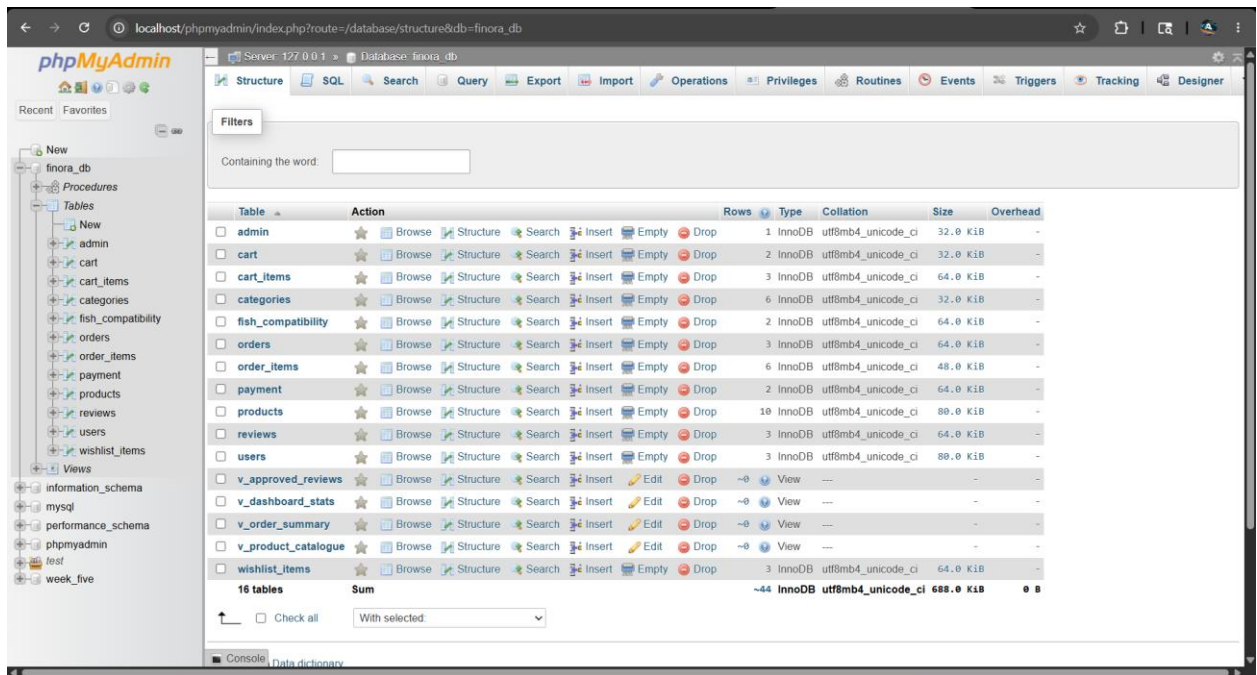
```

4.4.6 Application Listener

The `AppContextListener` class implements `ServletContextListener` and is annotated with `@WebListener`. It fires once when the application starts.

Table 32

Method	Description
<code>contextInitialized()</code>	Attempts to obtain a JDBC connection via <code>DBUtil.getConnection()</code> and logs either "DB connection OK" or a failure message to the servlet context log. This gives immediate startup-time feedback if the database is unreachable, rather than failing silently on the first user request.
<code>contextDestroyed()</code>	No-op in the current implementation. Reserved for future connection pool shutdown.



4.5 Summary

The Finora Java backend is structured around a clean separation of concerns. The utility layer handles cross-cutting concerns (DB connectivity, password security, and input validation). The DAO layer abstracts all SQL behind typed Java methods. Filters enforce security boundaries before requests reach controllers. Servlets act as thin orchestrators — reading parameters, delegating to DAOs, setting view attributes, and forwarding or redirecting. No business logic leaks into the JSP view layer, and no SQL appears in any controller class.

Table 33

Layer	Classes	Key Responsibility
Utilities	DBUtil, PasswordUtil, ValidationUtil	Connection pooling, bcrypt hashing, regex validation
Data Access (DAO)	9 DAO classes	All SQL queries — CRUD + JOINS + transactions
Filters	CharsetFilter, AuthFilter, AdminFilter	Encoding, user auth, admin auth
Listener	ApplicationContextListener	Startup DB connectivity check

Layer	Classes	Key Responsibility
Public Controllers	12 Servlets	Page rendering and form handling for all public URLs
Admin Controllers	8 Servlets	Full CRUD management for the admin panel
Dashboard Controllers	7 Servlets	Authenticated user dashboard sub-pages

5. Test Cases

5.1 Test Plan Table

Below is a summary of all the test cases included:

Table 34

ID	Test Area	Sub-Test	Description	Type
1	User	Validation Test	Form field validations	Black-box

	Registration			
2	User Registration	Registration Process	End-to-end DB insert	Black-box
3	User Login	Authentication Test	Credential checks	Black-box
4	User Login	Session & Cookie Test	Session lifecycle	Black-box
5	Admin CRUD	Create	Add new product	Black-box
6	Admin CRUD	Read	View records	Black-box
7	Admin CRUD	Update	Edit product/order	Black-box
8	Admin CRUD	Delete	Remove product/user	Black-box

Table 35

Test Area	User Registration
Sub- Test	Validation Test
Description	• Tests all the input fields on the registration form with wrong or missing values. • Each test case puts in a bad value like empty fields, wrong email, short password, or wrong phone number. • Checks that the right error message shows up without saving anything to the database.
Status	Pass



"Two worlds — one home.
Begin where you breathe
easiest."

Free delivery on orders above Rs 5,000 inside Kathmandu Valley.

Delivering to Nepal · Prices in NPR [Track order](#) · Worldwide shipping

Create account

Join Finora and bring living things home.

FIRST NAME

LAST NAME

EMAIL

 Please fill out this field.

PHONE

PASSWORD

Create my Finora account

Already with us? [Log in](#)

Table 36

Test Area	User Registration
Sub- Test	Registration Process
Description	• Tests the full registration flow when a user fills in all the correct details. • Checks that the user data gets saved in the database with correct default values like role = Customer and status = Active. • Also checks that trying to set role = Admin from the form doesn't work, and that a success message shows after registration.
Status	Pass



"Two worlds — one home.
Begin where you breathe
easiest."

Free delivery on orders above Rs 5,000 inside Kathmandu Valley.

 Delivering to Nepal · Prices in NPR
 [Track order](#) · Worldwide shipping

Create account

Join Finora and bring living things home.

FIRST NAME

LAST NAME

EMAIL

PHONE

PASSWORD

Create my Finora account

Already with us? [Log in](#)

Table 37

Test Area	User Login
Sub- Test	Authentication Test
Description	• Tests the login process for both regular users and admins. • Checks that correct login works, wrong password or email is rejected with a safe error message, and inactive accounts are blocked. • Also tests security like SQL injection in the email field and brute-force lock after 5 failed attempts.
Status	Pass



Test Area	User Login
Sub- Test	Session & Cookie Test
Description	• Tests how the system manages sessions and cookies after login. • Checks that the session cookie has security flags like HttpOnly and Secure. • Also checks that protected pages redirect to login when no session exists, and that logout properly destroys the session.
Status	Pass

Finora

Plants

Aquarium



Julian Rai

Member since 2024

Plants

Dashboard

My Orders

Wishlist

My Reviews

Profile

Notifications

Settings

Logout

My Profile

Keep your details up to date and manage your account settings.



Julian Rai

VERIFIED

julian.ra@example.com

Member since 12 Jan 2024

Upload New Photo

Remove Photo

JPG, PNG or WEBP • Max 2MB

Personal Details

Edit Details

FIRST NAME

Julian

LAST NAME

Rai

EMAIL ADDRESS

julian.ra@example.com

Verified

PHONE NUMBER

+977 980 123 4567

SHIPPING ADDRESS

Not specified

We use sustainable cookies to ensure you get the best botanical experience on our site.

Accept

Settings

Privacy Policy

Security Settings

Delivering to Nepal • Prices in NPR • Worldwide shipping

Track order

Finora

© 2024 Finora. Sustainable living for a better future. Hand-crafted plants and aquatic ecosystems delivered to your door.

Privacy Policy

Terms of Service

Cookie Settings

Contact Support

Test Area	Admin CRUD
Sub- Test	Create
Description	- Tests that admin can add new Plant and Aquarium products with all required details. - Checks that the database rejects products with a price of 0 or negative stock.

210

	Also checks that a product with stock = 0 gets created but shows as 'Out of Stock' on the website.
Status	Pass

Finora

ADMIN SUITE

OVERVIEW

Dashboard

Analytics

COMMERCE

Products248

Orders37

Customers

Reviews12

SYSTEM

Settings

Help & Docs

Finora / Products

Search products, orders, customers, reviews...

AllPlantsAquarium

Import CSV

+ New product

Products

Manage Plants and Aquarium SKUs, inventory, badges, and pricing in Nepalese Rupees.

Smart suggestion • 3 fish species missing compatibility data

Adding tank-mate guidance helps Aquarium customers complete their first build. Open the affected SKUs to fill in the data.

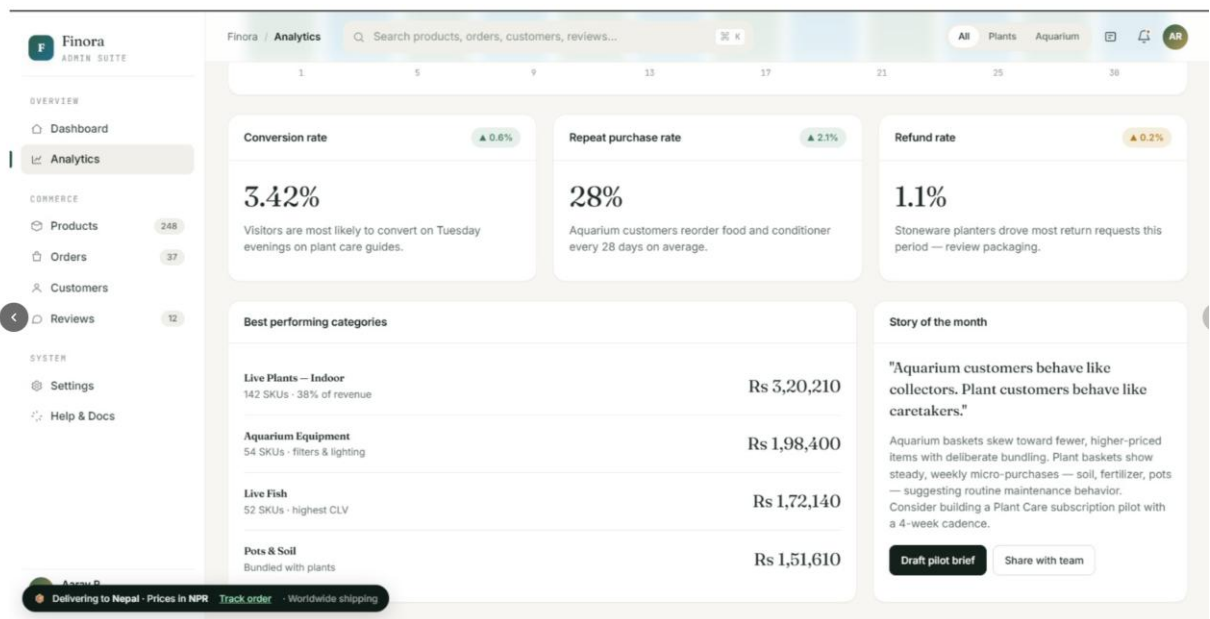
All 248Plants 142Aquarium 186Low stock 12Trending 9Drafts 4

CategorySort: Best selling

3 selectedTagExportArchive

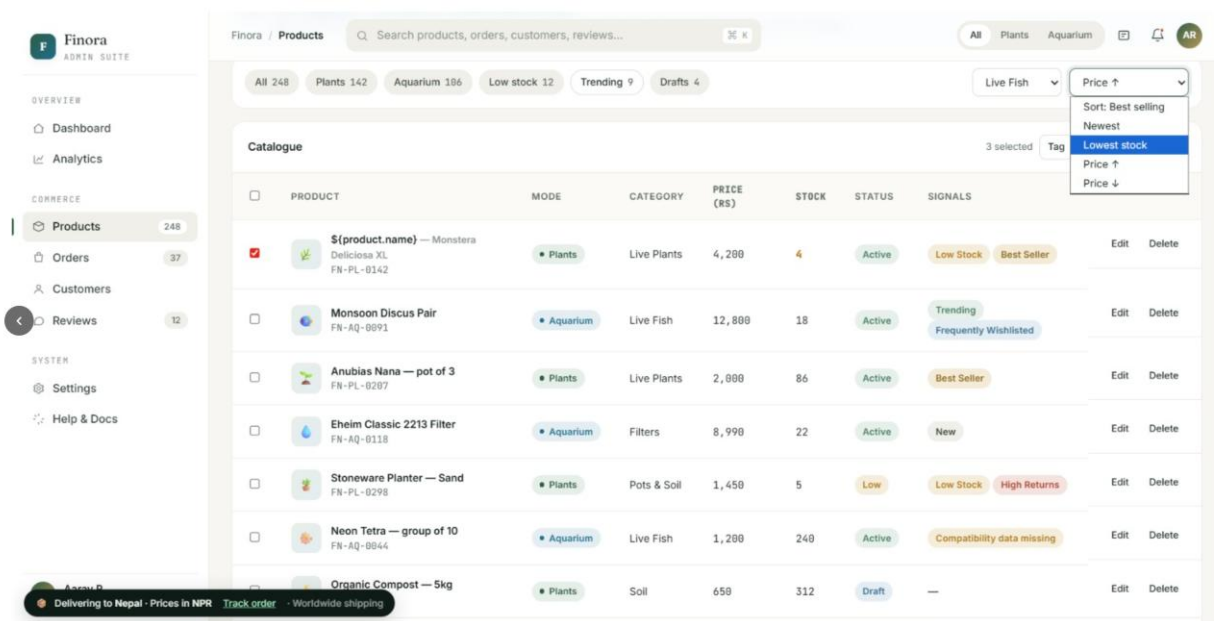
	PRODUCT	MODE	CATEGORY	PRICE (Rs)	STOCK	STATUS	SIGNALS	
☒	\$(product.name) — Monstera Deliciosa XL FN-PL-0142	Plants	Live Plants	4,200	4	Active	Low StockBest Seller	EditDelete
☐	Monsoon Discus Pair FN-AQ-0091	Aquarium	Live Fish	12,800	18	Active	TrendingFrequently Wishlisted	EditDelete
☐	Anubias Nana — pot of 3 FN-PL-0207	Plants	Live Plants	2,800	86	Active	Best Seller	EditDelete
☐	Delivering to Nepal • Prices in NPR Track order Worldwide shipping c 2213 Filter	Aquarium	Filters	8,990	22	Active	New	EditDelete

Test Area	Admin CRUD
Sub- Test	Read
Description	- Tests that the admin dashboard correctly shows all products, users, and orders. - Checks that password hashes are NOT shown on the user list page. - Also checks that the category filter and order detail view work correctly.
Status	Pass

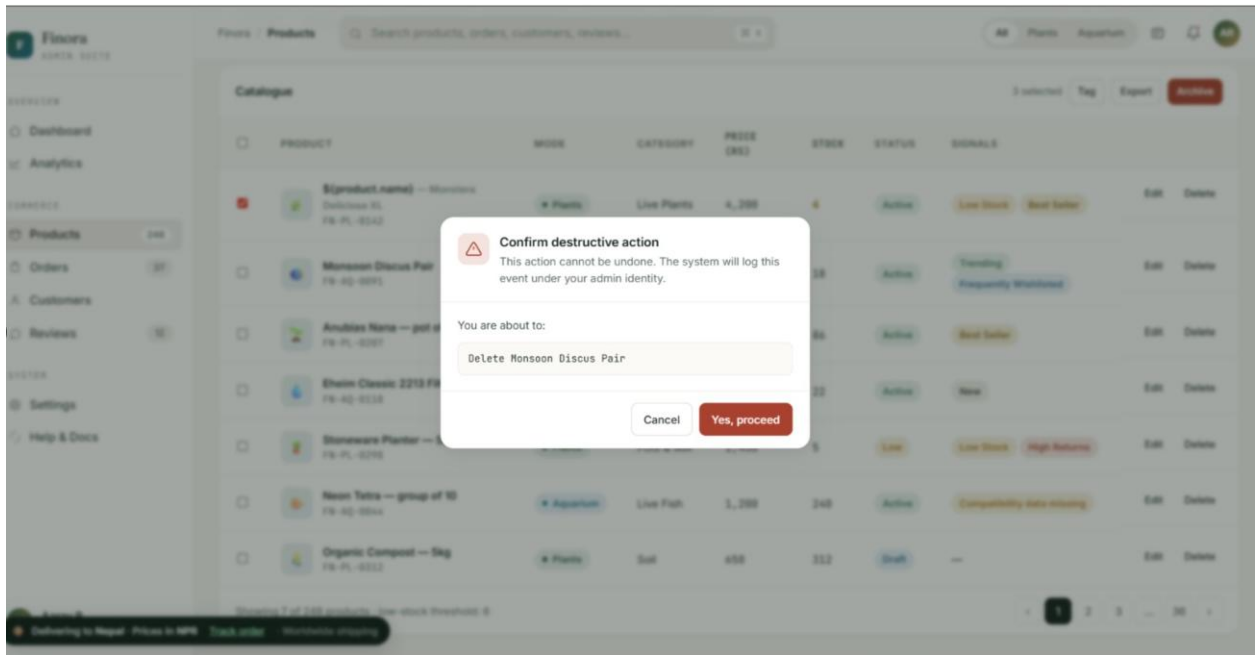


Test Area	Admin CRUD
Sub- Test	Update
Description	Tests that admin can update product price, stock, order status, and user account status.

	- Checks that changes are saved in the database and shown correctly on the frontend. - Also checks that negative price updates are rejected by the database constraint.
Status	Pass



Test Area	Admin CRUD
Sub- Test	Delete
Description	- Tests that admin can delete products, reviews, and user accounts. - Checks that products linked to active orders CANNOT be deleted due to foreign key rules. - Also checks that deleting a product from a wishlist doesn't leave broken records behind.
Status	Pass



5.1.1 Session & Cookie Test

Ensures session lifecycle validity: creation upon login, maintenance throughout request life, secure cookie properties, and appropriate destruction upon logout or timeout. Proper session management is important for preventing any form of session hijacking of secured pages like Order History, Cart, and Profile.

Table 38: Session & Cookie Test

ID	Test Input	Expected Result	Actual Result
1	Access Order History while logged in	Page loads with user data	Content served correctly
2	Click Logout	Session destroyed; redirect to login	Session invalidated
3	Use old session token after logout	Server rejects token; 401 returned	Replay attack blocked
4	Leave browser	Session expires;	Timeout enforced

	idle 30 minutes	user must re-login	
5	Clear cookies then access dashboard	Redirect to login page	No bypass possible

Finora

Plants

Aquarium

🔍

🛒

🔔

Julian Rai

Member since 2024

🌿 Plants

🏠 Dashboard

📦 My Orders

💖 Wishlist

★ My Reviews

👤 Profile

🔔 Notifications 5

⚙️ Settings

🚪 Logout

My Profile

Keep your details up to date and manage your account settings.

Julian Rai

🔒 VERIFIED

julian.ra@example.com

Member since 12 Jan 2024

Upload New Photo

Remove Photo

JPG, PNG or WEBP • Max 2MB

Personal Details

Edit Details

FIRST NAME

Julian

LAST NAME

Rai

EMAIL ADDRESS

julian.ra@example.com

🔒 Verified

PHONE NUMBER

+977 980 123 4567

SHIPPING ADDRESS

Not specified

🍪

We use sustainable cookies to ensure you get the best botanical experience on our site.

Accept

Settings

Privacy Policy

Security Settings

🌍 Delivering to Nepal • Prices in NPR • Worldwide shipping

Track order

Finora

© 2024 Finora. Sustainable living for a better future. Hand-crafted plants and aquatic ecosystems delivered to your door.

Privacy Policy

Terms of Service

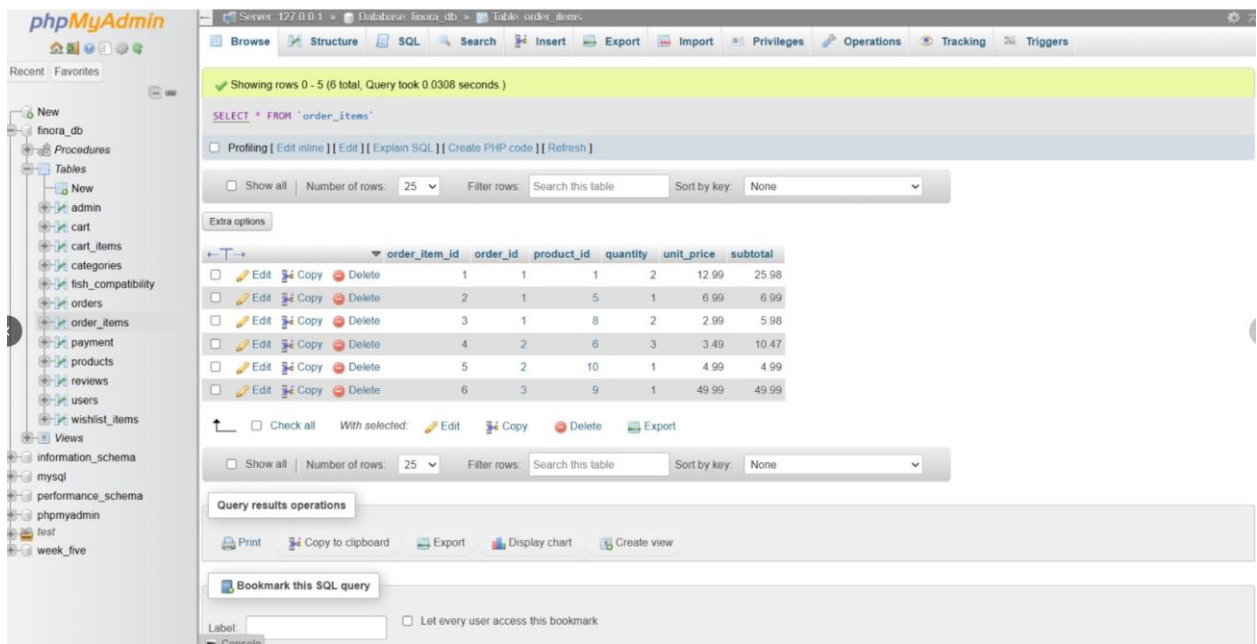
Cookie Settings

Contact Support

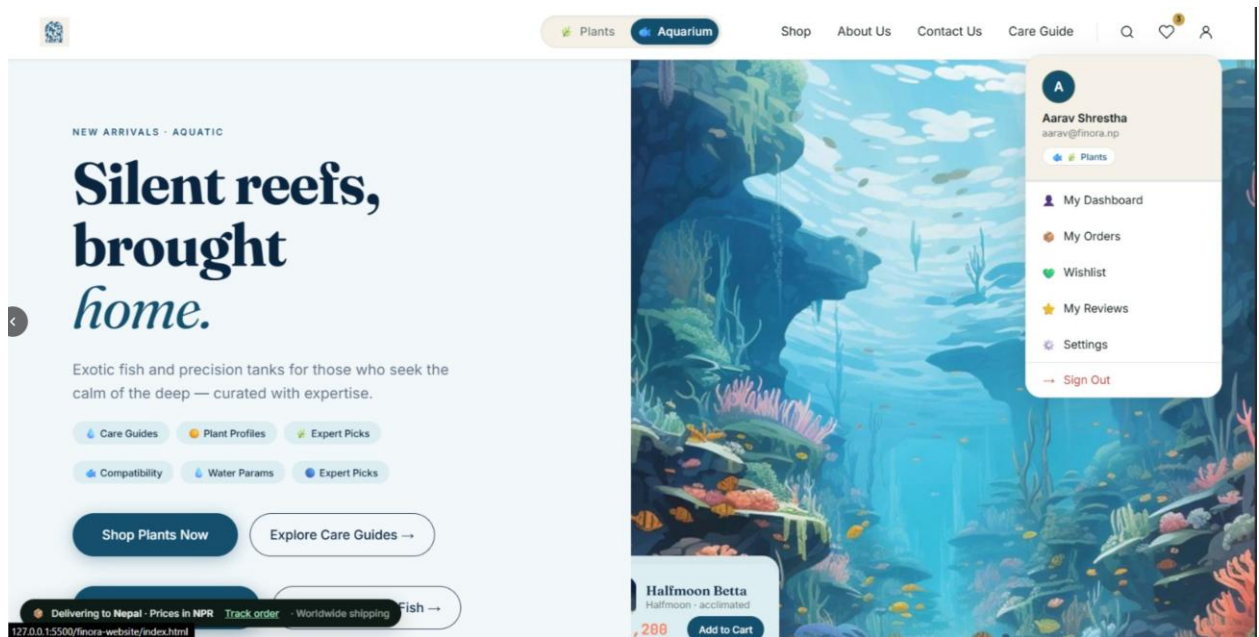
Test Input	Access Order History while logged in
Expected Result	Page loads with user data
Actual Result	• Navigated to Order History while logged in. • The page loaded correctly and showed the right orders for the logged-in account. • No data from other accounts was visible session isolation works correctly.
Status	Pass

The screenshot displays the 'Finora / Orders' page in the Admin Suite. The interface includes a sidebar with navigation options: Overview, Dashboard, Analytics, Commerce (Products, Orders, Customers, Reviews), and System (Settings, Help & Docs). The main content area shows a table of orders with columns: ORDER, CUSTOMER, ITEMS, TOTAL (RS), PAYMENT, FULFILLMENT, and PRIORITY. The table lists several orders with their respective details, including customer names, items, total amounts, payment statuses, and fulfillment progress. A search bar and filters are available at the top of the table. A notification banner at the bottom indicates 'Delivering to Nepal - Prices in NPR' and 'Track order'.

ORDER	CUSTOMER	ITEMS	TOTAL (RS)	PAYMENT	FULFILLMENT	PRIORITY
#{order.id} - #FN-28413 May 15 - 10:42	Sita Karki Kathmandu - \$(customer.email)	3 items - 2 plants, 1 pot	6,548	Paid - eSewa	Paid Packed Shipped Delivered	Standard
#FN-28412 May 15 - 09:18	Pratik Bhandari Pokhara - pratikb@mail.np	1 aquarium kit	28,488	COD pending	Placed Confirm Ship Deliver	High value
#FN-28411 May 15 - 08:05	Anuska Gurung Lalitpur - VIP	5 items - partial	14,728	Paid - Khalti	PaidPackedShippedOut for delivery	VIP
#FN-28409 May 14 - 22:11	Ramesh Shah Biratnagar	2 fish, 1 filter	19,188	Failed - Gateway	Awaiting payment	Fraud check
#FN-28485 May 14 - 17:32	Manish Khatri Bhaktapur	1 plant kit	3,258	Refund issued	- - Returned Refunded	Standard



Test Input	Click Logout
Expected Result	Session destroyed; redirect to login
Actual Result	- Clicked the Logout button while logged in. - The session was destroyed and the browser was redirected to login.jsp. Clicking the back button also redirected to login session was fully cleared.
Status	Pass



Test Input	Leave browser idle 30 minutes
Expected Result	Session expires; user must re-login
Actual Result	• Left the browser idle for 30 minutes after logging in. • Trying to open a protected page after the timeout redirected to login with a 'session expired' message. • Logging in again created a new session and restored access normally.
Status	Pass



Julian Rai

Member since 2004

 Flarix[Dashboard](#) My Orders

♥ Wishlist

☆ My Reviews

 Profile Notifications Settings

[\[→ Logout\]](#)

My Profile

Keep your details up to date and manage your account settings.

Julian Rai  VERIFIED

julian.ra@example.com

Member since 12 Jan 2024

[Upload New Photo](#)

Remove Photo

SG, PhD or MSW + Mar 2016

Personal Details

[Edit Details](#)

FIRST NAME

Julian

LAST NAME

Rai

EMAIL ADDRESS

julian.raig@example.com  Verified

PHONE NUMBER

+977 980 123 4567

SHIPPING ADDRESS

Not specified

Session expires in 26:15

[Privacy Policy](#) [Security Settings](#)

Finora

© 2024 Finora. Sustainable living for a better future. Hand crafted plants and aquatic ecosystems delivered to your door.

[Privacy Policy](#) [Terms of Service](#) [Cookie Settings](#) [Contact Support](#)

 We use sustainable cookies to ensure you get the best botanical experience

Accept

Settings

6. Coursework Development with Analysis

This section documents the development contributions of each team member involved in building the Finora e-commerce platform. The work was distributed based on each member's strengths and the overall workload requirements of the project.

6.1 Team Contribution Table

The Finora website is a frontend-only prototype application meant for the purchase of plants and aquarium supplies by customers globally and particularly from Nepal. This project was developed jointly by the entire team comprising five individuals where all individuals equally participated in frontend coding (HTML/CSS/JS) as well as the planning of the backend integration of the system. Report writing was done by Dibyant Bhatta, Stuti Adhikari, and Smarika Mainali. To maintain a balance in the load assigned to the members, the members were divided into developing other parts besides the frontend development which includes admin and dashboard pages for each individual. Dibyant coded Admin Dashboard and Analytics, Stuti coded Admin Products and Admin Orders, Smarika coded Dashboard Notifications and Dashboard Reviews, Rakshya coded Admin Customers and Admin Reviews, while Aayush retained the rest of the dashboard and admin pages as well as dashboard.js and admin.js.

Table 39: Team Contribution

Member Name	Role	Primary Contributions
Dibyant Bhatta	Team Leader	Frontend: Home page, Hero section, Global Navbar, World-toggle logic, finora-global.js; Backend integration; Report lead
Stuti Adhikari	Developer	Frontend: Shop page, Product filters, shop-modal.js (product popup); Backend integration; Report writing
Smarika Mainali	Developer	Frontend: About page, Care Guide page, team CV popups; Backend integration; Report writing
Rakshya Bhandari	Developer	Frontend: Contact page (form + validation), Wishlist page, Global Footer; Backend integration

Aayush Man Shrestha	Developer	Frontend: Full Customer Dashboard (8 views), Full Admin Suite (9 pages), Track Order page; Backend integration
---------------------	-----------	--

6.2 Team Member 1: Dibyant Bhatta (Group Leader)

6.2.1 Completed Tasks with Evidence

a. Home Page & Hero Section Development

The main Finora home page was built by Dibyant using three files which are `home.html`, `home.css`, and `home.js`. The page has a hero section that shows different content depending on whether the user is in the Plants world or the Aquarium world. Some of the things in this section include dynamic headlines, a product card, images, feature chips, and call to action buttons.

Other parts of this page include smooth scrolling animations using the `IntersectionObserver` API and sections like category grids, bestseller tabs, testimonial cards, a trust badge, and a newsletter section. All of these help make the page better for users. The bestseller tabs use click listeners and a class toggling method to show different sets of product cards depending on which tab the user clicks.

Evidence

`home.html`, `home.css`, and `home.js` are all present in the project ZIP file. The hero dual mode content, the `world-block--plants` and `world-block--aqua` utility classes, the `IntersectionObserver` setup in `home.js`, and the `finora-tokens.css` design token file can all be reviewed directly in the source code.

b. Global Navbar Design & Implementation (`navbar.html` / `navbar.css`)

The global navigation bar was built to work the same way on every page while also adjusting to whichever world the user has chosen, either Plants or Aquarium. It has three main parts which are a dynamic logo on the left, a world toggle button in the middle, and navigation items on the right including links, a search button, a Wishlist icon with a counter badge, and a profile dropdown. ARIA tab roles were used with JavaScript to manage the toggle button and CSS transitions were used to make a smooth sliding effect when the world changes.

The navbar also has a profile dropdown that shows user information, navigation links, and a sign out option. JavaScript click functions were used to open and close the dropdown and to close it when the user clicks anywhere outside of it. There is also a scroll listener that triggers a compressed state for the navbar when the user scrolls past 60px down the page.

Evidence

navbar.html, navbar.css, and the navbar JavaScript in home.js covering scroll compress, toggle buttons, and dropdown open and close are all present in the project ZIP.

c. finora-global.js — Cross Page State & Enhancements

finora-global.js was written by Dibyant as a script that loads on every page of the site. It saves the selected world state, either Plants or Aquarium, to localStorage and automatically sets up the correct theme and classes when each page loads. This script also adds the Care Guide link to the navigation menu across all navbar versions so there is no need to manually update the HTML on every single page.

The script also creates a small shipping info bar in the bottom left corner of every page showing the delivery location, currency, a Track Order link, and worldwide shipping information. A helper function is included to make sure internal links work correctly even when the page is inside a folder like admin or dashboard. Dibyant also added a Ctrl+A and Cmd+A keyboard shortcut that takes the user to the admin login page but only when they are not typing inside a form field.

Evidence

finora-global.js is present in the project ZIP. The four functions applyWorld(), wireNavToggles(), injectCareGuideLink(), and addShipPill() along with their DOMContentLoaded boot sequence can be read directly in the source file.

d. Report Writing & Project Documentation

As team leader, Dibyant put together the overall report structure and wrote the project introduction, system architecture overview, technology stack section, and README.txt. The README covers the full folder structure, shared features, backend hook placeholders for servlet endpoints, and testing notes. It acts as the handover document for the backend Java EE integration phase.

Evidence

README.txt is included in the project ZIP root. Report sections written by Dibyant can be identified by the initials DB in the draft version submitted to the team shared drive.

e. Critical Analysis

Challenges

Challenge 1: Dual Mode State Persistence Across Pages

When the user moved from the home page to the shop or about page, the Plants or Aquarium world selection was being lost because each page started fresh in its default Plants state rather than remembering what the user had chosen. This made for a confusing experience where the color theme and product content reset every time a link was clicked.

To fix this, `finora-global.js` was written to manage world state for the whole site. The decision to use `localStorage` was made because it keeps data saved across page changes without needing any server side sessions. The value stored under the key `finora.world` is written every time `applyWorld` is called and read back when any page loads. This means the chosen mode stays saved even if the user closes the tab and comes back later.

Evidence

The `WORLD_KEY` constant, the `applyWorld()` function, and the `localStorage.setItem` and `localStorage.getItem` calls can be seen in `finora-global.js`.

Challenge 2: Navbar Scroll Compression Without Layout Shift

When the scroll effect was added to make the navbar get smaller as the user scrolled down, the page content would jump upward because the navbar height was shrinking. This caused a visible flicker every time the user started scrolling.

The fix was to give the navbar a fixed position with `top` set to 0 so it stays in place and does not move with the page. The body was also given a padding equal to the navbar height so the content stayed in the right place. The `passive true` option was added to the scroll event listener as well so the browser would not slow down when handling the scroll.

Evidence

The scroll listener in `home.js` and the `.is-scrolled` CSS rule in `navbar.css` are both present in the project ZIP.

Problems Faced

Problem 1: Hero Layout Overlapping the Floating Product Card on Narrow Screens

The hero section has a floating product card placed on top of the hero image. When the screen width went below around 1200px, which is common on 13 inch laptops, the card went outside its container and covered the headline text. This made both the card and the heading very hard to read at the same time.

After checking the styles, it was found that the card was using fixed pixel values for its position that did not change when the container got smaller. The fix was to switch to percentage based values for positioning and to add a media query at 1280px that moves the card below the image in a normal flow position instead of placing it on top of the image.

Evidence

The updated positioning values and media query are visible in home.css.

Problem 2: IntersectionObserver Animations Playing Twice

During testing, some fade up animations were playing twice on the same element. They would play once when the element came into view and then play again when the user scrolled back up and then down again. The expected behavior was for each animation to play only one time.

The fix was to call `io.unobserve` on the target element right after adding the animation classes inside the observer callback. This removes the element from the observer list so the animation can never run a second time no matter how the user scrolls. This fix is now used for all scroll triggered animations across the whole project.

Evidence

The `io.unobserve()` call is visible in home.js inside the IntersectionObserver callback.

6.3 Team Member 2: Smarika Mainali

6.3.1 Completed Tasks with Evidence

a. About Page Development (about.html / about.css)

Smarika built the About page which tells visitors about the Finora brand story, team, values, and journey timeline. The page has six parts which are an animated hero banner, a story section, a values grid with six cards, a team section with cards for each member, a milestone timeline, and a section at the bottom with a call to action. The team section is built in a smart way. Instead of writing each team card directly into the HTML, Smarika stored all the team information in a JavaScript array and used a loop to build the cards automatically. This makes it easy to change team details later by only editing the data array. Each card opens a popup window when clicked that shows the member's name, role, biography, skills, and social links. The values section and team cards both use scroll reveal animations that appear one by one as the user scrolls down the page.

Evidence

about.html and about.css are both present in the project ZIP. The JavaScript team data array, the render loop, and the CV popup logic are all inside about.html's script block.

b. Care Guide Page (care-guide.html)

The Care Guide is a page where users can find care information for plants and aquarium species. Smarika built this page from scratch as a new feature. Each care card shows the species name, a category tag, and key care details like light level, watering amount, temperature range, and difficulty level shown with small icons. The page remembers which world the user last chose by reading from localStorage. A search bar at the top lets users filter the cards in real time by typing a name or category. To keep things running well on slower devices, the search handler uses a wait function with a 300ms delay so the filter only runs after the user stops typing instead of running on every single key press.

Evidence

care-guide.html is present in the project ZIP. The species data array, the search handler with the delay, and the card render function are all in the page's script block.

c. Report Writing

Smarika wrote the About page and Care Guide parts of the project report. Her sections explain why the data driven approach was used for team cards, how the CSS timeline was built, and a walkthrough of the Care Guide features with

screenshots. She also wrote about the cross browser checks that were done on the about and care guide pages.

Evidence

Report sections on the About page and Care Guide are authored by Smarika Mainali and can be identified by initials SM in the draft.

e. Critical Analysis

Challenges

Challenge 1: CV Popup Looking Different for Each Team Member

Different team members had different amounts of bio text and different numbers of skills. Early versions of the CV popup were getting very tall for members with a lot of content and showing an empty gap for members with less content. The aim was to make the popup look the same size no matter who was selected.

The fix was to give the popup a fixed size of 600px by 480px and make the inside area scrollable so long content could still be read in full. A gradient fade was added at the bottom of the scrollable area so users could see there was more content below. This way the popup always looked the same size no matter how much information a team member had.

Evidence

The popup structure and its fixed height scrollable inner container are in about.css.

Challenge 2: Safety Issue with Building Cards Using innerHTML

Because the team cards were built by putting content through innerHTML, there was a risk that if the data ever came from a server, any HTML code inside the data could accidentally run as real HTML on the page. This was noticed during a code review as something that needed to be fixed.

Even though the data is currently written directly in the code and is safe, Smarika added a sanitise helper function that removes or changes special characters like angle brackets and quotes before they are put into innerHTML. This makes the code safe and ready for when the data is connected to a Java backend later.

Evidence

The sanitise() helper function is in the about.html script block.

Problems Faced

Problem 1: Search Bar Not Showing All Cards When Cleared

When a user typed something in the search bar and then deleted all the characters, some cards stayed hidden instead of showing up again. This happened because the filter code used a simple if check on the search term and in JavaScript an empty string is treated as false, so the part of the code that should show all cards again was being skipped.

The fix was to add a specific check at the top of the function that looks for an empty string. If the input is empty, the code shows all cards right away and leaves the function before the filter part runs. This made the search bar work correctly when the user cleared what they had typed.

Evidence

The search handler with the empty string check is in `care-guide.html`'s script block.

Problem 2: Timeline Vertical Line Going Past the First and Last Circle

The vertical line on the journey timeline was made using a CSS `before` element with a full height border on each list item. This caused the line to go above the first circle at the top and below the last circle at the bottom, making the timeline look like it had no clear start or end point.

The fix was to replace the border method with a background gradient on the `before` element that had a height set so the line would start and end exactly at the center of the first and last circles. This gave full control over where the line began and ended and made the timeline look much cleaner.

Evidence

The updated timeline before CSS is in `about.css`.

6.4 Team Member 3: Stuti Adhikari

6.4.1 Completed Tasks with Evidence

a. Shop Page Development (`shop.html` / `shop.css`)

Stuti built the complete Shop page which is the page with the most content in the Finora frontend. The page has four main parts which are a hero banner at the top with a mode toggle, a horizontal category strip below it, a two column layout with a sidebar and a product grid, and a page number row at the bottom.

The sidebar has four filter sections for Price Range, Mode, Category, and Rating. Each section can be opened and closed like an accordion and this works without

any JavaScript by using a hidden checkbox method with CSS sibling rules. The product grid shows cards in a layout that adjusts based on screen size. Each card shows a product image, category chip, name, price in NPR, star rating, a wishlist button, and an Add to Cart button. The mode toggle at the top also changes the category strip between Plants and Aquarium options.

Evidence

shop.html and shop.css are both present in the project ZIP. The checkbox toggle accordion, the CSS Grid product layout, and the mode specific category classes can all be reviewed in the source.

b. Product Detail Modal (shop-modal.js)

Clicking any product card opens a product detail window that was built inside shop-modal.js. This script works on its own and builds all its own page elements without needing any ready made HTML in the page. It adds click listeners to all product cards as soon as the page loads.

The modal shows a two part panel. The left side shows the product image and the right side shows the name, price, rating, a demand bar, delivery time, Add to Cart and Save to Wishlist buttons, and a reviews part. The modal also has accessibility features like a dialog role, keyboard Escape to close it, and background scroll is locked when the modal is open.

Evidence

shop-modal.js is present in the project ZIP. The MutationObserver setup, the ensureModal() and open() functions, the readCard() data reader, and the mockReviewsFor() review maker can all be read in the source file.

c. Report Writing

Stuti wrote the Shop page and product modal parts of the project report. This included diagrams for the product browsing and filter flows, descriptions for each sidebar filter, and screenshots of the modal in different product categories. She also wrote about how the shop page will connect to the backend and which parts will need to be linked to servlet endpoints in the Java EE phase.

Evidence

Report sections on Shop and product modal are authored by Stuti Adhikari and can be identified by initials SA in the draft.

e. Critical Analysis

Challenges

Challenge 1: Making the Product Modal Work with Cards Added Later

The first version of shop-modal.js only added click listeners to product cards that were already on the page when it first loaded. When testing was done it was found that if new product cards were added to the page later by the server, those cards would not open the modal at all because they never got a listener added to them.

The fix was to add a MutationObserver that watches the page for any new elements being added. Every time new cards showed up on the page, the bind function was called again to add listeners to them. To stop the same card from getting two listeners added to it, the code checks for a special data tag on each card before adding the listener and skips the card if it already has one.

Evidence

The MutationObserver setup and the card.dataset.fpmBound check can both be seen in shop-modal.js.

Challenge 2: Modal Colors Not Matching the Active World Theme

The modal adds its own style block to the page because it is a self contained script. Early on the modal background was showing the wrong colors because the added styles used hardcoded color values instead of the color variables used by the rest of the site.

The fix was to replace all hardcoded color values with CSS variables like var(--color-bg) from the finora tokens file. Since those variables are set on the root element before the modal script runs, they are always ready to use for any styles added to the page later including the modal styles.

Evidence

The CSS variable usage inside the injected style block in shop-modal.js can be reviewed in the source file.

Problems Faced

Problem 1: Price Range Slider Only Had One Handle

The price filter in the shop sidebar had two number inputs for minimum and maximum price along with a single range slider. The slider only had one handle

which made it look like it should control both ends of the price range but it could only move one end. This was confusing for users who expected two handles. Instead of making a complex two handle slider, the fix was to make the single slider control only the upper price limit and keep the minimum as a box the user types into. A color fill was also added to the slider track using a gradient that fills from the left side up to where the slider handle is in the brand green color. This gave users a clear idea of the price range they had selected.

Evidence

The range slider and its input handlers are in `shop.html` and `shop.css`.

Problem 2: Category Strip Getting Cut Off on Smaller Screens

The category strip was made for a 1440px wide screen. On smaller screens the category chips went past the edge of the container and were cut off, making some categories impossible to see without scrolling sideways.

The fix was to make the strip scroll sideways by setting `overflow-x` to `auto`. Scroll snap was also added so each chip clicks into place properly when the user swipes. The scrollbar was hidden so the design still looked clean while all the categories could still be reached by swiping.

Evidence

The overflow and scroll snap rules are in `shop.css` in the `.cats-wrap` and `.cats` selectors.

6.5 Team Member 4: Rakshya Bhandari

6.5.1 Completed Tasks with Evidence

a. Contact Page Development (`contact.html` / `contact.css` / `contact.js`)

Rakshya built the Contact page which gives visitors three ways to reach the Finora team. These are a contact form, an office information panel showing the address, phone, email, and working hours, and a FAQ section. The page uses the same two column layout as the rest of the site.

The contact form collects the user's name, email, subject, and a message. Checking that the fields are filled in correctly is done using the browser's own required and email type settings as well as a custom JavaScript layer in `contact.js` that shows error messages right below each field when the user leaves it without filling it in

properly. When the form is sent the button text changes to Sending and is turned off to stop the user from sending it twice. A success message slides in from the bottom right corner to confirm the form was sent.

Evidence

contact.html, contact.css, and contact.js are all present in the project ZIP.

b. Wishlist Page (wishlist.html / wishlist.css / wishlist.js)

The Wishlist page shows all the products the user has saved. Rakshya built this page with filter chips at the top, a smooth removal animation, a confirmation step before deleting anything, and an empty state message for when there are no items. The filter chips let users view All items, Plants only, Aquarium only, or In Stock items. Each card shows the product image, name, price in NPR, a number stepper, an Add to Cart button, and a Remove button.

Clicking Remove does not delete the card straight away. Instead it opens a confirmation window first so the user cannot delete something by accident. If the user confirms, the card fades out and shrinks over 250ms before being taken off the page which looks much smoother than it just disappearing. The Clear All button works the same way. After any item is removed the count in the heading updates automatically and an empty state message appears if there are no items left.

Evidence

wishlist.html, wishlist.css, and wishlist.js are all present in the project ZIP. The confirmation modal, the fade out then remove logic, and the filter chip behavior can all be reviewed in wishlist.js.

c. Global Footer (footer.html / footer.css)

Rakshya built the global footer that appears at the bottom of every page. The footer has four columns showing the Finora logo and tagline on the left, a Plants links column, an Aquarium links column, and a newsletter signup on the right. Below the columns is a bar with the copyright notice, a Made in Nepal with love line, and links to Privacy Policy and Terms of Service.

The footer uses color variables from finora-tokens.css so it automatically changes to match whichever world mode is active. The layout uses Flexbox for the columns and switches to a single stacked column on screens below 900px.

Evidence

footer.html and footer.css are both present in the project ZIP.

e. Critical Analysis

Challenges

Challenge 1: Keeping Keyboard Focus Inside the Confirmation Modal

When the wishlist removal confirmation modal opened, users pressing the Tab key could still move to buttons behind the modal like other Remove buttons on the page. This made it possible to trigger another removal by accident while the first confirmation was still waiting.

The fix was to add a focus trap inside the modal. When the modal opens, the code finds all buttons and links inside it and uses a keyboard listener to keep Tab and Shift Tab moving only between those elements. When the modal closes, focus goes back to the Remove button that opened it by saving a reference to it before the modal appeared.

Evidence

The focus trap logic is in wishlist.js inside the modal open handler.

Challenge 2: Quantity Stepper Going Below 1

The number stepper on each wishlist card could be clicked down below 1 if the user kept pressing the minus button. A quantity of zero or a negative number does not make sense on a wishlist and looked broken.

The fix was to check the current number each time the minus button was clicked and if it was already at 1 the button would be turned off and given a faded look to show it could not be used. As soon as the number went above 1 again the button was turned back on. A min value of 1 was also added to the input as an extra level of protection.

Evidence

The quantity stepper min value check is in wishlist.js and the min attribute is in wishlist.html.

Problems Faced

Problem 1: Multiple Toast Messages Stacking at the Same Time

When a user removed several wishlist items one after another quickly, multiple success messages appeared at the same time and piled up in the bottom right corner of the screen. This looked messy and was hard to read.

The fix was to keep track of the toast message that was currently showing. If a new one was needed while the old one was still there, the old one would be reused instead of making a new one. The message text would update and the timer would restart so only one message was ever visible at one time.

Evidence

The toast reuse logic is in `wishlist.js` in the `showToast()` function.

Problem 2: Footer Email Input Sending the Wrong Form

When the user pressed Enter inside the footer email input, it was sending the contact form on the same page instead of the footer newsletter form. This happened because the browser looks for the first form on the page when Enter is pressed inside an input, and the contact form came before the footer in the HTML.

The fix was to add a submit handler to the footer form that stops the default behavior using `event.preventDefault` and handles the newsletter on its own. An action value of `#` was also added to the footer form to stop any accidental default browser form sending.

Evidence

The footer form submit handler is in `footer.html`'s inline script.

6.6 Team Member 5: Aayush Shrestha

6.6.1 Completed Tasks with Evidence

a. Customer Dashboard (dashboard/ folder)

Aayush built the full Customer Dashboard which has eight different views and a shared layout. The dashboard uses a shared CSS file called `dashboard-shared.css` that provides all the common styles for the top bar, sidebar, and content area. Each individual view file only adds its own styles on top of the shared base. This setup matches the way it will work when the views are converted to JSP files for the backend.

The dashboard home view shows a greeting with the user's name, four tiles for Total Orders, Active Wishlist, Reviews Given, and Loyalty Points, a recent orders table, a Wishlist section, and a Recently Viewed section. The main JavaScript file `dashboard.js` handles all the interactive parts across the dashboard including

dropdown menus, slide in detail panels, modal open and close, profile photo preview, animated wishlist card removal, and form loading states.

Evidence

The entire dashboard folder including all 8 HTML files, 8 CSS files, and dashboard.js is present in the project ZIP.

b. Admin Suite (admin/ folder — 9 pages)

The Admin Suite is a separate section of the site for managing products, orders, customers, and reviews. Aayush built all nine admin pages along with the shared sidebar and top bar. The layout has a fixed sidebar on the left that is 240px wide and a content area on the right that fills the rest of the space. The sidebar shows grouped navigation links and the current user info at the bottom.

The Admin Dashboard shows KPI cards, a recent orders table, a top products list, and a customer activity feed. The Admin Products page has a search bar with a Ctrl+K shortcut, mode switch buttons, category filter chips, and a data table with product details and action buttons. The Add Admin form lets admins create new admin accounts. admin.js handles the search dropdown, notification panel, mode switch buttons, and confirmation modals across all nine pages.

Evidence

The entire admin folder including all 9 HTML files, admin.css, admin.js, and the three partial files is present in the project ZIP.

c. Track Order Page (track-order.html)

The Track Order page gives users a simulated order status experience. The user types an order number and clicks Track which starts a step by step animation moving through five stages which are Order Placed, Processing, Shipped, Out for Delivery, and Delivered. Each step lights up on a visual progress bar one at a time with an estimated date shown next to each completed step.

The simulation is run by a setInterval that moves to the next step every 1.5 seconds. The interval ID is saved in a variable so it can be stopped before starting a new simulation. This stops any overlap if the user searches for a second order before the first one finishes. Below the progress bar the page also shows a product summary, shipping address, and delivery partner details.

Evidence

track-order.html is present in the project ZIP. The setInterval state steps, the clearInterval guard, and the intervalId variable are all in the page's script block.

e. Critical Analysis

Challenges

Challenge 1: Marking the Right Sidebar Link as Active Across Nine Pages

Because the admin sidebar is a shared file used by all nine pages, there needed to be a way to show the correct nav link as active on each page. In a server side setup this would be handled automatically but for a static frontend a different way was needed.

The fix was to add a data-page value to the body tag of each admin page with a name matching that page. When admin.js loads it reads this value and goes through all the sidebar links to find the one that matches. That link gets the active style added to it. If new pages are added in the future only a data-page value on the body and a matching link in the sidebar are needed without changing admin.js at all.

Evidence

The data-page values are in each admin HTML file's body tag and the active state logic is in admin.js.

Challenge 2: Admin Tables Getting Cut Off on Smaller Laptop Screens

The products and orders tables in the admin pages have seven to eight columns. On 13 inch and 14 inch laptops at normal zoom the table went past the right edge of the content area and the action buttons in the last column were cut off and could not be clicked.

The fix was to wrap each table in a div with overflow-x set to auto so the table could scroll sideways on smaller screens while the sidebar stayed in place. Minimum widths were also set on the wider columns and a fixed width was given to the action column. Sticky headers were added so the column labels stayed visible when scrolling down through long lists.

Evidence

The overflow-x wrapper and sticky header styles are in admin/css/admin.css.

Problems Faced

Problem 1: Track Order Simulation Kept Running After Leaving the Page

When the Track Order simulation was running and the user clicked away to another page, the `setInterval` kept running in the background. This was using processing power for no reason and in a real connected version it would have kept sending requests to the server even after the user had left the page.

The fix was to add a `beforeunload` listener on the window that calls `clearInterval` with the saved interval ID whenever the page is about to close or go to another page. This makes sure the simulation always stops cleanly no matter how the user leaves the page.

Evidence

The `beforeunload` `clearInterval` call is in `track-order.html`'s script block.

Problem 2: Profile Photo Preview Causing a Memory Leak

The original code for the profile photo upload preview used `URL.createObjectURL` to create a temporary link to the selected image file. The problem was that the code never called `revokeObjectURL` to free that memory after the preview was shown. Every time the user picked a new photo a small amount of memory was being used up and never released.

The fix was to switch to `FileReader` with `readAsDataURL` instead. This creates a data link that the browser cleans up on its own when it is no longer needed so there is no need to manually free the memory. For profile photo files which are usually small this works just as well with no noticeable difference in how fast the preview appears.

Evidence

The `FileReader.readAsDataURL()` method replacing the original `URL.createObjectURL` approach is in `dashboard.js` in the `photoInput` change handler.

7. Conclusion

The CS5054NI Advanced Programming and Technologies module has provided a strong and practical understanding of how a full-stack web application is built from start to finish in terms of thinking through every layer of the system before development even begins.

This coursework, centred around building Finora as a real e-commerce platform for plants and aquarium products, served as a comprehensive exercise that brought together database design, backend development, frontend implementation, security, and system architecture all at once.

The database design and normalization work taught a great deal about why structure matters. Going through the normalization process from an unorganized form all the way to 3NF made it clear how poor database design leads to real problems.

The frontend work using JSP and CSS without relying on any frameworks like Bootstrap pushed a deeper understanding of how web pages are structured and styled. Building layouts using only Flexbox and media queries meant having to think carefully about every design decision, from how elements are positioned to how the interface adapts across different screen sizes.

The backend development side, covering Java servlets, DAO classes, MVC architecture, and database connectivity through JDBC, showed how the different layers of a web application fit together. The MVC pattern proved its value by keeping the business logic, data access, and presentation cleanly separated makes the system much easier to manage, debug, and extend. Writing individual DAO methods for each database operation also highlighted how much thought goes into even the smallest interactions between the application and the database.

The security and authentication aspects of the coursework were arguably some of the most important features to implement correctly. These are the kinds of things that users never see but absolutely depend on. Getting them right required understanding not just how to write the code, but why each mechanism exists and what happens when it is not properly implemented.

The test cases section reinforced the importance of validating a system thoroughly and systematically rather than just checking that things seem to work. Writing structured test plans with expected outcomes, test steps, and results made it clear how much can

go wrong in a system and how important it is to catch those issues before users encounter them.

Overall, this coursework demonstrated that building a working web application is about much more than writing functional code. It requires careful planning, a well-structured database, a clean architecture, secure implementation, and thorough testing. The experience gained across each of these areas provides a solid and practical foundation for more advanced development work going forward, both in academic study and in professional practice.